FOUNDATION OF EMBODIED AI

中文课程讲义

具身智能基础 · 全套课程讲义

13 个专题

源文件以课程仓库的构建清单为准

zengxiong111 / Foundation_of_Embodied_AI

目录

点击章节标题可跳转到正文。

01	Git 基础	3
02	ROS 2 基础	26
03	相机标定	48
04	视觉感知与状态估计	66
05	机器人动力学	70
06	Isaac / MuJoCo 仿真	87
07	具身强化学习	104
08	运动控制 RL	120
09	PPO	144
10	SAC	163
11	数据采集与模仿学习	180
12	Flow Matching	184
13	GR00T 与 SONIC	203

各章保留原始公式、插图与引用；独立 PDF 页码与全书页码可能不同。

Git 使用教程：从零基础到科研项目协作

课程首页 · English · PDF

项目	说明
适用范围	Git 2.x；Ubuntu 终端；GitHub 协作示例
文档核验日期	2026-10-18
许可	CC BY 4.0 · Xiong Zeng and contributors

核验边界： 文档整理与上游引用快照核对；未进行教程环境、训练或硬件验证。 版本与固定源码引用见版本说明。原创教材采用 CC BY 4.0，第三方引用、图片及代码遵循其原有许可；注明来源不等于拥有再许可权。

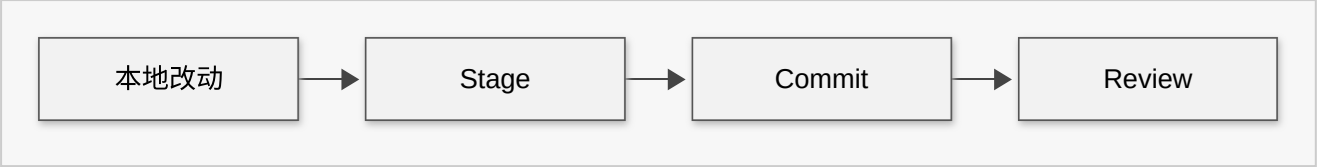
目录

- 1. Git 是什么：概念与工作区模型
  - 1.1 Git 能解决哪些问题?
  - 1.2 Git、GitHub、GitLab 和 VS Code
  - 1.3 理解工作区、暂存区、本地仓库、远程仓库
  - 1.4 最先记住的 10 个命令
- 2. Ubuntu 安装与初始配置
  - 2.1 安装 Git
  - 2.2 设置提交者身份
  - 2.3 统一新仓库默认分支名
  - 2.4 VS Code
- 3. 创建第一个本地 Git 项目
  - 第 1 步：创建目录和文件
  - 第 2 步：初始化仓库
  - 第 3 步：查看文件状态
  - 第 4 步：加入暂存区
  - 第 5 步：提交
  - 第 6 步：查看历史
- 4. 连接 GitHub：SSH、上传、克隆和同步
  - 4.1 配置 SSH 密钥
  - 4.2 将本地项目上传到 GitHub
  - 4.3 从 GitHub 下载开源项目
  - 4.4 更新本地仓库：Fetch 与 Pull
- 5. Git 分支管理
  - 5.1 查看、创建与切换分支
  - 5.2 在功能分支开发并提交
  - 5.3 合并到主分支

    - 5.4 删除已完成的分支
- 6. 多人协作与 Pull Request
    - 6.1 从最新主分支开始
    - 6.2 开发、检查并提交
    - 6.3 上传分支并创建 PR
- 7. 查看状态、差异和历史
    - 7.1 查看工作区状态
    - 7.2 比较代码
    - 7.3 查看提交图
- 8. 撤销操作与恢复旧版本
    - 8.1 尚未暂存：放弃某个文件的修改
    - 8.2 已 `git add`，尚未 commit：取消暂存
    - 8.3 已 commit，尚未共享：撤销提交并保留改动
    - 8.4 已 push / 其他人可能已拉取：用 Revert
- 9. 合并冲突与处理步骤
    - 9.1 解决冲突
    - 9.2 VS Code 解决冲突
- 10. `.gitignore`：忽略科研产物
    - 10.1 推荐模板
    - 10.2 `.gitignore` 的作用范围
    - 10.3 模型、数据与代码分离
- 11. 常用进阶命令：Stash、Rebase、Show、Blame
    - 11.1 `git stash`：临时收起未提交改动
    - 11.2 `git rebase`：把实验分支移到新基线
    - 11.3 查看某个提交、追踪某行代码
    - 11.4 查看所有分支
- 12. 完整科研开发工作流
    - 阶段 1：克隆自己的仓库
    - 阶段 2：从最新 `main` 创建实验分支
    - 阶段 3：开发并检查变化
    - 阶段 4：提交第一个独立功能
    - 阶段 5：继续增加算法功能
    - 阶段 6：将分支上传 GitHub
    - 阶段 7：测试并合并
- 13. Git 命令速查表
- 14. 常见错误及排查
    - 14.1 `fatal: not a git repository`
    - 14.2 `Permission denied (publickey)`
    - 14.3 `Updates were rejected`
    - 14.4 `Your local changes would be overwritten`

    - 14.5 常见错误处理原则
- 15. 在 VS Code 中使用 Git
- 16. 科研项目最佳实践与实验复现
    - 16.1 按任务创建分支
    - 16.2 每次训练记录 Commit ID
    - 16.3 把代码、配置和大文件管理分开
    - 16.4 新手先熟练掌握的一条完整路线
    - 16.5 官方参考资料

章节概念图



1. Git 是什么：概念与工作区模型

Git 是分布式版本控制系统（Version Control System, VCS）。它记录代码的修改历史，使你能够比较版本、恢复旧版本、在独立分支上尝试新算法，并与团队成员合并工作。

1.1 Git 能解决哪些问题?

- **版本管理**：每次提交都对应一个可追溯的代码状态。
- **安全试验**：在实验分支上尝试新奖励函数或训练算法，不干扰稳定版本。
- **错误恢复**：找回被误改的代码，或撤销引入 Bug 的提交。
- **团队协作**：独立开发 PPO、SAC、视觉感知和 Sim2Real 模块，再合并。
- **跨机器同步**：借助 GitHub/GitLab 在本地电脑与 GPU 服务器同步源码。
- **实验复现**：记录某次训练对应的 Commit ID，与配置、随机种子、数据版本关联。

1.2 Git、GitHub、GitLab 和 VS Code

工具	作用
Git	在本地管理版本、提交、分支和合并；大部分操作无需联网。
GitHub	在线托管 Git 仓库，提供 Pull Request、代码审查和自动化功能。
GitLab	Git 托管与协作平台的另一种选择。
VS Code	编辑器；内置 Source Control 界面，可操作 Git。

Git 不是 GitHub。即使没有 GitHub 账号，你仍能运行 `git init`、`git add`、`git commit`、`git branch` 等命令。

1.3 理解工作区、暂存区、本地仓库、远程仓库



三条最常用的命令含义不同：

```
git add .
git commit -m "Implement PPO training"
git push
```

- `git add`：将**选定的文件变化**放入暂存区。
- `git commit`：将**暂存内容**记录为本地提交。
- `git push`：将**本地提交**发送到远程仓库。

一个 Commit 可以理解为项目在某个时刻的一份版本快照；其哈希 ID 用于唯一标识该提交。工作区修改若未提交，**不会因为执行 `git push` 就自动上传**。

1.4 最先记住的 10 个命令

命令	用途
`git init`	初始化本地仓库
`git clone URL`	克隆远程仓库
`git status`	查看当前状态
`git add PATH`	暂存文件变化
`git commit -m "..."`	创建本地提交
`git push`	推送提交
`git pull`	获取并整合远程变更
`git log --oneline`	查看简短提交历史
`git switch -c NAME`	创建并切换分支
`git merge NAME`	合并指定分支

2. Ubuntu 安装与初始配置

2.1 安装 Git

打开 Ubuntu 终端：

```
sudo apt update
sudo apt install git
git --version
```

安装成功后会输出类似 `git version 2.x.y` 的版本信息；具体版本取决于 Ubuntu 发行版和软件源。

2.2 设置提交者身份

```
git config --global user.name "Your Name"
git config --global user.email "your_email@example.com"
```

这两个值写入新提交的作者身份；建议邮箱与你希望关联的 GitHub 邮箱一致。查看全局配置：

```
git config --global --list
```

也可以只给**当前仓库**设置不同的身份（移除 `--global`）：

```
git config user.name "Lab Account"
git config user.email "lab@example.com"
```

2.3 统一新仓库默认分支名

```
git config --global init.defaultBranch main
```

它影响之后通过 `git init` 创建的新仓库，不会自动重命名已有仓库。

2.4 VS Code

VS Code 内置 Git 集成功能；安装 Git 并在 VS Code 中打开项目即可使用 Source Control。若 `code` 命令已添加到 PATH，可通过：

```
code .
```

打开当前项目。Ubuntu 下日常操作建议使用“**终端执行命令 + VS Code 比较代码与解决冲突**”的组合。

---

3. 创建第一个本地 Git 项目

假设要开发一个机器人强化学习项目：

```
robot-policy/
├── train.py
├── evaluate.py
├── configs/
│   └── ppo.yaml
├── policies/
│   ├── ppo.py
│   └── sac.py
└── README.md
```

第 1 步：创建目录和文件

```
mkdir robot-policy
cd robot-policy
echo 'print("Hello Git")' > train.py
```

第 2 步：初始化仓库

```
git init
```

此时会出现隐藏目录 `.git/`，其中存放仓库对象、历史和配置。不要随意删除或手动编辑它。

第 3 步：查看文件状态

```
git status
```

可能看到：

```
On branch main
Untracked files:
  train.py
```

`Untracked` 表示文件尚未被 Git 跟踪。

第 4 步：加入暂存区

```
git add train.py
# 或暂存当前目录下的所有变化
git add .
git status
```

状态中的 `Changes to be committed` 表示文件已经暂存。`git add .` 很方便，但实验目录中应先确认 `.gitignore`，避免误加大型模型或数据。

第 5 步：提交

```
git commit -m "Initial commit"
```

以后每完成一个明确的开发任务就可以提交一次：

```
git commit -m "Implement PPO training"
git commit -m "Add SAC algorithm"
git commit -m "Fix observation normalization"
```

提交信息应说明“本次改动做了什么”，而不是只写 `update` 或 `fix`。

第 6 步：查看历史

```
git log
git log --oneline
```

示例：

```
f72bc15 Fix PPO reward computation
d43a812 Implement PPO training
a81c32f Initial commit
```

现在你已经可以在**离线环境**下完成 Git 版本管理。

---

4. 连接 GitHub：SSH、上传、克隆和同步

4.1 配置 SSH 密钥

在 Ubuntu 上长期开发，推荐为自己的 GitHub 账号配置 SSH。

先检查是否已有密钥：

```
ls -al ~/.ssh
```

如果没有可用的 Ed25519 密钥，创建一个：

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

按照提示保存密钥并设置口令。若密钥文件已存在，**不要不加判断地覆盖原文件**。

启动 SSH Agent，添加私钥：

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

输出并复制**公钥**：

```
cat ~/.ssh/id_ed25519.pub
```

在 GitHub 网页进入 **Settings → SSH and GPG keys → New SSH key**，粘贴公钥并保存。注意：`id_ed25519` 是私钥，**绝不能上传或公开**；只能分享 `id_ed25519.pub`。

测试 SSH 连接：

```
ssh -T git@github.com
```

首次连接时要核对 GitHub 服务器指纹并确认。连接成功后通常会看到认证成功但不提供 Shell 访问的提示。

参考：GitHub 官方 SSH 配置文档。

4.2 将本地项目上传到 GitHub

在 GitHub 网站创建一个名为 `robot-policy` 的**空仓库**；如果本地已经提交过代码，先不要在网页上添加 README、`.gitignore` 或 License，避免产生两份独立初始历史。

```
cd robot-policy
git remote add origin git@github.com:YOUR_USERNAME/robot-policy.git
git remote -v
git branch -M main
git push -u origin main
```

解释：

- `origin` 是远程仓库的常用**别名**，不是特殊服务器。
- `git remote -v` 显示读取与推送地址。
- `git branch -M main` 将当前分支命名为 `main`。
- `-u` 为本地分支建立上游跟踪关系，以后通常可以直接运行 `git push`。

4.3 从 GitHub 下载开源项目

例如：

```
git clone https://github.com/ajwagen/dsrl.git
cd dsrl
```

如果你使用 SSH 且具有相应访问权限，也可以克隆仓库提供的 SSH 地址：

```
git clone git@github.com:ajwagen/dsrl.git
```

`clone` 会下载仓库历史、创建项目目录，并默认配置 `origin` 远程仓库。克隆后无需再次 `git init`。

HTTPS	SSH
公共仓库一般可匿名克隆	通常需要配置 SSH 身份验证
使用 `https://...` URL	使用 `git@github.com:...` URL
推送时需要适当的账号认证	配置密钥后推送较方便

4.4 更新本地仓库：Fetch 与 Pull

```
cd dsrl
git fetch origin
```

fetch 只获取远程最新对象和远程跟踪分支，**不会直接合并到当前工作分支**。

如果当前位于跟踪 origin/main 的 main 分支，可执行：

```
git merge origin/main
```

把更新合并到当前分支。另一种常见写法：

```
git pull
```

pull 相当于先 fetch，再按配置采用 merge 或 rebase 整合。为了避免不期望的自动合并，新手在更新稳定分支时可以用：

```
git pull --ff-only
```

它只接受快进更新；若本地和远程历史已分叉，会停止并要求你明确处理。

> **注意**：在开源项目原仓库上通常没有推送权限。想提交自己的修改，可以先 Fork 到个人 GitHub，再克隆自己的 Fork，或者在团队仓库中通过授权分支发起 Pull Request。

---

5. Git 分支管理

分支允许你同时维护稳定版本和实验性代码。例如 main 保存稳定 PPO，feature/sac 用于新的 SAC 实现：

```
A --- B ---------------- M      main
       \               /
        C --- D -------/        feature/sac
```

其中 M 代表一次可能的合并提交；如果满足快进条件，Git 也可能不创建 M。

5.1 查看、创建与切换分支

```
git branch                      # 列出本地分支
git branch feature/sac          # 只创建分支
git switch -c feature/sac       # 创建并切换（择一使用）
git switch main                 # 切回 main
git switch feature/sac          # 切回实验分支
```

如果已经用 git branch feature/sac 创建了分支，就用 git switch feature/sac 切换，而不是再次运行同名的 switch -c。

Git 在切换分支时会更新工作区文件；如果未提交的修改与目标分支冲突，切换会被阻止。应先提交或使用 git stash。

5.2 在功能分支开发并提交

```
git switch -c feature/sac
# 编辑 policies/sac.py
git add policies/sac.py
git commit -m "Implement SAC policy"
```

这时新的 Commit 只会推进 `feature/sac`，不会直接改变 `main`。

5.3 合并到主分支

```
git switch main
git merge feature/sac
```

若没有冲突即可完成；若有冲突，参见第 9 节。

5.4 删除已完成的分支

```
git branch -d feature/sac
```

`-d` 会检查分支是否已安全合并。删除对应远程分支：

```
git push origin --delete feature/sac
```

在多人项目中，更推荐通过 GitHub Pull Request 将功能分支合并到受保护的 `main`。

---

6. 多人协作与 Pull Request

科研团队可按模块建立分支：

```
main
 ├── feature/ppo
 ├── feature/sac
 ├── feature/pose-estimation
 └── feature/sim2real
```

推荐采用 **Feature Branch → Push → Pull Request → Review → Merge** 流程。

6.1 从最新主分支开始

```
git switch main
git pull --ff-only
git switch -c feature/pose-estimation
```

6.2 开发、检查并提交

```
# 修改视觉位姿估计模块
git status
git diff
git add .
git commit -m "Implement visual pose estimation"
```

6.3 上传分支并创建 PR

```
git push -u origin feature/pose-estimation
```

在 GitHub 仓库页面创建 Pull Request，通常以 `feature/pose-estimation` 为源分支、`main` 为目标分支。团队成员可以在 PR 中检查差异、审阅、讨论、运行 CI 测试，最后合并。

团队协作的基本习惯：**不要在共享主分支直接进行未验证的大型算法改动**；让新特性先在隔离分支上通过基本测试。

---

7. 查看状态、差异和历史

调试模型训练问题时，知道代码**实际改了什么**往往比记住“我应该改了什么”更重要。

7.1 查看工作区状态

```
git status
```

可以看到当前分支、已暂存修改、未暂存修改和未跟踪文件。

7.2 比较代码

```
git diff                    # 工作区 vs 暂存区
git diff --staged           # 暂存区 vs 最近提交
git diff -- train.py        # 只看某个文件
```

差异示例：

```
- learning_rate = 3e-4
+ learning_rate = 1e-4
```

- 是旧内容，+ 是新内容。

7.3 查看提交图

```
git log --oneline
git log --oneline --graph --all
```

例如：

```
* a34bd21 (HEAD -> main) Merge SAC implementation
|\
| * d72f8a1 Implement SAC algorithm
| * f61b2c9 Add replay buffer
|/
* b83fc92 Implement PPO
* 28ad613 Initial commit
```

建议经常组合使用 `git status`、`git diff`、`git log --oneline --graph --all`。每次提交前，先确认没有误加入临时文件、模型权重或敏感信息。

---

8. 撤销操作与恢复旧版本

恢复操作必须区分：修改**是否已暂存、是否已提交、是否已共享**。下面的命令不等价。

8.1 尚未暂存：放弃某个文件的修改

```
git restore train.py
```

把工作区的 `train.py` 恢复到暂存区对应的内容。**未另行保存的工作区修改可能丢失**。

8.2 已 `git add`，尚未 commit：取消暂存

```
git restore --staged train.py
```

修改仍保留在工作区，只是不再参与下一次提交。

8.3 已 commit，尚未共享：撤销提交并保留改动

假设最近两次提交：

```
a82bc31 Bad reward modification
b41d2ab Working PPO version
```

撤销最近提交，但仍保留暂存：

```
git reset --soft HEAD~1
```

撤销最近提交并取消暂存，但保留工作区改动：

```
git reset HEAD~1
```

连同相应工作区修改一起丢弃：

```
git reset --hard HEAD~1
```

`reset --hard` 是破坏性操作。 执行前务必确认代码、实验结果和未提交内容不需要保留。

8.4 已 push / 其他人可能已拉取：用 Revert

```
git revert a82bc31
```

revert 创建一个新的提交，尝试抵消某个旧提交所引入的改动，保留可审计的共享历史。若修改已经与后续提交交织在一起，可能需要解决冲突。

操作	影响	典型场景
git restore FILE	丢弃文件未暂存修改	放弃本地临时改动
git restore --staged FILE	取消暂存，保留改动	文件被误加到暂存区
git reset --soft HEAD~1	回退分支指针，保留暂存	整理未共享的提交
git reset HEAD~1	回退分支指针，保留工作区	重新选择修改进行提交
git reset --hard HEAD~1	回退并丢弃相应修改	确认无需保留时
git revert HASH	增加反向提交	撤销已经共享的提交

基本原则：**本地未共享的历史可以谨慎 reset；共享历史优先 revert**。

---

9. 合并冲突与处理步骤

两个分支修改了同一处内容时，Git 可能无法自动判断保留哪个版本。

例如同一个 train.py：

```
<<<<<<< HEAD
learning_rate = 0.0001
=======
learning_rate = 0.0003
>>>>>>> feature/sac
```

其中：

- <<<<<<< HEAD 到 =======：当前分支中的版本；
- ======= 到 >>>>>>> feature/sac：传入分支中的版本。

9.1 解决冲突

1. 运行 git status，定位冲突文件。
2. 打开文件，理解两边修改的实际意图。
3. 选择一边，或者编写兼容两边需求的最终代码。
4. **删除所有冲突标记**（<<<<<<<、=======、>>>>>>>）。
5. 运行测试，确认逻辑正确。
6. git add 标记该文件已解决，完成合并提交。

例如决定使用：

```
learning_rate = 0.0001
```

随后：

```
git add train.py
git commit
```

若想取消当前 merge：

```
git merge --abort
```

如果冲突发生在 rebase 中，使用的是 `git rebase --continue`、`git rebase --abort` 等命令，而不是 `merge --abort`。

9.2 VS Code 解决冲突

VS Code 的 Merge Editor 可以比较当前版本、传入版本和合并后的结果。无论通过图形界面还是终端解决，最终都应运行测试，尤其是奖励函数、动作尺度、机器人坐标系等难以仅凭文本差异判断的代码。

---

10. .gitignore：忽略科研产物

机器学习项目常见数 GB 的数据集、模型权重、录像和训练日志，它们通常**不适合直接提交到普通 Git 仓库**。

10.1 推荐模板

在项目根目录创建文件：

```
touch .gitignore
nano .gitignore
```

示例：

```
# Python cache
__pycache__/
*.pyc

# Local environments
.venv/
venv/

# Secrets
.env
*.pem

# Model artifacts
*.pt
*.pth
*.ckpt
checkpoints/

# Training outputs
outputs/
logs/
```

```
wandb/

# Large datasets
data/
datasets/

# ROS recordings
*.bag
*.db3

# Optional IDE state
.vscode/
```

根据项目调整规则：例如某些项目确实需要跟踪小型测试权重，或者共享部分 `.vscode/` 配置，不应机械地全部忽略。

10.2 .gitignore 的作用范围

规则主要影响**尚未跟踪**的文件。若 `checkpoint.pt` 以前已经被 Git 跟踪，即使现在写了 `*.pt`，它也不会自动退出版本控制。

要停止跟踪、但保留本地文件：

```
git rm --cached checkpoint.pt
git commit -m "Stop tracking checkpoint"
```

这不会清除历史提交中的旧副本。若意外上传 API Key、口令或其他机密，应该立即撤销/轮换凭据，并评估是否需要清理公开历史。

10.3 模型、数据与代码分离

建议：

- Git：源码、配置、脚本、文档、小型测试资源。
- 对象存储/实验平台：模型检查点、视频、训练日志和大型数据集。
- 实验元数据：保存代码 Commit ID、数据集版本、权重文件哈希和训练参数。

需要版本化大文件时，可以另行评估 Git LFS、DVC 或专门的模型/数据管理方案。

---

11. 常用进阶命令：Stash、Rebase、Show、Blame

11.1 `git stash`：临时收起未提交改动

你正在开发 SAC，却需要立即切换回 `main` 修复问题：

```
git stash
```

默认主要收起已跟踪文件的改动；若需要一起保存未跟踪文件：

```
git stash push -u -m "WIP SAC training"
```

然后：

```
git switch main
# 修复并提交问题
git switch feature/sac
git stash pop
```

命令	用途
`git stash list`	查看 Stash 记录
`git stash show -p`	查看最近 Stash 的差异
`git stash pop`	恢复修改；成功后移除该条 Stash
`git stash apply`	恢复修改，但保留该条 Stash

Stash 恢复也可能出现冲突，不能把它当作长期可靠备份。

11.2 `git rebase`：把实验分支移到新基线

原始历史：

```
main:    A -- B -- C
feature: A -- B -- D -- E
```

运行：

```
git switch feature
git rebase main
```

概念上变为：

```
main:    A -- B -- C
feature: A -- B -- C -- D' -- E'
```

`rebase` 会重放提交并生成新的提交 ID；适合整理**自己的私有分支**，不要随意重写其他人正在使用的共享历史。

11.3 查看某个提交、追踪某行代码

```
git show <commit-id>       # 查看一次提交的详细改动
git blame train.py         # 查看每行最近由哪次提交修改
```

`git blame` 在追踪“奖励缩放何时被修改”“哪个提交改变了控制频率”时很实用，但它是定位历史的工具，不等于判断责任。

11.4 查看所有分支

```
git branch -a
```

可能显示：

```
* main
  feature/sac
  remotes/origin/main
  remotes/origin/feature/ppo
```

远程跟踪分支（`remotes/origin/...`）代表本地记录的远程状态；执行 `git fetch` 可以更新这些记录。

---

12. 完整科研开发工作流

目标：在强化学习项目中开发 `residual_velocity.py`，新增 Advantage-Guided Velocity Loss，测试后合并。

阶段 1：克隆自己的仓库

```
git clone git@github.com:YOUR_USERNAME/robot-policy.git
cd robot-policy
```

如果仓库来自别人的开源项目，需要先 Fork，或按照项目允许的贡献流程操作。

阶段 2：从最新 `main` 创建实验分支

```
git switch main
git pull --ff-only
git switch -c experiment/residual-velocity
```

阶段 3：开发并检查变化

目录结构可以是：

```
robot-policy/
├── policies/
│   ├── sac.py
│   ├── ppo.py
│   └── residual_velocity.py
├── configs/
│   └── residual_velocity.yaml
└── train.py
```

编辑之后：

```
git status
git diff
```

阶段 4：提交第一个独立功能

```
git add policies/residual_velocity.py
git add configs/residual_velocity.yaml
git add train.py
git commit -m "Implement residual velocity adaptation"
```

阶段 5：继续增加算法功能

```
# 实现 Advantage-Guided Loss 并测试
git add .
git commit -m "Add advantage-guided velocity loss"
```

实验分支会形成清晰历史：

```
A --- B --- C           experiment/residual-velocity
      |     |
      |     └── Add advantage-guided velocity loss
      └──────────── Implement residual velocity adaptation
```

阶段 6：将分支上传 GitHub

```
git push -u origin experiment/residual-velocity
```

后续更新该分支通常只需：

```
git push
```

阶段 7：测试并合并

个人仓库可采用：

```
git switch main
git pull --ff-only
git merge experiment/residual-velocity
git push origin main
```

团队仓库建议在 GitHub 创建 Pull Request，经审核、测试和权限检查后合并。**合并成功不代表实验可复现**，还需要记录依赖版本、数据集版本、种子和检查点对应的 Commit ID。

---

13. Git 命令速查表

分类	命令	用途
初始化	`git init`	创建本地仓库
克隆	`git clone URL`	下载仓库
状态	`git status`	查看当前状态
差异	`git diff`	查看未暂存改动
历史	`git log --oneline`	查看提交历史
暂存	`git add .`	暂存当前目录变化
提交	`git commit -m "msg"`	创建提交

分类	命令	用途
上传	`git push`	推送本地提交
获取	`git fetch`	下载远程状态，不自动合并
同步	`git pull --ff-only`	仅允许快进更新
分支	`git branch`	列出本地分支
分支	`git switch -c name`	新建并切换分支
分支	`git switch name`	切换已有分支
合并	`git merge name`	合并指定分支
撤销	`git restore file`	丢弃未暂存的文件改动
撤销	`git restore --staged file`	取消暂存
撤销	`git revert HASH`	生成反向提交
暂存	`git stash`	临时收起改动
暂存	`git stash pop`	恢复 Stash
信息	`git show HASH`	查看指定提交
信息	`git blame file`	追踪文件行的提交历史

---

14. 常见错误及排查

14.1 `fatal: not a git repository`

当前目录及其父目录都找不到有效 Git 仓库。检查：

```
pwd
ls -a
```

确认是否进入了项目目录；只有确实要创建**新仓库**时才运行 `git init`。

14.2 `Permission denied (publickey)`

SSH 身份验证失败。检查：

```
ssh -T git@github.com
```

确认本地 SSH Agent 已加载正确密钥，并且公钥已添加到 GitHub 对应账号。

14.3 `Updates were rejected`

远程通常有本地不存在的提交，直接推送无法快进。先检查：

```
git fetch origin
git status
```

对于你自己的功能分支且当前上游配置正确，可根据团队规范选择：

```
git pull --rebase
# 解决可能出现的冲突并测试
git push
```

不要为了消除报错直接 `git push --force`；这可能覆盖别人的提交。

14.4 `Your local changes would be overwritten`

当前未提交修改可能被切换/同步操作覆盖。先二选一：

```
# 方案 A：提交当前工作
git add .
git commit -m "Save work in progress"
```

```
# 方案 B：临时保存（包括未跟踪文件）
git stash push -u
```

然后再重复原操作。

14.5 常见错误处理原则

遇到不熟悉的 Git 报错时，先运行 `git status`，阅读 Git 给出的后续操作提示，再判断是处于 merge、rebase 还是普通状态。不要在不清楚仓库状态时连续执行 `reset --hard`、`clean -fd` 或强制推送。

---

15. 在 VS Code 中使用 Git

在项目目录运行：

```
code .
```

左侧打开 **Source Control（源代码管理）**，快捷键通常是 `Ctrl + Shift + G`。

操作	VS Code 中的位置或方式
查看修改文件	Source Control 文件列表
比较修改前后代码	点击列表中的修改文件
暂存	文件旁的 + / Stage Changes
提交	输入提交信息并选择 Commit
推送	Push 或 Sync Changes
切换分支	状态栏分支名称或命令面板

操作	VS Code 中的位置或方式
解决冲突	Merge Editor

假设你将：

```
reward = success_reward
```

改成：

```
reward = success_reward + progress_reward
```

VS Code 的 Diff Editor 会直观显示删除和新增的行。对于涉及多文件配置的强化学习实验，在提交前检查一次完整 Diff，通常可以发现意外的超参数修改。

推荐搭配：终端更适合准确输入和记录 Git 命令；VS Code 更适合比较改动、定位冲突、审查大段代码。

---

16. 科研项目最佳实践与实验复现

16.1 按任务创建分支

典型科研场景	推荐方式
新增 PPO / SAC 算法	`feature/ppo` / `feature/sac`
探索新的奖励函数	`experiment/reward-v2`
实现在线 Residual Velocity 适配	`experiment/residual-velocity`
修改位姿估计或相机标定	`feature/pose-estimation`
修复线上机器人控制 Bug	`fix/control-latency`
论文复现实验	用 Commit ID 或 Tag 固定发布代码

16.2 每次训练记录 Commit ID

```
git rev-parse HEAD
```

建议和实验配置一起保存，例如：

```
experiment:
  name: residual_velocity_v3
  git_commit: a82bc31
  random_seed: 42
training:
  learning_rate: 0.0003
  batch_size: 256
```

上述 Commit ID 是演示用短哈希，正式记录最好写入命令输出的**完整哈希**。

> 重要：Commit ID 只标记已经提交的代码状态。如果实验启动时存在未提交修改，仅保存哈希并不能完整复现。正式运行前请检查 `git status`，必要时记录补丁或先建立实验提交。

16.3 把代码、配置和大文件管理分开

- **Git 仓库**：源码、配置、训练入口、评测脚本、README、环境说明。
- **实验输出目录**：TensorBoard/W&B 日志、视频、权重和大型缓存；通常加入 `.gitignore`。
- **复现记录**：Commit ID、环境与依赖版本、数据集版本、训练随机种子、仿真器版本、GPU/驱动信息。
- **机器人部署**：在发布稳定控制策略时，可创建 Tag（如 `v1.0-deploy`）并记录对应的控制参数和硬件版本。

例如为经过验证的版本添加 Tag：

```
git tag -a v1.0-deploy -m "Validated robot deployment"
git push origin v1.0-deploy
```

16.4 新手先熟练掌握的一条完整路线

```
# 1. 克隆项目
git clone <repo_url>
cd <repo_name>

# 2. 查看状态
git status

# 3. 创建实验分支
git switch -c experiment/new-method

# 4. 修改并检查代码
git diff

# 5. 提交代码
git add .
git commit -m "Implement new method"

# 6. 第一次上传分支
git push -u origin experiment/new-method

# 7. 后续修改并上传
git add .
git commit -m "Improve training"
git push
```

掌握这条路线，就足以支持大多数个人科研项目的日常代码管理；再逐步学习冲突解决、Rebase、Pull Request 和团队约定。

16.5 官方参考资料

- Pro Git（中文）
- Pro Git（English）
- Git 官方命令参考
- GitHub：通过 SSH 连接

- GitHub：Pull Request 指南

结语：对于科研代码，Git 最重要的价值不是“把代码上传到 GitHub”，而是将**源代码版本、实验变更与可复现结果**联系起来。先保持“小步提交、分支实验、审查 Diff、避免上传大文件”的习惯，再逐步扩展到团队协作。

} 器人学习中的304　完整教程

课程首页 · English · PDF

项目	说明
适用范围	Ubuntu 24.04；ROS 2 Jazzy；Python 示例
文档核验日期	2026-10-18
许可	CC BY 4.0 · Xiong Zeng and contributors

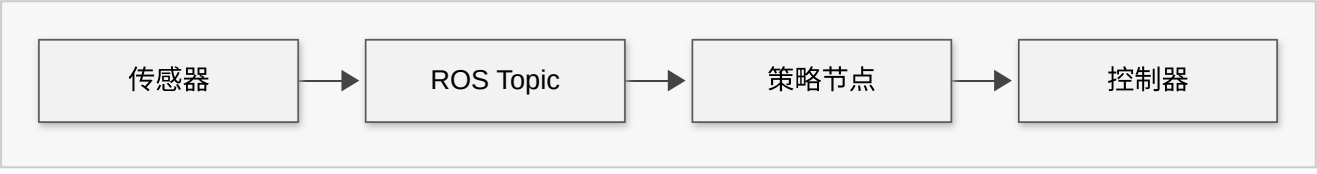
核验边界： 文档整理与上游引用快照核对；未进行教程环境、训练或硬件验证。 版本与固定源码引用见版本说明。原创教材采用 CC BY 4.0，第三方引用、图片及代码遵循其原有许可；注明来源不等于拥有再许可权。

目录

- 一、ROS 是什么？为什么机器人学习需要 ROS?
  - 1.1 ROS 的定义
  - 1.2 一个典型的机器人学习系统
  - 1.3 机器人学习一定需要 ROS 吗?
- 二、ROS 1 与 ROS 2，以及发行版选择
  - 2.1 ROS 1 与 ROS 2 的主要区别
  - 2.2 2026 年该选哪个 ROS 版本?
- 三、ROS 2 的核心概念
  - 3.1 Node：节点
  - 3.2 Topic：发布 / 订阅
  - 3.3 Message：消息格式
  - 3.4 Service：请求 / 响应
  - 3.5 Action：需要进度反馈的长任务
  - 3.6 Parameter：节点配置
  - 3.7 TF2：机器人坐标系管理
  - 3.8 QoS：通信质量配置
- 四、Ubuntu 24.04 安装 ROS 2 Jazzy
  - 4.1 检查系统、添加软件源
  - 4.2 安装 ROS 2 和开发工具
  - 4.3 配置环境并验证
  - 4.4 第一个 ROS 通信测试
  - 4.5 必须掌握的 ROS 命令
- 五、实操：用 Python 编写机器人学习策略节点
  - 5.1 创建 Workspace
  - 5.2 创建 Python Package
  - 5.3 编写策略节点

    - 5.4 声明依赖和命令入口
    - 5.5 编译并运行
    - 5.6 模拟状态输入并验证输出
    - 5.7 替换成 PPO / SAC 网络推理
- 六、ROS 2 如何用于机器人数据采集?
    - 6.1 遥操作数据采集架构
    - 6.2 什么是 rosbag2?
    - 6.3 记录与回放数据
    - 6.4 对机器人学习有用的数据字段
    - 6.5 多频率同步与数据集转换
- 七、URDF、TF2、RViz 和 ros2_control
    - 7.1 URDF：机器人结构描述
    - 7.2 URDF 与 TF2 的关系
    - 7.3 RViz 2：可视化工具
    - 7.4 ros2_control：连接高层命令与硬件
- 八、ROS 如何连接 Isaac Lab / MuJoCo 强化学习?
    - 8.1 仿真训练通常不需要 ROS
    - 8.2 真机部署需要连接各个模块
    - 8.3 Sim2Real 最关键的是接口语义一致
    - 8.4 控制频率与延迟
- 九、Tron2 + Revo3 灵巧操作项目架构
    - 9.1 推荐节点与信息流
    - 9.2 推荐 Workspace 组织
    - 9.3 机械臂与灵巧手的动作分发
    - 9.4 FoundationPose 的 ROS 接入
- 十、机器人学习中的 ROS 常见问题
    - 10.1 ROS 2 与 Conda 的关系
    - 10.2 与真实机器人相关的安全边界
- 十一、推荐学习路线
- 十二、总结与实践建议
- 十三、官方文档与参考资料

章节概念图



一、304 是什么？为什么} 器人学习需要 304？

1.1 ROS 的定义

ROS（Robot Operating System）虽然叫“机器人操作系统”，但并不是 Ubuntu、Windows 这样的传统操作系统。它是机器人软件开发框架及其生态系统，核心作用是让机器人中的不同软件模块通信、共享数据并协作运行。

一套机器人通常包含相机、机械臂、灵巧手、物体位姿估计器、神经网络策略和底层控制器。它们可能用不同语言开发、运行在不同进程中，甚至分布在不同计算机上。ROS 通过统一的消息、通信和工具链连接这些组件。

1.2 一个典型的机器人学习系统

假设 Tron2 + Revo3 需要完成“拿起螺丝刀并旋转”的任务，闭环结构如下：



必须区分以下工具的职责：

组件	在机器人学习中的主要作用
ROS 2	模块通信、系统集成、坐标变换、数据记录与调试
PyTorch	训练、加载和推理神经网络
PPO / SAC	强化学习算法
Isaac Lab / MuJoCo	物理仿真与强化学习训练环境
ros2_control	硬件抽象、控制器管理及命令接口
MoveIt 2	运动规划、运动学和碰撞检查
RViz 2	机器人状态、传感器及 TF 可视化
rosbag2	消息记录和回放

结论：ROS 负责“集成与通信”，学习框架负责“训练与推理”，控制器负责“执行”。 三者互补，而非相互替代。

1.3 机器人学习一定需要 ROS 吗?

不一定。训练任务如果只需要 Python 和物理仿真器，ROS 并非必需。

场景	是否需要 ROS	原因
Isaac Lab 中并行训练 PPO	不需要	直接调用仿真 API，减少额外开销
MuJoCo 中训练 SAC	不需要	环境与算法可以在 Python 内交互
离线训练 Diffusion Policy 或 VLA	不需要	直接使用数据加载器和 PyTorch
使用机器人厂商 SDK 控制单台机械臂	不一定	可以直接调用 SDK
多相机 + 机械臂 + 灵巧手集成	推荐	方便模块解耦、监控和同步
遥操作数据采集	推荐	方便多模态数据记录与回放
强化学习策略真机部署	推荐但非必需	规范 Observation/Action 与设备接口
多机器人或分布式系统	推荐	便于跨进程、跨计算机通信

> **实用判断**：高吞吐量的仿真训练环路通常不放进 ROS；真实机器人的传感器、状态估计、控制接口、数据记录和部署，则适合使用 ROS 2。

二、ROS 1 与 ROS 2，以及发行版选择

2.1 ROS 1 与 ROS 2 的主要区别

ROS 1 是早期生态，ROS 2 面向现代机器人系统重新设计，在分布式发现、QoS、生命周期管理、跨平台和实时控制集成方面提供了更完整的基础设施。

特性	ROS 1	ROS 2
发现机制	通常依赖 ROS Master	常见 DDS 中间件支持分布式发现
Python 接口	`rospy`	`rclpy`
C++ 接口	`roscpp`	`rclcpp`
记录工具	`rosbag`	`rosbag2`
参数	ROS Parameter Server	由节点管理参数
QoS	较有限	支持可靠性、历史、深度、期限等
控制框架	`ros_control`	`ros2_control`
新项目	一般不推荐从 ROS 1 开始	推荐使用 ROS 2

需要澄清：ROS 2 并非所有情况下都“自动实时”；它提供有利于实时系统集成的机制，真正的确定性仍取决于操作系统、控制线程、驱动和部署设计。

2.2 2026 年该选哪个 ROS 版本?

截至 **2026 年 10 月**，ROS 2 **Lyrical Luth** 已于 2026 年 5 月 22 日发布；但应当按当前 Ubuntu 系统和机器人驱动的兼容性选型。

Ubuntu	对应的常用 ROS 2 版本	备注
Ubuntu 22.04	Humble Hawksbill	长期支持至 2027 年 5 月
Ubuntu 24.04	**Jazzy Jalisco**	长期支持至 2029 年 5 月
Ubuntu 26.04	Lyrical Luth	长期支持至 2031 年 5 月

本教程使用 Ubuntu 24.04 + ROS 2 Jazzy。 如果厂商驱动只支持指定发行版，必须优先核对驱动兼容性。

三、ROS 2 的核心概念

3.1 Node：节点

Node 是 ROS 的基本计算单元，可以理解为一个独立的软件功能模块。节点并不必然等于独立进程；多个节点也可以被组合在同一进程中。

```
ROS 2 System
 ├── camera_node          # 读取图像
 ├── robot_state_node     # 读取关节状态
 ├── object_pose_node     # 估计物体 6D 位姿
 ├── rl_policy_node       # 策略推理
 └── controller_node      # 执行动作
```

Node 分工清晰，可以独立测试与替换，例如只更新 FoundationPose 而不修改机械臂驱动。

3.2 Topic：发布 / 订阅

Topic 适合连续、异步数据流：

```
camera_node --Publish--> /camera/image_raw --Subscribe--> perception_node
```

发送方无需知道接收方身份，一个 Topic 可以有多个订阅者。常见的机器人学习 Topic：

Topic 示例	内容
`/camera/color/image_raw`	RGB 图像
`/camera/depth/image_rect_raw`	深度图像
`/joint_states`	机器人关节状态
`/object_pose`	物体位姿

Topic 示例	内容
/policy/action	策略输出动作
/tf、/tf_static	动态、静态坐标变换

Topic 的核心用途就是在感知、Observation 构建、Policy 和控制器之间传输数据。 实际 Topic 名取决于驱动及命名空间。

3.3 Message：消息格式

Message 规定每条数据的字段与类型。常见类型：

Message 类型	用途
sensor_msgs/msg/Image	图像
sensor_msgs/msg/CameraInfo	相机内参和标定相关信息
sensor_msgs/msg/JointState	关节名称、位置、速度、作用力
sensor_msgs/msg/Imu	IMU 数据
geometry_msgs/msg/Pose	位置和姿态
geometry_msgs/msg/PoseStamped	带时间戳、坐标系的位姿
geometry_msgs/msg/Twist	线速度和角速度
trajectory_msgs/msg/JointTrajectory	关节轨迹
std_msgs/msg/Float64MultiArray	简单浮点数组

例如 sensor_msgs/msg/JointState 的主要字段：

```
header       # 时间戳及头信息
name         # 关节名称，如 ['joint_1', 'joint_2']
position     # 关节位置
velocity     # 关节速度
effort       # 关节力矩或作用力
```

Revo3 是否直接发布 JointState、各关节的名字与顺序是什么，需要由实际驱动或包装层确定。**不要只凭数组下标假定不同设备的关节顺序一致**。

3.4 Service：请求 / 响应

Service 适合短时间的、需要响应的一次性操作：

```
policy_node --Reset Request--> reset_server
policy_node <--Reset Result----- reset_server
```

典型操作有机器人 Reset、查询设备信息、切换控制模式、开始标定。Service 类似远程函数调用，但网络与服务端执行仍可能失败或超时。

3.5 Action：需要进度反馈的长任务

Action 适合“提交任务 → 反馈进度 → 返回最终结果”的过程，例如执行机械臂轨迹、导航到指定位置、完成抓取技能。Action 支持取消目标。

机制	最适合的通信模式	机器人学习例子
Topic	连续异步流	相机、状态、策略动作
Service	单次请求与响应	Reset、查询、切换模式
Action	长任务、反馈、结果	执行一个抓取动作或轨迹

对于 50 Hz 之类的周期性闭环动作指令，通常采用 Topic 或直接进程内接口，而不是每个周期都新建一个 Action。

3.6 Parameter：节点配置

参数用于调整节点配置，不必修改主程序：

```
policy_node:
  ros__parameters:
    control_frequency: 50.0
    use_camera: true
    observation_dim: 32
    policy_path: "/home/robot/policy.pt"
```

常见参数包括控制频率、相机名称、策略权重路径、目标坐标系、动作缩放与关节限位。

3.7 TF2：机器人坐标系管理

典型机器人包含多个坐标系：

```
world
  └── base_link
      ├── arm_base
      │     └── ee_link
      │           └── hand_base
      └── head_link
            └── camera_link
                  └── camera_optical_frame
```

例如 FoundationPose 计算物体在相机坐标系中的位姿 `T(C,O)`，抓取任务需要它在基座坐标系中的位姿：

坐标变换： `T(B,O) = T(B,C) × T(C,O)`

这里 `B` 表示机器人基座，`C` 表示相机，`O` 表示物体。TF2 允许查询相应坐标系之间、相应时间戳上的变换关系。

关键注意：TF2 本身不负责相机标定。 必须通过 URDF、手眼标定、外参测量等获得正确的 `T(B,C)`，然后 TF2 才能维护与使用这一关系。Tron2 头部相机、腕部相机、yaw/pitch 关节中心都涉及这一问题。

3.8 QoS：通信质量配置

QoS（Quality of Service）控制消息交付行为：

属性	含义
Reliability	`Reliable` 或 `Best Effort`
History / Depth	消息保留方式与队列长度
Durability	新订阅者是否可能收到历史保留数据
Deadline	期望更新期限；也可用于超时检测

传感器图像经常使用 `Best Effort`，避免追传过时帧；一些关键状态或控制指令可能使用 `Reliable`，但仍需避免旧指令积压。发布者和订阅者的 QoS 不兼容时，可能出现 Topic 存在却收不到数据的现象。

四、Ubuntu 24.04 安装 ROS 2 Jazzy

下面是基于官方 **Jazzy Ubuntu deb 包安装文档**的示例。安装命令需要联网、管理员权限，建议先确认机器尚未安装冲突的 ROS 发行版。

4.1 检查系统、添加软件源

```
lsb_release -a                 # 确认 Ubuntu 24.04
locale                         # 确认当前 locale 支持 UTF-8

sudo apt update
sudo apt install software-properties-common curl -y
sudo add-apt-repository universe

export ROS_APT_SOURCE_VERSION=$(curl -s \
  https://api.github.com/repos/ros-infrastructure/ros-apt-source/releases/latest \
  | grep -F 'tag_name' | awk -F\" '{print $4}')

curl -L -o /tmp/ros2-apt-source.deb \
  "https://github.com/ros-infrastructure/ros-apt-source/releases/download/${ROS_APT_SOURCE_VERSION}/ros2-apt-
source_${ROS_APT_SOURCE_VERSION}.$(. /etc/os-release && echo ${UBUNTU_CODENAME:-${VERSION_CODENAME}})_all.deb"

sudo dpkg -i /tmp/ros2-apt-source.deb
```

如果系统没有 UTF-8 locale，可参考官方文档设置：

```
sudo apt install locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
```

4.2 安装 ROS 2 和开发工具

```
sudo apt update
sudo apt install ros-jazzy-desktop ros-dev-tools
```

`ros-jazzy-desktop` 包含 RViz、示例程序和常用工具。如果不需要 GUI，也可使用更精简的 `ros-jazzy-ros-base`。

4.3 配置环境并验证

```
source /opt/ros/jazzy/setup.bash
echo $ROS_DISTRO              # 应输出 jazzy
```

若希望 Bash 新终端自动加载：

```
echo 'source /opt/ros/jazzy/setup.bash' >> ~/.bashrc
```

不要在同一终端误加载多个 ROS 发行版。新开终端仍需要确认环境已正确 source。

4.4 第一个 ROS 通信测试

终端 A：

```
source /opt/ros/jazzy/setup.bash
ros2 run demo_nodes_cpp talker
```

终端 B：

```
source /opt/ros/jazzy/setup.bash
ros2 run demo_nodes_py listener
```

预期可以看到 `Publishing: Hello World` 与 `I heard: Hello World` 等输出，说明 Python 和 C++ 节点能够互相通信。

4.5 必须掌握的 ROS 命令

```
ros2 node list
ros2 node info /camera_node
ros2 topic list
ros2 topic echo /joint_states
ros2 topic type /joint_states
ros2 topic hz /joint_states
ros2 topic bw /camera/color/image_raw
ros2 topic info -v /joint_states
ros2 interface show sensor_msgs/msg/JointState
ros2 service list
ros2 action list
ros2 run tf2_ros tf2_echo base_link camera_link
```

以上节点、Topic 和 frame 名均为示例。真机调试时，优先掌握 `ros2 topic list`、`echo`、`hz`、`info -v`，以确认消息内容、频率与 QoS。

五、实操：用 Python 编写机器人学习策略节点

目标是构建一条不连接真实电机的、可复现的最小链路：

```
/demo/joint_states
       | (JointState)
       ▼
```

```
    Policy Node
       | (Float64MultiArray)
       ▼
/demo/policy_target
```

5.1 创建 Workspace

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws
```

Workspace 的常见目录：

```
ros2_ws/
├── src/      # 源代码
├── build/    # 构建文件（编译时生成）
├── install/  # 安装后的程序（编译时生成）
└── log/      # 构建日志（编译时生成）
```

5.2 创建 Python Package

```
cd ~/ros2_ws/src
ros2 pkg create \
    --build-type ament_python \
    --license Apache-2.0 \
    --node-name demo_policy \
    robot_learning_demo
```

结构大致为：

```
robot_learning_demo/
├── package.xml
├── setup.py
├── setup.cfg
├── resource/
└── robot_learning_demo/
    ├── __init__.py
    └── demo_policy.py
```

`package.xml` 定义依赖；`setup.py` 定义 Python 包安装与可执行命令入口；`demo_policy.py` 实现策略节点。

5.3 编写策略节点

编辑 `~/ros2_ws/src/robot_learning_demo/robot_learning_demo/demo_policy.py`：

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import JointState
from std_msgs.msg import Float64MultiArray


class PolicyNode(Node):
    def __init__(self):
        super().__init__("policy_node")

        # Subscribe: virtual robot observation
```

```
        self.subscription = self.create_subscription(
            JointState,
            "/demo/joint_states",
            self.state_callback,
            10,
        )

        # Publish: demonstration action (not for real hardware)
        self.publisher = self.create_publisher(
            Float64MultiArray,
            "/demo/policy_target",
            10,
        )

    def state_callback(self, msg):
        q = list(msg.position)
        if not q:
            return

        # An illustrative mapping, not a trained RL policy
        action = [0.5 * x for x in q]

        command = Float64MultiArray()
        command.data = action
        self.publisher.publish(command)


def main(args=None):
    rclpy.init(args=args)
    node = PolicyNode()
    try:
        rclpy.spin(node)
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == "__main__":
    main()
```

这个演示策略为 `a_t = 0.5 q_t`，展示如何从 Observation 到 Action，不是安全的机器人控制策略。`Float64MultiArray` 也只用于快速演示；真实设备必须定义关节顺序、动作模式、单位、限位及命令超时处理。

5.4 声明依赖和命令入口

在 `package.xml` 内添加以下依赖（已有则不要重复）：

```
<exec_depend>rclpy</exec_depend>
<exec_depend>sensor_msgs</exec_depend>
<exec_depend>std_msgs</exec_depend>
```

确认 `setup.py` 的 `console_scripts` 包含：

```
'demo_policy = robot_learning_demo.demo_policy:main'
```

5.5 编译并运行

```
cd ~/ros2_ws
colcon build --symlink-install
source install/setup.bash
ros2 run robot_learning_demo demo_policy
```

节点启动后会等待状态消息。注意 `colcon` 构建前的终端应已加载 Jazzy 环境。

5.6 模拟状态输入并验证输出

新终端，首先加载环境：

```
source /opt/ros/jazzy/setup.bash
```

发布两维虚拟关节状态，频率 5 Hz：

```
ros2 topic pub -r 5 \
  /demo/joint_states \
  sensor_msgs/msg/JointState \
  "{name: ['j1', 'j2'], position: [0.4, -0.2]}"
```

另一个终端：

```
source /opt/ros/jazzy/setup.bash
ros2 topic echo /demo/policy_target
```

预期看到：

```
data:
- 0.2
- -0.1
```

这样就完成了“发布状态 → 订阅状态 → 执行策略 → 发布动作”的最小闭环通信验证。

5.7 替换成 PPO / SAC 网络推理

训练完成后，可以把示例映射：

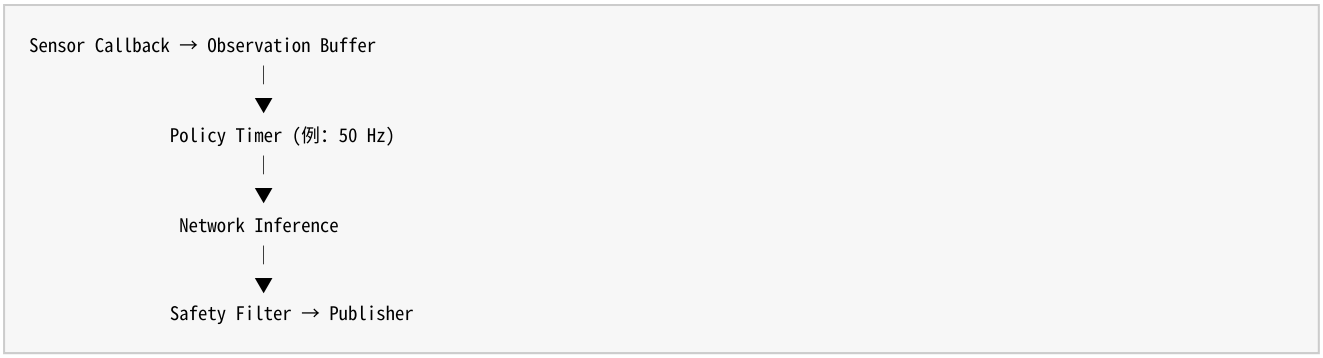
```
action = [0.5 * x for x in q]
```

替换为 PyTorch 推理（示意代码）：

```
with torch.inference_mode():
    action_tensor = policy(observation_tensor)
    action = action_tensor.cpu().numpy().tolist()
```

实际部署需要加载权重、构造正确 Observation、归一化、关节顺序映射、Action Scaling、处理设备和 dtype、限幅以及失败回退。

此外，更推荐把“消息接收”和“固定频率的推理周期”分开：



50 Hz 仅为示例，真实频率由模型推理时间、设备能力和任务决定。接收回调中无条件执行推理，可能引起频率漂移、消息积压和延迟。

六、ROS 2 如何用于机器人数据采集？

6.1 遥操作数据采集架构

以 PICO 遥操作 Tron2 + Revo3 为例：



6.2 什么是 rosbag2?

rosbag2 是 ROS 2 的消息记录与回放工具。它可以保存 Topic 消息及相关元数据，常见后端包括 SQLite3 和 MCAP。rosbag2 尤其适合重现数据采集过程，排查传感器和算法问题。

6.3 记录与回放数据

假设以下 Topic 已经存在：

```
/joint_states
/camera/color/image_raw
/camera/depth/image_rect_raw
/tf
/tf_static
```

记录指定 Topic：

```
ros2 bag record -o demo_001 \
  /joint_states \
  /camera/color/image_raw \
  /camera/depth/image_rect_raw \
  /tf \
  /tf_static
```

按 `Ctrl+C` 结束，然后查看或回放：

```
ros2 bag info demo_001
ros2 bag play demo_001
```

安全提示：回放 rosbag 可能重新发布原有控制命令。带有真实机械臂的系统应隔离控制 Topic 或断开执行通道，避免无意运动。

6.4 对机器人学习有用的数据字段

模仿学习常用轨迹可以写作：

$D = \{ (o_t, a_t) \}_{t=0,...,T}$

建议记录：

类别	示例
图像	头部 RGB、腕部 RGB、头部 Depth
机器人状态	机械臂与手指关节位置、速度、末端位姿
动作	实际下发的命令、目标关节位置或速度
时间	每个数据源的原始时间戳
标定	相机内参、外参、TF、机器人 URDF 版本
Episode 信息	任务、开始/结束时间、成功与否
可选	触觉、力矩、电流、物体 Pose

必须区分：

- **Commanded Action**：遥操作或策略发出的目标动作。
- **Executed State**：机器人真实传感器测量到的执行状态。

二者可能因延迟、控制器限制、饱和或碰撞而不同，不能直接混同。

6.5 多频率同步与数据集转换

常见管线：

```
ROS Bag
  ▼
Read Topics
  ▼
Timestamp Alignment
  ▼
Episode Segmentation
  ▼
Observation / Action Pairing
  ▼
HDF5 / Parquet / LeRobot-compatible Dataset
  ▼
Policy Training
```

示意频率（并非 Tron2 / Revo3 的设备规格）：

模态	示例频率
RGB / Depth 相机	30 Hz
机器人关节状态	100 Hz
触觉	100 Hz
Policy Action	50 Hz

需要选择参考时间轴，对状态插值、对图像选择对应帧或按具体策略进行时序堆叠。**不能只根据消息到达顺序进行同步；应优先使用有意义的消息时间戳，并考虑硬件时钟偏差**。

七、URDF、TF2、RViz 和 ros2_control

7.1 URDF：机器人结构描述

URDF（Unified Robot Description Format）描述机器人连杆、关节与几何、惯性等信息。例如：

```
Tron2
 ├── Shoulder Joint
 ├── Elbow Joint
 ├── Wrist Joint
 └── Revo3 Hand
     ├── Finger / Joint 1
     ├── Finger / Joint 2
     └── ...
```

典型 XML 元素：

```
<link>      <!-- 刚体连杆 -->
<joint>     <!-- 关节和连接关系 -->
<visual>    <!-- 显示几何 -->
```

```
<collision>  <!-- 碰撞几何 -->
<inertial>   <!-- 质量和惯量 -->
```

URDF 可为 ROS 可视化、运动学和仿真模型构建提供基础，但导入仿真器时可能还需要格式转换和补充参数。

7.2 URDF 与 TF2 的关系

URDF 是结构描述；TF2 是带时间关系的坐标变换系统。

```
          URDF
            |
            ▼
robot_state_publisher ← /joint_states
            |
            ▼
         TF Tree
            |
            ▼
           RViz
```

在 Tron2 项目中，这一流程适合检查机械臂末端、Revo3 手基座、头部相机及腕部相机的位置关系。URDF 外参错误不会被 TF2 自动修复。

7.3 RViz 2：可视化工具

启动：

```
rviz2
```

RViz 能显示 Robot Model、TF、相机图像、点云、关节状态、姿态标记和运动轨迹。对于 FoundationPose 结果，可以同时检查物体 Pose 与机器人基座的相对关系。

7.4 ros2_control：连接高层命令与硬件

```
High-Level Policy
        ▼
Controller (ros2_control)
        ▼
Hardware Interface
        ▼
Vendor SDK / Driver
        ▼
Motor / Robot
```

Hardware Interface 将读写硬件能力映射为控制框架接口；Controller 则执行轨迹、位置、速度或其他控制逻辑。

常见控制器或概念	用途
Joint Trajectory Controller	执行关节轨迹
Forward Command Controller	将指令传给指定关节接口
Joint Position / Velocity 命令接口	发送位置或速度目标，取决于配置

常见控制器或概念	用途
Diff Drive Controller	差速移动底盘控制

安装：

```
sudo apt install \
  ros-jazzy-ros2-control \
  ros-jazzy-ros2-controllers
```

启动兼容的 `controller_manager` 之后可查看：

```
ros2 control list_controllers
```

重要：安装 ros2_control 本身不代表已经具备 Tron2 / Revo3 的硬件适配。 必须找到兼容 Hardware Interface，或基于厂商 SDK 编写适配器。也可以在不使用 ros2_control 的情况下封装独立 ROS 2 驱动节点。

八、ROS 如何连接 Isaac Lab / MuJoCo 强化学习?

8.1 仿真训练通常不需要 ROS

Isaac Lab、MuJoCo 中的 PPO/SAC 训练往往直接调用环境 API：

```
observations = env.reset()
for step in range(num_steps):
    actions = policy(observations)
    observations, rewards, dones, info = env.step(actions)
```

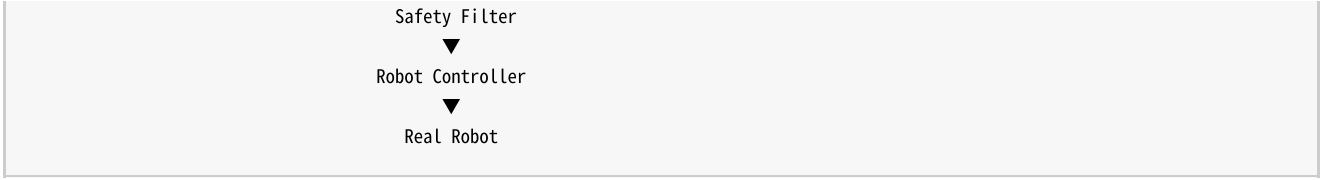
上面是概念示意，不对应所有 Gymnasium / Isaac Lab 版本的真实返回签名。大量并行环境需要高吞吐量，因此通常不把每个仿真步都转换为 ROS Topic。

8.2 真机部署需要连接各个模块

```
Isaac Lab / MuJoCo
        ▼
  PPO / SAC Training
        ▼
   Exported Policy
        ▼
  ROS 2 Policy Node
        ▼
  Safety / Controller
        ▼
    Real Robot
```

典型真机闭环：

```
Camera ──────────────┐
Joint States ────────┼──→ Observation Builder → RL Policy
Object Pose ─────────┘                              |
                                                    ▼
```



8.3 Sim2Real 最关键的是接口语义一致

项目	训练端	真机端应做的事
Joint Order	仿真模型的关节顺序	显式映射为实际驱动顺序
Joint Units	常为 rad	核对 SDK 是否使用 rad、degree 等
Observation	归一化状态	保持相同归一化与拼接顺序
Action	目标位置、增量或速度等	正确映射到控制模式
Frequency	仿真 timestep / decimation	明确 Policy 与 Controller 周期
Coordinate Frames	Sim World、Base 等	明确真实基座与世界系关系

ROS 能统一通信接口，却不会自动解决关节顺序、相机外参、归一化、动力学误差或延迟问题。

8.4 控制频率与延迟

需要分别考虑：

- `f_camera`：相机采样频率。
- `f_policy`：神经网络策略推理与动作更新频率。
- `f_control`：底层控制器频率。

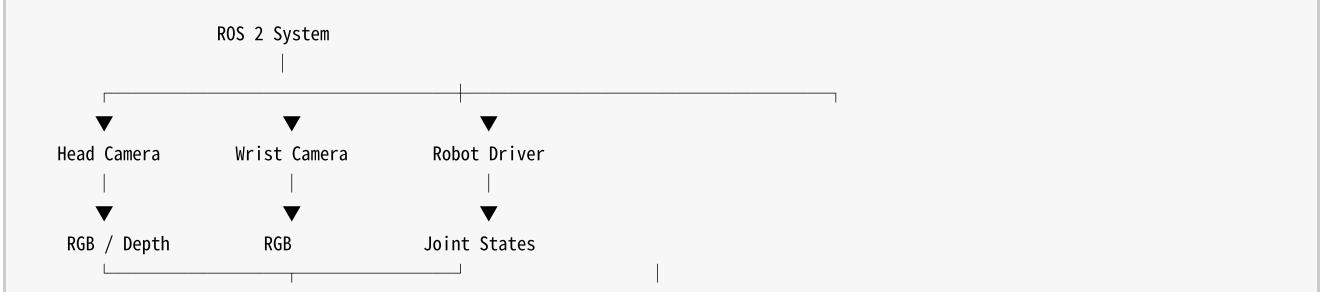
这些频率可以不同。例如 30 Hz 的图像、50 Hz 的策略与更高频率的底层控制；这只是示意而非设备约束。

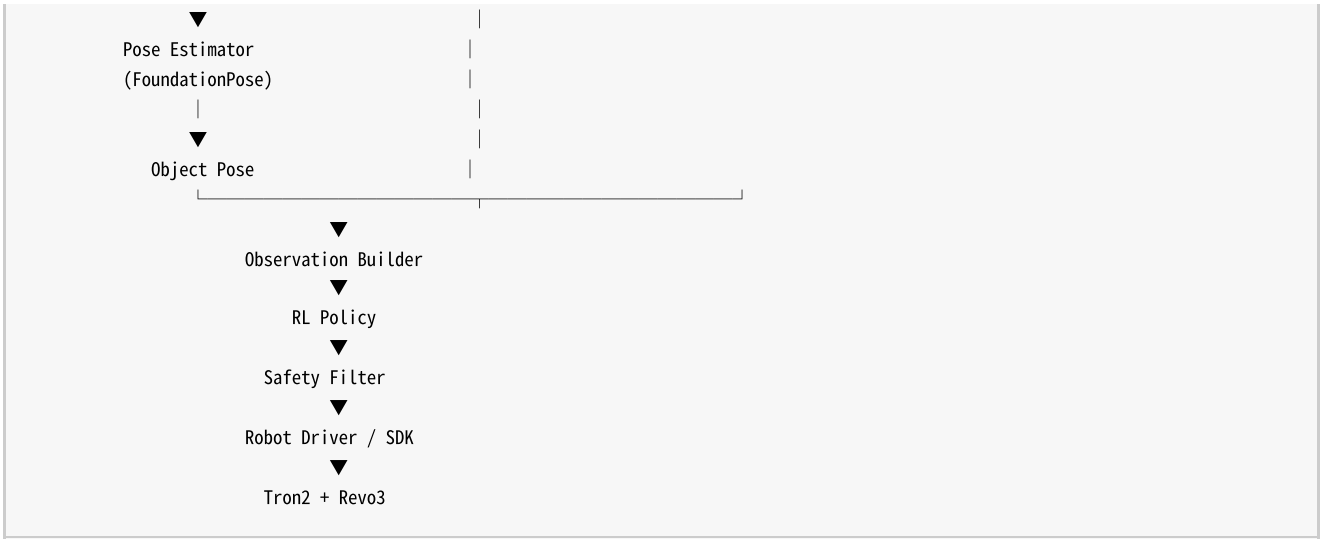
Python 的 ROS 2 节点不是天然硬实时系统。为减少延迟与提高稳定性，可分离 GPU 推理与伺服控制，使用 C++ / 实时线程完成关键控制，防止控制线程阻塞，并设置命令超时、限位和急停。

九、Tron2 + Revo3 灵巧操作项目架构

任务：**机器人从桌面拿起螺丝刀，移动到指定位置，完成旋转**。 系统包含 Tron2、BrainCo Revo3、头部 RGB-D、腕部 RGB、FoundationPose 和 PPO/SAC 策略。

9.1 推荐节点与信息流





9.2 推荐 Workspace 组织

```
robot_learning_ws/
└── src/
    ├── tron2_driver/
    ├── revo3_driver/
    ├── camera_driver/
    ├── robot_description/
    ├── pose_estimation/
    ├── observation_builder/
    ├── rl_policy/
    ├── safety_controller/
    ├── teleoperation/
    └── data_recorder/
```

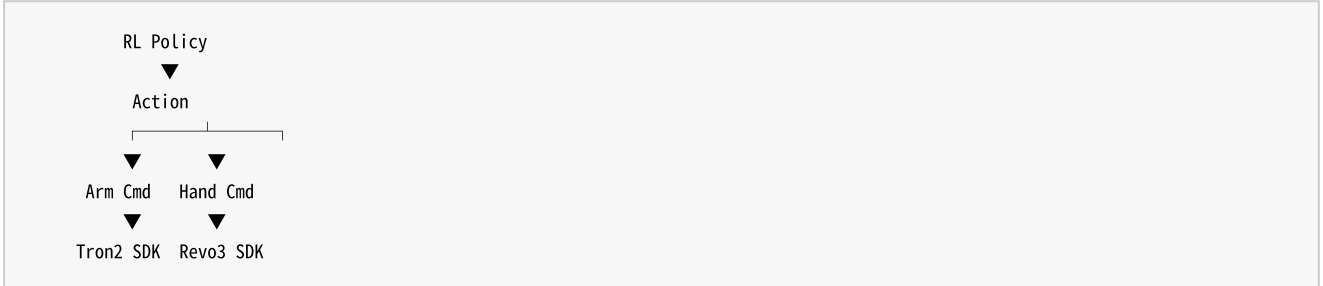
这是建议的工程目录，不代表厂商提供同名官方 Package。如果厂商只提供 Python/C++ SDK，可以编写 ROS Wrapper 转换为统一的消息接口。

9.3 机械臂与灵巧手的动作分发

联合策略可以输出：

$a_t = [\ a_t^{arm},\ a_t^{hand}\ ]$

其中机械臂动作维度为 n_a，灵巧手动作维度为 n_h。分发方案：



可用两个独立命令 Topic，也可以定义统一 Robot Action 消息由一个控制节点分发。无论哪种方案，都需要明确两个设备的控制时钟、关节顺序、目标模式和异常处置。

9.4 FoundationPose 的 ROS 接入

```
Head RGB-D / Wrist RGB
          ▼
     FoundationPose
          ▼
      /object_pose
          ▼
     TF Transform
          ▼
Object Pose in Robot Base
          ▼
       RL Policy
```

位姿估计模块可以独立于控制器开发，只需保证输出的 PoseStamped 时间戳与 frame_id 准确，必要时考虑遮挡、估计丢失后的重置与时间滤波。

十、机器人学习中的 ROS 常见问题

现象	可能原因	排查方向
`ros2 topic list` 看不到设备	驱动未启动、网络发现或域配置问题	检查节点、DDS、`ROS_DOMAIN_ID`
Topic 存在但收不到数据	QoS 不兼容、订阅类型不一致	`ros2 topic info -v`
相机和关节数据不同步	不同频率、时钟不一致	时间戳、硬件同步、缓存和插值
物体 Pose 偏移	外参、TF 链或参考系定义有误	检查标定与 optical frame
RL 真机动作异常	关节顺序、归一化、动作模式不一致	对比训练与部署 Observation/Action
端到端延迟大	相机、推理、通信、SDK 或控制器延迟	分段计时、测量队列积压
ROS 与 Conda 冲突	系统 Python 与 Conda 的 ABI/依赖不匹配	分离进程或使用兼容环境
rosbag 文件过大	多路 RGB-D 带宽较高	压缩、MCAP/存储设置、拆分
RViz 模型不对	URDF、JointState、TF 不一致	逐级核对 Robot Model / TF

10.1 ROS 2 与 Conda 的关系

机器人学习经常在 Conda 环境中使用 PyTorch、CUDA、Isaac Lab 和 RL 框架；而通过 apt 安装的 `rclpy` 通常面向 Ubuntu 系统 Python。直接在任意 Conda 环境里 `import rclpy` 可能遇到 ABI 或动态库问题。

一种稳妥结构是：

```
 ROS 2 Driver Process
          |
      Observation
          ▼
PyTorch Inference Process
    (Conda / CUDA)
          |
        Action
```

```
        ▼
ROS 2 Control Process
```

可以通过 ROS 2 Topic、独立通信接口或共享内存传递数据，而非强行把全部依赖安装在同一个 Python 环境。

10.2 与真实机器人相关的安全边界

学习策略的输出不是可直接信任的底层电机命令。真实系统至少需要：关节限位、速度与加速度约束、命令超时、停止与急停、通信失联回退，以及在初始调试时从低速、无负载、无接触场景开始。ROS 本身不会替代这些安全机制。

十一、推荐学习路线

阶段	重点	可检验的完成标准
1	Node、Topic、Message	能查看并解释机器人的 Topic
2	`rclpy`、Publisher、Subscriber	能运行自己的 Python 策略节点
3	rosbag2	能完整记录并回放一个 Episode
4	URDF、TF2、RViz	能核对机器人与相机坐标系
5	机器人 SDK、ros2_control	能安全读取状态并下发测试目标
6	PyTorch + ROS	能在真机运行 PPO/SAC 推理
7	QoS、同步与延迟分析	能量化传感器到动作的端到端延迟

对于已经有机器人学习、控制和 Python 基础的研究者，不必从 ROS 的每个组件入手，应围绕 **数据采集 → 策略部署 → 可复现实验** 逐步学习。

十二、总结与实践建议

ROS 2 在机器人学习中的价值可以归为三类：

方向	ROS 负责的工作	典型应用
数据采集	采集与记录相机、机器人状态和遥操作动作	BC、Diffusion Policy、VLA
策略部署	连接 Observation、Policy、Controller	PPO、SAC、Sim2Real
系统集成	TF、硬件通信、可视化、诊断	灵巧操作、视觉闭环、真机实验

对于 **Tron2 + Revo3**，最优先掌握的五项是：

1. **Topic + rclpy**：连接关节状态、相机与 RL 策略。
2. **rosbag2**：采集遥操作数据，构建离线训练数据集。
3. **TF2 + URDF**：管理头部相机、腕部相机、末端和物体坐标系。

4. **Robot SDK / Hardware Interface**：可靠控制 Tron2 与 Revo3。
5. **Sim2Real Deployment**：将 Isaac Lab / MuJoCo 中的策略转换为真机闭环控制。

最终目标不是“在训练中使用更多 ROS”，而是建立可观测、可复现、便于维护的“传感器 → 策略 → 控制器 → 机器人”系统。

十三、官方文档与参考资料

1. ROS 2 Jazzy 官方教程
2. ROS 2 Jazzy Ubuntu deb 安装说明
3. ROS 2 Python Publisher / Subscriber 教程
4. ROS 2 TF2 教程
5. ros2_control 官方文档
6. MoveIt 2 官方文档
7. rosbag2 GitHub
8. ROS 2 发行版本信息
9. ROS 2 Lyrical 官方发行记录

文档说明：本教程为学习与工程设计材料，代码中的虚拟动作 Topic 仅供无硬件示例测试；真实机器人部署必须结合厂商接口文档和设备安全要求检查。

机器人学习中的相机标定教程

课程首页 · English · PDF

项目	说明
适用范围	OpenCV 4.x 概念；ROS 2 Jazzy；设备 SDK 版本需另行匹配
文档核验日期	2026-10-18
许可	CC BY 4.0 · Xiong Zeng and contributors

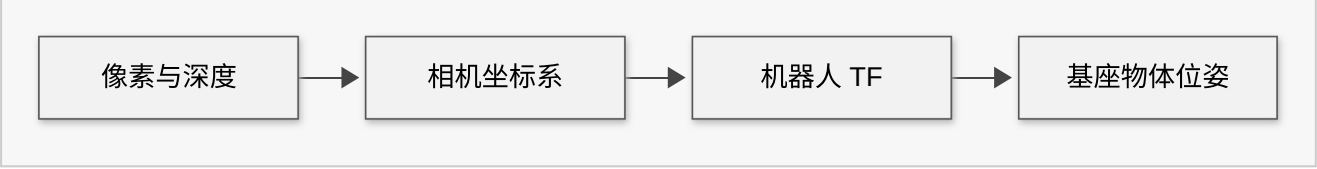
核验边界： 文档整理与上游引用快照核对；未进行教程环境、训练或硬件验证。 版本与固定源码引用见版本说明。原创教材采用 CC BY 4.0，第三方引用、图片及代码遵循其原有许可；注明来源不等于拥有再许可权。

目录

- 一、相机标定的分类与用途
- 二、最重要的数学概念与坐标系
  - 2.1 坐标系约定
  - 2.2 机器人坐标系与相机 Optical Frame
- 三、相机内参标定（Intrinsic Calibration）
  - 3.1 相机矩阵和畸变
  - 3.2 内参有什么用?
  - 3.3 标定板选择
  - 3.4 内参标定实际流程
- 四、相机外参标定（Extrinsic Calibration）
  - 4.1 外参到底是什么?
  - 4.2 已知标定板位姿时：PnP + 坐标变换
- 五、手眼标定（Hand-Eye Calibration）
  - 5.1 两种常见布局
  - 5.2 Eye-in-Hand：腕部相机
  - 5.3 Eye-to-Hand：固定相机、末端标定板
- 六、Tron2 头部与腕部相机的特殊情况
  - 6.1 可转动头部相机
  - 6.2 腕部相机
  - 6.3 头部相机与腕部相机之间的变换
- 七、RGB-D 深度相机标定与对齐
  - 7.1 设备内部坐标系
  - 7.2 RGB 像素到机器人三维点
  - 7.3 深度相机需要重新标定吗?
- 八、OpenCV 完整手眼标定代码
  - 8.1 数据格式

    - 8.2 统一支持 Eye-in-Hand / Eye-to-Hand
    - 8.3 标定结果验证
- 九、ROS 2 中的标定工具与基本操作
    - 9.1 常用工具
    - 9.2 相机内参标定
    - 9.3 ROS 2 手眼标定
- 十、将标定结果写入 ROS TF2 / URDF
    - 10.1 推荐的 TF 结构
    - 10.2 静态外参发布
    - 10.3 动态 TF 检查
- 十一、相机标定如何进入机器人学习
    - 11.1 物体 6D 位姿估计与视觉抓取
    - 11.2 模仿学习与 VLA
    - 11.3 强化学习与 Sim-to-Real
    - 11.4 视觉闭环控制与时间同步
- 十二、标定精度与结果验证
    - 12.1 重投影误差
    - 12.2 三维定位误差
    - 12.3 实际抓取测试
- 十三、常见问题与排查
- 十四、推荐开源工具与官方资料
- 十五、针对 Tron2 + Revo3 的实施路线
    - 阶段一：设备内参与深度检查
    - 阶段二：头部相机几何标定
    - 阶段三：腕部相机与 TCP
    - 阶段四：视觉闭环与学习部署
- 十六、总结与检查清单

章节概念图



一、相机标定的分类与用途

在机器人学习中，“相机标定”不仅指计算相机内参，还包括建立相机、机器人和环境之间的空间及时序关系。

标定类型	英文名称	主要目的	常见应用
相机内参标定	Intrinsic Calibration	确定成像几何与畸变	2D–3D 投影、PnP、6D Pose

标定类型	英文名称	主要目的	常见应用
相机外参标定	Extrinsic Calibration	确定相机相对参考系的位姿	机器人抓取、空间定位
手眼标定	Hand-Eye Calibration	确定相机与末端/基座之间的刚体变换	腕部/固定相机标定
多相机标定	Multi-Camera Calibration	确定多相机之间的相对位姿	视觉融合、多视角感知
深度相机标定	RGB-D / Depth Calibration	校验 RGB–Depth 对齐和深度尺度	点云、三维定位
时间标定	Temporal Calibration	对齐图像、关节状态等传感器时间	动态追踪、闭环控制

与之密切相关的还有 **机器人运动学标定**（kinematic calibration）和 **相机–IMU 标定**（camera–IMU calibration）。

重要区分： RealSense 相机通常已有出厂内参；安装到 Tron2 上以后，仍需要确定相机相对于机器人运动学链的位置和朝向。对于可转动的头部云台，相机到基座的变换还是随关节角度变化的。

二、最重要的数学概念与坐标系

2.1 坐标系约定

符号	名称	说明
W	World frame	世界/仿真坐标系
B	Base frame	机器人基座坐标系
G	Gripper/end-effector frame	机械臂末端坐标系
C	Camera optical frame	相机光学坐标系
O	Object frame	物体坐标系
T	Target frame	标定板坐标系

以下统一采用明确的方向定义：

$$
{}^{A}T_{B} = \begin{bmatrix} {}^{A}R_{B} & {}^{A}t_{B} \\ 0 & 1 \end{bmatrix}
$$

表示**把在 B 中表达的点坐标变换为在 A 中表达的坐标**。其中旋转 $R \in SO(3)$，平移 $t \in \mathbb{R}^3$，齐次刚体变换 $T \in SE(3)$。

三维点的齐次坐标满足

$$
\bar{p}_A = {}^{A}T_{B}\bar{p}_B.
$$

求逆与组合规则分别为

$$
{}^{B}T_{A} = ({}^{A}T_{B})^{-1}, \qquad {}^{A}T_{C} = {}^{A}T_{B}\,{}^{B}T_{C}.
$$

例如 FoundationPose 得到物体在相机中的位姿 ${}^{C}T_{O}$，控制器想要物体在基座中的位姿，则有

$$
\boxed{{}^{B}T_{O} = {}^{B}T_{C}\,{}^{C}T_{O}.}
$$

相机标定的关键任务之一，就是获取足够准确的 ${}^{B}T_{C}$。

2.2 机器人坐标系与相机 Optical Frame

ROS 常见约定（REP 103）如下：

轴	机器人机体坐标系	相机光学坐标系
X	前	图像右
Y	左	图像下
Z	上	朝相机前方

因此，`camera_link` 不一定等同于 `camera_optical_frame`。从普通相机机体坐标系到 optical frame，通常还有一段固定旋转。不要直接把 OpenCV 输出的光学坐标当作机器人机体坐标使用。参考：ROS REP 103。

三、相机内参标定（Intrinsic Calibration）

3.1 相机矩阵和畸变

理想针孔模型中，内参矩阵是

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}.$$

其中 f_x, f_y 是以像素为单位的焦距，c_x, c_y 是主点（principal point）。对已校正畸变的相机坐标 (X, Y, Z)：

$$u = f_x \frac{X}{Z} + c_x, \qquad v = f_y \frac{Y}{Z} + c_y.$$

真实相机还有镜头畸变。OpenCV 中常见的五参数模型为

$$D = [k_1, k_2, p_1, p_2, k_3],$$

前两类参数分别描述径向畸变和切向畸变；鱼眼等镜头需要相应的其他模型。

3.2 内参有什么用?

任务一：三维点投影到图像。 给定相机坐标系中 $p_C = (X, Y, Z)$，利用 K 计算其像素位置 (u, v)。用于可视化、重投影、视觉控制和仿真—真实图像对齐。

任务二：RGB-D 反投影。 已知去畸变像素 (u, v) 及其光学轴深度 Z，恢复

$$\boxed{p_C = \begin{bmatrix} (u - c_x)Z/f_x \\ (v - c_y)Z/f_y \\ Z \end{bmatrix}.}$$

如果使用原始畸变图像，应先去畸变或使用正确的反投影函数；如果深度图已经对齐到 RGB 图，则使用对齐后数据对应的相机模型。

任务三：6D 位姿估计。 PnP、FoundationPose、MegaPose 和基于 CAD 模型的跟踪常使用内参。注意：仅凭普通 RGB 图像中的一个像素，无法独立恢复该点的绝对深度。

3.3 标定板选择

标定板	优点	注意事项	典型用途
Checkerboard（棋盘格）	精度高、成本低	需要足够清晰的角点	基础内参标定
ArUco / AprilTag	易检测、有 ID	单个标记角点精度受分辨率影响	PnP、位姿跟踪
ChArUco	棋盘角点 + ID，可应对部分遮挡	打印和角点对应关系需准确	高质量内参、手眼标定

对于普通机器人实验，优先建议 **ChArUco**；也可使用检测稳定的棋盘格。OpenCV 的 ArUco/ChArUco 指南 提供标板检测示例。

3.4 内参标定实际流程

1. **制作标定板**。 例如 7×5 方格的 ChArUco 板，方格边长 30 mm、标记边长 20 mm；这些只是示例。打印必须按真实物理尺寸，固定在刚性平板上，避免卷曲。
2. **采集图像**。 以 20–40 张清晰、有效图像作为初始数据集；让标定板出现在中心及四角，覆盖不同距离、倾斜角和尺度；固定对焦、曝光或其他关键成像设置。
3. **检测角点**。 可调用 `cv2.findChessboardCornersSB()` 或相应的 ChArUco 接口。确保 2D–3D 角点索引对应。
4. **估计内参与畸变**。 优化各帧相机姿态、K、D，最小化重投影残差：

$$\min_{K,D,\{R_j,t_j\}} \sum_{j,i} \|p_{ij} - \pi(K, D, R_j P_i + t_j)\|_2^2 .$$

5. **独立验证**。 查看 RMS 重投影误差、图像各区域的残差、去畸变后的直线，以及未用于标定的图像。约 0.3–0.5 px 可以作为高质量标定的参考范围，但绝不是所有分辨率、镜头和任务通用的合格线，也不代表毫米级定位误差。

四、相机外参标定（Extrinsic Calibration）

4.1 外参到底是什么？

相机内参回答“**三维点怎样投影到图像**”；外参回答“**相机相对某个参考坐标系在哪里**”。对于 Tron2 头部相机，一个典型目标是求

$$^{B}T_{C} = \begin{bmatrix} ^{B}R_{C} & ^{B}t_{C} \\ 0 & 1 \end{bmatrix}.$$

4.2 已知标定板位姿时：PnP + 坐标变换

如果已经准确知道标定板在机器人基座中的位姿 $^{B}T_{T}$，通过相机内参和图像角点，PnP 可估计 $^{C}T_{T}$。于是

$$^{B}T_{T} = {}^{B}T_{C}\,{}^{C}T_{T},$$

从而

$$\boxed{^{B}T_{C} = {}^{B}T_{T}({}^{C}T_{T})^{-1}.}$$

实操步骤： 固定标定板；通过测量或机器人接触法确定 $^{B}T_{T}$；获取图像；检测对应角点并运行 PnP；计算 $^{B}T_{C}$；使用独立观测验证。

```
import cv2
import numpy as np

# N×3 的标定板三维角点，单位：m
object_points = np.asarray(..., dtype=np.float64)
# 对应的 N×2 图像角点，单位：pixel
image_points = np.asarray(..., dtype=np.float64)
K = np.asarray(..., dtype=np.float64)
dist = np.asarray(..., dtype=np.float64)

ok, rvec, tvec = cv2.solvePnP(
    object_points, image_points, K, dist
)
if not ok:
    raise RuntimeError("PnP failed")

R, _ = cv2.Rodrigues(rvec)
T_C_T = np.eye(4)
T_C_T[:3, :3] = R
T_C_T[:3, 3] = tvec.reshape(3)

# 必须来自独立测量，而不是凭空假设
T_B_T = np.asarray(..., dtype=np.float64)
T_B_C = T_B_T @ np.linalg.inv(T_C_T)
print(T_B_C)
```

方向提醒： `solvePnP` 返回的是**标定板到相机**的变换 ${}^{C}T_{T}$，不是其逆。

限制： 如果标定板只是随意摆在桌面上，且不知道 ${}^{B}T_{T}$，一张图像通常不足以得到 ${}^{B}T_{C}$；此时可使用下一节的手眼标定。

五、手眼标定（Hand-Eye Calibration）

5.1 两种常见布局

项目	Eye-in-Hand	Eye-to-Hand
相机	安装在机器人末端，随机械臂运动	固定于基座/环境
标定板	通常固定在场景中	通常固定在末端
典型未知量	${}^{G}T_{C}$	${}^{B}T_{C}$
常见用途	腕部 RGB / RGB-D 相机	外部/顶视相机

注意，**可随 yaw/pitch 转动的 Tron2 头部相机，并不是严格意义上的固定 Eye-to-Hand 相机**；只能在云台姿态固定时暂时如此处理。

5.2 Eye-in-Hand：腕部相机

希望求解固定变换 ${}^{G}T_{C}$。将标定板固定在桌上，驱动机械臂到不同姿态，每次记录：

- 由机器人 FK 或 TF 得到末端在基座中的位姿 ${}^{B}T_{G,i}$；
- 由 PnP 得到标定板在相机中的位姿 ${}^{C}T_{T,i}$。

因为标定板不动，有

$$^{B}T_{G,i}\,^{G}T_{C}\,^{C}T_{T,i} = {}^{B}T_{T}.$$

不同采样构成经典的 $AX = XB$ 手眼标定问题，其中 $X = {}^{G}T_{C}$。需要包含多轴旋转的丰富运动；单纯沿三个方向平移，或只绕同一旋转轴转动，通常不足以充分约束问题。

采样建议： 从约 20–30 个有效且角度多样的姿态开始，确保标定板可见、关节状态与图像同步，排除模糊和检测失败样本。姿态质量往往比机械增加样本数更重要。

OpenCV 包含 Tsai、Park、Horaud、Andreff、Daniilidis 等求解选项；参考 `calibrateHandEye`。

5.3 Eye-to-Hand：固定相机、末端标定板

现在相机固定在环境中，标定板刚性安装在机器人末端。每次观察满足

$$^{B}T_{G,i}\,^{G}T_{T} = {}^{B}T_{C}\,^{C}T_{T,i}.$$

其中相机位姿 $^{B}T_{C}$ 和末端—标定板位姿 $^{G}T_{T}$ 为固定未知量。根据多组姿态可求 $^{B}T_{C}$，而不必事先知道标定板放在桌面上的绝对位置。

关键： 使用 `cv2.calibrateHandEye()` 做 Eye-to-Hand 时，机器人位姿输入应转换为 $^{G}T_{B} = (^{B}T_{G})^{-1}$，标定板位姿仍输入 $^{C}T_{T}$；输出才可解释为 $^{B}T_{C}$。直接照搬 Eye-in-Hand 的输入可能造成错误的变换方向。

六、Tron2 头部与腕部相机的特殊情况

6.1 可转动头部相机

定义 H 为头部运动学链的末端安装坐标系，C_h 为头部相机光学坐标系，q_h 为头部关节角。则

$$\boxed{^{B}T_{C_h}(q_h) = {}^{B}T_{H}(q_h)\,{}^{H}T_{C_h}.}$$

- $^{B}T_{H}(q_h)$：头部运动学模型和关节角决定的**动态变换**；
- $^{H}T_{C_h}$：相机安装支架决定的**固定变换**，需要标定或测量。

推荐方法：把 ChArUco 板固定在桌面；采集多个 yaw/pitch 姿态的图像和关节角；用 FK 求 $^{B}T_{H}$，由 PnP 求 $^{C_h}T_{T}$，利用 Eye-in-Hand 的形式求 $^{H}T_{C_h}$；最后用保留的姿态验证。

yaw/pitch 旋转轴不相交时

不能简单地写成 $R_z(q_y)R_y(q_p)$ 并忽略关节中心之间的平移。应使用完整的刚体变换链：

$$^{B}T_{C_h} = {}^{B}T_{Y}\,T_{Y}(q_y)\,{}^{Y}T_{P}\,T_{P}(q_p)\,{}^{P}T_{C_h}.$$

其中 $^{Y}T_{P}$ 包含 yaw、pitch 轴之间的固定旋转和平移。优先核查 URDF 里的每个关节 `origin`、`axis`。如果机械臂/云台运动学本身有系统性偏差，单独修正相机安装外参**不一定**能完全消除各个姿态下的误差。

对于当前固定头部视角的螺丝刀抓取实验，可先固定云台姿态完成一组基座—相机外参，再逐步扩展到动态头部模型。

6.2 腕部相机

定义 C_w 为腕部相机，q_a 为机械臂关节角，则

$$\boxed{{}^{B}T_{C_w}(q_a) = {}^{B}T_{G}(q_a)\,{}^{G}T_{C_w}.}$$

其中 ${}^{B}T_{G}(q_a)$ 来自 FK，${}^{G}T_{C_w}$ 由 Eye-in-Hand 标定。若腕相机安装在 `wrist_link` 而非用于运动学记录的 G 坐标系，应使用一致的链接并正确组合固定变换。

更换 Revo3 灵巧手的影响

必须区分

$$ {}^{B}T_{G},\quad {}^{G}T_{C_w},\quad {}^{G}T_{\mathrm{TCP}}. $$

第一项是机械臂末端位姿；第二项是腕部相机外参；第三项是 TCP 相对于末端的安装变换。更换灵巧手而**没有改变相机相对末端的安装位置**时，相机外参原则上仍有效，但 TCP 与指尖几何可能变化。若相机支架移动、末端坐标系定义改变或安装发生松动，应重新检查相机标定。

6.3 头部相机与腕部相机之间的变换

一旦两台相机都在机器人基座中有位姿，就可以计算：

$$\boxed{{}^{C_h}T_{C_w} = ({}^{B}T_{C_h})^{-1}\,{}^{B}T_{C_w}.}$$

因此，两台相机不一定要单独做一轮“双目标定”；但通过共同观察标定板来验证两路外参与运动学的一致性仍然非常有价值。由于两条运动学链会运动，${}^{C_h}T_{C_w}$ 通常**不是常数**。

七、RGB-D 深度相机标定与对齐

7.1 设备内部坐标系

D435、D455 等 RGB-D 设备通常包含 RGB、左右红外/深度等不同的光学坐标系。RGB 与 Depth 的光学中心通常不完全重合，因而需要已标定的

$$ {}^{C_{\mathrm{rgb}}}T_{C_{\mathrm{depth}}}. $$

RealSense SDK 一般能够读取相机出厂标定的内参、外参和深度尺度，通常先**读取并验证**即可。

7.2 RGB 像素到机器人三维点

基本流水线：

```
RGB image
   -> Detect pixel (u, v)
   -> Align depth to RGB
   -> Read depth Z
   -> Deproject using intrinsics
   -> Point p_C in optical camera frame
   -> Transform using T_B_C
   -> Point p_B in robot base frame
```

在 RealSense SDK 中，对齐与反投影的一个示例如下：

```
import pyrealsense2 as rs
import numpy as np

pipeline = rs.pipeline()
config = rs.config()
config.enable_stream(rs.stream.depth)
config.enable_stream(rs.stream.color)
pipeline.start(config)

align = rs.align(rs.stream.color)
try:
    frames = pipeline.wait_for_frames()
    aligned = align.process(frames)

    depth = aligned.get_depth_frame()
    color = aligned.get_color_frame()
    if not depth or not color:
        raise RuntimeError("Missing RGB-D frame")

    u, v = 320, 240  # 示例像素，须检查图像边界
    z = depth.get_distance(u, v)  # 米
    if z <= 0:
        raise RuntimeError("Invalid depth")

    intr = depth.profile.as_video_stream_profile().get_intrinsics()
    point_C = rs.rs2_deproject_pixel_to_point(
        intr, [float(u), float(v)], z
    )
    print("Point in camera optical frame:", point_C)
finally:
    pipeline.stop()
```

随后使用已经标定的相机到基座变换：

```
point_C_h = np.r_[np.asarray(point_C), 1.0]
point_B_h = T_B_C @ point_C_h
print("Point in base frame:", point_B_h[:3])
```

这就完成了“RGB 像素 → 相机三维点 → 机器人基座三维点”。

7.3 深度相机需要重新标定吗?

先确认出厂数据和实际表现，再决定是否进行设备的深度重新标定。建议检查：

检查项	目的
RGB 内参	保证 RGB 像素投影正确
Depth 内参	保证深度反投影正确
RGB–Depth 外参	保证对齐准确
深度尺度	确保单位正确
目标工作距离处的深度精度	确保抓取距离可靠
相机—机器人外参	确保视觉点能正确进入基座坐标系

对于螺丝刀等细长或反光物体，要注意深度空洞、边缘错配和多路径/反光等误差；它们不一定能通过重新标定内参解决。参考：RealSense SDK。

八、OpenCV 完整手眼标定代码

8.1 数据格式

每个姿态记录两组匹配的 4×4 齐次变换，并以同一时间点对应：

```
base_T_gripper.npy   shape = (N, 4, 4)
camera_T_target.npy  shape = (N, 4, 4)
```

即分别存放 ${}^{B}T_{G,i}$ 和 ${}^{C}T_{T,i}$，平移采用米。保存的矩阵应满足旋转正交、行列式约等于 1，底行等于 `[0, 0, 0, 1]`。

8.2 统一支持 Eye-in-Hand / Eye-to-Hand

```
import cv2
import numpy as np

# ^B T_G : 机器人末端 -> 机器人基座
T_B_G = np.load("base_T_gripper.npy")
# ^C T_T : 标定板 -> 相机光学坐标系
T_C_T = np.load("camera_T_target.npy")

assert T_B_G.shape == T_C_T.shape
assert T_B_G.ndim == 3 and T_B_G.shape[1:] == (4, 4)
assert len(T_B_G) >= 3

mode = "eye_in_hand"  # 或 "eye_to_hand"

if mode == "eye_in_hand":
    robot_poses = T_B_G
elif mode == "eye_to_hand":
    robot_poses = np.linalg.inv(T_B_G)
else:
    raise ValueError("Unknown calibration mode")

R_robot = [T[:3, :3] for T in robot_poses]
t_robot = [T[:3, 3] for T in robot_poses]
R_target = [T[:3, :3] for T in T_C_T]
t_target = [T[:3, 3] for T in T_C_T]

R, t = cv2.calibrateHandEye(
    R_robot, t_robot, R_target, t_target,
    method=cv2.CALIB_HAND_EYE_PARK
)

X = np.eye(4)
X[:3, :3] = R
X[:3, 3] = t.reshape(3)

if mode == "eye_in_hand":
    print("^G T_C (gripper <- camera):")
else:
    print("^B T_C (base <- camera):")
```

```
print(X)
np.save("handeye_result.npy", X)
```

该示例展示接口用法，但实际部署前还需要剔除坏样本、检查运动激励是否充分，并在独立姿态上验证结果。尤其必须确保 `camera_T_target` 使用的是**光学相机坐标系**而非 `camera_link`。

8.3 标定结果验证

Eye-in-Hand：对每一组采样计算

$$\hat{T}_{BT,i} = {}^{B}T_{G,i}\,{}^{G}T_{C}\,{}^{C}T_{T,i}.$$

如果目标板真的固定，所有 $\hat{T}_{BT,i}$ 应彼此接近。

Eye-to-Hand：对每一组采样计算

$$\hat{T}_{GT,i} = ({}^{B}T_{G,i})^{-1}\,{}^{B}T_{C}\,{}^{C}T_{T,i}.$$

如果标定板刚性安装在末端，所有 $\hat{T}_{GT,i}$ 应彼此接近。

实践中应分别报告平移残差、旋转角残差，并保留未参加拟合的姿态做独立测试，而不能只目测最终矩阵是否“差不多”。

九、ROS 2 中的标定工具与基本操作

9.1 常用工具

工具	功能
`camera_calibration`	基于棋盘格的相机内参标定
`image_pipeline`	图像去畸变和校正
`tf2`	机器人坐标变换管理
`robot_state_publisher`	用 URDF 和关节角发布 TF
`easy_handeye2`	ROS 2 手眼标定工作流
MoveIt Calibration	手眼标定 GUI
Kalibr	多相机/相机–IMU 时空标定

9.2 相机内参标定

在支持的 ROS 2 发行版中：

```
sudo apt install \
  ros-${ROS_DISTRO}-camera-calibration \
  ros-${ROS_DISTRO}-image-pipeline

ros2 topic list
```

假设相机 Topic 为 `/camera/image_raw`、`/camera/camera_info`，可运行：

```
ros2 run camera_calibration cameracalibrator \
  --size 8x6 \
  --square 0.025 \
  image:=/camera/image_raw \
  camera:=/camera
```

这里 `8x6` 是**内部角点数**，`0.025` 表示方格边长为 0.025 m；实际参数必须与标定板一致。不同相机驱动的 Topic 名称可能不同。参考：image_pipeline。

9.3 ROS 2 手眼标定

可参考 easy_handeye2 或 MoveIt Calibration。典型工作流是：

1. ROS 发布机器人末端 TF；
2. 检测目标板并发布标定板 TF；
3. 在多个机器人姿态下采样；
4. 选择 Eye-in-Hand / Eye-to-Hand；
5. 计算、验证并保存手眼结果；
6. 将结果接入机器人 URDF/TF 图。

只需要一次性计算时，OpenCV Python 脚本通常已足够；反复做团队实验时，ROS 工具更有利于标准化。

十、将标定结果写入 ROS TF2 / URDF

10.1 推荐的 TF 结构

以下仅为示例，具体链接名字以 Tron2 URDF 为准：

```
base_link
├── arm_link_1
│   └── ...
│       └── wrist_link
│           └── wrist_camera_link
│               └── wrist_camera_optical_frame
└── head_yaw_link
    └── head_pitch_link
        └── head_camera_link
            └── head_camera_optical_frame
```

相机固定安装外参应与腕部/头部关节的动态 FK 分开：

```
URDF + joint_states + robot_state_publisher
              +
      calibrated camera mount
               |
               v
              TF2
               |
               v
      Base-to-camera transform
```

10.2 静态外参发布

下面的平移与旋转只是**命令格式示意**，不是 Tron2 标定数据：

```
ros2 run tf2_ros static_transform_publisher \
  --x 0.03 --y 0.00 --z 0.08 \
  --qx 0 --qy 0 --qz 0 --qw 1 \
  --frame-id wrist_link \
  --child-frame-id wrist_camera_optical_frame
```

注意： 实际光学坐标系的旋转一般不是单位四元数。若变换已由 URDF + `robot_state_publisher` 发布，不要重复发布相同的父子 TF 边。

10.3 动态 TF 检查

```
ros2 run tf2_ros tf2_echo \
  base_link wrist_camera_optical_frame

ros2 run tf2_ros tf2_echo \
  base_link head_camera_optical_frame
```

机器人关节运动时，相机相对于基座的变换应该随之更新。对采集时的图像，查询 TF 应使用**图像时间戳**，而不是不加区分地取机器人最新状态。

十一、相机标定如何进入机器人学习

11.1 物体 6D 位姿估计与视觉抓取

假设 FoundationPose 从头部 RGB-D 估计到螺丝刀位姿 ${}^{C_h}T_O$，那么

$$ {}^{B}T_O = {}^{B}T_{C_h}(q_h)\,{}^{C_h}T_O. $$

流程：

```
Head RGB-D
  -> Object pose estimation
  -> ^C T_O
  -> Calibrated ^B T_C
  -> ^B T_O
  -> Grasp / task planning
  -> IK / motion control
  -> Real robot + feedback
```

必须区分视觉估计器误差与相机外参误差。即使 6D Pose Estimator 很准，错误的 ${}^{B}T_C$ 也会带来目标在基座下的系统性偏移。

11.2 模仿学习与 VLA

动作可能以 TCP 增量 $a_t = \Delta T_{\text{TCP}}$，或者关节序列 $a_t = q_{t:t+H}$ 表示。是否显式依赖外参取决于任务：

任务	是否需要显式外参
RGB 到关节命令的端到端行为克隆	不一定
RGB + proprioception 的端到端策略	不一定
通过像素与深度恢复三维物体位置	需要
视频轨迹重定向为机器人 SE(3) 轨迹	通常需要
多相机的三维融合	需要
仿真/真机相机几何对齐	需要
FoundationPose + IK	需要

端到端 VLA 可以隐式学习视觉—动作映射，但准确、统一的几何关系通常有助于跨相机、跨机器人和跨场景迁移。

11.3 强化学习与 Sim-to-Real

视觉策略可以写成

$$a_t = \pi(I_t, q_t).$$

若仿真与真机的相机位置、视角、内参、裁剪/缩放方式不同，便会引入观测分布偏移。应优先对齐：

$$K_{\text{sim}} \approx K_{\text{real}}, \qquad {}^{B}T_{C,\text{sim}} \approx {}^{B}T_{C,\text{real}}.$$

之后可在训练时对残余标定误差施加 Camera Domain Randomization，例如

$$\delta t \sim \mathcal{N}(0, \Sigma_t), \qquad \delta\theta \sim \mathcal{N}(0, \Sigma_R).$$

随机化用于增强鲁棒性，不应替代尽可能准确的基础标定。

11.4 视觉闭环控制与时间同步

对目标物体和当前物体位姿，可以构造 $SE(3)$ 位姿误差：

$$e_t = \operatorname{Log}\left((T_O^*)^{-1} T_{O,t}\right).$$

但实时视觉闭环的关键不只有几何外参，还包括**时间对齐**。若图像曝光发生在 t_{image}，应使用同一时刻的关节状态计算

$${}^{B}T_C(t_{\text{image}}),$$

而不是随意使用当前最新机器人姿态。移动中的头部/腕部相机尤为如此。

十二、标定精度与结果验证

必须同时评估图像层、几何层与任务层的质量。

验证层次	关键指标	目的
图像层	重投影 RMS / 分布	检查内参与目标板角点
几何层	平移残差、旋转角残差	检查外参与手眼约束
任务层	三维定位误差、抓取成功率	检查整套机器人闭环系统

12.1 重投影误差

检测角点 p_i 与投影预测 $\hat{p}_i$ 之间的欧氏误差为

$$e_i = \|p_i - \hat{p}_i\|_2,$$

其 RMS 为

$$e_{\mathrm{RMS}} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} e_i^2}.$$

图像里的像素误差不能直接等同于机器人末端的毫米级误差。

12.2 三维定位误差

对真实位置已知的测试点 p_B^*，比较视觉估计 $\hat{p}_B$：

$$\boxed{e_p = \|\hat{p}_B - p_B^*\|_2.}$$

分别测试工作空间中心、边缘、不同高度、不同观察方向，以及不同 yaw/pitch 姿态。如果头部一转动误差就显著增大，应优先检查头部运动学、转轴位置、时间同步和相机安装变换。

12.3 实际抓取测试

还可以检查 TCP 的目标位置误差

$$e_{\mathrm{grasp}} = \|p_{\mathrm{TCP,actual}} - p_{\mathrm{TCP,target}}\|_2$$

及螺丝刀抓取成功率、重复性、目标姿态误差和扰动恢复能力。要注意这些指标还受到 FK、IK、执行器、TCP 几何和物体位姿估计的共同影响；抓取失败**不等于**相机标定失败。

十三、常见问题与排查

现象	可能原因	排查方式
目标位置整体偏移	外参平移偏差	检查 ${}^{B}T_{C}$
误差随视角增加	旋转外参或运动学误差	多关节姿态验证
头部旋转后漂移	yaw/pitch 轴与偏移建模不准确	检查 URDF/实际机构
腕部手眼标定不稳定	旋转激励不足或坏样本	增加多轴姿态、剔除异常值
深度距离不准	深度尺度、反光或深度噪声	测量已知距离

现象	可能原因	排查方式
RGB 与深度图错位	对齐或帧模型错误	检查 alignment 和内参
图像方向与机器人方向不一致	optical frame 混用	检查 TF 坐标轴
机器人运动时标定误差变大	图像与关节时间戳不一致	查询曝光时刻的 TF
更换灵巧手后抓取偏移	TCP/法兰坐标变更	检查 ${}^{G}T_{\mathrm{TCP}}$
仿真正常、真机失败	相机/动力学/控制存在 gap	分别验证视觉与执行

手眼采样的简单保守策略是：**运动到位** → **等待稳定** → **同时记录图像和对应机器人状态** → **保存样本**。这样可以减少运动模糊、时间偏移及末端弹性形变影响。

十四、推荐开源工具与官方资料

工具	主要功能	使用建议
OpenCV	内参、PnP、手眼标定	优先掌握
ROS image_pipeline	ROS 内参及图像处理	使用 ROS 时推荐
easy_handeye2	ROS 2 手眼标定	可用于可重复工作流
MoveIt Calibration	GUI 标定流程	需要可视化交互时
Kalibr	多相机、相机–IMU、时间标定	复杂多传感器系统
RealSense SDK	设备参数、深度对齐、反投影	D435/D455 等设备
ROS REP 103	ROS 坐标系规范	检查 optical frame

对于只有头部和腕部两台相机的操作系统，不必一开始就引入复杂的相机–IMU 联合标定工具。优先把机器人 TF 链、PnP 和手眼标定做好。

十五、针对 Tron2 + Revo3 的实施路线

建议分成四个阶段进行，避免一开始把所有误差来源混合在一起。

阶段一：设备内参与深度检查

- 读取 RealSense RGB/Depth 内参和畸变；
- 对腕部 RGB 图像进行必要的内参验证或重标定；
- 检查 RGB–Depth 对齐、深度尺度和典型抓取距离上的误差；
- 确认图像分辨率、裁剪及内参与视觉模型输入匹配。

此阶段通常不需要机械臂运动。

阶段二：头部相机几何标定

目标：获得 ${}^{B}T_{C_h}(q_h)$。

1. 核查 Tron2 的 yaw/pitch 关节轴、固定偏移和 URDF；
2. 固定 ChArUco 板，采集多个云台姿态；
3. 记录对应 $q_{h,i}$，使用 FK 求 ${}^{B}T_{H}(q_{h,i})$；
4. 使用 PnP 求 ${}^{C_h}T_{T,i}$；
5. 标定固定相机安装变换 ${}^{H}T_{C_h}$；
6. 在未参加求解的 yaw/pitch 姿态下测试三维定位误差。

若当前只使用一个头部角度，可先将云台固定并完成简化版标定。

阶段三：腕部相机与 TCP

目标：获得 ${}^{G}T_{C_w}$ 和 ${}^{G}T_{\mathrm{TCP}}$。

1. 固定标定板，在 20–30 个多样姿态下采样；
2. 保存末端 FK 与相机看到的板位姿；
3. 运行 `calibrateHandEye()`；
4. 用未见姿态验证固定目标板的一致性；
5. 核对更换 Revo3 之后的工具坐标系和指尖几何。

阶段四：视觉闭环与学习部署

统一数据流：

```
Head RGB-D / Wrist RGB
  -> Object perception
  -> Camera-frame object pose
  -> Calibrated camera transform
  -> Robot-base object pose
  -> Policy / grasp planner
  -> IK / control
  -> Tron2 + Revo3
  -> Visual feedback / replanning
```

优先完成三个关键变换：

$$\boxed{{}^{B}T_{C_h}(q_h)}, \qquad \boxed{{}^{G}T_{C_w}}, \qquad \boxed{{}^{G}T_{\mathrm{TCP}}}.$$

得到这三个结果后，再结合 FK、TF2 和时间同步，即可把头部感知、腕部视觉及抓取控制整合到统一坐标系统中。

十六、总结与检查清单

核心问题	对应技术	机器人学习中的作用
图像如何对应三维几何?	内参与畸变、PnP、RGB-D	深度反投影、位姿估计
相机相对于机器人在哪里?	外参、手眼标定	抓取坐标变换
机器人运动后变换是否仍然有效?	FK、URDF、TF2、时间同步	主动视觉、视觉闭环

实际部署前建议逐项确认：

- [ ] 使用正确的图像分辨率与内参，并明确像素是否已去畸变。
- [ ] 所有变换都标清方向（如 ${}^{B}T_{C}$ 而非模糊的 `camera_extrinsic`）。
- [ ] 确认 `camera_link` 与 `camera_optical_frame` 的区别。
- [ ] 所有机器人 FK、PnP、手眼采样使用一致的长度单位和帧定义。
- [ ] 头部 yaw/pitch 轴、关节固定偏移以及腕部相机安装外参正确。
- [ ] RGB-D 已对齐，且知道深度代表的轴向距离和尺度。
- [ ] 对外参使用独立的多姿态数据验证，而非只看训练数据残差。
- [ ] 动态实验时，图像与关节角/TF 时间戳严格对应。
- [ ] 更换灵巧手后已核查 TCP 与指尖坐标。
- [ ] 最终使用三维位置测试和真实抓取表现闭环验证。

对于 Tron2 操作系统，最值得优先掌握的是 OpenCV PnP、Hand-Eye Calibration、机器人 FK 和 ROS TF2。 这四项结合起来，就能构建不依赖 Vicon、适合头部/腕部视觉感知与灵巧手闭环操作的相机标定基础设施。

说明：本教程中的尺寸、采样数量、路径、TF 名称及部分命令参数为示例或工程建议，并非 Tron2 的厂家参数或经现场标定的真实结果。实际部署必须以设备、URDF、采集日志与实验验证为准。

视觉感知与状态估计

课程首页 · English · PDF

项目	说明
适用范围	RGB-D、六维位姿、时序跟踪与恢复方法
文档核验日期	2026-10-18
许可	CC BY 4.0 · Xiong Zeng and contributors

核验边界： 概念方法与实验设计；没有真机或精度验证。 版本与固定源码引用见版本说明。原创教材采用 CC BY 4.0，第三方引用、图片及代码遵循其原有许可；注明来源不等于拥有再许可权。

目录

- 1. 为什么机器人需要状态估计
  - 1.1 先区分观测、估计状态与真实状态
  - 1.2 什么任务需要 6D 位姿
- 2. 从像素到机器人基座坐标系
  - 2.1 头部与腕部相机不能混用固定外参
  - 2.2 时间戳延迟会变成空间误差
- 3. 视觉估计器的模块化流水线
  - 3.1 估计与跟踪要区别设计
  - 3.2 对称物体可能没有唯一姿态
- 4. 时序滤波、置信度与失败恢复
  - 4.1 置信度驱动安全动作
  - 4.2 丢失与重新捕获是基本能力
- 5. 训练时如何处理估计误差
  - 5.1 评估误差应与控制目标关联
- 6. Panda 方块抓取的最小闭环实验
  - 6.1 验收清单
- 7. 课程衔接与参考资料

学习目标： 理解位姿估计链路；处理延迟和丢失；对接视觉闭环机器人控制。

1. 为什么机器人需要状态估计

视觉观测是部分、延迟且有噪声的。抓取策略希望知道物体位姿、末端位姿以及是否保持接触，但相机直接测得的仅是像素或深度。状态估计负责把图像、关节状态、已知几何和时间历史整合为可用于闭环控制的状态与不确定性。



1.1 先区分观测、估计状态与真实状态

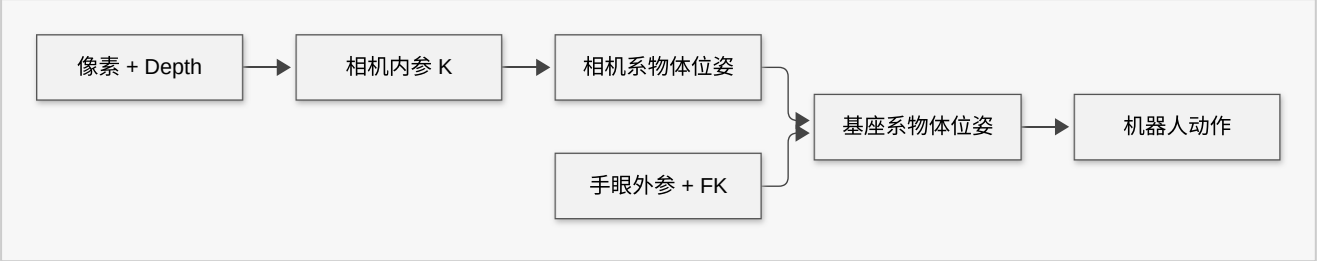
真实环境状态 s_t 通常不可完全测得；相机观测 o_t 随遮挡、曝光、视角变化而变化。估计器输出 $\hat{s}_t$ 与可信度，策略动作依赖这些估计。仿真中应区分 ground-truth state、rendered observation 和 estimated state，避免训练部署信息不一致。

1.2 什么任务需要 6D 位姿

精细插装、工具旋转与抓取对物体的三维位置和姿态可能敏感，适合显式估计 SE(3) 位姿。对目标对称、被遮挡或纹理匮乏的场景，可采用任务相关关键点、轮廓、相对位姿或直接视觉策略；**不是每个 VLA 都需要显式 6D 位姿**。

2. 从像素到机器人基座坐标系

先完成内参、畸变、深度尺度和外参标定。若相机估计 T_C^O，机器人位姿链是 $T_B^O = T_B^C T_C^O$。腕部相机的 $T_B^C(t)$ 随机器人关节角变化，必须使用同一时间戳下的正向运动学和手眼外参。



2.1 头部与腕部相机不能混用固定外参

固定头部相机一般有常量外参；若头部有云台，则 $T_B^C(t)$ 仍是时变的。腕部相机通常由 $T_B^E(q_t)T_E^C$ 构造。相机到关节轴的实际几何误差不会因 URDF 已存在而自动消失，需要独立标定或验证。

2.2 时间戳延迟会变成空间误差

机器人移动时，用当前关节角解释过去的腕部图像，会引入系统性物体位姿偏差。记录相机曝光/到达时间、机器人状态时间、推理输出时间和动作生效时间，并用历史 TF 插值。测得的端到端延迟应纳入控制器稳定性与动作幅度评估。

3. 视觉估计器的模块化流水线

典型模块包括目标检测或分割、物体对应关系与位姿候选、姿态细化、时间跟踪、异常检测、置信度估计。分割 mask 不是 6D 位姿；PnP 需要可靠的 2D-3D 对应；FoundationPose 可用于新物体的位姿估计与跟踪，但具有其规定的 CAD 或参考图像等输入条件。

3.1 估计与跟踪要区别设计

初始化可从分割、深度和几何模型产生多个候选；跟踪在上一帧姿态附近优化，因此通常更快，但也更容易漂移。遮挡、出画面或误检后，必须允许回到全局重初始化模式，不能无限制沿用旧位姿。

3.2 对称物体可能没有唯一姿态

圆柱或无纹理工具绕对称轴的旋转可能无法从视觉唯一确定。评估时采用对称性感知的姿态误差，或将任务必需的接触关键点纳入状态。如果反馈本身不可观测，调大模型不一定能解决问题。

4. 时序滤波、置信度与失败恢复

推荐维护“预测 → 测量更新 → 有效性检验 → 恢复”的闭环。预测来自上一帧估计及机器人/物体运动先验，观测更新来自当前图像。可用 EKF、粒子滤波、学习型递推网络或简单时序平滑，但不能只报告滤波后的漂亮轨迹：需要测量延迟、误差和重新初始化次数。

4.1 置信度驱动安全动作

给估计输出附带不确定性或质量分数。连续低置信度时可降低速度、保持安全姿态、重新观测或请求人工干预；**不要在未知物体位姿下继续高力接触操作**。置信度校准需要用留出的场景验证，不能只凭网络 softmax 作为误差界。

4.2 丢失与重新捕获是基本能力

设计显式状态机：TRACKING、UNCERTAIN、LOST、RELOCALIZING。通过遮挡、主动移动相机、物体掉落和视角突变测试转换规则，并测量重定位成功率与时间。应记录轨迹 ID 的持续性，避免误把另一个相似物体当作原目标。

5. 训练时如何处理估计误差

特权状态的仿真教师策略不等于视觉部署策略。可采用视觉蒸馏或在状态输入上注入统计匹配的位姿噪声，但应考虑有偏误差、时间相关噪声、估计延迟、漏检和突发错误。独立比较 oracle pose、noisy pose 和 estimated pose 三种输入，才能判断性能瓶颈来自感知还是控制。

5.1 评估误差应与控制目标关联

记录平移误差、旋转误差、轨迹丢失率、置信度可靠性、推理延迟、遮挡场景的恢复时间，以及使用估计器之后的抓取成功率。姿态数值误差更小，不一定意味着最终操作更稳；应同时报告任务指标。

6. Panda 方块抓取的最小闭环实验

在共同 Panda + 方块 + 桌面场景中，先用模拟 ground truth 定义基线，再开启固定相机的 RGB-D 位姿估计，最后增加腕部相机及其时变外参。控制器保持不变，只改变视觉反馈链路。设置三个场景：正常光照、遮挡、初始位姿随机；扩展可加入主动相机视角恢复。

6.1 验收清单

检查项	判断标准
坐标闭环	已知物体能正确投到 base frame
时间对齐	图像和 FK 在相同采样时间对齐
无遮挡误差	在留出场景上定量测量
遮挡恢复	状态机可重定位，不输出长期过期姿态
控制安全	不确定度过高时可限速或停止
任务指标	记录抓取成功率、延迟与恢复率

7. 课程衔接与参考资料

本章接续相机标定，并为模仿学习、运动控制 RL与视觉 VLA 部署提供状态接口。

- FoundationPose，CVPR 2024
- SAM 2
- OpenCV Camera Calibration and 3D Reconstruction
- ROS 2 tf2

核验边界： 内容以概念、方法选择及实验设计为主，并未验证某特定相机、GPU 或模型在机器人上的实时性能。

机器人动力学：面向具身智能的理论与仿真实践

课程首页 · English · PDF

项目	说明
适用范围	刚体动力学理论；MuJoCo 与 Isaac 示例为概念说明，非已测试的跨引擎组合
文档核验日期	2026-10-18
许可	CC BY 4.0 · Xiong Zeng and contributors

核验边界： 文档整理与上游引用快照核对；未进行教程环境、训练或硬件验证。 版本与固定源码引用见版本说明。原创教材采用 CC BY 4.0，第三方引用、图片及代码遵循其原有许可；注明来源不等于拥有再许可权。

目录

- 阅读指南：先理解物理链，再学习公式
- 第一章 从一个具身任务看动力学
  - 1.1 运动学告诉你“在哪里”，动力学告诉你“为什么动”
  - 1.2 状态、动作与观测不能混为一谈
  - 1.3 一条必须掌握的核心方程
- 第二章 只学动力学需要的运动学
  - 2.1 为什么需要关节坐标与末端坐标
  - 2.2 Jacobian 是从运动通向力的桥梁
  - 2.3 为什么 IK 不能代替动力学
- 第三章 从一根杆理解质量、惯量与重力
  - 3.1 平移：质量抵抗加速度
  - 3.2 旋转：惯量不仅取决于质量
  - 3.3 单关节摆：最简单的机器人动力学
  - 3.4 刚体旋转中的速度耦合
- 第四章 从单关节推广到整台机器人
  - 4.1 为什么会出现质量矩阵
  - 4.2 从能量得到标准方程
  - 4.3 正向动力学与逆动力学
  - 4.4 能量解释为什么模型值得信任
- 第五章 二连杆机械臂：一遍算懂 M、速度项和重力
  - 5.1 几何与约定
  - 5.2 从动能看关节耦合
  - 5.3 速度项与重力项
  - 5.4 四个最有用的思考实验
- 第六章 复杂机器人如何快速计算动力学
  - 6.1 机器人为什么不用手工推导所有连杆

    - 6.2 重力补偿是逆动力学的特例
- 第七章 执行器：策略动作如何真正作用于机器人
    - 7.1 位置动作不等于立即到达目标位置
    - 7.2 必须考虑的非理想因素
    - 7.3 强化学习动作空间与物理接口对应关系
- 第八章 接触动力学：抓住、推动、滑动和掉落
    - 8.1 运动受限时，接触力从哪里来
    - 8.2 为什么接触是单边约束
    - 8.3 摩擦锥决定物体能不能被握住
    - 8.4 碰撞是速度的快速变化
    - 8.5 抓取中要区分四种接触模式
- 第九章 灵巧操作：机械臂、手与物体是一个耦合系统
    - 9.1 为什么只训练手指不一定够
    - 9.2 什么时候可以把物体当作额外负载
    - 9.3 对 Tron2 + Revo3 的具体启示
- 第十章 人形机器人：从固定基座到浮动基座
    - 10.1 为什么人形机器人的身体不能像关节一样直接控制
    - 10.2 站稳需要外部支持力
    - 10.3 质心动力学与全身控制只保留最重要部分
- 第十一章 动力学控制：哪些方法对具身智能最有用
    - 11.1 PD 与重力补偿
    - 11.2 计算力矩控制
    - 11.3 阻抗控制用于柔顺接触
    - 11.4 MPC/WBC 的位置
- 第十二章 MuJoCo 与 Isaac Sim / Isaac Lab：如何理解差异
    - 12.1 两套软件分别承担什么角色
    - 12.2 不能只对齐 URDF 文件名
    - 12.3 必须区分三个时间尺度
- 第十三章 无代码实践：先验证自由空间，再引入接触
    - 实验 1｜单关节杆的重力与惯量
    - 实验 2｜二连杆的质量矩阵与关节耦合
    - 实验 3｜正向与逆动力学一致性
    - 实验 4｜PD 与重力补偿的差异
    - 实验 5｜动作频率与饱和
- 第十四章 无代码实践：接触、灵巧手与人形机器人
    - 实验 6｜方块落桌：法向接触与碰撞
    - 实验 7｜斜面滑动：验证摩擦直觉
    - 实验 8｜灵巧手抓取、旋转与掉落恢复
    - 实验 9｜人形站立：足底力与质心
    - 实验 10｜跨平台与 Sim2Real 参数敏感性

    - 实验报告最小模板
- 第十五章 从仿真走向真实机器人：检查清单
    - 15.1 模型首先要物理自洽
    - 15.2 Sim2Real 优先随机化什么
    - 15.3 四个常见误区
    - 15.4 一页公式速查
- 附录 A：课程知识点与本教程的取舍
- 附录 B：符号、阅读建议与参考资料

本章知识地图



阅读指南：先理解物理链，再学习公式

机器人动力学不是一套孤立的数学公式，它解决的是具身智能中最直接的问题：**策略输出一个动作以后，机器人和环境为什么会这样运动？** 本教程以一个贯穿始终的任务——机械臂配合灵巧手从桌面抓起螺丝刀、旋转并在掉落后重新抓取——组织内容。一个人形机器人站立并伸手抓物体，也服从同样的物理规律，只是多了浮动基座与足底接触。

本教程参考张巍教授的 MEE5114 Advanced Control for Robotics 中刚体运动、动力学与控制的教学顺序，但**不是原课程讲义的翻译**；同时吸收 *Modern Robotics*、Featherstone 以及 MuJoCo / Isaac Lab 官方文档中与机器人学习直接相关的知识。[1–8]

学习主线：动作命令 a → 执行器产生关节力矩 τ → 刚体动力学决定加速度 $\ddot{q}$ → 接触决定机器人与物体的相互作用 → 积分得到下一状态 → 相机、编码器等产生新的观测 o → 策略更新动作。

$$\boxed{a_t \longrightarrow \tau_t \longrightarrow (\ddot{q}_t, \lambda_t) \longrightarrow s_{t+1} \longrightarrow o_{t+1}.}$$

保留的内容：最必要的坐标与 Jacobian、质量/惯量、标准机器人动力学、正逆动力学、接触与摩擦、执行器、浮动基座、控制、仿真与 Sim2Real。

有意略去的内容：Lie 群证明、完整螺旋理论和 POE 展开、空间向量叉乘算子的详细矩阵推导、复杂 RNEA/ABA 的逐行实现、通用最优控制与 SDP 理论，以及软件安装命令和任何代码。需要研究级算法推导时，可直接查阅文末参考资料。

使用方式：第 1–6 章建立动力学直觉，第 7–11 章解释接触和控制，第 12–14 章在 MuJoCo、Isaac Sim / Isaac Lab 中设计实验并联系真实机器人。公式属于可验证的物理模型；实验描述是实践方案，并非已完成的仿真结果。

第一章 从一个具身任务看动力学

1.1 运动学告诉你“在哪里”，动力学告诉你“为什么动”

设机械臂关节位置为 $q \in \mathbb{R}^n$，关节速度为 $\dot{q}$。运动学回答：已知 q，末端手掌的位姿在哪里？已知 $\dot{q}$，手指运动有多快？它**不需要**质量、惯量或电机力矩。

动力学回答：施加指定力矩后，关节加速度是多少？接触桌面后，支持力如何改变运动？物体脱离手指后，为什么会自由下落？因此动力学的输入不只是位置，而是**当前状态、驱动力及外力**。

任务中的问题	主要理论	为什么需要
手指应该移动到哪里?	正/逆运动学	将物体位姿转成目标关节位姿
电机到底如何推动手指?	执行器 + 刚体动力学	解释力矩、加速度和控制延迟
螺丝刀为什么不滑落?	接触 + 摩擦	判断接触力是否处于摩擦锥内
为什么换装灵巧手后控制变差?	质量、惯量、重力补偿	负载与质心发生变化
人形机器人为什么需要脚下支持力?	浮动基座 + 质心动力学	内部关节力不能直接悬停整个身体

1.2 状态、动作与观测不能混为一谈

真实物理状态通常包括机器人和物体的位姿、速度及接触模式，可抽象为 $s = (q, \dot{q}, x_o, v_o, \text{contacts}, \dots)$。策略只获得观测 o，如 RGB-D、关节编码器、手指电流估计；相机中没有直接出现质量矩阵或真实摩擦系数。

动作 a 的物理含义取决于控制接口：它可能是目标关节位置、目标关节速度、力矩，也可能是手部目标位姿。**同一个动作向量，经过不同低层伺服控制器，会产生不同的物理行为**。这是理解机器人学习论文中 action space 的首要前提。

1.3 一条必须掌握的核心方程

对固定基座、完全驱动且关节坐标独立的刚性机械臂，常用模型为

$$\boxed{M(q)\ddot{q} + C(q,\dot{q})\dot{q} + g(q) = \tau + J_c(q)^T\lambda.}$$

其中 M 表示构型相关的惯性，$C\dot{q}$ 表示速度耦合，g 表示抵消重力所需的广义力，τ 是关节驱动力，λ 是环境对机器人的接触力，$J_c^T\lambda$ 将接触力换算为广义力。

记 $h(q,\dot{q}) = C(q,\dot{q})\dot{q} + g(q)$，便得到更紧凑的形式：

$$M(q)\ddot{q} + h(q,\dot{q}) = \tau + J_c^T\lambda.$$

真实设备还可能存在摩擦、传动弹性、力矩饱和与延迟；这些效应必须在解释实验差异时重新加入。

第二章 只学动力学需要的运动学

2.1 为什么需要关节坐标与末端坐标

对于 n 个关节，$q = (q_1, \dots, q_n)$ 表示关节配置。末端位置可写为 $p = f(q)$；若需表示物体或夹具的完整六自由度位姿，则使用

$$T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}, \qquad R^TR = I,\ \det R = 1.$$

R 表示朝向，p 表示位置。坐标系必须明确，例如 T_{AB} 把在坐标系 B 中表示的点转换到 A。对于具身学习，更重要的是**保证相机、机械臂、物体、桌面使用一致的坐标系**，而不是证明 $SO(3)$ 或 $SE(3)$ 的全部代数性质。

2.2 Jacobian 是从运动通向力的桥梁

若末端位置是 $p(q)$，则

$$\boxed{\dot{p} = J_p(q)\dot{q}, \qquad \ddot{p} = J_p(q)\ddot{q} + \dot{J}_p(q,\dot{q})\dot{q}.}$$

这个 Jacobian 首先告诉我们关节速度如何转成末端速度。更关键的是：末端所受的力 f 可映射成关节广义力

$$\boxed{\tau_{\text{ext}} = J_p(q)^T f.}$$

证明只需用**虚功相等**：$f^T \delta p = f^T J_p \delta q = \tau_{\text{ext}}^T \delta q$。接触桌面时，桌面对指尖的力就是沿着这个通道影响每个关节。

对于六自由度手掌运动，可使用包含角速度与线速度的六维速度（Twist）及其六维 Jacobian。此处只需记住：**速度表示、参考点和力矩表示必须配对一致**，不必先掌握空间伴随变换的完整推导。

2.3 为什么 IK 不能代替动力学

IK 将目标末端位姿转成关节目标，但不会自动考虑物体重量、关节惯性或接触力。轨迹看起来“几何上可达”，不等于执行器在给定时间内能够实现。例如，抓起重物后，目标位置不变，维持姿态所需的力矩也可能明显改变。

第三章 从一根杆理解质量、惯量与重力

3.1 平移：质量抵抗加速度

刚体质心遵循牛顿第二定律：

$$\boxed{f = ma_C.}$$

同样大小的外力，质量更大的物体通常产生更小的质心加速度。这个直觉在抓取中很直接：同一抓取力策略面对不同重量的螺丝刀，会出现不同的加速与滑落表现。

3.2 旋转：惯量不仅取决于质量

对于绕固定轴旋转的刚体，可写成 $\tau = I_O \ddot{q}$（不计其他力矩）。转动惯量 I_O 取决于**质量相对于轴的位置**。若质心距关节轴为 r，则平行轴定理给出

$$\boxed{I_O = I_C + mr^2.}$$

同质量的灵巧手，若装得更远或质心更偏离旋转轴，关节的等效惯量仍可能更大。这解释了“只是换了末端工具，却改变整条机械臂的动态响应”。

3.3 单关节摆：最简单的机器人动力学

考虑在竖直平面内转动的均匀刚性杆，q 从水平方向向上计正，质心距轴 r，绕关节轴惯量为 I_O。忽略摩擦，其动力学为

$$\boxed{I_O\ddot{q} + mgr\cos q = \tau.}$$

这一个方程已经包含三个关键概念：惯性项 $I_O\ddot{q}$，重力项 $mgr\cos q$，驱动力矩 τ。当杆水平且静止时，所需重力补偿力矩为 mgr；杆竖直时，静态重力力矩为零。

重要区分：机器人所谓“重力补偿”，补的是构型决定的重力力矩，不是简单给每个关节加一个常数。静止不代表力矩为零。

3.4 刚体旋转中的速度耦合

三维刚体旋转更一般地满足（在刚体固连坐标系内）

$$\tau_C = I_C\dot{\omega} + \omega \times (I_C\omega).$$

第二项说明旋转会改变角动量方向。理解其存在即可；在具身学习实践中，更常直接让多刚体求解器计算这类速度耦合项，而不手动展开每根连杆的三维方程。

第四章 从单关节推广到整台机器人

4.1 为什么会出现质量矩阵

机械臂上某个关节运动时，可能带动整条下游连杆，因此每个关节的等效惯量取决于全身构型。把所有连杆平动与转动动能相加，总动能必定可以写成

$$\boxed{K(q,\dot{q}) = \tfrac{1}{2}\dot{q}^T M(q)\dot{q}.}$$

$M(q)$ 是**广义质量矩阵**，一般不是对角矩阵。非对角项表示一个关节的加速度会影响另一个关节所需力矩。对独立广义坐标、非退化的刚性系统，$M(q)$ 对称正定。

4.2 从能量得到标准方程

将重力势能记作 $U(q)$，定义拉格朗日函数 $L = K - U$。欧拉—拉格朗日方程为

$$\frac{d}{dt}\frac{\partial L}{\partial \dot{q}_i} - \frac{\partial L}{\partial q_i} = \tau_i.$$

代入二次型动能，整理得

$$\boxed{M(q)\ddot{q} + C(q,\dot{q})\dot{q} + g(q) = \tau, \qquad g(q) = \nabla_q U(q).}$$

$C\dot{q}$ 中出现的是不同关节速度的乘积和平方。它不是接触力，也不是人为设置的阻尼，而是刚体运动本身带来的惯性耦合。通常直接使用 $h = C\dot{q} + g$；C 矩阵的具体表示并不唯一。

4.3 正向动力学与逆动力学

正向动力学：给定当前位置、速度和力矩，求下一个加速度：

$$\boxed{\ddot{q} = M(q)^{-1}[\tau - h(q,\dot{q})] \quad \text{(无接触)}.}$$

逆动力学：给定期望加速度，求需要的力矩：

$$\boxed{\tau = M(q)\ddot{q} + h(q,\dot{q}).}$$

物理仿真更频繁地使用正向问题；重力补偿、轨迹前馈与某些全身控制会使用逆向问题。这里的逆矩阵是数学表达，实际实现通常求解线性系统而不直接显式求逆。

4.4 能量解释为什么模型值得信任

无摩擦、无外接触时，总机械能 $E = K + U$ 的变化满足

$$\boxed{\dot{E} = \tau^T \dot{q}.}$$

即“电机输出的机械功率改变系统机械能”。当加上耗散性摩擦时，部分功率变为热；当有外部接触且接触点运动时，还要计入接触做功。这是分析仿真能量漂移、调试控制器和理解柔顺性的基本工具，不必在本教程中深入 Lyapunov 定理证明。

第五章 二连杆机械臂：一遍算懂 M、速度项和重力

5.1 几何与约定

考虑竖直平面内的两关节旋转臂：q_1 从水平向上计正，q_2 是第二杆相对第一杆的角度；第一杆长 l_1，质量 m_1，质心距第一关节 r_1，惯量 I_1；第二杆的对应参数为 m_2, r_2, I_2。第二杆相对世界的方向角为 $q_1 + q_2$。

质心坐标为

$$\begin{aligned} p_1 &= (r_1 \cos q_1,\ r_1 \sin q_1)^T, \\ p_2 &= (l_1 \cos q_1 + r_2 \cos(q_1 + q_2), \\ &\qquad l_1 \sin q_1 + r_2 \sin(q_1 + q_2))^T. \end{aligned}$$

这里的 I_1, I_2 是各连杆绕质心、垂直运动平面的惯量。

5.2 从动能看关节耦合

总动能为

$$K = \frac{1}{2} m_1 \|\dot{p}_1\|^2 + \frac{1}{2} I_1 \dot{q}_1^2 + \frac{1}{2} m_2 \|\dot{p}_2\|^2 + \frac{1}{2} I_2 (\dot{q}_1 + \dot{q}_2)^2.$$

整理成 $K = \frac{1}{2} \dot{q}^T M(q) \dot{q}$，得到

$$\begin{aligned} M_{11} &= I_1 + I_2 + m_1 r_1^2 + m_2 (l_1^2 + r_2^2 + 2 l_1 r_2 \cos q_2), \\ M_{12} &= M_{21} = I_2 + m_2 (r_2^2 + l_1 r_2 \cos q_2), \\ M_{22} &= I_2 + m_2 r_2^2. \end{aligned}$$

读懂这个矩阵：M_{11} 随 q_2 改变；M_{12} 通常非零。这分别对应构型相关惯量与关节之间的惯性耦合。

5.3 速度项与重力项

令 $b = m_2 l_1 r_2$，速度相关项是

$$C(q,\dot{q})\dot{q} = \begin{bmatrix} -b \sin q_2 (2\dot{q}_1 \dot{q}_2 + \dot{q}_2^2) \\ b \sin q_2 \dot{q}_1^2 \end{bmatrix}.$$

重力势能为

$$U = m_1 g r_1 \sin q_1 + m_2 g [l_1 \sin q_1 + r_2 \sin(q_1 + q_2)],$$

因此

$$g(q) = \begin{bmatrix} (m_1 r_1 + m_2 l_1) g \cos q_1 + m_2 r_2 g \cos(q_1 + q_2) \\ m_2 r_2 g \cos(q_1 + q_2) \end{bmatrix}.$$

最终得到 $\tau = M(q)\ddot{q} + C(q,\dot{q})\dot{q} + g(q)$。这个例子展示了从质心位置 → 质心速度 → 动能与势能 → 质量矩阵与重力项的完整逻辑，不需要深入更复杂的三维符号计算，就足以理解实际机器人的多体动力学结构。

5.4 四个最有用的思考实验

1. **静止、无接触**：令 $\dot{q} = \ddot{q} = 0$，关节力矩必须满足 $\tau = g(q)$。
2. **低速运动**：速度项相对较小，惯性与重力是主要影响。
3. **高速摆动**：速度乘积项增大，忽略 $C\dot{q}$ 会出现预测偏差。
4. **换更重的第二杆**：M 与 g 都发生变化；同一套位置伺服参数可能需要重新整定。

第六章 复杂机器人如何快速计算动力学

6.1 机器人为什么不用手工推导所有连杆

对于几十个关节的系统，逐项符号展开 M, C, g 成本高、容易出错。现代动力学库通常基于机械结构树递归计算。需要认识三种算法**分别解决什么问题**，但不需要先背熟空间向量递归公式。

算法	输入	输出	机器人学习中的用途
RNEA（递归 Newton–Euler）	$q, \dot{q}, \ddot{q}$	逆动力学广义力	重力补偿、力矩前馈
CRBA（复合刚体算法）	q	$M(q)$	分析关节耦合、模型控制
ABA（关节化刚体算法）	$q, \dot{q}, \tau$	$\ddot{q}$	正向动力学、物理仿真中的基础计算

RNEA 先由基座向末端传播速度和加速度，再由末端向基座回传力；ABA 利用关节化刚体惯量求解加速度；CRBA 组合下游连杆惯性得到质量矩阵。在无接触、统一模型和一致坐标约定下，正向与逆向结果应当互相验证。

6.2 重力补偿是逆动力学的特例

将目标速度、加速度设为零，理想刚性机械臂满足

$$\boxed{g(q) = \mathrm{RNEA}(q, 0, 0).}$$

这比“为每个关节设固定力矩”更符合真实机器人随姿态变化的负载需求。对于有负载的机械臂，必须先保证手和物体的质量、质心、惯量进入模型；否则算法即使计算正确，预测仍会不准确。

第七章 执行器：策略动作如何真正作用于机器人

7.1 位置动作不等于立即到达目标位置

机器人学习策略经常输出 q_{target}，但真实电机不是瞬间把机器人“传送”到这个位置。在一个典型的关节 PD 执行器模型中，

$$\boxed{\tau_{\text{servo}} = K_p(q_{\text{target}} - q) + K_d(\dot{q}_{\text{target}} - \dot{q}).}$$

然后由动力学方程决定 $\ddot{q}$。因此系统实际是“目标位置 → 伺服力矩 → 加速度 → 速度 → 位置”。有些机器人将此闭环隐藏在驱动器内部，策略只看到位置接口。

7.2 必须考虑的非理想因素

因素	真实效果	常见训练后果
力矩/电流限幅	目标过大也无法无限加速	策略要求的动作实际执行不到
速度限制	快速动作被约束	跟踪滞后与接触时序偏差
摩擦与齿隙	小力矩不一定带来运动	低速精细操作不准
延迟与滤波	命令晚于观测生效	闭环出现振荡或超调
电机/负载惯量	换手或拿物体后惯性变化	同样控制增益表现不同
传动柔顺	手指或关节存在弹性	接触力响应与刚性模型不同

对于减速比为 N 的理想传动，电机转子惯量 I_m 反射到关节侧近似为 $N^2 I_m$；这说明只看连杆质量并不足以确定执行器动态。实际装置还可能有电流环、速度环、位置环和电机温度保护。

7.3 强化学习动作空间与物理接口对应关系

策略输出	优点	关键风险
关节目标位置	稳定、易用，常用于模仿学习	结果依赖 PD 增益与命令周期
关节位置增量	适合残差或跟踪型策略	积分漂移、限位处理
关节目标速度	可直接描述运动方向	积分误差、执行器低层差异
关节力矩	可显式探索动力学交互	训练难、饱和和安全要求高
末端位姿增量	任务含义直观	需要 IK / Jacobian 与接触处理

结论：比较两个强化学习方法时，必须同时报告低层执行器模型，否则对同一个 action 的物理解释可能不一致。

第八章 接触动力学：抓住、推动、滑动和掉落

8.1 运动受限时，接触力从哪里来

假设指尖与桌面保持固定接触，接触点速度为零：

$$J_c(q)\dot{q} = 0, \qquad J_c(q)\ddot{q} + \dot{J}_c(q,\dot{q})\dot{q} = 0.$$

动力学需要同时满足接触产生的广义力：

$$\boxed{M(q)\ddot{q} + h(q,\dot{q}) = \tau + J_c(q)^T\lambda.}$$

因此加速度和接触力**相互耦合**：机器人试图往桌面内部移动时，桌面通过法向支持力阻止穿透；关节所需的实际力矩随之变化。上述“接触点不动”的模型仅适用于持续粘着、无相对滑动的接触模式。

8.2 为什么接触是单边约束

定义物体之间的法向间隙 $\phi(q) \geq 0$、法向接触力 $\lambda_n \geq 0$，理想刚性单边接触满足

$$\boxed{\phi(q)\lambda_n = 0.}$$

它表达：**要么分离且无接触力，要么接触且允许压力**。普通桌面不会在螺丝刀离开后继续“拉住”它。物体粘接、吸盘和磁吸是另行建模的情况。

8.3 摩擦锥决定物体能不能被握住

对于库仑摩擦，切向接触力 λ_t 与法向力 λ_n 必须满足

$$\boxed{\|\lambda_t\| \leq \mu\lambda_n.}$$

μ 是摩擦系数。用更大法向夹持力，一般能够提供更大的最大静摩擦力，但也受到手指结构、驱动能力、物体变形和安全限制。物体滑落可能是夹持法向力不足、摩擦不足、重力方向改变，或动态惯性载荷突然增大。

8.4 碰撞是速度的快速变化

当手指快速触碰物体，接触力可能在极短时间内很大。采用冲量 P，碰撞前后广义速度近似满足

$$\boxed{M(q)(v^+ - v^-) = J_c(q)^T P.}$$

接触求解器不仅处理持续支持力，也处理碰撞时的速度更新。不同仿真器可能在接触柔顺、离散步长、摩擦近似和约束求解方式上不同，因此相同的质量与摩擦系数不保证完全相同的碰撞曲线。

8.5 抓取中要区分四种接触模式

模式	可观测现象	对策略的意义
无接触	指尖与物体有间隙	应继续靠近而不是施加抓力
接触且粘着	接触点相对速度接近零	可以建立稳定的操作约束
接触且滑动	接触点沿切向相对运动	需调整夹持、轨迹或重新规划
分离/掉落	接触力消失，物体独立运动	需要重新定位、接近与抓取

这也是为什么灵巧操作中的“掉落恢复”不是普通的轨迹跟踪：**系统的接触模式已经切换，必须重新建立新的接触关系**。

第九章 灵巧操作：机械臂、手与物体是一个耦合系统

9.1 为什么只训练手指不一定够

设机械臂与灵巧手的广义坐标为 q_r，物体位姿为 x_o。手指接触力同时作用于机器人和物体，满足作用力与反作用力关系。机器人侧满足

$$M_r(q_r)\ddot{q}_r + h_r = \tau_r + J_r^T\lambda,$$

物体侧满足质心线动量和角动量方程，其外力包括重力以及手指接触力。接触力的符号必须依各自方向定义；两侧不能都把同一个力视为“环境给自己的正向作用力”。

如果物体由手掌整体平移，机械臂负载的质量、质心和惯量起决定性作用；如果物体在指尖滚动或转动，手指接触点、摩擦和物体姿态也成为主导因素。因此末端机械臂运动与灵巧手运动应放在一**个耦合系统**中看待。

9.2 什么时候可以把物体当作额外负载

若物体被稳定夹持，且相对于手掌几乎不发生运动，可以近似把物体视为刚性附加负载，更新末端总质量、质心和惯量。这个近似很适合重力补偿和缓慢搬运。

但当物体发生指尖转动、滑动、切换接触点，或离手掉落时，该近似立即失效；这时需要显式模拟物体独立自由度与接触约束。

9.3 对 Tron2 + Revo3 的具体启示

更换灵巧手后，至少应重新检查：安装坐标和工具中心点（TCP）；末端负载质量、质心及惯量；关节控制增益与限幅；指尖碰撞几何；指尖—物体摩擦；观测频率和执行延迟。

TCP 的改变主要影响几何与 Jacobian，**不等于**改变物理惯量；更换真实灵巧手才会改变负载惯性。头部/腕部相机能够观测物体位姿，但不会自动提供精确接触力；没有触觉时，可结合手指电流、关节状态和视觉变化推断接触，须考虑其不确定性。

第十章 人形机器人：从固定基座到浮动基座

10.1 为什么人形机器人的身体不能像关节一样直接控制

固定在桌面的机械臂基座不动；人形机器人的躯干有六个自由运动维度，却通常没有直接驱动“世界坐标下身体平移和旋转”的电机。若关节数为 n，浮动基座广义速度维度为 $n+6$；采用四元数表示姿态时，配置变量维度通常为 $n+7$，二者不能直接混用。

动力学写成

$$\boxed{M(q)\dot{v} + h(q,v) = S^T\tau + J_c(q)^T\lambda,}$$

其中 S 仅将可驱动关节的力矩放入完整广义力向量。基座运动依赖关节与环境接触之间的耦合。

10.2 站稳需要外部支持力

对于整个机器人质心 c，外部合力满足

$$\boxed{m\ddot{c} = m\mathbf{g} + \sum_i f_i.}$$

站立时 $\ddot{c} \approx 0$，足底接触力之和必须近似抵消重力。若脚离地，机器人不能单靠内部关节力矩维持质心在空中不下落。手部抓物、人形走路和四足行走在物理上都需要处理**接触力与可驱动自由度的关系**。

10.3 质心动力学与全身控制只保留最重要部分

质心角动量变化由外部接触力矩决定：

$$\dot{L}_c = \sum_i (r_i - c) \times f_i + \sum_i m_i^{\text{ext}}.$$

Whole-Body Control（WBC）把全身任务和约束放在一起考虑：末端位置、躯干平衡、足底固定、力矩限幅、摩擦锥以及关节限位。基于模型的 WBC 可显式优化加速度与接触力；强化学习 locomotion policy 则可通过仿真和奖励学到相应行为。**两者都依赖同一套底层物理**，但学习到的策略不一定在部署时显式求解逆动力学或 QP。

第十一章 动力学控制：哪些方法对具身智能最有用

11.1 PD 与重力补偿

给定目标关节位置 q_d 和速度 $\dot{q}_d$，典型力矩控制为

$$\tau = K_p(q_d - q) + K_d(\dot{q}_d - \dot{q}).$$

加入模型重力补偿得到

$$\boxed{\tau = K_p(q_d - q) + K_d(\dot{q}_d - \dot{q}) + g(q).}$$

它适合解释位置动作的低层闭环，也有助于分析机器人换负载后为何需要更新参数。若接触较复杂，重力补偿不足以自动保证接触稳定。

11.2 计算力矩控制

在自由空间、模型较准确、力矩可控时，定义 $e = q_d - q$ 和

$$\ddot{q}^* = \ddot{q}_d + K_d\dot{e} + K_p e,$$

令

$$\boxed{\tau = M(q)\ddot{q}^* + h(q,\dot{q}),}$$

则理想情况下误差满足 $\ddot{e} + K_d\dot{e} + K_p e = 0$。这一推导说明动力学模型如何把非线性运动补偿为简单的跟踪误差动力学。真实机器人仍受饱和、摩擦、建模误差和接触扰动影响。

11.3 阻抗控制用于柔顺接触

灵巧操作中，不应仅追求无限硬的末端位置跟踪。简化的笛卡尔阻抗目标力为

$$F_d = K_x(x_d - x) + D_x(\dot{x}_d - \dot{x}),$$

通过 Jacobian 映射到关节力矩，再结合重力与必要的动力学补偿。与刚性位置跟踪相比，阻抗控制允许末端在接触力作用下适度偏移，更容易实现安全柔顺的物理交互。

11.4 MPC/WBC 的位置

MPC 用动力学预测未来若干步并优化控制输入；WBC 则更侧重同时满足全身任务和接触约束。对于本教程的目标，无需详细展开非线性优化求解器；需要理解的是：**物理约束决定哪些目标加速度、力矩和接触力可同时实现**。高层 VLA 发出的目标不一定天然满足这些约束。

第十二章 MuJoCo 与 Isaac Sim / Isaac Lab：如何理解差异

12.1 两套软件分别承担什么角色

MuJoCo 适合做动力学直觉验证、参数敏感性分析、接触行为分析、较轻量的机械臂与灵巧手实验。它以多刚体、执行器与约束求解为核心，不应因为它有快速仿真能力，就忽视接触模型和求解器设置。

Isaac Sim 提供机器人与场景仿真能力；**Isaac Lab** 在其仿真能力之上组织机器人学习环境、批量训练、动作/观测/奖励与随机化。二者不是彼此独立的两个物理引擎。具体使用的物理后端、版本及执行器模型，应以本地版本对应的官方文档为准。[4–7]

研究问题	MuJoCo 重点	Isaac Sim / Isaac Lab 重点
为什么杆会下落?	最小模型、逐项理解力矩	验证物理属性和关节驱动配置
为什么抓取时会滑动?	接触几何、摩擦与数值参数	多接触、并行环境、随机化
为什么策略真机表现下降?	精简参数对照与诊断	批量 DR、强化学习训练与迁移
怎样评价同一个控制器?	可解释的单环境基线	完整机器人和任务规模扩展

12.2 不能只对齐 URDF 文件名

真正公平的跨平台对照，需要对齐：连杆质量及质心、惯量张量及惯量坐标系、关节轴和限位、基座是否固定、碰撞几何、重力方向、执行器增益和限幅、物理积分步长、控制周期、初始条件、接触与摩擦设置。

即使这些条件尽可能一致，两个物理后端的约束求解、接触柔顺和数值积分仍可能不同，所以**先比较无接触自由空间，再逐渐加入接触**。不要把某个引擎当作另一引擎的“真值”。

12.3 必须区分三个时间尺度

物理仿真步长 Δt_{phys}、执行器或控制器更新间隔 Δt_{ctrl}、策略输出周期 Δt_{π} 可能不同。若策略以较低频率输出位置目标，底层 PD 可以在两次策略动作之间持续更新，物理求解器还会执行多个更小的积分步。

比较策略时必须说明三个频率以及动作保持方式，否则看似相同的策略可能遭遇完全不同的接触响应与延迟。

第十三章 无代码实践：先验证自由空间，再引入接触

以下实验以**同一根单关节杆、同一套二连杆机械臂**作为两个平台的共同起点。所有实验都采用“目标 → 设置 → 观测量 → 预期解释”的结构；不包含软件命令或程序片段。

实验 1｜单关节杆的重力与惯量

目标：区分质量、质心与转动惯量的作用。**设置**：建立绕固定轴转动的竖直平面单杆，先在水平姿态自由释放，再改变杆质量、质心位置与惯量。**记录**：$q(t)$、$\dot{q}(t)$、$\ddot{q}(t)$ 和关节重力力矩。**预期**：改变质心位置同时改变静态重

力力矩与惯量；静止水平杆需要非零保持力矩。

检查点：不同平台需采用一致的转轴、重力方向和角度零位；否则重力曲线相位可能不同。

实验 2｜二连杆的质量矩阵与关节耦合

目标：认识 $M(q)$ 为什么依赖构型。**设置**：使用第五章二连杆参数，将 q_1 固定并缓慢改变 q_2，考察等效关节惯量及静态保持力矩。**记录**：质量矩阵的对角/非对角项，$g(q)$。**预期**：M_{11} 与 M_{12} 随 $\cos q_2$ 变化，关节之间存在耦合。

检查点：只比较质量矩阵时先关闭接触和关节驱动干扰；确认质量单位为 kg、长度为 m。

实验 3｜正向与逆动力学一致性

目标：验证 $M\ddot{q} + h = \tau$。**设置**：选择相同的非奇异机械臂构型、关节速度和目标加速度，先计算实现该加速度所需广义力，再用该力计算自由空间正向加速度。**记录**：目标与计算得到的加速度差、动力学残差。**预期**：统一模型和坐标约定下应足够接近；离散积分和传动模型会带来误差。

检查点：不要把位置伺服目标直接当成关节力矩；先明确是否启用了额外控制器。

实验 4｜PD 与重力补偿的差异

目标：理解低层伺服与模型前馈。**设置**：让二连杆分别在水平和下垂姿态保持静止，比较只有 PD 与 PD+重力补偿。**记录**：稳态误差、最大关节力矩、恢复时间。**预期**：在适当增益和准确模型下，重力补偿能降低姿态相关偏差；不能消除所有执行器延迟或未建模摩擦。

实验 5｜动作频率与饱和

目标：理解为什么相同策略目标可能产生不同运动。**设置**：保持相同目标轨迹，改变物理步长、控制更新周期、策略周期、力矩限幅和 PD 增益。**记录**：跟踪误差、峰值力矩、超调、动作滞后。**预期**：较低策略频率或更严格限幅可能增加跟踪误差；过大增益在接触与粗时间步下可能不稳定。

第十四章 无代码实践：接触、灵巧手与人形机器人

实验 6｜方块落桌：法向接触与碰撞

目标：看懂从自由下落到支持接触的模式变化。**设置**：在 MuJoCo 与 Isaac Sim 中让同质量、同碰撞几何的方块从相同高度落到桌上，分别使用不同接触设置。**记录**：高度、垂直速度、最大穿透、接触法向力/冲量以及静止后抖动。**预期**：两个平台的碰撞细节可能不同；静置后平均支持力应接近物体重量。

实验 7｜斜面滑动：验证摩擦直觉

目标：研究摩擦系数与滑动阈值。**设置**：让方块在缓慢增大倾角的斜面上运动，逐组调整静摩擦参数。**记录**：开始滑动的角度、滑动速度和接触法向力。**预期**：理想库仑静摩擦模型下临界倾角满足 $\tan\theta_c \approx \mu$；真实求解器的静摩擦近似和离散时间会造成偏差。

实验 8｜灵巧手抓取、旋转与掉落恢复

目标：把质量、惯量、摩擦和接触模式放到一个任务中。**设置**：固定螺丝刀形状，分别改变物体质量/质心、摩擦、初始方向、抓取位置以及操作速度；设计抓取→抬起→旋转→扰动→必要时重抓的阶段。**记录**：成功率、物

体姿态误差、滑移距离、接触持续时间、掉落后恢复率、动作次数。

预期：更快的旋转提高惯性载荷，低摩擦使滑落概率提高；恢复成功并不自动说明抓取策略最优，需要检查是否真正识别了接触模式变化。对真实 Tron2 + Revo3，应先核查末端安装、负载和指尖材料，再谈随机化与视觉闭环。

实验 9｜人形站立：足底力与质心

目标：验证浮动基座不能靠内部力矩悬浮。**设置**：选一个有足底接触的人形或四足机器人，比较双足稳定站立、身体轻微前倾和抬起一只脚。**记录**：质心投影、足底接触力、躯干姿态与关节力矩。**预期**：支持力重新分配，稳定控制需要接触条件和质心动态共同满足。

实验 10｜跨平台与 Sim2Real 参数敏感性

目标：找出训练环境中最敏感的物理误差。**设置**：从无接触二连杆开始校准两平台，再逐渐增加摩擦、负载、关节延迟和接触；每次仅改变一组参数，最后联合随机化。**记录**：动力学残差、轨迹误差、接触持续时间、任务成功率、鲁棒性曲线。**预期**：无接触条件下更容易获得可比结果；接触和执行器差异通常是更难匹配的部分。

实验报告最小模板

建议每个实验只用一页记录六项信息：**研究问题、系统状态与动作定义、主要物理参数、干预变量、记录指标、结论与局限**。至少绘制位置/速度随时间变化曲线，接触实验再记录法向力或冲量；跨平台曲线必须使用相同的时间轴、单位、初始条件和事件标注。

第十五章 从仿真走向真实机器人：检查清单

15.1 模型首先要物理自洽

动力学参数不要完全独立地随意随机化。质量、质心、转动惯量之间存在物理关联；如果修改负载位置，转动惯量也通常需要相应改变。惯量矩阵应对称并满足物理可实现性。碰撞几何、惯性参考系和可视化网格也应保持清楚的对应关系。

15.2 Sim2Real 优先随机化什么

建议按实际误差来源排序：末端负载质量/质心、关节摩擦、执行器增益和饱和、延迟、指尖—物体摩擦、接触几何误差，以及物体初始位姿。物理随机化的目标是覆盖**合理真实分布**，并非范围越宽越好。

当观察来源包括 RGB-D 时，还需独立考虑视觉噪声、标定误差和遮挡：视觉误差改变策略对状态的估计；动力学误差改变动作后的真实运动。这两类误差都重要，但不应混为一谈。

15.3 四个常见误区

- **“URDF 正确，动力学一定正确。”** 错。碰撞、执行器、摩擦和数值求解器同样影响运动。
- **“策略输出位置，所以不需要动力学。”** 错。位置目标仍经由执行器力矩实现。
- **“重力补偿可以解决抓取不稳定。”** 不充分。接触摩擦、负载变化和动态扰动仍起作用。
- **“两个仿真器结果不同，必有一个错误。”** 不一定。不同接触模型与离散求解方法可能产生不同的有效物理响应。

15.4 一页公式速查

场景	核心关系	物理含义
末端运动	$\dot{p} = J_p \dot{q}$	关节速度决定末端速度
末端受力	$\tau_{\text{ext}} = J_p^T f$	笛卡尔接触力映射为关节力
无接触机械臂	$M\ddot{q} + h = \tau$	力矩决定关节加速度
含接触机械臂	$M\ddot{q} + h = \tau + J_c^T \lambda$	接触力改变运动
关节重力补偿	$\tau = g(q)$（静止且无接触）	姿态决定保持力矩
摩擦锥	$\lvert\lambda_t\rvert \le \mu\lambda_n$	限制无滑动接触
浮动基座	$M\dot{v} + h = S^T\tau + J_c^T\lambda$	基座欠驱动，依赖外力
质心平动	$m\ddot{c} = m\mathbf{g} + \sum f_i$	站立和抓取受外部支持力支配

附录 A：课程知识点与本教程的取舍

MEE5114 对应内容	本教程位置	处理原则
Lecture 2–7：位姿、Twist、POE、Jacobian	第 2 章	保留坐标系与 Jacobian，省略 POE 详细推导
Lecture 8：Rigid Body Dynamics	第 3–4 章	保留刚体惯量与受力直觉，省略空间算子细节
Lecture 9：Dynamics of Open Chains	第 4–6 章	保留多体方程及 RNEA / CRBA / ABA 作用
Lecture 14：Robot Motion Control	第 7、11 章	保留伺服、重力补偿、计算力矩及阻抗
额外扩展：接触与操作	第 8–9 章	具身任务必需，重点保留
额外扩展：浮动基座	第 10 章	面向人形和腿式机器人
额外扩展：双平台实验与 Sim2Real	第 12–15 章	实践目标与评价指标，无代码

附录 B：符号、阅读建议与参考资料

基本符号：$q, \dot{q}, \ddot{q}$：关节位置/速度/加速度；M：关节空间质量矩阵；$C\dot{q}$：科氏和离心速度项；$g(q)$：重力广义力；$h = C\dot{q} + g$：非线性偏置项；τ：驱动广义力；J：Jacobian；λ：环境接触力；μ：摩擦系数；v：浮动基座广义速度；S：关节驱动选择矩阵。

推荐最短路径：先理解第 1–5 章并完成实验 1–4，再学习第 7–9 章并完成实验 5–8；如果研究人形或四足机器人，再扩展第 10 章与实验 9。实验 10 用于检验跨平台模型一致性与 Sim2Real 误差敏感性。

1. MEE5114 — Advanced Control for Robotics，Wei Zhang：课程总体结构；重点阅读 Lecture 8、9、14。
2. Modern Robotics — Chapter 8: Dynamics of Open Chains：拉格朗日动力学、正逆动力学、约束与执行器。
3. Modern Robotics — Chapter 11: Robot Control：关节和任务空间控制。

4. Featherstone — Rigid Body Dynamics Algorithms / Spatial Vector Resources：理解 RNEA、CRBA、ABA 的进一步材料。
5. MuJoCo — Computation：多刚体、执行器、接触和约束求解。
6. Isaac Lab — Articulations：固定基座与浮动基座、多体结构、动作接口。
7. Isaac Lab — Actuators：显式/隐式执行器与模型差异；以实际安装版本为准。
8. MIT — Underactuated Robotics: Multibody Dynamics：复杂多体与欠驱动系统的拓展阅读。

Isaac Sim、Isaac Lab 与 MuJoCo：机器人仿真和强化学习完整使用教程

课程首页 · English · PDF

项目	说明
适用范围	文档路线：Isaac Lab 2.3.2 / Isaac Sim 5.1；Lab 3.0 Early Access / Sim 6.1。MuJoCo、MJX、MJWarp 需按任务选择版本
文档核验日期	2026-10-18
许可	CC BY 4.0 · Xiong Zeng and contributors

核验边界： 文档整理与上游引用快照核对；未进行教程环境、训练或硬件验证。 版本与固定源码引用见版本说明。原创教材采用 CC BY 4.0，第三方引用、图片及代码遵循其原有许可；注明来源不等于拥有再许可权。

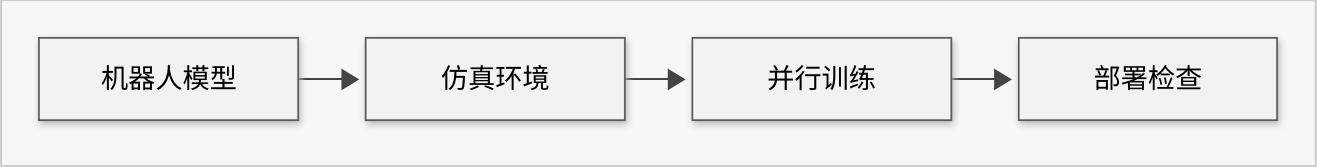
目录

- 1. 三个平台究竟是什么？
  - 1.1 角色分工
  - 1.2 研究任务怎么选？
- 2. 版本、安装方案与硬件注意事项（不含命令）
  - 2.1 两条版本路线
  - 2.2 计算资源如何分配
  - 2.3 环境验收，而不只是“能够启动”
- 3. 不写代码，如何真正学会两个平台？
  - 3.1 MuJoCo 的入门顺序
  - 3.2 Isaac Sim / Isaac Lab 的入门顺序
  - 3.3 推荐统一练习：Panda + 方块 + 桌面
- 4. 机器人建模：URDF、MJCF、USD 的对应关系
  - 4.1 不存在无损“一键转换”
  - 4.2 导入后必须做的十项测试
- 5. 动力学、控制接口与仿真时间
  - 5.1 共同的动力学语言
  - 5.2 五种常见控制方式
  - 5.3 三个时间尺度必须分开
  - 5.4 跨平台控制一致性
- 6. 接触、摩擦、视觉与传感器：最容易影响结果的环节
  - 6.1 为什么灵巧操作难以仿真？
  - 6.2 标准接触验收测试
  - 6.3 视觉闭环的四类误差

    - 6.4 何时需要视觉，何时先用状态?
- 7. 强化学习训练前：把任务说清楚
    - 7.1 强化学习任务的六个要素
    - 7.2 推荐任务层次
- 8. PPO、SAC 与其他训练路线如何选?
    - 8.1 PPO 与 SAC 的工程区别
    - 8.2 推荐的选型规则
    - 8.3 训练样本效率与总效率不是一回事
- 9. 设计可训练的 RL 环境：逐项操作指南
    - 9.1 观测空间（Observation）
    - 9.2 动作空间（Action）
    - 9.3 奖励函数（Reward）
    - 9.4 终止、时间上限与恢复
    - 9.5 Reset 与初始状态随机化（RSI）
    - 9.6 课程学习（Curriculum）
- 10. Isaac Lab 强化学习训练：完整工作流
    - 10.1 从已知任务到自定义任务
    - 10.2 Isaac Lab 框架选择
    - 10.3 Isaac Lab 常见陷阱
- 11. MuJoCo 强化学习训练：CPU、MJX 与 MuJoCo Warp
    - 11.1 经典 MuJoCo：验证逻辑优先
    - 11.2 MJX / MJWarp：高吞吐量训练
    - 11.3 MuJoCo RL 的十步流程
- 12. PPO 与 SAC：调参顺序和容易犯的错误
    - 12.1 PPO 调参应先检查什么?
    - 12.2 SAC 调参应先检查什么?
    - 12.3 三种常见“训练成功”的假象
- 13. 训练效率：环境数量、视觉成本和算力瓶颈
    - 13.1 至少测量五个数字
    - 13.2 并行环境并非越多越好
    - 13.3 视觉 RL 的两条路线
- 14. 域随机化与鲁棒训练：按因果关系逐步加入
- 15. 评测、消融与复现：论文级实验怎么做?
    - 15.1 核心指标
    - 15.2 最小消融矩阵
    - 15.3 如何避免不可信结论
- 16. Sim-to-Real：从仿真策略到真机
    - 16.1 部署前的六个校准层
    - 16.2 真机验证的阶梯
- 17. 综合案例：Tron2 + Revo3 的螺丝刀操作

    - 17.1 研究任务定义
    - 17.2 建议的技术路线
    - 17.3 关键建模决策
    - 17.4 推荐阶段性指标
- 18. 常见问题与排查顺序
- 19. 建议的六周学习计划与交付物
- 20. 最终检查清单
- 参考资料（官方资料优先）

章节知识地图



1. 三个平台究竟是什么？

1.1 角色分工

- **Isaac Sim：** 基于 NVIDIA 仿真技术的机器人应用平台，擅长场景编辑、USD 资产、PhysX 动力学、RTX 渲染、相机及 ROS 2 集成。它解决的是“场景和机器人怎样在仿真中运行与感知”的问题。
- **Isaac Lab：** 建立在仿真后端之上的机器人学习框架，提供任务接口、并行环境、观测、动作、奖励、终止、重置、域随机化以及与 RL 框架的连接。它解决的是“怎样系统地训练和评估策略”的问题。
- **MuJoCo：** 动力学与接触仿真引擎，适合研究刚体动力学、约束、执行器、控制与模型诊断。经典 MuJoCo 通常在 CPU 上运行；MJX、MuJoCo Warp（MJWarp）则面向批量和 GPU 仿真。

重要更新： Isaac Lab 3.0 Early Access 已引入多物理后端，包含基于 Newton／MuJoCo Warp 的运行路线。因此，不应再把“Isaac Lab = 只能用 PhysX”或“MuJoCo = 只能在 CPU 上使用”当成绝对结论。新架构与旧版 API 的兼容性仍需逐项确认。[1–4]

维度	Isaac Sim	Isaac Lab	MuJoCo 及其 GPU 生态
核心定位	场景、传感器、数字孪生	机器人学习与批量环境	动力学、接触、控制及批量仿真
常见资产	USD、可导入 URDF	使用后端可接受的资产	MJCF，可导入 URDF
可视化	RTX 渲染能力强	沿用所选渲染后端	轻量查看器；GPU 批量渲染能力依实现而异
RL 集成	需要任务／训练层	RSL-RL、RL-Games、skrl 等	MuJoCo Playground、Gymnasium 或自定义训练栈
最适合入门	场景和传感器	并行 RL	动力学和控制
易踩的坑	驱动、显存、光线追踪要求	版本/API、奖励与重置	执行器、接触参数、CPU/GPU 差异

1.2 研究任务怎么选?

研究目标	优先选择	选择理由
单机器人动力学、PD/MPC、逆动力学	MuJoCo	状态和力学机制易于检查，单步调试方便
多关节灵巧手 PPO	Isaac Lab 或 MuJoCo Playground	大规模并行与成熟任务结构
使用 replay buffer 的 SAC	支持 SAC 的 MuJoCo RL 栈或对应 Isaac Lab 集成	必须确认训练库确实实现 SAC 与向量环境适配
头部＋腕部 RGB-D、遮挡、光照变化	Isaac Sim / Isaac Lab	真实感渲染和相机系统较完整
高速状态策略、接触任务的批量仿真	Isaac Lab 的适合后端，或 MJX/MJWarp	取决于任务、GPU、接触复杂度和数据搬运
真机上线前复核	两个引擎交叉验证	减少对单一接触模型和执行器假设的依赖

核心原则： 首先根据任务选择物理模型与观测类型，其次才根据峰值仿真帧率决定工具。视觉、接触和真机一致性可能比原始 FPS 更重要。

2. 版本、安装方案与硬件注意事项（不含命令）

2.1 两条版本路线

稳定复现路线：Isaac Lab 2.3.2 + Isaac Sim 5.1。 适合复现基于 2.x 开发的现有论文、RSL-RL 和 PhysX 操作任务；旧项目最好锁定 Git 标签、Python、PyTorch、扩展及驱动版本。

新框架路线：Isaac Lab 3.0 Early Access + Isaac Sim 6.1（需要完整 Isaac Sim 功能时）。 3.0 引入无 Kit 的部分后端工作流和多后端抽象，但存在破坏性变化；截至本文日期，属于 Early Access，不能认为所有旧任务与插件直接兼容。尤其注意四元数约定由 2.x 常见的 **wxyz** 转向 3.0 的 **xyzw**，并且部分数据接口已变化。[1][2]

安装决策步骤： ① 确定复现目标与所需功能；② 根据官方兼容表选择版本；③ 将 Isaac 与 MuJoCo 放在独立环境；④ 先运行官方最小场景；⑤ 验证物理步进、资产导入和 GPU 使用；⑥ 再安装训练框架。不要先混装一堆深度学习依赖再查错。

2.2 计算资源如何分配

- **RTX 4090（24 GB）：** 适合 Isaac Lab 物理训练和受限分辨率的视觉任务；复杂场景仍需控制显存、环境数和相机像素。
- **RTX 3090（24 GB）：** 可用于兼容的部分实验和 MuJoCo GPU 训练，但 Isaac Sim 5.1 官方最低性能档列为 RTX 4080；在 3090 上的工作流应视为需要自行验证的配置。
- **A100／A800：** 适合神经网络训练与部分不需要 RTX 渲染的计算；**A100 不属于完整 Isaac Sim 5.1 官方支持的 RTX GPU**，不要把高算力误判为 Isaac Sim 渲染兼容。
- **CPU：** 经典 MuJoCo 的小批量调试、MPC 或动力学检查往往很合适；当任务大量并行时再评估 MJX/MJWarp。

官方的 Isaac Sim 5.1 系统要求给出了 Ubuntu、内存、存储、RTX GPU 和驱动的具体兼容条件；依赖和驱动不应根据“最新”二字盲目升级。[5]

2.3 环境验收，而不只是“能够启动”

验收项目	通过标准
基础场景	地面、重力、机器人正常出现，不发生非预期穿透
关节运动	逐关节测试方向、限位、零位与速度
机器人状态	位姿、速度、力矩与真实模型定义一致
渲染／相机	图像方向、深度类型、时间戳和分辨率正确
物理稳定性	静止机器人无持续震荡，物体自由落体合理
训练接入	重置、观测、动作、奖励与终止流程能够闭环
复现记录	记录软件版本、模型文件哈希、GPU/驱动与随机种子

3. 不写代码，如何真正学会两个平台？

3.1 MuJoCo 的入门顺序

1. **理解模型层：** 将机器人视为由 body、joint、geom、actuator、sensor 及约束组成。MJCF 的“外观”与“物理碰撞”可以分开定义。
2. **查看模型：** 在查看器中逐一检查关节转轴、零位、碰撞几何、惯性参数和末端参考点。
3. **进行自由运动实验：** 让单关节摆杆或机械臂受重力运动，比较物理直觉与仿真行为。
4. **进行伺服实验：** 分别观察位置、速度和转矩驱动的差异，明确输入是目标值、力矩还是执行器内部信号。
5. **加入接触：** 放入桌面和小物体，比较自由落体、平面滑动、夹持和释放行为。
6. **进行 RL 准备：** 明确环境状态、动作、重置与随机化接口后再连接训练器。

适合的首个完整机器人： Franka Panda。模型公开、自由度适中，兼具末端控制和抓取任务，可使用 MuJoCo Menagerie 的维护版本作为起点。[6]

3.2 Isaac Sim / Isaac Lab 的入门顺序

1. **Isaac Sim 场景编辑：** 熟悉 Stage、Prim、USD、视口、时间轴、物理材质及刚体属性。
2. **导入机器人：** 查看 URDF 导入后的 articulation 根节点、驱动、质量、关节限位和碰撞体。
3. **构建基本场景：** 地面、桌面、固定机械臂、抓取物体及灯光。
4. **验证传感器：** 放置头部与腕部相机，检查内参、外参、深度含义、遮挡和同步。
5. **学习 Isaac Lab 任务：** 了解场景、机器人资产、观测、动作、奖励、终止和 Event／Domain Randomization。
6. **运行官方基准任务：** 从 Cartpole、Franka Reach、Franka Lift，再进阶至灵巧手重定向。
7. **进入 RL：** 先跑无视觉、低随机化的状态策略，再增加视觉与接触复杂度。

Manager-based 与 Direct 的取舍： Manager-based 适合快速复用标准观测、奖励、重置和随机化模块；Direct 适合高度定制的接触过程或算法研究。两种模式都能实现强化学习环境，区别在工程组织，而不是理论上的 RL 能力。[7]

3.3 推荐统一练习：Panda + 方块 + 桌面

不要一开始就把不同平台中的两个机器人拿来比较。建议固定机器人、物体、桌高、末端点、初始位姿与动作接口，完成四次递进实验：**单关节响应 → 末端轨迹 → 方块抓取 → 加扰动的重新抓取**。这一套场景随后可以替换为 Tron2 + Revo3，而评测逻辑保持一致。

4. 机器人建模：URDF、MJCF、USD 的对应关系

4.1 不存在无损“一键转换”

内容	URDF	MJCF	USD / Isaac
Link、Joint、惯性	标准描述能力	可表达	可表达
视觉和碰撞模型	支持	支持且可细化接触	支持且适合渲染
电机／执行器	标准本身描述有限	多种 actuator 和 tendon	关节驱动与物理属性
复杂接触设置	通常依赖扩展	接触求解器参数丰富	PhysX 接触及材质
机器人场景组织	比较基础	适合物理模型	支持大规模场景组合

建议维护一份**物理参数主表**：关节名称、轴方向、关节零位、限位、link 质量、质心、惯量、TCP、碰撞体、驱动模式及传感器外参。它比仅维护一份 URDF 更能保证跨平台一致。

4.2 导入后必须做的十项测试

1. 机器人基座坐标系与世界系是否一致；2. 关节名称与顺序是否匹配；3. 关节正方向是否正确；4. 角度与长度单位是否一致；5. 末端 TCP 定义是否一致；6. 质量与质心是否正确；7. 惯量是否物理可行；8. 碰撞体是否合理简化；9. 自碰撞过滤是否符合真实机构；10. actuator 的限幅、刚度、阻尼及控制频率是否一致。

灵巧手额外要求： 检查关节耦合、差动机构、腱传动、欠驱动、指尖材料与手指关节摩擦。不能仅凭运动学外形一致就认为手部接触可信。

5. 动力学、控制接口与仿真时间

5.1 共同的动力学语言

对于刚体机器人，可用概念关系表达为：

惯性效应 + 科氏／离心与重力等偏置项 = 执行器广义力 + 接触／约束广义力。

在 MuJoCo 和 PhysX 中，数值求解实现不完全相同；比较两者时，应先确认模型、执行器和接触条件一致，再比较轨迹或物体位姿。

5.2 五种常见控制方式

控制方式	策略输出	要警惕的问题
关节位置目标	各关节的期望角度	背后 PD 增益、力矩限幅、速度限幅
关节位置增量	相对当前／历史目标的增量	累积漂移、增量缩放、动作连续性
关节速度	期望速度	停止与接触时的稳定性、积分误差
关节转矩	执行器力矩	重力补偿、摩擦、饱和与真机安全
末端位姿或阻抗	TCP 位姿／力学参数	IK 可解性、奇异位形、末端坐标系

关键提醒： “策略输出关节位置”与“仿真中把关节直接设置到该位置”完全不同。前者应经由真实可比的控制器和执行器动力学实现，后者可能绕过物理约束，形成严重的 Sim-to-Real 偏差。

5.3 三个时间尺度必须分开

- **物理步长：** 用于积分碰撞、动力学和接触；越小通常越稳定，但成本更高。
- **控制频率：** 位置伺服、阻抗环或 IK 更新速率。
- **策略／相机频率：** RL 推理和视觉观测速率，可能互不整除。

示例：物理 500 Hz、策略 50 Hz，则每个策略动作维持 **10 个物理步**；若相机为 30 Hz，就必须明确图像缓存、时间戳和延迟，而不能假设策略每步都会得到新帧。真实机器人部署时还要考虑通信、估计、执行器响应和动作滤波的总延迟。

5.4 跨平台控制一致性

比较前固定四件事：**相同任务空间或关节空间动作定义、相同动作缩放、相同低层控制器语义、相同有效采样周期**。 不满足这些条件的性能差异不能简单归因于物理引擎。

6. 接触、摩擦、视觉与传感器：最容易影响结果的环节

6.1 为什么灵巧操作难以仿真?

接触是高度非线性过程，碰撞几何、摩擦、接触柔顺性、积分步长、控制刚度、惯量以及求解器共同决定“是否抓得住”。不同引擎即使使用同一个摩擦系数，也未必给出同样的滑移行为。

建议分离诊断： 先做单物体落地测试，再做已知法向力下的滑动测试，然后做静止夹持，最后才做边移动边抓取和旋转。不要以“增加摩擦系数”作为所有接触问题的默认答案。

6.2 标准接触验收测试

测试	观察量	常见异常
自由落体	落地时刻、回弹、静止位置	穿透、异常弹跳
平面滑动	减速、停止距离	非真实滑移、震荡
指尖夹持	滑移、接触力、持物时长	过度抓稳或突然弹飞

测试	观察量	常见异常
在手旋转	物体相对手的姿态轨迹	接触点不稳定、抓取失效
受扰动恢复	掉落后搜索、重新抓取	任务逻辑把掉落当成直接终止

6.3 视觉闭环的四类误差

1. **几何误差：** 内参、畸变、外参、TCP、物体 CAD／mesh 的尺度。
2. **坐标误差：** 世界系／基座系／光学系、左乘与右乘、四元数顺序。
3. **观测误差：** 深度缺失、光照、遮挡、运动模糊、位姿估计漂移。
4. **时序误差：** 图像与关节数据不同步、网络延迟、控制保持与旧帧复用。

Isaac Lab 中沿光轴的深度与到相机中心的欧氏距离是不同概念。训练与部署时必须对齐深度定义，不能仅以变量名“depth”推断含义。[8]

6.4 何时需要视觉，何时先用状态?

若研究目标是验证控制算法、奖励和动力学鲁棒性，优先使用仿真状态。若研究目标是头／腕相机闭环，最终必须使用部署时可获得的视觉观测。推荐 **privileged-state teacher → visual student**：先证明任务在理想状态下可学，再引入感知误差和部分可观测性。切忌在最终 actor 观测中偷偷保留真机无法获得的物体真值位姿。

7. 强化学习训练前：把任务说清楚

7.1 强化学习任务的六个要素

- **任务目标：** 哪个物体、从哪里开始、做到什么状态，成功保持多久。
- **观测：** 机器人知道什么；哪些信息仅训练时的 critic 可见。
- **动作：** 控制机器人什么量，动作周期、限幅和滤波如何定义。
- **奖励：** 哪些行为值得鼓励，以及是否存在投机取巧。
- **终止／截断：** 何时真正失败，何时只是到达时间上限。
- **初始分布与评测分布：** 训练多样性和泛化边界是什么。

不要把“奖励增加了”误认为“任务学会了”。RL 策略可能通过不合理接触、碰撞穿透、停在容易拿到奖励的位置来获得高回报。

7.2 推荐任务层次

Level A： 到达／跟踪（Reach）；**Level B：** 接近并接触（Pre-grasp）；**Level C：** 稳定抓取与抬升（Grasp & Lift）；**Level D：** 物体位姿调整（Reorient）；**Level E：** 多阶段操作（Pick–Move–Rotate）；**Level F：** 物体被打落后的恢复（Recovery）。

每完成一级都应独立评测，再进入下一阶段。这样失败时能知道是控制、接触、阶段切换还是策略本身出了问题。

8. PPO、SAC 与其他训练路线如何选?

8.1 PPO 与 SAC 的工程区别

维度	PPO	SAC
数据使用	典型 on-policy：依赖当前策略收集新 rollout	off-policy：可重用 replay buffer
常见优势	大量并行环境下实现成熟、训练稳定性较易管理	数据复用率高，适合交互数据昂贵的场景
主要成本	同样的数据通常不能长期重复使用	critic、温度参数和分布偏差的调试更复杂
典型适用场景	Isaac Lab 状态控制、大并行灵巧操作	数据有限、需要 replay 的连续控制
常见失败源	奖励尺度、优势归一化、更新幅度	Q 值过估计、replay 分布偏移、熵／更新比例

不要认为“MuJoCo 必须用 SAC，Isaac Lab 必须用 PPO”。 两个平台都可以连接不同算法；算法可用性取决于具体 RL 库、任务和后端。Isaac Lab 的集成中，RSL-RL 常用于 GPU PPO；RL-Games 等库提供其他算法，但并非任意任务切换训练器都无需适配。[9]

8.2 推荐的选型规则

- **先做大并行状态任务：** 优先 PPO，利用现成环境建立可复现基线。
- **训练交互样本昂贵，或需要大量 replay：** 考虑 SAC，但必须验证并行环境与 replay 的吞吐和正确性。
- **视觉编码器很大：** 先冻结视觉骨干或蒸馏为轻量策略，再判断 PPO/SAC 是否是主要瓶颈。
- **只有少量专家演示：** 可以先做行为克隆（BC），再用 RL 微调；不能认为 RL 必须从零开始。
- **具身基础模型后训练：** 还需要区分策略网络、动作生成器、奖励／价值模型与实时控制环，不能直接复用低维控制任务的超参数。

8.3 训练样本效率与总效率不是一回事

离策略方法通常能重复利用数据，但机器人项目的总成本还包括：仿真步进、图像渲染、视觉编码、网络更新、数据传输、场景重置和真实机器人交互。必须同时统计 **环境步数、总 GPU 小时、墙钟时间及成功率**。

9. 设计可训练的 RL 环境：逐项操作指南

9.1 观测空间（Observation）

低维基础观测： 关节位置、关节速度、前一动作、末端位姿、物体相对目标的位姿差、必要的接触／夹持信息。

仅训练时的特权信息： 真实物体位姿、精确速度、接触力、摩擦参数、外力扰动等；这些可供 teacher 或 asymmetric critic 使用，但部署 actor 不应依赖缺失信息。

视觉观测： RGB、深度、分割或位姿估计结果。需要明确图像频率、分辨率、历史帧数量、遮挡策略和时间戳。对于目标部分可见的任务，历史信息或状态估计通常比单帧假设更可靠。

注意三件事： 所有数值量做合理归一化；避免不同尺度的观测互相淹没；防止把未来目标、下一个接触标签等训练信息泄漏到 actor。

9.2 动作空间（Action）

推荐从 **关节位置增量 + 真实可比的位置伺服器** 或 **TCP 位姿增量 + IK + 灵巧手关节目标** 开始。动作每个分量都应有物理单位、范围、滤波方式与安全约束。

动作大小过大导致碰撞和不可达姿态；过小导致探索不足。训练时的目标速度、加速度、力矩限幅，应该与真机驱动的可达能力一致。

9.3 奖励函数（Reward）

将奖励分为五类：**任务成功奖、阶段进展奖、接触与抓取奖、平滑与能耗惩罚、安全与失败惩罚**。

奖励项	鼓励的行为	可能的 reward hacking
接近目标	缩短手与物体距离	只靠近、不抓取
抓取稳定	维持合理接触	用非真实高摩擦夹住物体
物体位姿误差	实现目标平移／旋转	只优化简单的一个方向
成功保持	达到并稳定维持目标	反复触发阈值赚取奖励
动作平滑	降低振荡与冲击	因惩罚过重而拒绝移动

建议记录每个奖励分项，而不是只记录总回报。稀疏成功信号应与适度的密集 shaping 配合，并在收敛后逐步检查策略是否仍依赖不合理的 shaping。

9.4 终止、时间上限与恢复

- **成功终止：** 达成任务并满足保持时间，结束该回合。
- **失败终止：** 真正不可恢复或违反安全边界，例如机器人严重碰撞／关节越界。
- **时间截断：** 达到规定时长但物理任务尚未自然终止；训练器应正确处理 bootstrap 与时间上限语义。
- **恢复任务特殊要求：** 若目标是“掉落后重新抓取”，**掉落不能立即触发 episode 终止**；应允许策略继续观察、定位、接近并重新抓取，除非物体已离开可恢复工作区。

9.5 Reset 与初始状态随机化（RSI）

建议分层采样：正常起始、即将接触、稳定抓取、旋转中、滑移中、掉落后。接触与释放附近可以采用 **event-biased RSI** 增采样；同时保留 uniform RSI 防止分布过窄。

使用参考轨迹时，随机化物体初始位置／朝向后，要**一致地变换后续参考轨迹和目标**，否则观测与奖励会自相矛盾。每次重置还要检查物体穿透、关节突然跳变以及接触求解器是否稳定。

9.6 课程学习（Curriculum）

推荐的课程进展：固定初始位 → 轻微位置变化 → 旋转变化 → 更大工作空间 → 参数扰动 → 位姿估计误差 → 掉落与重抓。以成功率和失败模式作为晋级条件，而不是简单按训练步数强行升级。

10. Isaac Lab 强化学习训练：完整工作流

10.1 从已知任务到自定义任务

第 1 步：选任务。 从 Cartpole 了解训练闭环，从 Franka Reach 验证动作与观测，从 Lift 进入接触，最后再做灵巧手多阶段任务。

第 2 步：确认物理后端与版本。 2.x 既有 PhysX 任务与 3.0 多后端任务不能默认混用；先用官方支持的参考配置完成基准。

第 3 步：定义场景。 机器人、桌面、物体、关节驱动、物理材质、初始位置、相机均应记录参数。

第 4 步：定义 RL 五大接口。 观测、动作、奖励、终止、重置；确认每次 reset 后环境状态和缓存同步。

第 5 步：用单环境“过拟合”验证。 固定物体、固定目标、无随机化，确保策略确实能够成功。失败时先查环境，不急于扩大网络。

第 6 步：增加并行环境。 从显存可承受的少量环境开始，逐步观察每秒环境步数、GPU 利用率、显存和接触稳定性；并行数越大并不保证整体训练越快。

第 7 步：训练 PPO 基线。 固定种子、学习率、rollout 长度、网络结构、奖励尺度和控制周期。每次只改一类因素。

第 8 步：进行独立评估。 关闭训练探索噪声或按固定评估协议采样，保存成功率、轨迹、碰撞和失败录像。

第 9 步：加入真实感变化。 动力学随机化、执行器延迟、观测噪声、视觉遮挡，逐项验证边际效果。

第 10 步：保存完整可复现包。 模型资产、任务配置、奖励权重、归一化统计、训练器版本、checkpoint、评估脚本版本和关键日志均需要保留。

10.2 Isaac Lab 框架选择

训练框架	推荐用途	注意事项
RSL-RL	GPU 大并行 PPO、状态控制、蒸馏	检查 actor／critic 观测配置与网络兼容
RL-Games	已有 RL-Games 实验，或需要其支持的算法	不同版本的算法与任务配置存在差别
skrl	需要更广的算法支持或 PyTorch/JAX 选项	额外适配与可复现性验证
Stable-Baselines3	小规模基线、熟悉的分析工作流	大规模 GPU 向量环境未必最合适

10.3 Isaac Lab 常见陷阱

显存： 相机数量 × 分辨率 × 环境数会显著提高显存占用，加入视觉之后应重新进行吞吐基准。**Headless：** 不打开交互式窗口不等于自动关闭全部离屏渲染；也不等于视觉训练无需配置相机输出。**Reset：** 并行环境各自独立结束，必须正确处理环境索引及历史观测清空。**奖励：** 物理频率或策略 decimation 改变后，要检查按时间积累的奖励是否被无意缩放。

11. MuJoCo 强化学习训练：CPU、MJX 与 MuJoCo Warp

11.1 经典 MuJoCo：验证逻辑优先

经典 MuJoCo 适合单机控制实验、动力学分析、小规模 Gymnasium 环境和 CPU 并行 rollout。优点是调试直接、单步延迟较低；但大规模训练若每步都需 CPU→GPU 搬运观测，可能出现通信瓶颈。[3]

建议路线：先在经典 MuJoCo 中完成模型、奖励、重置、接触和控制正确性验证，再评估是否需要 GPU 批量实现。

11.2 MJX / MJWarp：高吞吐量训练

MJX-JAX 利用 JAX/XLA 做批量计算；**MJWarp** 针对 NVIDIA GPU 并行场景优化，尤其适合许多独立机器人环境的高吞吐量仿真。不要把“总吞吐量更高”误解成“单个仿真步的延迟更低”：MJWarp 侧重吞吐，经典 MuJoCo 常更适合单环境低延迟控制。当前 MJX-Warp 路线不以自动微分作为能力目标。[3][4]

MuJoCo Playground 已提供运动控制、灵巧操作与部分视觉环境，以及基于 MJX/MJWarp 的 RL 基线；适合对照 Isaac Lab 的状态策略与灵巧操作训练。[10]

11.3 MuJoCo RL 的十步流程

1. 从 Menagerie 或可靠 MJCF 模型出发；2. 确认关节／执行器与真机一致；3. 构建最小操作场景；4. 验证接触与自由运动；5. 定义观测动作与归一化；6. 设计奖励、终止、重置；7. 用小样本和固定初态测试；8. 选择 CPU、MJX-JAX 或 MJWarp；9. 使用 PPO 或受支持的其他训练器；10. 在不同初态、物理参数和仿真器中独立评估。

并行后端注意事项： GPU 版本的求解能力与模型特性并非完全等价；迁移时重点检查 tendon、复杂约束、接触数量限制、传感器，以及批量重置行为。并非所有 CPU MuJoCo 模型都能无修改地获得同样物理结果。

12. PPO 与 SAC：调参顺序和容易犯的错误

12.1 PPO 调参应先检查什么?

推荐顺序是：**任务正确性 → 奖励尺度 → 动作尺度 → rollout 数据量 → 更新强度 → 探索／熵 → 网络容量**。

参数或指标	起步思路（仅经验范围，不是统一最优值）	异常征兆
学习率	常从约 1e−4 至 3e−4 开始试验	剧烈震荡、损失发散
Clip 范围	常见起点约 0.1–0.2	更新过小或策略突变
GAE λ	可从约 0.95 起步	方差高或学习缓慢
Discount γ	常见 0.99–0.995，需匹配实际控制时间尺度	过短视或回报难估计
每环境 rollout 长度	常见 16–64 策略步，按任务调整	轨迹太短无法覆盖关键阶段
Policy update epochs	常见约 3–5，需结合 KL 和数据量	过拟合一批 rollout

尤其重要：并行环境数 × 每环境 rollout 长度才是每轮的样本规模。增加并行数后，如果没有相应调整 minibatch 与优化次数，算法行为可能不再相同。对多阶段长时任务，应关注有效视野而不是机械套用短时运动控制任务配置。

12.2 SAC 调参应先检查什么?

SAC 重点核查：**replay buffer 的样本覆盖、采样比例、critic 稳定性、target network 更新、熵温度和 actor/critic 更新节奏**。

从已有数据开始训练时，必须区分演示、旧策略数据和当前策略数据，避免回放分布与当前任务不一致。视觉任务还要监控图像增强、编码器梯度和重放存储成本。SAC 不是“打开 replay buffer 就自动更省样本”。

12.3 三种常见“训练成功”的假象

- **Reward hacking：** 总回报上涨，但任务成功率没有同步提高。
- **Reset overfitting：** 只会在固定初态或少数 RSI 状态成功。
- **Privileged information leakage：** 训练 actor 使用真实物体位姿，而实际部署只能获得有噪声的图像估计。

每种假象都需要单独设计评测，而不只是继续训练。

13. 训练效率：环境数量、视觉成本和算力瓶颈

13.1 至少测量五个数字

环境步／秒、完整物理步／秒、策略更新耗时、峰值显存、达到既定成功率所需的墙钟时间。 视觉任务还应统计渲染时间、编码器前向时间与数据搬运时间。

不同文献的 FPS／SPS 可能统计口径不同：单环境物理帧、所有环境总步数或策略步数不相等。报告训练速度必须同时给出环境数、物理／控制频率、GPU 型号和相机设置。

13.2 并行环境并非越多越好

环境数较小时 GPU 未饱和；环境数过大时显存、接触求解、相机渲染、数据搬运和策略更新可能成为瓶颈。应从小规模开始逐倍增加，用真实的成功率／小时和 GPU 使用数据确定工作点，而非凭经验固定为几千环境。

13.3 视觉 RL 的两条路线

- **端到端视觉 RL：** 直接从图像训练，简单直观但计算成本高，对遮挡、视觉域差异更敏感。
- **状态 teacher → 视觉 student：** 先学习可达的控制策略，再把视觉表征或估计状态用于学生训练；更适合用来排除“物理任务不会做”和“视觉看不懂”的混杂因素。

如果视觉 teacher/student 的控制频率不同，蒸馏损失必须对齐动作时间戳与输出语义，不能只对齐向量维度。

14. 域随机化与鲁棒训练：按因果关系逐步加入

随机化类别	建议变量	验证目的
几何	物体初始位姿、台面高度、小尺度偏差	空间泛化
动力学	质量、惯量、摩擦、关节阻尼	物理模型偏差
执行器	延迟、死区、刚度、限幅、低通滤波	实际伺服差异
观测	关节噪声、位姿估计偏差、丢帧	感知与同步鲁棒性
视觉	光照、材质、背景、遮挡、相机外参	图像域变化
扰动	外力、滑移、掉落、目标移动	恢复和纠错能力

推荐次序： 无随机化学会任务 → 扩展初始状态 → 动力学与执行器 → 状态估计噪声 → 视觉域随机化 → 外力和掉落恢复。随机化幅度以真实机器人的可观测误差或合理系统辨识区间为基础，避免生成大量真机永远不会遇到

的极端状态。

训练集与评测集应当分离。 评测时需要固定未参与训练的物体姿态、质量、摩擦、照明和相机偏差组合，才能判断真正泛化，而不是只在训练随机化内插成功。

15. 评测、消融与复现：论文级实验怎么做？

15.1 核心指标

指标	说明
Success rate	符合客观判定条件的回合比例
Time-to-success	达到成功的步数或真实时间
Drop / slip rate	掉落／滑移比例
Recovery success	出现可恢复扰动后的重新成功比例
Pose / tracking error	物体或末端位姿误差
Unsafe event rate	越界、过载、碰撞等事件比例
Environment steps / wall time	样本效率与训练总效率
Real-time latency	部署时策略、感知与控制链时延

建议至少使用多个随机种子，并报告均值及不确定性区间；不能仅选择最好的一个训练 checkpoint。跨方法比较时固定模型、环境、动作接口、训练预算和评估分布。

15.2 最小消融矩阵

1. 无域随机化／有域随机化。
2. 均匀 RSI／接触事件偏置 RSI／混合 RSI。
3. 仅本体感知／带物体真值／有噪声位姿估计／图像观测。
4. 关节动作／TCP 增量动作。
5. 只训练正常抓取／增加掉落恢复。
6. 只在训练引擎评测／跨 MuJoCo 与 Isaac 引擎评测。

如果希望比较 PPO 与 SAC，还必须对齐：交互样本预算、并行度、网络参数量、奖励与终止、评估方式以及墙钟时间；只对齐 update 次数通常不公平。

15.3 如何避免不可信结论

- **跨引擎比较不等于严格动力学等价**。 不同求解器的摩擦和接触实现不同，必须报告这种差异。
- **相同成功率不等于相同鲁棒性**。 应分阶段统计接触、旋转、释放、恢复的失败模式。
- **训练帧数不等于交互样本数**。 物理步、策略步和并行总步数必须分别定义。
- **不要只保存最终权重**。 还要保存观测归一化统计、动作缩放和依赖版本。

16. Sim-to-Real：从仿真策略到真机

16.1 部署前的六个校准层

1. **几何：** 关节零位、TCP、机器人底座与桌面关系。
2. **动力学：** 手部负载质量、质心、惯量、摩擦和重力补偿。
3. **执行器：** 控制模式、位置／速度／力矩限制、响应延迟。
4. **视觉：** 头部与腕部相机的内外参、图像时间戳与深度单位。
5. **软件：** ROS 2 或控制 SDK 的频率、缓存、丢包和异常恢复。
6. **安全：** 急停、工作空间、碰撞阈值、手指限流与物体脱落处理。

16.2 真机验证的阶梯

空载关节运动 → 无接触轨迹 → 慢速接近 → 低力抓取 → 物体抬升 → 物体旋转 → 人为小扰动 → 可控掉落与重抓。 每一级先设置限幅、保护和人工监护，再进行下一项。

真机性能低于仿真时的诊断顺序： 先查坐标系与观测同步，再查动作语义与伺服频率，然后查接触和动力学参数，最后才考虑扩大 RL 网络或调算法超参数。

17. 综合案例：Tron2 + Revo3 的螺丝刀操作

17.1 研究任务定义

机器人从桌面上**不同位置和朝向**拿起螺丝刀，将其移动到目标区域，完成规定的旋转／重定向；出现滑移或掉落时，能根据观测定位物体、重新接近并完成任务。部署传感器假设为头部与腕部相机以及机器人自身关节状态。

17.2 建议的技术路线

阶段	平台	实验重点	通过标准
A. 模型验证	MuJoCo	Tron2/Revo3 运动学、质量、碰撞、执行器	单关节响应、FK、接触不异常
B. 无视觉控制	MuJoCo 或 Isaac Lab	给定物体真值位姿的抓取与旋转	固定初态稳定成功
C. 大规模 RL	Isaac Lab / MJWarp	RSI、课程学习、PPO 训练	随机初态成功率稳定
D. 鲁棒性	两个仿真引擎	位姿误差、延迟、摩擦、掉落扰动	OOD 和恢复能力可量化
E. 视觉蒸馏	Isaac 的视觉后端	头／腕图像→状态估计或视觉策略	无特权 actor 可成功
F. 真机	Tron2 + Revo3	标定、动作限幅、闭环重抓	按安全协议完成真实任务

17.3 关键建模决策

动作空间： 机械臂优先采用 TCP 位姿增量与 IK，灵巧手采用关节位置目标；待稳定后再研究力控制或残差控制。**观测：** 初期以物体位姿真值训练 teacher，后续使用头／腕视觉估计或视觉 student。**重置：** 将接触前、接触后、旋转中、滑移中和掉落后的状态均纳入 RSI。**终止：** 若希望学会掉落恢复，掉落本身不得直接结束所有训练回合。**域随机化：** 优先考虑物体位姿、估计偏差、摩擦、执行器延迟和关节跟踪误差。

17.4 推荐阶段性指标

除了最终成功率，还应分别统计：第一次抓取成功率、在手旋转成功率、掉落概率、掉落后重新抓取成功率、完成时间，以及 **头部相机可见／不可见、腕部相机遮挡** 等条件下的分组指标。

18. 常见问题与排查顺序

现象	优先检查	不建议立刻采取的做法
机器人启动后抽搐	关节零位、刚度、碰撞、物理步长	直接提高控制增益
方块穿模	碰撞几何、接触设置和步长	只提高摩擦
PPO 奖励上涨但任务失败	奖励分项、成功判定、观察录像	继续盲目增加训练步数
SAC Q 值不稳定	replay 分布、critic、目标网络、奖励尺度	只增大网络
GPU 利用率低	CPU-GPU 拷贝、渲染、环境数、同步点	立即堆到最多环境数
视觉训练极慢	相机数、分辨率、渲染成本、编码器	先上全分辨率图像
真机手指抓不牢	接触几何、材料、伺服控制、模型误差	只增加仿真摩擦系数
物体掉落后无法恢复	终止逻辑、RSI、视觉覆盖、目标定义	仅增加成功奖励
机械臂追踪误差大	IK、TCP、延迟、动作缩放、限速	直接改 PPO 超参数
Isaac 版本升级后行为变化	依赖锁定、四元数顺序、传感器／后端	直接沿用旧 checkpoint 作结论

排查总顺序： 模型正确性 → 坐标与单位 → 执行器控制 → 物理和接触 → 观测与延迟 → 奖励与 reset → 优化算法。

19. 建议的六周学习计划与交付物

周次	学习与实验内容	验收交付物
第 1 周	了解三个工具；完成 Panda 场景与单关节测试	模型参数表、关节映射及截图
第 2 周	MuJoCo 中实现轨迹、接触与抓取验证	自由落体／滑移／夹持测试报告
第 3 周	Isaac Lab 基准任务、环境定义和 PPO	固定初态成功训练及学习曲线
第 4 周	多初态 RSI、课程学习、域随机化	随机初态评测与第一组消融
第 5 周	MJWarp／跨引擎评估，视觉 teacher-student	OOD 与跨引擎差异报告
第 6 周	Tron2 + Revo3 映射、限幅与真机测试	真机安全检查与成功／恢复指标

时间只是建议，实际节奏取决于模型成熟度、任务复杂度与传感器质量。每一周最好保持相同机器人参数记录和固定评测集，避免多个变量同时变化。

20. 最终检查清单

模型与物理： 关节顺序、轴向、限位、质量、惯量、TCP、碰撞体、执行器、物理步长、摩擦均已核查。
RL 任务： 成功判据明确；观测无泄漏；动作限幅正确；奖励分项可解释；终止与 timeout 区分；RSI 覆盖关键事件。
训练： 已跑通固定初态；记录种子、网络、训练器与环境版本；报告训练样本、吞吐、墙钟时间与峰值显存。
评估： 多种子、独立初态、OOD 扰动、跨引擎、失败模式、drop recovery 均有记录。
真机： 坐标与相机校准、速率和延迟匹配、力／速／位限幅、监护与急停机制准备完毕。

> **一句话总结：** MuJoCo 负责把物理模型和控制接口“解释清楚”；Isaac Sim 负责把机器人、场景和视觉“搭建可信”；Isaac Lab 或 MuJoCo Playground 负责把 RL “训练系统化”；最后用跨引擎、OOD 与真机结果验证策略，而不是只看训练奖励。

参考资料（官方资料优先）

1. Isaac Lab Releases（含 3.0 Early Access 与迁移说明）：https://github.com/isaac-sim/IsaacLab/releases
2. Isaac Lab 3.0 Early Access 安装与架构：https://isaac-sim.github.io/IsaacLab/v3.0.0-EA/source/setup/installation/index.html
3. MuJoCo Warp 官方文档：https://mujoco.readthedocs.io/en/latest/mjwarp/
4. MuJoCo XLA / MJX 官方文档：https://mujoco.readthedocs.io/en/stable/mjx.html
5. Isaac Sim 5.1 系统要求：https://docs.isaacsim.omniverse.nvidia.com/5.1.0/installation/requirements.html
6. MuJoCo Menagerie 机器人模型：https://github.com/google-deepmind/mujoco_menagerie
7. Isaac Lab 2.3.2 Direct RL Environment：https://isaac-sim.github.io/IsaacLab/v2.3.2/source/tutorials/03_envs/create_direct_rl_env.html
8. Isaac Lab 2.3.2 传感器文档：https://isaac-sim.github.io/IsaacLab/v2.3.2/source/api/lab/isaaclab.sensors.html
9. Isaac Lab RL 框架对比：https://isaac-sim.github.io/IsaacLab/main/source/overview/reinforcement-learning/rl_frameworks.html
10. MuJoCo Playground：https://github.com/google-deepmind/mujoco_playground
11. MuJoCo 官方基础文档：https://mujoco.readthedocs.io/en/stable/
12. Isaac Sim 官方文档：https://docs.isaacsim.omniverse.nvidia.com/

资料使用提醒： 官方项目的版本、名称和 API 可能继续变化。本文刻意不提供命令行安装细节，以免把只适用于某个版本的命令误用到其他版本。

身智能中的 化学习教程：算法分类、任务分类与应用场景

课程首页 · English · PDF

项目	说明
适用范围	算法与任务分类；不规定统一框架运行环境
文档核验日期	2026-10-18
许可	CC BY 4.0 · Xiong Zeng and contributors

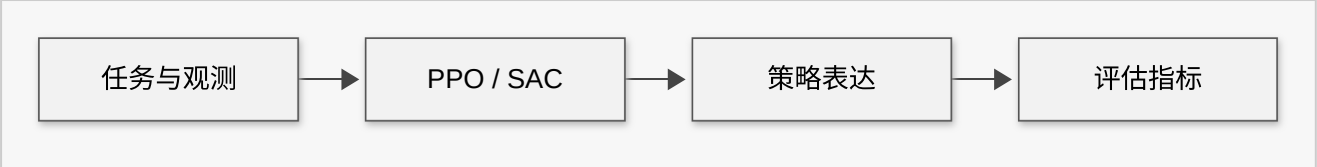
核验边界： 文档整理与上游引用快照核对；未进行教程环境、训练或硬件验证。 版本与固定源码引用见版本说明。原创教材采用 CC BY 4.0，第三方引用、图片及代码遵循其原有许可；注明来源不等于拥有再许可权。

目Ö

- 一、具身强化学习的总体分类
  - 1.1 分类框架
  - 1.2 核心分类表
- 二、按算法分类：PPO 与 SAC
  - 2.1 On-policy RL：PPO
  - 2.2 Off-policy RL：SAC
  - 2.3 PPO 与 SAC 的比较
- 三、按任务分类：机器人运动控制强化学习
  - 3.1 Locomotion RL：机器人运动控制
  - 3.2 Manipulation RL：机器人操作强化学习
- 四、按任务分类：具身基础模型强化学习后训练
  - 4.1 什么是具身基础模型 RL Post-training?
  - 4.2 RL 后训练主要解决什么问题?
  - 4.3 具身基础模型 RL 后训练的五类方法
  - 4.4 五类方法的总结
- 五、PPO、SAC 在两类任务中的具体用途
  - 5.1 PPO + Locomotion
  - 5.2 SAC + Manipulation
  - 5.3 PPO 类方法 + VLA
  - 5.4 SAC / Off-policy Critic 类方法 + VLA
- 六、如何根据任务选择 RL 方法?
- 七、两个研究方向的典型实验路线
  - 7.1 机器人运控 RL
  - 7.2 具身基础模型 RL 后训练

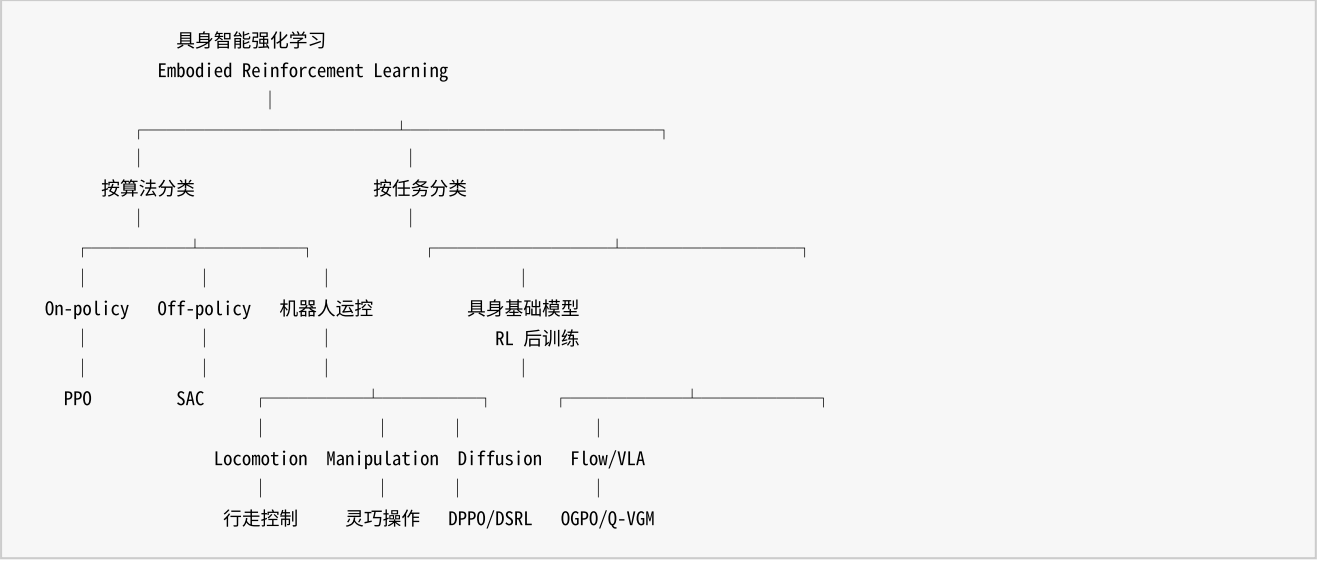
- 八、最后总结

章节知识地图



一、具身强化学习的总体分类

1.1 分类框架



其中，机器人操作（Manipulation）既可以采用传统的低层控制策略，也可以采用 Diffusion Policy 或 VLA。

因此，**同一个操作任务可以属于传统机器人控制 RL，也可以属于具身基础模型 RL 后训练，关键取决于策略模型和训练目标**。

1.2 核心分类表

分类维度	类别	代表算法或方法	主要用途
算法	On-policy	PPO	大规模仿真训练、运动控制
算法	Off-policy	SAC	连续控制、数据高效学习、在线改进
任务	Locomotion RL	PPO、SAC	双足、四足、人形机器人运动
任务	Manipulation RL	PPO、SAC	抓取、手内操作、灵巧控制
任务	Foundation Model RL	DPPO、DSRL、OGPO 等	改进预训练 Diffusion / Flow / VLA 策略

需要注意：DPPO、DSRL 和 OGPO 是针对生成式策略设计的方法，不能全部直接归类为标准 PPO 或 SAC。

二、按算法分类：PPO 与 SAC

2.1 On-policy RL：PPO

PPO（Proximal Policy Optimization） 是机器人运控中广泛使用的强化学习算法。

其主要特点是：

- 使用当前策略采集的交互数据进行训练；
- 每轮采样后对策略进行有限次数的更新；
- 训练通常比较稳定；
- 容易结合 GPU 大规模并行仿真；
- 不适合长期重复使用大量旧数据。

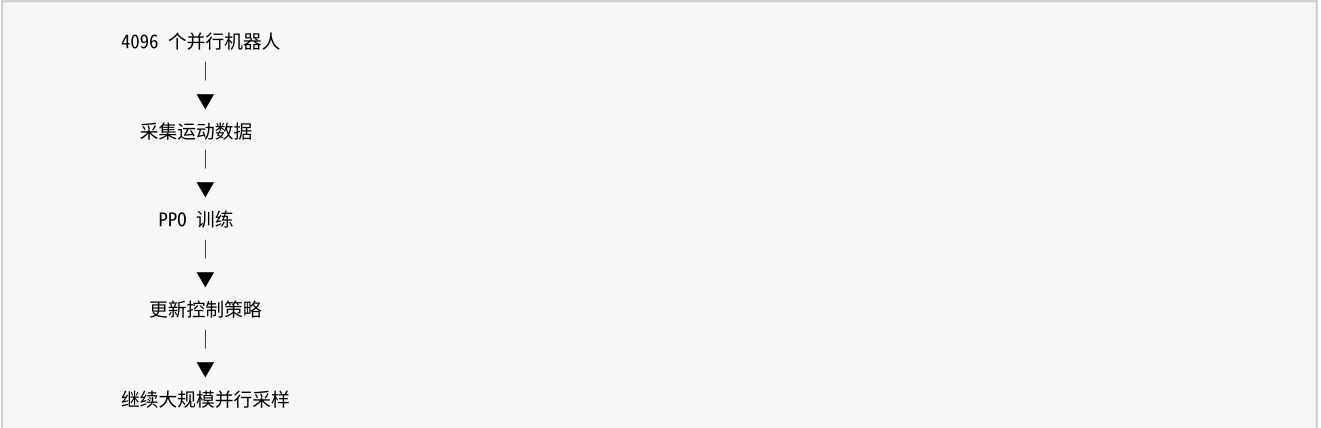
PPO 主要用于哪些任务？

应用	具体任务
四足机器人	行走、奔跑、跳跃、复杂地形穿越
人形机器人	双足行走、平衡、抗扰恢复
Whole-Body Control	全身协调控制、动作跟踪
灵巧操作	灵巧手旋转物体、抓取、接触操作
Sim-to-Real	仿真训练控制策略并部署到真机
生成式策略后训练	DPPO 等针对 Diffusion Policy 的策略梯度方法

为什么运控领域喜欢 PPO？

因为运控任务通常可以在 Isaac Lab 等仿真器中同时运行数千个机器人。

例如：



在这样的设置下，环境交互数据容易大量获得，因此 PPO 的 On-policy 数据使用方式通常是可以接受的。

典型技术组合：

```
Isaac Lab + RSL-RL + PPO
```

Isaac Lab 官方文档也将 RSL-RL 作为新机器人 RL 项目的一种推荐起点。(Isaac Lab 文档)

2.2 Off-policy RL：SAC

SAC（Soft Actor-Critic） 是机器人连续控制中具有代表性的 Off-policy 算法。

主要特点：

- 可以重复使用历史交互数据；
- 使用 Replay Buffer 存储经验；
- 通常具有较高的环境交互样本效率；
- 适合在线策略改进；
- 可以利用 Critic 指导动作选择；
- 对 Critic 的学习质量比较敏感。

SAC 主要用于哪些任务？

应用	具体任务
机械臂控制	抓取、放置、推物体
灵巧操作	手内旋转、物体重定位、精密操作
真实机器人 RL	利用有限真实交互数据改进策略
Residual RL	在已有控制器上学习动作修正
Latent RL	学习预训练生成策略的潜变量
具身模型后训练	基于 Critic 的 Off-policy 策略优化

为什么 SAC 适合真实机器人学习？

因为真实机器人交互比较昂贵。

例如，一次灵巧手抓取可能涉及：

- 机械臂移动；
- 手指接触；
- 抓取执行；
- 任务成功或失败判断；
- 机器人复位。

如果每次策略更新都需要重新采集大量数据，训练成本会非常高。

SAC 可以通过 Replay Buffer 反复利用已有经验，因此常用于研究数据高效的机器人在线学习。

但需要强调：**SAC 的样本效率优势并不意味着它在所有机器人任务上都比 PPO 更好**。 特别是在拥有大规模并行仿真资源时，PPO 可能获得更高的训练吞吐量。

2.3 PPO 与 SAC 的比较

维度	PPO	SAC
类型	On-policy	Off-policy
训练数据	主要来自当前策略	当前和历史策略数据
历史数据复用	有限	较充分
Replay Buffer	通常不需要长期 Replay Buffer	需要
样本效率	通常较低	通常较高
并行仿真	非常成熟	同样可以使用
训练稳定性	通常容易控制	依赖 Critic 质量
真实交互成本高的场景	通常不占优势	往往更有优势
典型任务	Locomotion	Manipulation、Online RL
代表框架	RSL-RL	Stable-Baselines3、其他 SAC 实现

如何选择？

选择 PPO：

机器人仿真速度快、可以运行大量并行环境、希望从零训练一个稳定的运动控制策略。

选择 SAC：

真实机器人数据昂贵、需要重复利用历史经验、希望持续在线提升策略性能。

不过，实际选择还取决于策略结构。如果策略是大型 Diffusion 或 Flow Matching 模型，就不能不加修改地直接使用标准 PPO/SAC。

三、按任务分类：机器人运动控制强化学习

机器人运动控制强化学习主要目标是：

让机器人通过与环境交互，学会稳定、高效地执行具体的物理动作。

通常直接训练一个相对轻量的控制策略，并将其部署到机器人控制系统中。

3.1 Locomotion RL：机器人运动控制

Locomotion RL 主要针对机器人移动与全身运动能力。

典型应用

方向	任务	主要用途
Quadruped Locomotion	四足行走、奔跑、跳跃	机器狗运动控制

方向	任务	主要用途
Humanoid Locomotion	双足行走、上下楼梯	人形机器人运动
Motion Tracking	跟踪人类或参考运动	人形机器人动作模仿
Whole-Body Control	全身协调运动	移动操作、复杂全身动作
Push Recovery	外部推力扰动恢复	提高机器人稳定性
Terrain Adaptation	斜坡、碎石、楼梯	复杂地形适应

典型训练方式

```
机器人模型
    |
    ▼
构建物理仿真环境
    |
    ▼
定义运动任务与奖励
    |
    ▼
使用 PPO 训练
    |
    ▼
Domain Randomization
    |
    ▼
Sim-to-Real
    |
    ▼
真实机器人部署
```

这一方向主要研究什么？

1. 运动能力

让机器人完成越来越复杂的运动任务，例如：

- 从平地行走扩展到楼梯；
- 从慢速行走扩展到奔跑；
- 从简单步态扩展到全身动作跟踪。

2. 鲁棒性

提高机器人应对不同环境的能力，例如：

- 外部推力；
- 摩擦变化；
- 未知负载；
- 执行器误差。

3. Sim-to-Real

减少仿真与真实机器人的差异。

主要技术包括：

- Domain Randomization；
- System Identification；
- Teacher-Student Learning；
- Privileged Learning；
- Online Adaptation。

典型技术栈

模块	常见选择
仿真	Isaac Lab、MuJoCo
RL	PPO
实现	RSL-RL
控制策略	MLP、RNN
低层控制	PD / Torque Control
训练方式	大规模并行仿真
部署	机器人实时控制系统

这一方向的特点：

通常可以在仿真中从零训练，主要瓶颈是物理仿真、奖励设计、任务复杂度以及真实部署的鲁棒性。

3.2 Manipulation RL：机器人操作强化学习

Manipulation RL 主要针对机械臂和灵巧手操作。

虽然它与 Locomotion RL 都属于机器人控制层面的强化学习，但操作任务经常涉及更复杂的接触动力学。

典型应用

方向	任务	主要用途
Reaching	机械臂到达目标位置	基础末端控制
Grasping	抓取不同物体	机器人抓取
In-hand Manipulation	手内旋转与重定位	灵巧手精细操作
Object Reorientation	调整物体姿态	灵巧操作
Tool Manipulation	使用螺丝刀等工具	工具操作
Contact-Rich Manipulation	插孔、旋转、装配	精密接触操作
Bimanual Manipulation	双臂协同	双手操作与装配

与 Locomotion RL 的区别

Locomotion 更多关注：

- 身体平衡；
- 速度跟踪；
- 地面接触；
- 周期性运动。

Manipulation 更多关注：

- 多点接触；
- 物体位姿；
- 抓取稳定性；
- 接触切换；
- 手指协调；
- 精密操作。

使用 PPO 还是 SAC？

PPO：

适合在仿真中并行训练大量灵巧操作任务。

例如，给定不同物体初始姿态，训练灵巧手完成物体旋转。

SAC：

适合研究真实机器人在线学习，以及需要较高环境交互样本效率的操作任务。

例如，在已有初始操作策略的基础上，通过少量真实交互逐步提高抓取成功率。

重要区别

这里训练的可能仍然是传统低层控制策略：

```
Observation
    ↓
MLP Actor
    ↓
Joint Position / Torque
    ↓
Robot
```

它与下一部分的 VLA 强化学习后训练不是同一个问题设定。

四、按任务分类：具身基础模型强化学习后训练

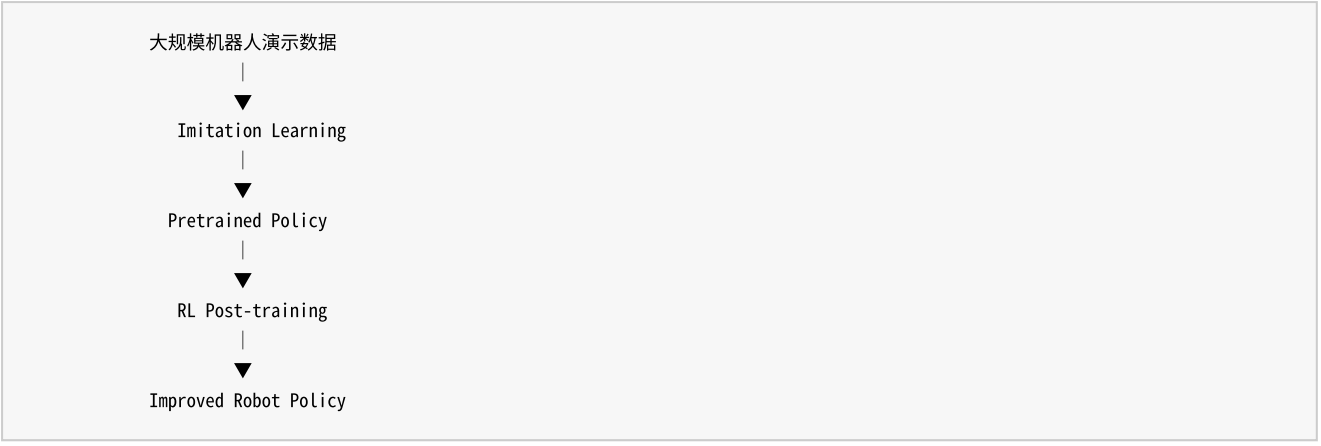
4.1 什么是具身基础模型 RL Post-training?

具身基础模型通常先利用大量演示数据进行预训练，再利用强化学习提高任务表现。

例如：

- Diffusion Policy；
- Flow Matching Policy；
- π0 / π0.5 等 VLA；
- 其他视觉语言动作生成模型。

训练过程通常是：



它与从零训练 Locomotion Policy 的主要区别是：

后训练阶段已经有一个具备一定任务能力的预训练策略。

强化学习的目标不再是从零学习基本动作，而是在已有能力基础上进一步提升性能。

4.2 RL 后训练主要解决什么问题?

问题	RL 后训练的作用
成功率不足	提高抓取、放置、插孔等任务成功率
动作速度慢	缩短任务完成时间
精度不足	提升末端定位和精密操作能力
遇到新环境失败	在线适应环境变化
模仿学习性能受限	利用自主经验超越演示策略
失败后无法恢复	学习错误恢复和重新尝试
真实任务缺少演示	通过自主探索进一步改进策略

例如：

一个预训练 VLA 已经能够完成螺丝刀抓取，但成功率只有 70%。

我们希望通过强化学习，提高成功率、减少不必要的动作，并提高对初始姿态变化的鲁棒性。

4.3 具身基础模型 RL 后训练的五类方法

按照**强化学习究竟更新模型的哪个部分**，可以分为五类。

第一类：Full Policy Fine-tuning

直接更新预训练生成策略。

代表方法：

- DPPO；
- OGPO。

其中 DPPO 通过策略梯度微调 Diffusion Policy；OGPO 则利用 Off-policy Critic 和适配生成过程的策略优化目标，实现对生成式控制策略的完整微调。(DPPO)

主要用途：

当预训练策略的能力明显不足，希望真正改变其动作生成能力时，采用完整策略微调。

优点是修改范围大，能力提升空间也可能更大。

局限是训练成本较高，并且需要防止破坏原有的预训练能力。

第二类：Latent / Noise-Space RL

冻结预训练模型，只优化其输入潜变量或噪声分布。

代表方法：

- DSRL；
- LP-DS。

DSRL 通过学习 Diffusion Policy 的初始噪声分布实现策略改进，而不需要修改基础策略参数。(DSRL)

主要用途：

当预训练模型已经具备较好的动作生成能力，只需要选择更优的动作模式时。

优点是：

- 不需要微调整个大模型；
- 在线训练相对轻量；
- 可以复用预训练策略。

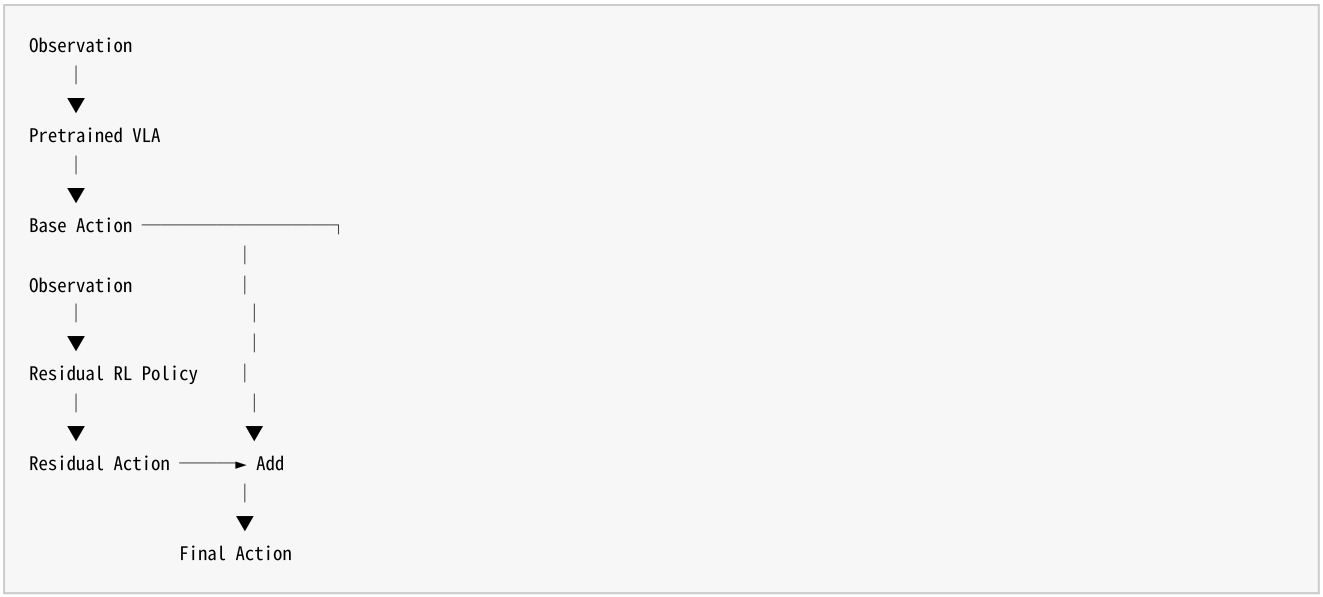
局限是：

无法轻易突破冻结生成器本身的表达能力限制。

第三类：Action Residual RL

在预训练策略输出的动作上增加 RL 修正。

例如：



主要用途：

预训练策略基本正确，但末端误差、姿态误差或接触控制仍然不够精确。

例如：

- 提高插孔精度；
- 修正抓取姿态；
- 对工具操作增加局部控制；
- 改善接触阶段的动作。

这类方法通常不需要重新训练基础 VLA。

第四类：Feature / Token-Level RL

利用预训练 VLA 的内部特征，训练轻量 RL 模块。

代表方法：

- RL Token。

RL Token 通过预训练 VLA 提供紧凑的任务相关表示，并训练轻量 Actor-Critic 模块实现在线策略改进。(RL Token)

主要用途：

大型 VLA 的在线强化学习。

例如：

- 精密装配；
- 插孔；
- 工具操作；
- 在线提升操作速度。

这类方法适合希望保留基础模型表征、降低在线训练开销的场景。

第五类：Velocity Field Fine-tuning

直接优化 Flow Matching Action Expert 的 Velocity Field。

代表研究：

- Q-VGM；
- Reward / Advantage-Guided Velocity Matching 相关方法。

Q-VGM 使用 Critic 提供的价值梯度信号，通过局部 Velocity Matching 改进预训练 Flow Matching Policy。(Q-VGM)

主要用途：

希望真正改进 Flow Matching 动作生成模型，但又不希望每次策略更新都对完整生成轨迹进行昂贵反向传播。

这是一条与大型 Flow Matching VLA 后训练高度相关的研究路线。

4.4 五类方法的总结

方法	代表	更新内容	主要用途
Full Fine-tuning	DPPO、OGPO	整个生成策略或主要 Action Expert	提升模型的动作生成能力
Latent RL	DSRL、LP-DS	输入噪声或潜变量策略	轻量级在线改进
Action Residual RL	Residual SAC 等	输出动作残差	精密控制、局部动作修正
Feature / Token RL	RL Token	轻量 Actor-Critic 模块	大型 VLA 的高效在线学习
Velocity Field RL	Q-VGM 等	Flow Matching Velocity Field	直接改进 Flow Matching Action Expert

其中，前四类的主要区别是优化参数的位置，而第五类进一步利用了 Flow Matching 的生成结构。

这些类别并非完全互斥。例如，可以只训练 Velocity Field 的残差模块，它同时属于 Residual Adaptation 和 Velocity Field Fine-tuning。

五、PPO、SAC 在两类任务中的具体用途

将两个分类维度组合，可以得到下面这张核心表。

	机器人运动控制	具身基础模型 RL 后训练
On-policy PPO	大规模仿真训练运控策略	DPPO 等生成策略微调
Off-policy SAC	灵巧操作、真实机器人在线学习	DSRL 等基于 SAC 或 Off-policy Critic 的适配方法

5.1 PPO + Locomotion

典型任务：

人形机器人学会上楼梯。

常用方式：

Isaac Lab + RSL-RL + PPO。

主要优势：

充分利用大规模仿真环境。

5.2 SAC + Manipulation

典型任务：

机器人通过在线交互学习抓取和物体操作。

主要优势：

复用 Replay Buffer 中的经验，减少获得新训练数据的需求。

5.3 PPO 类方法 + VLA

典型任务：

使用 DPPO 对 Diffusion Policy 进行强化学习后训练。

主要优势：

直接通过策略梯度改进生成策略。

5.4 SAC / Off-policy Critic 类方法 + VLA

典型任务：

使用 DSRL 优化预训练生成策略的潜变量，或者使用 OGPO、Q-VGM 等方法改进生成策略。

主要优势：

利用历史交互经验提高数据效率。

这里尤其需要区分：

OGPO 和 Q-VGM 都属于 Off-policy 生成策略优化路线，但并不等价于标准 SAC。

六、如何根据任务选择 RL 方法？

下面给出实际项目中比较有用的选型参考。

项目目标	优先考虑	原因
从零训练四足机器人行走	PPO	并行仿真成熟
训练 G1 等人形机器人运控	PPO	成熟的运动控制 RL 工作流
训练灵巧手在仿真中旋转物体	PPO	可利用并行仿真
真机灵巧手在线改进操作能力	SAC 或基于演示初始化的 Off-policy RL	可以复用历史数据

项目目标	优先考虑	原因
微调已有 Diffusion Policy	DPPO、OGPO	直接改进生成策略
不修改已有 Diffusion/Flow 模型	DSRL	冻结生成模型，优化噪声策略
提升 VLA 的精密操作能力	Residual RL、RL Token	训练轻量模块
直接微调 Flow Matching 的 Velocity Field	Q-VGM 等	利用价值信号更新生成模型
希望避免完整生成链的反向传播	Latent RL、Token RL、局部 Velocity Matching	降低在线优化成本

实际选型还要考虑模型大小、仿真是否可用、奖励是否稀疏，以及真实交互和计算资源的预算。

七、两个研究方向的典型实验路线

7.1 机器人运控 RL

推荐路线：

```
Isaac Lab / MuJoCo
        |
        ▼
    PPO / SAC
        |
        ▼
  训练运控策略
        |
        ▼
Domain Randomization
        |
        ▼
  Teacher-Student
        |
        ▼
    Sim-to-Real
        |
        ▼
  真实机器人控制
```

这条路线主要关注：

如何让机器人运动得更稳定、更灵活，并具备更强的物理环境适应能力。

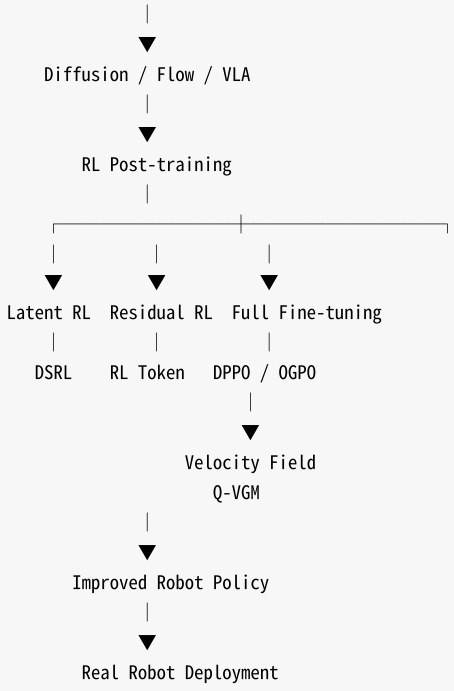
推荐入门：

Isaac Lab + RSL-RL

7.2 具身基础模型 RL 后训练

推荐路线：

```
    Robot Demonstrations
            |
            ▼
       Pretraining
```



这条路线主要关注：

如何利用强化学习，以较少的机器人交互数据提升预训练基础模型的任务能力。

推荐入门：

DSRL → DPPO → OGPO → Q-VGM

八、最后总结

对于具身智能中的强化学习，建议采用以下三层分类来理解整个领域。

分类层次	主要类别	核心问题
第一层：算法	PPO、SAC	如何利用环境反馈改进策略?
第二层：任务	Locomotion、Manipulation、Foundation Model Post-training	强化学习要解决什么机器人任务?
第三层：优化对象	低层控制策略、Residual、Latent、Action Expert、Velocity Field	强化学习究竟更新策略的哪个部分?

其中：

机器人运控强化学习以 PPO、大规模物理仿真、运动控制和 Sim-to-Real 为主要技术路线。

具身基础模型强化学习后训练以 Diffusion/Flow/VLA 为基础，重点研究如何利用 RL 改进预训练模型，包括 Latent RL、Residual RL、Token-Level RL 和 Full Policy Fine-tuning。

二者最核心的差异在于：

- **运控 RL：** 主要解决机器人如何获得稳定的运动与操作控制能力。
- **基础模型 RL 后训练：** 主要解决如何让已经学会操作的模型表现得更好，并进一步提高成功率、速度、精度和鲁棒性。

因此，具身强化学习并不只是 PPO 和 SAC 的选择问题，还包括**任务类型、策略架构、训练数据来源和策略更新范围**的选择。

具身智能中的运动控制强化学习：完整教程

课程首页 · English · PDF

项目	说明
适用范围	Isaac Lab / RSL-RL 工作流为说明性示例；任务、训练器、资产和执行器版本需一起绑定
文档核验日期	2026-10-18
许可	CC BY 4.0 · Xiong Zeng and contributors

核验边界： 文档整理与上游引用快照核对；未进行教程环境、训练或硬件验证。 版本与固定源码引用见版本说明。原创教材采用 CC BY 4.0，第三方引用、图片及代码遵循其原有许可；注明来源不等于拥有再许可权。

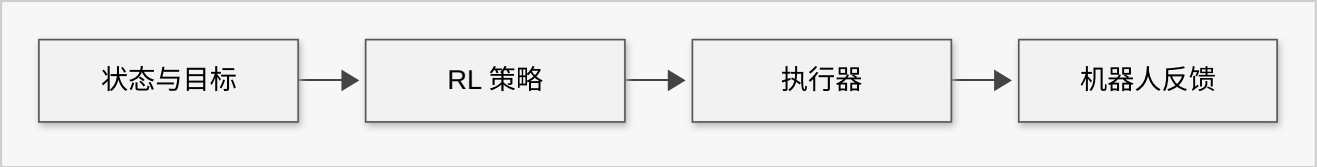
目录

- 第一章：什么是运动控制强化学习?
  - 1.1 运动控制 RL 在具身智能系统中的位置
  - 1.2 运动控制 RL 与具身基础模型 RL 后训练的区别
- 第二章：运动控制强化学习的任务分类
  - 2.1 按机器人任务分类
  - 2.2 按训练方法分类
- 第三章：运动控制强化学习的算法
  - 3.1 PPO：大规模仿真训练的常见基线
  - 3.2 SAC：强调交互数据复用
  - 3.3 PPO 与 SAC 对比
- 第四章：如何设计一个运动控制强化学习任务?
  - 4.1 Observation：机器人能观察到什么?
  - 4.2 Action：强化学习输出什么?
  - 4.3 Reward：机器人应该优化什么?
  - 4.4 Reset：从什么状态开始训练?
- 第五章：训练完整的运动控制策略
  - 5.1 标准训练流程
  - 5.2 第一步：建立机器人仿真模型
  - 5.3 第二步：设置控制频率
  - 5.4 第三步：并行训练
- 第六章：运动模仿强化学习
  - 6.1 基本流程
  - 6.2 Motion Tracking Reward
  - 6.3 DeepMimic
  - 6.4 AMP：Adversarial Motion Priors
  - 6.5 Motion Tracking 与 AMP 的区别

    - 6.6 从单条轨迹到通用运动控制
- 第七章：教师–学生强化学习
    - 7.1 为什么需要教师–学生？
    - 7.2 标准训练过程
    - 7.3 Asymmetric Actor-Critic 与 Teacher–Student 的区别
    - 7.4 RMA：Rapid Motor Adaptation
- 第八章：案例一——使用 PPO 训练 Unitree G1 行走
    - 8.1 任务定义
    - 8.2 Observation
    - 8.3 Action
    - 8.4 Reward
    - 8.5 Curriculum Learning
    - 8.6 开源实验入口
- 第九章：案例二——机械臂与灵巧手运动控制 RL
    - 9.1 任务定义
    - 9.2 建议的控制架构
    - 9.3 三种训练路线
    - 9.4 Observation 设计
    - 9.5 Reward 设计
    - 9.6 训练路线
- 第十章：Sim-to-Real
    - 10.1 为什么仿真成功不保证真机成功？
    - 10.2 Domain Randomization
    - 10.3 Sim-to-Sim
    - 10.4 真机部署检查清单
- 第十一章：如何评估运动控制 RL？
    - 11.1 腿式机器人指标
    - 11.2 Motion Tracking 指标
    - 11.3 灵巧手操作指标
- 第十二章：开源框架与工具
    - 12.1 Isaac Lab
    - 12.2 RSL-RL
    - 12.3 MuJoCo Playground
    - 12.4 如何选择？
- 第十三章：代表性论文与阅读顺序
- 第十四章：学习与研究路线
    - 阶段一：基础运动控制 RL
    - 阶段二：Motion Tracking RL
    - 阶段三：Teacher–Student 与 Sim-to-Real
    - 阶段四：复杂运动控制

    - 阶段五：通用运动控制器
- 第十五章：总结与关键结论
- 参考资料与开源链接

章节知识地图



第一章：什么是运动控制强化学习？

1.1 运动控制 RL 在具身智能系统中的位置

在具身智能中，可以将机器人控制系统粗略分为三个层次：

层次	主要任务	常见技术
高层：任务规划	决定机器人应该做什么	LLM、VLM、VLA、任务规划
中层：运动规划与技能控制	决定机器人如何运动	Motion Planning、Motion Tracking、RL
底层：执行器控制	让电机准确执行指令	PID、PD、阻抗控制、电流控制

运动控制 RL 主要应用于中层，也可能直接学习部分底层控制。例如，机器人的高层控制器要求“以 0.5 m/s 向前走”，运动策略决定关节动作，底层控制器则负责执行。

```
用户指令：“走到桌子前，拿起杯子”
                |
                v
          VLM / VLA / Planner
                |
                v
          目标速度 / 目标位姿
                |
                v
          RL Motion Policy <--- 传感器反馈
                |
                v
         关节目标 / 末端指令
                |
                v
          PD / IK / 电机驱动
                |
                v
             真实机器人
```

一个目标条件运动策略可以写为：

$$a_t = \pi_\theta(o_t, c_t),$$

其中，o_t 是关节、IMU、视觉等观测；c_t 是目标速度、目标姿态或技能指令；a_t 是输出的关节位置、力矩或末端动作；π_θ 是学习得到的策略。

传统控制通常由工程师显式设计控制律，运动控制 RL 则让策略通过与动力学环境的交互学习控制规律。但它们并不互斥：现代系统经常把 RL 与 PD、IK、MPC、WBC（Whole-Body Control）等技术组合。

1.2 运动控制 RL 与具身基础模型 RL 后训练的区别

维度	运动控制 RL	具身基础模型 RL 后训练
主要目标	学习稳定、灵活的运动技能	提高预训练策略的任务成功率
常见输入	关节、IMU、目标状态等	图像、语言、机器人状态等
常见输出	关节控制指令、末端控制指令	Action Chunk、动作分布等
典型模型	MLP、GRU、Transformer	VLA、Diffusion Policy、Flow Matching
常用方法	PPO、SAC、模仿 RL	PPO、SAC、Offline RL、专门的后训练方法
主要交互来源	大量仿真交互	预训练数据、仿真或真机交互
典型任务	行走、跳跃、爬楼梯、灵巧操作	视觉/语言条件下的操作任务
重点难题	稳定性、鲁棒性、运动性能	泛化、样本效率、在线适应

例如，训练 Unitree G1 在速度指令下行走属于运动控制 RL；对已经能抓取的 $\pi_{0.5}$ 通过 RL 提升新环境抓取成功率属于具身基础模型 RL 后训练。两者边界并不绝对：机械臂操作的低层控制 RL 与 VLA 后训练有时优化的是同一机器人动作，只是模型结构、初始化和目标不同。

第二章：运动控制强化学习的任务分类

2.1 按机器人任务分类

类别	英文	典型任务	主要挑战
腿式运动控制	Legged Locomotion	行走、奔跑、转向	平衡、接触稳定性
高动态运动控制	Agile Locomotion	跳跃、爬楼梯、越障、翻滚	动态接触、强非线性
全身动作跟踪	Whole-Body Motion Tracking	跳舞、动作模仿、轨迹跟踪	全身协调、动作保真度
机械臂操作控制	Manipulation Control	抓取、搬运、插入	接触、精度、力控制
灵巧手操作控制	Dexterous Manipulation	手内旋转、重抓取、工具操作	高维动作、多点接触
移动操作与全身协同	Loco-Manipulation	边走边抓、搬运、推门	行走与操作协同

这些任务并不互斥。人形机器人搬箱子通常同时涉及行走、全身协调和操作。

A. Legged Locomotion：腿式运动控制

目标是根据给定指令稳定运动。常见速度指令为：

$$c_t = (v_x^{\text{cmd}}, v_y^{\text{cmd}}, \omega_z^{\text{cmd}}),$$

分别对应前后、横向与转向速度。RL 输出关节动作，以跟踪指定速度。四足和双足机器人已经有较成熟的大规模仿真训练范式。代表工作 *Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning* 利用并行仿真训练 ANYmal 运动策略并进行实机验证。[1]

B. Agile Locomotion：高动态运动控制

高动态任务包括跳跃、爬楼梯、跨越障碍、奔跑、跌倒恢复和空中姿态调整。区别于普通行走，策略必须主动利用动力学与接触，例如在爬楼梯过程中协调足部抬升、落脚时序与重心转移。

C. Whole-Body Motion Tracking：全身动作跟踪

给机器人参考动作 $q^{\mathrm{ref}}(t)$，通过 RL 尽量准确地跟踪，同时维持动力学可行性。

```
人类视频 / 动捕 -> 动作重建 -> 机器人重定向
                                  |
                                  v
                            参考机器人动作
                                  |
                                  v
                          RL Motion Tracking
                                  |
                                  v
                            物理可实现的动作
```

代表性路线包括 DeepMimic、AMP、HOVER、BeyondMimic；它们分别涉及参考轨迹模仿、运动先验学习、全身多模式控制和复杂人形动作。[2][3][4]

D. Manipulation Control：机械臂操作控制

目标是完成抓取、搬运、插入等物体操作。RL 可以输出关节动作，也可以输出末端位置或姿态增量，随后交给 IK、操作空间控制（OSC）或关节控制器。通常更关注末端精度、接触力、成功率、碰撞安全与扰动恢复。

E. Dexterous Manipulation：灵巧手运动控制

多指灵巧手需要通过多个关节协同控制接触：

$$a_t = [a_t^{\mathrm{finger1}}, a_t^{\mathrm{finger2}}, \ldots, a_t^{\mathrm{fingerN}}]^\top.$$

任务包括稳定抓取、手内旋转、重抓取、工具使用以及物体打滑后的恢复。核心困难是高维动作空间与复杂的手指–物体接触动力学。

F. Loco-Manipulation：移动操作

这类任务将运动与操作联动，例如边走边搬运、扶墙跨越障碍、行走时双手操作物体和推门。重点从单个技能转向对身体、双臂、腿和接触模式的整体协调。

2.2 按训练方法分类

方法	英文	核心思想	适用场景
从零开始强化学习	RL from Scratch	用任务奖励直接探索	基础平衡、行走

方法	英文	核心思想	适用场景
动作模仿强化学习	Imitation RL	参考运动奖励引导 RL	跳舞、动作复现
残差强化学习	Residual RL	在已有控制器上学习修正	轨迹修复、接触补偿
分层强化学习	Hierarchical RL	高层选技能、低层控制	多阶段任务
教师–学生训练	Teacher–Student	先训练特权教师，再蒸馏	视觉控制、Sim-to-Real
自适应控制策略	Adaptive RL	根据历史观测适应变化	摩擦、负载、动力学变化

这些方法可以组合：先通过模仿强化学习训练教师，再将策略蒸馏到真实传感器可观测的学生模型，最后进行部署。

第三章：运动控制强化学习的算法

3.1 PPO：大规模仿真训练的常见基线

PPO（Proximal Policy Optimization）属于 On-policy 强化学习，在四足、双足、动作跟踪和灵巧操作中使用广泛。典型训练循环：

```
初始化 Actor / Critic
         |
         v
创建数千个并行仿真环境
         |
         v
Actor 交互并收集 (Observation, Action, Reward)
         |
         v
计算 Advantage
         |
         v
更新 Actor 和 Critic -> 重复
```

PPO 特别适合结合 GPU 并行物理仿真：虽然历史交互数据复用有限，但数千个环境可以快速产生大量样本。例如，4096 个环境同时模拟各自的机器人。[1]

常见实现是 RSL-RL，提供面向机器人控制的 PPO、相关网络与训练组件。[5]

3.2 SAC：强调交互数据复用

SAC（Soft Actor-Critic）是 Off-policy 算法，使用 Replay Buffer 反复利用以往交互数据。

```
Robot / Simulator
        |
        v
   Experience
        |
        v
  Replay Buffer
     /     \
    v       v
Critic     Actor
Update     Update
```

```
    \     /
     v   v
    New Policy
```

对于真实机器人，交互成本高，SAC 的数据复用能力有吸引力。它可用于连续机械臂操作、有限真机数据在线改进、灵巧手及残差控制。但在高吞吐 GPU 仿真下，PPO 可能更易实现、调试和扩展；不能脱离环境与交互预算直接断言 SAC 总是更好。

3.3 PPO 与 SAC 对比

特性	PPO	SAC
类型	On-policy	Off-policy
历史样本复用	低	高
Replay Buffer	通常不需要	需要
大规模并行仿真	非常成熟	可行，但系统设计更复杂
高维连续动作	支持	支持
真机样本效率	通常较低	通常有潜在优势
工程实现	相对直接	相对复杂
腿式运动控制	很常见	相对较少
机械臂与灵巧手	常见	也常见/可用
推荐起点	仿真运动控制基线	数据昂贵的连续控制

建议：双足/四足仿真运动控制先复现 PPO；如果关心有限真实数据下的在线策略改进，进一步比较 SAC。算法比较应固定环境、交互次数与评估协议。

第四章：如何设计一个运动控制强化学习任务？

完整环境通常包括：

$$\text{Environment} = \{\text{Observation}, \text{Action}, \text{Reward}, \text{Reset}, \text{Dynamics}\},$$

目标条件任务还需要 Command 或 Goal。实际表现往往比单纯更换 PPO 网络结构更依赖环境建模。

4.1 Observation：机器人能观察到什么？

观测	含义	典型来源
Joint Position	关节位置	编码器
Joint Velocity	关节速度	编码器/估计器
Base Angular Velocity	机身角速度	IMU
Projected Gravity	重力方向在机身坐标系中的表示	姿态估计
Previous Action	上一步控制动作	控制器

观测	含义	典型来源
Velocity Command	目标速度	上层控制
Contact State	足部/手指接触状态	接触估计
Height Map	地形高度	深度相机或其他传感器

腿式机器人的典型观测可以写为：

$$o_t = [q_t, \dot{q}_t, \omega_t, g_t, a_{t-1}, c_t].$$

要区分**仿真可获得的状态**与**真机实际可用的传感器观测**。机身线速度、接触状态和精确物体位姿等在真机上可能需要估计。

Observation History

运动控制往往部分可观测：当前关节角度不能直接确定之前是否滑动、是否受到扰动或执行器是否延迟。可以输入历史窗口：

$$o_t^{\text{hist}} = [o_{t-k}, \dots, o_{t-1}, o_t].$$

随后使用 MLP、GRU 或 Transformer 编码，帮助策略隐式估计隐藏动力学与接触状态。

4.2 Action：强化学习输出什么？

动作空间	输出	特点
Joint Position	关节目标位置	容易构建稳定基线
Joint Velocity	关节目标速度	适合速度控制接口
Joint Torque	关节目标力矩	表达更直接，训练与安全挑战更高
Cartesian Action	末端位置、姿态或速度	适合机械臂

A. Joint Position Control

最常见的一种形式：

$$q_t^{\text{target}} = q^{\text{default}} + s_a a_t, \qquad a_t = \pi_\theta(o_t).$$

由底层 PD 执行：

$$\tau_t = K_p(q_t^{\text{target}} - q_t) - K_d \dot{q}_t.$$

对动作跟踪，也可以设定：

$$q_t^{\text{target}} = q_t^{\text{ref}} + s_a a_t.$$

这对应参考运动上的残差动作。优势是减轻 RL 学习底层电机动态的负担。

B. Joint Torque Control

直接输出 $\tau_t = \pi_\theta(o_t)$，控制自由度更高，但必须显式考虑电流、力矩、温度、速度和关节限位约束。

C. Cartesian Control

机械臂可输出 $a_t = [\Delta x, \Delta y, \Delta z, \Delta R]$，通过 IK 或 OSC 将末端目标转换为关节指令。

```
RL Policy -> 末端增量/目标位姿 -> IK / OSC -> 关节命令 -> 机械臂
```

其中 ΔR 可由旋转向量等表示，实际应保证旋转参数化与控制频率一致。Isaac Lab 提供多种关节和笛卡尔动作接口。[6]

选型建议

机器人	建议初始动作空间
四足/双足	Joint Position Target
人形动作跟踪	Joint Position Target / Residual
常规机械臂	Cartesian Delta Pose
多指灵巧手	Finger Joint Position Target
机械臂 + 灵巧手	Arm Cartesian Action + Finger Joint Action

策略输出与驱动器最终执行的接口不必相同，例如策略输出末端位姿增量，底层电机仍执行关节位置控制。

4.3 Reward：机器人应该优化什么？

常见拆分：

$$r_t = r_t^{\text{task}} + r_t^{\text{stability}} - r_t^{\text{regularization}}.$$

以双足速度跟踪为例：

$$r_{\text{velocity}} = \exp\left(-\frac{\|v_t - v_t^{\text{cmd}}\|^2}{\sigma_v^2}\right),$$

$$r_{\text{orientation}} = \exp\left(-\frac{\|e_t^{\text{orientation}}\|^2}{\sigma_R^2}\right),$$

$$r_{\text{smooth}} = -\|a_t - a_{t-1}\|^2.$$

可组合成：

$$r_t = w_v r_{\text{velocity}} + w_R r_{\text{orientation}} - w_a\|a_t - a_{t-1}\|^2 - w_\tau\|\tau_t\|^2.$$

实际任务还可能加入：身体高度、足部滑动、碰撞、关节限位、能耗、接触稳定等项。只奖励移动速度可能导致身体前冲、异常高速摆腿或利用仿真接触误差；但惩罚过强则可能使机器人选择不运动。

原则：奖励既要反映任务是否完成，也要约束运动是否稳定、自然、可部署。

4.4 Reset：从什么状态开始训练？

普通行走可以从站立开始；动作跟踪和多阶段操作不能总从同一状态开始，否则难以学习中途恢复。

Uniform Reference State Initialization（Uniform RSI） 从参考运动不同时间点采样：

$$t_0 \sim U(0, T).$$

Event-biased RSI 在起跳、落地、接触切换、开始抓取或释放等关键事件附近增加采样密度。初始化应尽量满足动力学和接触一致性，避免把不合理的跳跃状态强加给策略。

第五章：训练完整的运动控制策略

5.1 标准训练流程

```
机器人模型 (URDF / MJCF / USD)
            |
            v
物理仿真器 (Isaac Lab / MuJoCo)
            |
            v
定义 Observation / Action / Reward / Reset
            |
            v
      PPO / SAC 训练
            |
            v
       仿真性能测试
            |
            v
 Domain Randomization / Curriculum
            |
            v
      Sim-to-Sim 验证
            |
            v
       Sim-to-Real
            |
            v
         真实机器人
```

5.2 第一步：建立机器人仿真模型

模块	应建模的内容
Robot Model	连杆质量、惯量、关节轴、关节限位
Actuator	电机力矩、速度限制、摩擦与响应
Contact	摩擦、碰撞几何、接触参数
Sensors	编码器、IMU、视觉传感器
Control	PD 增益、控制频率、动作延迟
Environment	地面、障碍物、操作物体

URDF 不仅是可视化模型，也是重要的动力学参数载体；但“文件里有参数”不代表“参数经过真机标定”。

5.3 第二步：设置控制频率

以下仅为示例，并非某一具体机器人的真实配置：

模块	示例频率
Physics Simulation	500 Hz
Joint PD Control	500 Hz
RL Policy	50 Hz
Vision Update	30 Hz
High-level Planning	5 Hz

若 $f_{\text{sim}} = 500\,\text{Hz}$、$f_{\text{RL}} = 50\,\text{Hz}$，每次 RL 输出后进行 10 个物理步长，即 Action Decimation。高频控制器负责执行，RL 在较低频率更新目标。

5.4 第三步：并行训练

一个**示例性** PPO 训练配置：

参数	示例配置
Parallel Environments	4096
RL Frequency	50 Hz
Rollout Length	24 steps/environment
Algorithm	PPO
Actor / Critic	MLP / MLP
Action	Joint Position Target
Observation	Proprioception + Command

一次 Rollout 的环境交互数为 $4096 \times 24 = 98{,}304$。这一数值是收集到的步数，不代表参数更新次数；视觉或高接触灵巧操作任务往往需要减少并行环境数。

Actor-Critic 结构：

```
Observation
   |
   +-------> Actor  -----> Action -----> Simulator
   |                                        |
   +-------> Critic -----> Value <------- Reward
```

训练期间更新 Actor 与 Critic，典型部署时只运行 Actor。

第六章：运动模仿强化学习

复杂跳跃、跳舞、体操动作和高维灵巧操作，直接从零探索可能非常困难，因此可以引入参考运动。

6.1 基本流程

```
Human Motion / Teleoperation
            |
            v
    Motion Reconstruction
            |
            v
     Motion Retargeting
            |
            v
    Robot Reference Motion
            |
            v
     Motion Tracking RL
            |
            v
      Robust Motion Policy
```

参考数据可以来自动捕、人体视频重建、遥操作、已有控制器以及运动规划。

6.2 Motion Tracking Reward

给定参考关节位置 q_t^{ref}，可以定义：

$$r_{\text{track}} = \exp\left(-\frac{\|q_t - q_t^{\text{ref}}\|^2}{\sigma_q^2}\right).$$

全身任务还可能跟踪 Root Position、Root Orientation、Joint Velocity、End-effector Position、Contact Schedule。仅关节角接近并不保证机器人不会摔倒；需要同时考虑动作相似性、动力学可行性与稳定性。

6.3 DeepMimic

DeepMimic 使用参考运动奖励与任务奖励，帮助物理角色学习复杂动作，是 Motion Tracking RL 的经典工作。[2]

```
Reference Motion -> Tracking Reward --+
                                      +--> RL Policy -> Physics
Task Specification -> Task Reward ----+
```

项目主页：https://xbpeng.github.io/projects/DeepMimic/

6.4 AMP：Adversarial Motion Priors

传统 Motion Tracking 要求在给定时刻跟踪指定轨迹；AMP 使用判别器从参考动作数据中学习运动风格奖励，使策略在完成任务目标的同时产生符合先验的动作。例如，学习自然的行走风格，而不严格逐帧复制某一条轨迹。[3]

6.5 Motion Tracking 与 AMP 的区别

特性	Motion Tracking RL	AMP
是否需要运动数据	通常需要参考轨迹	需要运动数据
是否显式跟踪某条轨迹	是	不一定
奖励来源	跟踪误差	学到的运动先验奖励
主要目标	准确复现参考运动	产生符合参考分布的运动
典型难点	时间同步、跟踪稳定性	判别器训练、奖励平衡

6.6 从单条轨迹到通用运动控制

HOVER 等工作探索将不同控制命令和动作模式集成到统一的全身控制策略中，使策略不再局限于固定轨迹的逐帧跟踪。[4]

趋势：Single Motion Tracking Policy → Multi-Mode / Universal Motion Controller。

第七章：教师–学生强化学习

7.1 为什么需要教师–学生？

在仿真中可以访问精确物体位置/速度、真实摩擦系数、接触力以及隐藏动力学参数，但这些信息在真机中未必可测。如果 Actor 直接依赖它们，就难以部署。

```
仿真完整状态：物体位姿、接触力、摩擦、动力学
                        |
                        v
                  Teacher Policy
                        |
                        v
                  Teacher Action
                        |
                  Distillation
                        |
                        v
真机观测: Camera / Encoder / IMU -> Student Policy
```

7.2 标准训练过程

先训练使用特权状态的教师：

$$a_t^T = \pi_T(s_t).$$

再训练只依赖可部署观测的学生：

$$a_t^S = \pi_S(o_t),$$

典型的动作蒸馏损失：

$$\mathcal{L}_{\text{distill}} = \mathbb{E}\left[\|a_t^S - a_t^T\|^2\right].$$

学生可借助观测历史和状态估计模块推断隐藏信息。实际蒸馏还需处理教师/学生访问状态分布不同的问题。

7.3 Asymmetric Actor-Critic 与 Teacher–Student 的区别

Asymmetric Actor-Critic： Actor 训练时就只使用可部署观测，而 Critic 可使用 Privileged Information。

Teacher–Student： Teacher Actor 可以使用 Privileged Information，再将教师策略迁移给仅使用有限观测的 Student Actor。

所以前者是**训练结构**，后者是**策略迁移方法**。两者可以组合。Isaac Lab 与 RSL-RL 支持相应的观测分组和训练机制。[5][6]

7.4 RMA：Rapid Motor Adaptation

RMA 将动力学适应问题与机器人运动策略结合，利用近期观测历史构建隐式自适应信息，使机器人应对地形、摩擦、负载及未知动力学变化。[7]

论文：https://arxiv.org/abs/2107.04034

第八章：案例一——使用 PPO 训练 Unitree G1 行走

8.1 任务定义

让 G1 根据目标速度 $c_t = [v_x^{\text{cmd}}, v_y^{\text{cmd}}, \omega_z^{\text{cmd}}]$ 稳定运动，包括前后、横向移动与转向。

8.2 Observation

建议先使用本体感知：Joint Position、Joint Velocity、Base Angular Velocity、Projected Gravity、Previous Action、Velocity Command。训练楼梯或复杂地形后，再逐步添加 Height Scan、接触估计、观测历史与视觉特征。

8.3 Action

推荐位置目标：

$$q_t^{\text{target}} = q^{\text{default}} + s_a \pi_\theta(o_t),$$

底层由 PD 执行，利于复用现有控制接口。

8.4 Reward

奖励项	目标
Linear Velocity Tracking	跟踪前进/横向速度
Angular Velocity Tracking	跟踪转向速度
Orientation Stability	维持合理身体姿态
Action Smoothness	减少动作震荡
Joint Limit Penalty	避免极限关节位置

奖励项	目标
Foot Slip Penalty	降低足底滑动
Termination Penalty	惩罚摔倒或失败

8.5 Curriculum Learning

阶段	任务	目的
1	站立	平衡
2	低速前进	基本步态
3	前后左右移动	扩大速度范围
4	转向	角速度跟踪
5	外力扰动	稳定与恢复
6	不平地形	接触适应
7	楼梯	复杂地形运动

可以连续扩展命令/地形分布，并不一定每阶段重训单独策略。

8.6 开源实验入口

Unitree RL Lab 基于 Isaac Lab，提供 Unitree 机器人的训练及部署相关代码。[8] 以下命令为该仓库接口的**示例**，执行前应核对当前版本的环境与任务名称：

```
# 查看可用任务
./unitree_rl_lab.sh -l

# 训练 G1 速度跟踪策略（示例任务名）
./unitree_rl_lab.sh -t --task Unitree-G1-29dof-Velocity

# 测试策略
./unitree_rl_lab.sh -p --task Unitree-G1-29dof-Velocity
```

第一个研究实验：先复现平地行走，再分别修改 Action Smoothness、Domain Randomization、Observation History 和 Foot Slip Penalty，观察训练收敛、速度误差和扰动恢复是否变化。

第九章：案例二——机械臂与灵巧手运动控制 RL

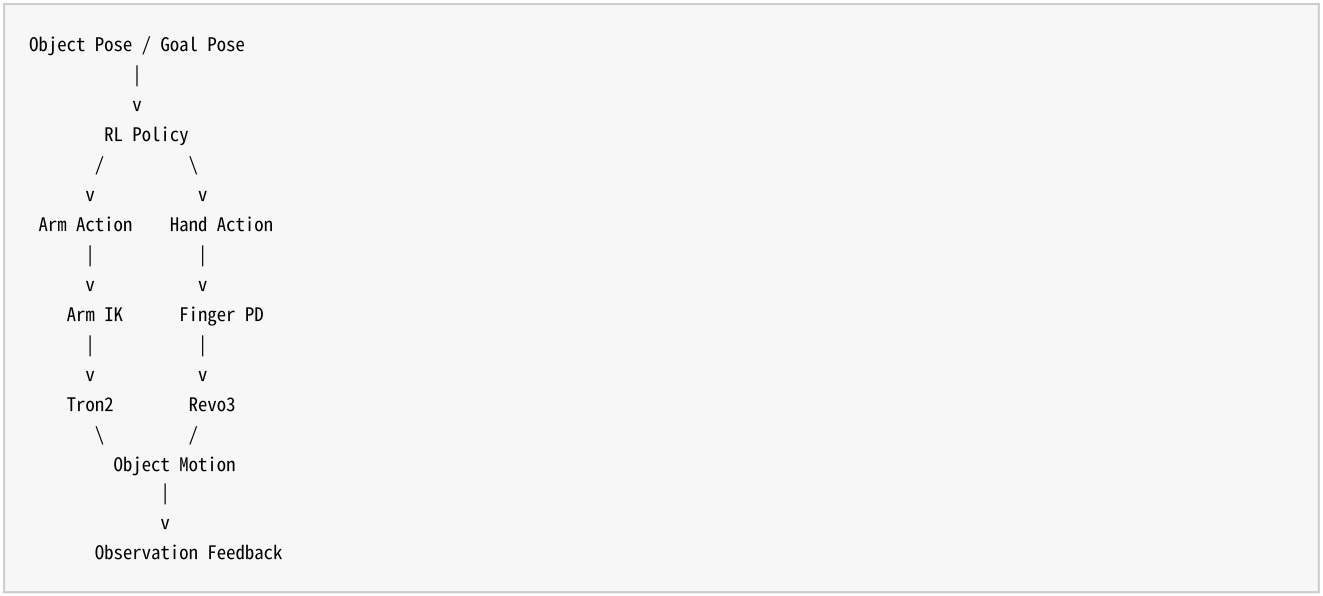
以 **Tron2 机械臂 + Revo3 灵巧手 + 螺丝刀操作** 为例，说明如何把腿式运动控制的环境设计方法迁移到复杂接触操作。

9.1 任务定义

1. 从桌面不同位置拾取螺丝刀。
2. 协调机械臂姿态和手指接触。
3. 移动螺丝刀到指定位置。

4. 完成指定角度旋转。
5. 在扰动后保持或恢复抓取。

9.2 建议的控制架构



可以先训练机械臂与手分离的控制器，再逐步扩展到联合策略。

9.3 三种训练路线

路线 A：从零开始训练。直接使用抓取、抬起、旋转和成功奖励：

$$r_t = r_{\text{grasp}} + r_{\text{lift}} + r_{\text{rotate}} + r_{\text{success}}.$$

优点是不依赖参考轨迹，困难在于高维接触探索。

路线 B：Motion Tracking RL。从人类视频或遥操作获得参考：$T^{\text{ref}}_{\text{hand}}(t)$、$q^{\text{ref}}_{\text{finger}}(t)$。可以降低探索难度，但单条轨迹不足以覆盖所有物体初始姿态和接触变化。

路线 C：Residual RL。在轨迹跟踪器、IK 或已有策略之上学习受约束的动作修正：

$$a_t = a_t^{\text{base}} + \Delta a_t^{\text{RL}}.$$

例如由基线控制机械臂末端跟踪，RL 修正手指接触，或对臂手都施加有限残差。实际必须约束残差幅值、动作变化率和接触力。

9.4 Observation 设计

$$o_t = [q_{\text{arm}}, q_{\text{hand}}, \dot{q}, \hat{T}_{\text{object}}, T_{\text{goal}}, a_{t-1}].$$

其中 $\hat{T}_{\text{object}}$ 由头部/腕部相机的位姿估计器提供。如果真机无法准确测量物体速度，可用历史观测/状态估计器推断。**Actor 不应无意中使用只在仿真中有的真实物体状态**；Critic 可以采用特权状态。

9.5 Reward 设计

奖励	作用
Reach	靠近目标物体
Grasp	形成稳定抓取
Lift	抬起物体
Position	移动到目标区域
Rotation	完成目标旋转
Stability	减少滑动和掉落
Smoothness	抑制动作震荡
Success	奖励完成完整任务

如果只有最终旋转成功奖励，从随机探索中发现合理抓取可能非常困难。建议使用课程学习与适度分阶段奖励。

9.6 训练路线

阶段	任务	目标
1	已抓住时维持稳定	基本接触控制
2	已抓住时旋转	手内操作
3	接近目标并抓取	抓取能力
4	抓取后旋转	多阶段技能
5	物体初始位姿随机化	泛化
6	加物体位姿估计噪声	视觉鲁棒性
7	加外力扰动	恢复能力
8	真机闭环测试	验证迁移

对于有参考轨迹的任务，可将 Uniform RSI 与 Event-biased RSI 结合，使训练更多覆盖接触建立、释放和物体转动等关键片段。

第十章：Sim-to-Real

10.1 为什么仿真成功不保证真机成功?

差异类型	典型来源
Mass / Inertia	质量与惯量误差
Friction	地面或物体摩擦不准确
Actuator	电机响应、死区、内部控制

差异类型	典型来源
Delay	通信/控制延迟
Observation Noise	编码器、IMU、视觉误差
Contact	接触求解器误差
Geometry	碰撞几何不准确
Calibration	关节零位、坐标系和外参误差

例如，仿真中电机对目标动作快速响应，而真实硬件有延迟，就可能导致振荡。

10.2 Domain Randomization

训练时随机化环境/动力学参数：

$$\xi \sim p(\xi),$$

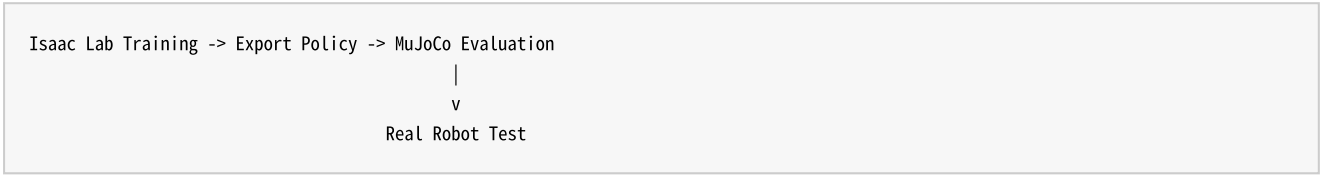
其中 ξ 可以包含质量、重心、摩擦、执行器强度、延迟和传感器噪声。

随机化	目的
Mass	适应质量误差
Center of Mass	适应重心偏差
Friction	适应接触表面变化
Motor Strength	适应执行器差异
Action Delay	适应控制延迟
Observation Noise	适应传感器误差
External Push	提高扰动鲁棒性

随机化范围要依据标定数据、硬件规格和物理常识设置，并不是越大越好。

10.3 Sim-to-Sim

可在 Isaac Lab 中训练，再在 MuJoCo 中测试（或反过来），验证策略是否利用了某个仿真器特有的数值行为。



如果同一控制策略在不同仿真器表现差异很大，应优先检查动力学、接触与动作接口。通过 Sim-to-Sim 并不等于实机安全。

10.4 真机部署检查清单

- 观测维度、顺序和标准化参数与训练一致。
- 关节映射、正负方向、单位、坐标系与零位一致。

- Action Scale、PD 增益、控制频率与仿真设定一致或经过验证。
- 位置、速度、力矩、温度设置保护约束。
- 通信中断、推理超时具有安全回退与紧急停止机制。
- 双足机器人配置防跌倒措施；灵巧手注意异常接触力、卡死和物体掉落。
- 初次部署从低风险、低幅值动作开始，逐步扩展任务范围。

进阶：学习真实与仿真之间的动力学差异

ASAP 等工作尝试利用真实机器人数据对齐仿真与真实动力学，改善敏捷全身动作的 Sim-to-Real。[9]

第十一章：如何评估运动控制 RL？

训练 Reward 上升不代表真实任务表现一定改善。必须构建独立测试集和性能指标。

11.1 腿式机器人指标

指标	含义
Success Rate	任务完成率
Fall Rate	摔倒率
Velocity Tracking Error	速度跟踪误差
Orientation Error	姿态误差
Energy Consumption	能耗
Foot Slip	足底滑动
Recovery Rate	扰动恢复率
Generalization	新地形/新参数下的表现

11.2 Motion Tracking 指标

指标	含义
Joint Tracking Error	关节轨迹误差
Root Position Error	根部位置误差
Root Orientation Error	根部方向误差
End-effector Error	末端误差
Contact Consistency	接触一致性
Motion Completion Rate	动作完成率
Recovery Rate	扰动后的恢复率

11.3 灵巧手操作指标

指标	含义
Task Success Rate	任务成功率
Object Pose Error	物体位姿误差
Grasp Stability	抓取稳定性
Object Drop Rate	物体掉落率
Contact Slip	滑动情况
Recovery Rate	扰动恢复率
Completion Time	任务完成时间
Generalization	新物体位姿下的表现

公平对比原则：相同环境与任务、相同交互预算、多个随机种子、未见过的测试状态，以及与训练奖励独立的性能指标。尤其要区分训练表现、仿真泛化和真机表现。

第十二章：开源框架与工具

框架	主要用途	建议
Isaac Lab	通用机器人仿真与 RL	优先学习
RSL-RL	PPO、运动控制和蒸馏	优先学习
Unitree RL Lab	Unitree 机器人训练与部署	G1/H1 适合
MuJoCo Playground	GPU 加速机器人学习	值得比较
legged_gym	经典腿式机器人 PPO 代码	理解经典实现
skrl	多种 RL 算法实现	算法对比可用

12.1 Isaac Lab

项目：https://github.com/isaac-sim/IsaacLab

支持 Legged Locomotion、Humanoid Locomotion、Manipulation、Dexterous Manipulation 和 Contact-rich Manipulation 等任务。代表性任务名（以特定版本官方环境目录为参考）：[6]

示例环境	任务
`Isaac-Velocity-Flat-G1-v0`	G1 平地速度跟踪
`Isaac-Velocity-Rough-G1-v0`	G1 复杂地形速度跟踪
`Isaac-Velocity-Flat-Unitree-Go2-v0`	Go2 平地行走
`Isaac-Reach-Franka-v0`	Franka 到达
`Isaac-Lift-Cube-Franka-v0`	Franka 举起方块

示例环境	任务
`Isaac-Repose-Cube-Allegro-v0`	Allegro 灵巧手物体重定向

任务名称可能随软件版本变化，以本地 task registry 为准。

12.2 RSL-RL

项目：https://github.com/leggedrobotics/rsl_rl

建议按 Runner → PPO → Actor/Critic → Rollout Storage → Distillation 的顺序读代码。若研究 PPO 运动控制的算法改进，可以直接从这套实现构建基线。[5]

12.3 MuJoCo Playground

项目：https://github.com/google-deepmind/mujoco_playground

提供面向四足、人形、灵巧手、机械臂的 GPU 加速强化学习环境与任务。某些批量动力学任务比完整渲染系统更轻量；实际速度取决于 GPU、接触数量、视觉与仿真后端。[10]

12.4 如何选择？

研究目标	推荐起点
学习 PPO 运动控制	Isaac Lab + RSL-RL
训练 G1 行走	Unitree RL Lab
GPU 大规模并行训练	Isaac Lab / MuJoCo Playground
灵巧手操作	Isaac Lab / MuJoCo Playground
视觉与运动控制结合	Isaac Lab
修改 PPO 算法	RSL-RL
比较 SAC 与 PPO	支持二者的通用 RL 实现
轻量级控制实验	MuJoCo Playground

第十三章：代表性论文与阅读顺序

论文	年份	核心主题	重点学习
DeepMimic	2018	动作模仿 RL	参考运动奖励、RSI
AMP	2021	对抗运动先验	从运动数据学习风格
RMA	2021	快速自适应	历史观测与动力学适应
Learning to Walk in Minutes	2022	并行 PPO	大规模运动训练
Learning Robust Perceptive Locomotion	2022	感知运动控制	地形感知与迁移
HOVER	2025	全身多模式控制	通用控制命令

论文	年份	核心主题	重点学习
ASAP	2025	动力学差异对齐	Sim-to-Real
BeyondMimic	2025	人形复杂动作	动作跟踪与技能泛化
MuJoCo Playground	2025	GPU 机器人学习	现代实验平台

建议优先精读四项：

1. **Learning to Walk in Minutes**：并行 PPO、奖励、课程学习与 Sim-to-Real。[1]
2. **DeepMimic**：参考动作、动作模仿奖励、RSI、物理动作跟踪。[2]
3. **RMA**：特权状态、历史观测、自适应以及部署。[7]
4. **HOVER**：全身控制、多命令模式、动作跟踪与蒸馏。[4]

第十四章：学习与研究路线

阶段一：基础运动控制 RL

目标：训练一个能够稳定行走的双足/四足机器人。

学习 Isaac Lab、RSL-RL、PPO、Observation/Action/Reward、Domain Randomization。先复现 G1 等平台的速度跟踪，再做奖励和观测消融。

阶段二：Motion Tracking RL

目标：跟踪参考动作，同时具备物理可行性与恢复能力。

学习 Motion Retargeting、Tracking Reward、RSI、Motion Curriculum、Residual RL。从单条参考动作开始，再逐步增加初始状态扰动和外部扰动。

阶段三：Teacher–Student 与 Sim-to-Real

目标：策略不依赖仿真中特有的完整状态。

学习 Asymmetric Actor-Critic、Privileged Information、蒸馏、Observation History 与 Domain Randomization。训练特权教师，将其迁移给可部署学生，并开展跨仿真器测试。

阶段四：复杂运动控制

路线	典型任务
Humanoid Locomotion	爬楼梯、越障、跳跃、跌倒恢复
Dexterous Manipulation	手内旋转、工具操作、扰动恢复

深入研究接触动力学、任务课程、强鲁棒性和真机闭环策略。

阶段五：通用运动控制器

探索一个支持多种命令的条件策略：

$$a_t = \pi_\theta(o_t, c_t).$$

其中 c_t 可编码目标速度、目标位姿、参考运动片段、目标物体运动或技能条件。对应方向包括 Goal-conditioned Motion Control、Universal Motion Tracking、Hierarchical Motion Policies、Residual Motion Adaptation 和 Learning-based Whole-Body Control。

第十五章：总结与关键结论

运动控制强化学习可以从**任务、训练方法和工程流程**三个维度理解。

任务	核心问题
Locomotion	如何稳定移动?
Agile Locomotion	如何完成高动态动作?
Motion Tracking	如何准确跟踪参考动作?
Manipulation	如何完成物体操作?
Dexterous Manipulation	如何协调多指接触?
Loco-Manipulation	如何协调整体移动与操作?

方法	重点解决的问题
PPO	如何利用大规模并行仿真学习控制?
SAC	如何增加交互数据的利用效率?
Imitation RL	如何利用参考动作减少探索难度?
Residual RL	如何在已有控制器之上改进?
Teacher–Student	如何从特权仿真状态迁移到真实观测?
Adaptive RL	如何适应未知动力学和环境变化?

完整开发流程为:

$$\boxed{\begin{array}{c}\text{Robot Modeling} \rightarrow \text{Task Design} \rightarrow \text{PPO/SAC Training} \\ \text{Curriculum} + \text{DR} \rightarrow \text{Sim-to-Sim} \rightarrow \text{Sim-to-Real}\end{array}}$$

结论一：任务建模与算法同样重要。 Observation、Action、Reward、Reset、仿真动力学和课程学习会显著影响性能。

结论二：PPO + 并行仿真是重要基线。 腿式运动与动作跟踪尤其成熟；真机数据宝贵时，应进一步考虑 SAC 等高数据复用方法。

结论三：通用、条件化、鲁棒的运动控制器是值得关注的方向。 一条有价值的研究路线是:

Motion Tracking RL → Residual RL → Robust Sim-to-Real → Goal-conditioned Universal Motion Control。

参考资料与开源链接

以下资料主要为教程中提及的论文主页、论文页面和官方开源仓库。论文年份与任务名称应以其最终发表版本/仓库当前版本为准。

1. Rudin et al., *Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning* (2022)
2. Peng et al., *DeepMimic* (2018)
3. Peng et al., *AMP: Adversarial Motion Priors* (2021)
4. HOVER: Versatile Neural Whole-Body Controller for Humanoid Robots；论文
5. RSL-RL 官方仓库
6. Isaac Lab 官方仓库；环境目录
7. RMA: Rapid Motor Adaptation for Legged Robots
8. Unitree RL Lab 官方仓库
9. ASAP: Aligning Simulation and Real-World Physics
10. MuJoCo Playground 官方仓库
11. legged_gym 官方仓库
12. skrl 官方仓库

文档版本：2026-10-15。示例参数用于教学，不能直接视为真实机器人安全部署配置。

PPO 完整推导与设计动机

课程首页 · English · PDF

项目	说明
适用范围	PPO-Clip / GAE 理论；PyTorch 风格示例为教学内容，非已测试训练包
文档核验日期	2026-10-18
许可	CC BY 4.0 · Xiong Zeng and contributors

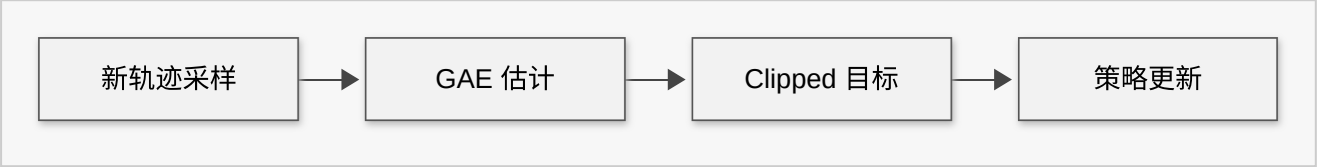
核验边界： 文档整理与上游引用快照核对；未进行教程环境、训练或硬件验证。 版本与固定源码引用见版本说明。原创教材采用 CC BY 4.0，第三方引用、图片及代码遵循其原有许可；注明来源不等于拥有再许可权。

目录

- 第一部分　从预期回报到策略梯度
  - 1. 定义强化学习优化问题
  - 2. 利用 Log-Derivative Trick 推导 Policy Gradient
  - 3. 因果性与 Reward-to-Go
- 第二部分　Baseline、Value 与 Advantage
  - 4. 为什么可以减去 Baseline?
  - 5. 为什么使用 Value Function 作为 Baseline?
- 第三部分　性能差分、Surrogate Objective 与重要性采样
  - 6. 旧数据为什么不能无限重复使用?
  - 7. Performance Difference Lemma 的逐步证明
  - 8. 将真实性能改进近似为 Surrogate Objective
  - 9. Importance Sampling 推导 Probability Ratio
- 第四部分　TRPO：用 Trust Region 控制更新
  - 10. 直接最大化 CPI 为什么危险?
  - 11. KL Divergence 与 TRPO 约束
  - 12. TRPO 局部二阶近似与 Natural Gradient
- 第五部分　PPO-Clip 的核心推导
  - 13. 从 KL Penalty 到 Clipped Surrogate
  - 14. 正 Advantage：好动作不能无限被奖励
  - 15. 负 Advantage：坏动作也不能无限被压低
  - 16. 为什么一定要有 `min`，而不是只用 `clip`?
  - 17. 从梯度看 PPO 在何处停止更新
  - 18. PPO 不能保证 Ratio 或 KL 被硬性约束
- 第六部分　GAE：Advantage 究竟如何估计?
  - 19. Monte Carlo Advantage 的动机与代价
  - 20. 由 Bellman 方程得到 One-Step TD Advantage

    - 21. 从 One-Step 推导 N-Step Advantage
    - 22. GAE 的指数权重与求和交换推导
    - 23. GAE 的 $O(T)$ 反向递推
    - 24. 用 GAE 构造 Critic 训练目标
- 第七部分　完整 Actor–Critic 损失
    - 25. Actor Loss
    - 26. Critic Loss
    - 27. Entropy Bonus 的动机
    - 28. PPO 的最终训练目标
- 第八部分　连续动作 PPO 与机器人控制
    - 29. Gaussian Policy 的对数概率与 Ratio
    - 30. Gaussian Policy 的梯度如何推动动作均值?
    - 31. 机器人 Residual PPO 示例
- 第九部分　一次 PPO Iteration 的完整执行过程
    - 32. 从采样到更新的十个步骤
    - 33. 简洁且可直接改造的 PyTorch 实现
- 第十部分　超参数与实际诊断
    - 34. 常见起始超参数及其含义
- 第十一部分　与 REINFORCE、Actor–Critic、TRPO 的关系
    - 35. 算法演化：每一步解决一个问题
- 第十二部分　最终公式总览与研究启示
    - 参考文献与延伸阅读

章节知识地图



第一部分　从预期回报到策略梯度

1. 定义强化学习优化问题

考虑马尔可夫决策过程（MDP）：

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma, \rho_0).$$

其中 $s_t \in \mathcal{S}$ 为状态，$a_t \in \mathcal{A}$ 为动作，$P(s_{t+1} \mid s_t, a_t)$ 为环境转移核，r_t 为奖励，ρ_0 是初始状态分布，$\pi_\theta(a \mid s)$ 是参数为 θ 的随机策略。学习目标是

$$\boxed{\theta^* = \arg\max_\theta J(\theta)}, \qquad \boxed{J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]}. \tag{1}$$

轨迹记为 $\tau = (s_0, a_0, s_1, a_1, \ldots)$，其折扣回报为 $R(\tau) = \sum_t \gamma^t r_t$。于是

$$J(\theta) = \int p_\theta(\tau) R(\tau)\, d\tau.$$

动机与困难：θ 首先改变动作分布，再通过环境动力学影响整条未来轨迹：

$$\theta \longrightarrow a_0 \longrightarrow s_1 \longrightarrow a_1 \longrightarrow \cdots \longrightarrow R(\tau).$$

对于碰撞、摩擦、接触切换和未知动力学，$R(\tau)$ 一般不能简单地通过环境对 θ 反向传播。因此需要一种**不要求环境动力学可微**的梯度估计方法。

2. 利用 Log-Derivative Trick 推导 Policy Gradient

对期望求梯度：

$$\nabla_\theta J(\theta) = \int \nabla_\theta p_\theta(\tau) R(\tau)\, d\tau.$$

使用恒等式

$$\boxed{\nabla_\theta p_\theta(\tau) = p_\theta(\tau) \nabla_\theta \log p_\theta(\tau)}, \tag{2}$$

得到

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\left[R(\tau) \nabla_\theta \log p_\theta(\tau) \right]}. \tag{3}$$

这称为 Likelihood-Ratio / Score-Function Trick。它把**概率密度的梯度**转为**对数概率的梯度**。下一个关键点是：MDP 轨迹概率恰好可以分解。

$$p_\theta(\tau) = \rho_0(s_0) \prod_{t=0}^{\infty} \pi_\theta(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t).$$

取对数并求导。如果初始分布、环境转移核不依赖 θ，则

$$\begin{aligned} \log p_\theta(\tau) &= \log \rho_0(s_0) + \sum_t \log \pi_\theta(a_t \mid s_t) + \sum_t \log P(s_{t+1} \mid s_t, a_t), \\ \boxed{\nabla_\theta \log p_\theta(\tau)} &= \boxed{\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)}. \end{aligned} \tag{4}$$

所以形式上有

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}\left[R(\tau) \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) \right]}. \tag{5}$$

这里严格处理无穷时域时需要适当的可积性与极限交换条件；也可以先在有限时域推导后取极限。

这一转换解决了什么？ 我们只需要对策略网络的 $\log \pi_\theta(a_t \mid s_t)$ 求梯度。机器人动力学和奖励仍然只用于前向采样，不必可微。

3. 因果性与 Reward-to-Go

式 (5) 的每个动作梯度都乘以整条轨迹回报。但 a_t 不能影响发生在 t 以前的奖励。利用 score function 的零均值性质，过去奖励对应的交叉项期望为零，于是可改写成

$$G_t = \sum_{k=t}^{\infty} \gamma^{k-t} r_k, \qquad \boxed{\nabla_\theta J(\theta) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t \nabla_\theta \log \pi_\theta(a_t \mid s_t) G_t\right]}. \tag{6}$$

动机：对 a_t 的更新应由它能够影响的未来后果决定。去除过去奖励通常可以降低估计方差，而不改变梯度期望。式中的 γ^t 与原始折扣目标严格一致；实际 PPO 代码经常直接对采样时间步平均，属于相应采样分布下的优化约定，不应混为完全相同的无穷时域表达式。

第二部分　Baseline、Value 与 Advantage

4. 为什么可以减去 Baseline?

虽然使用 Reward-to-Go 比整条轨迹回报更合理，但 G_t 仍会因未来随机性而显著波动。考虑引入只依赖状态的 Baseline $b(s_t)$：

$$g_t = \nabla_\theta \log \pi_\theta(a_t \mid s_t)\,[G_t - b(s_t)].$$

证明它不改变策略梯度的期望：

$$\begin{aligned} &\mathbb{E}_{a\sim\pi_\theta(\cdot|s)}[\nabla_\theta \log \pi_\theta(a \mid s) b(s)] \\ &= b(s) \int \pi_\theta(a \mid s) \nabla_\theta \log \pi_\theta(a \mid s)\, da \\ &= b(s) \int \nabla_\theta \pi_\theta(a \mid s)\, da \\ &= b(s) \nabla_\theta \int \pi_\theta(a \mid s)\, da \\ &= b(s) \nabla_\theta 1 = 0. \end{aligned} \tag{7}$$

因此

$$\boxed{\mathbb{E}[\nabla \log \pi_\theta\, G_t] = \mathbb{E}[\nabla \log \pi_\theta\,(G_t - b(s_t))]}. \tag{8}$$

动机：去除与“当前动作相对质量”无关的整体回报水平，从而减小随机波动。要注意：不是任何 Baseline 都能保证减小方差；严格的最优方差 Baseline 通常还与 score gradient 的平方范数有关。$V^\pi(s)$ 是实用而自然的选择。

5. 为什么使用 Value Function 作为 Baseline?

定义状态价值、动作价值和优势函数：

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi[G_t \mid s_t = s], \\ Q^\pi(s,a) &= \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a], \\ \boxed{A^\pi(s,a)} &= \boxed{Q^\pi(s,a) - V^\pi(s)}. \end{aligned} \tag{9}$$

$V^\pi(s)$ 衡量“从该状态按现有策略继续执行的平均回报”；$Q^\pi(s,a)$ 衡量“先执行指定动作再按策略继续执行的平均回报”。因此：

- $A^\pi(s,a) > 0$：这个动作比当前策略的平均动作更好，应倾向于提高其概率；
- $A^\pi(s,a) < 0$：这个动作比平均动作更差，应倾向于降低其概率；
- $A^\pi(s,a) = 0$：在该估计下没有相对优劣信号。

机器人示例：在同一个抓取状态，设 $V^\pi(s) = 10$。接触调整动作的 $Q = 15$，Advantage 为 $+5$；保持姿态的 $Q = 10$，Advantage 为 0；错误闭合手指的 $Q = 3$，Advantage 为 -7。PPO 学习的是这些动作**相对平均表现**的概率变化，而不是简单追求所有正奖励动作。

在准确的 Q^π, V^π 下，得到 Advantage Policy Gradient：

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}\left[\sum_t \gamma^t \nabla_\theta \log \pi_\theta(a_t \mid s_t) A^{\pi_\theta}(s_t, a_t)\right]}. \tag{10}$$

从而引出 Actor–Critic：Actor π_θ 负责产生动作；Critic V_ϕ 近似状态价值；Advantage 连接二者。使用不准确的 Value Baseline 本身不会改变理想 score-function 公式的期望，但使用自举构造 Advantage 时会引入额外估计偏差。

第三部分　性能差分、Surrogate Objective 与重要性采样

6. 旧数据为什么不能无限重复使用？

Policy Gradient 假设采样数据与当前策略一致。若使用 π_{θ_0} 采样后更新到 π_{θ_1}、π_{θ_2}，旧 Rollout 依然服从 π_{θ_0}，并不服从新策略。每次梯度步都重新采样又非常昂贵。

PPO 的目标之一是：**在一批由旧策略采集的数据上，进行多次相对安全的一阶更新**。要做到这一点，必须区分状态分布的变化和给定状态下动作分布的变化。

7. Performance Difference Lemma 的逐步证明

定义归一化折扣状态访问分布：

$$\boxed{d_\pi(s) = (1-\gamma)\sum_{t=0}^{\infty} \gamma^t \Pr(s_t = s \mid \pi)}. \tag{11}$$

对连续状态空间，上式应理解为对应概率测度/密度。对于任意新旧策略，性能差分引理给出**精确恒等式**：

$$\boxed{J(\pi_\theta) - J(\pi_{\text{old}}) = \frac{1}{1-\gamma}\mathbb{E}_{\substack{s\sim d_{\pi_\theta} \\ a\sim\pi_\theta}}[A^{\pi_{\text{old}}}(s,a)]}. \tag{12}$$

证明：由旧策略的 Bellman 方程，

$$A^{\pi_{\text{old}}}(s_t,a_t) = \mathbb{E}[r_t + \gamma V^{\pi_{\text{old}}}(s_{t+1}) - V^{\pi_{\text{old}}}(s_t) \mid s_t, a_t].$$

沿**新策略**的轨迹求折扣和：

$$\begin{aligned}
&\mathbb{E}_{\pi_\theta}\left[\sum_t \gamma^t A^{\pi_{\text{old}}}(s_t,a_t)\right] \\
&= \mathbb{E}_{\pi_\theta}\left[\sum_t \gamma^t r_t + \sum_t \gamma^{t+1} V^{\pi_{\text{old}}}(s_{t+1}) - \sum_t \gamma^t V^{\pi_{\text{old}}}(s_t)\right].
\end{aligned}$$

后两个级数望远镜相消，只剩 $-V^{\pi_{\text{old}}}(s_0)$（假设末端项消失），因此

$$\mathbb{E}_{\pi_\theta}\left[\sum_t \gamma^t A^{\pi_{\text{old}}}(s_t,a_t)\right] = J(\pi_\theta) - J(\pi_{\text{old}}).$$

再用 d_{π_θ} 的定义重写折扣时间求和，即得到式 (12)。

重要直觉：新策略的回报提升，等于它在自身将访问的状态、执行的动作上，取得的“相对于旧策略”的 Advantage 总和。

8. 将真实性能改进近似为 Surrogate Objective

式 (12) 的困难在于，期望中的状态来自新策略：

$$s \sim d_{\pi_\theta}.$$

但我们手上只有旧策略数据。如果新旧策略足够接近，可以作**局部近似**：

$$d_{\pi_\theta}(s) \approx d_{\pi_{\text{old}}}(s).$$

忽略正常数 $1/(1-\gamma)$ 后，定义

$$\boxed{L_{\text{sur}}(\theta) = \mathbb{E}_{\substack{s\sim d_{\pi_{\text{old}}} \\ a\sim\pi_\theta}}\left[A^{\pi_{\text{old}}}(s,a)\right]}. \tag{13}$$

它比直接优化新策略的整条轨迹分布容易得多：状态分布固定，只需要研究给定状态下动作概率的变化。在旧参数附近，准确 Advantage 对应的 Surrogate 与真实目标具有相同的一阶局部方向（差一个正常数）。

核心前提：只有当 π_θ 不远离 π_{old} 时，这种状态分布替换才比较可信。这为后面的 Trust Region 和 Clipping 提供了动机。

9. Importance Sampling 推导 Probability Ratio

式 (13) 仍然包含 $a \sim \pi_\theta$，而数据中的动作来自旧策略。给定状态 s，利用重要性采样：

$$\begin{aligned}\mathbb{E}_{a\sim\pi_\theta}[A^{\pi_{\text{old}}}(s,a)] &= \int \pi_\theta(a\mid s) A^{\pi_{\text{old}}}(s,a)\,da \\ &= \int \pi_{\text{old}}(a\mid s)\frac{\pi_\theta(a\mid s)}{\pi_{\text{old}}(a\mid s)} A^{\pi_{\text{old}}}(s,a)\,da.\end{aligned}$$

定义概率比：

$$\boxed{r_\theta(s,a) = \frac{\pi_\theta(a\mid s)}{\pi_{\text{old}}(a\mid s)}}. \tag{14}$$

于是得到 CPI（Conservative Policy Iteration）形式的代理目标：

$$\boxed{L^{\text{CPI}}(\theta) = \mathbb{E}_{\substack{s\sim d_{\pi_{\text{old}}} \\ a\sim\pi_{\text{old}}}}\left[r_\theta(s,a)A^{\pi_{\text{old}}}(s,a)\right]}. \tag{15}$$

实际估计写为 $\mathbb{E}_t[r_t(\theta)\hat{A}_t]$。

概率比的解释：若旧动作概率为 0.2、新概率为 0.3，则 $r=1.5$；若新概率变为 0.1，则 $r=0.5$。正 Advantage 鼓励 r 上升，负 Advantage 鼓励 r 下降。

边界与限制：上述重要性采样要求新策略对旧策略概率为零的动作不赋正概率（绝对连续性）；对于一般 Gaussian 分布，通常满足动作密度支持条件。重要性采样仅精确修正了**给定状态下的动作分布**，并未修正 d_{π_θ} 与 $d_{\pi_{\text{old}}}$ 的差异。因此，它没有消除式 (13) 的状态分布近似误差。

第四部分　TRPO：用 Trust Region 控制更新

10. 直接最大化 CPI 为什么危险?

对于正 Advantage 样本，$r_t\hat{A}_t$ 随 r_t 增长：若 $\hat{A}=5$，$r=1.2,2,10$ 对应 $6,10,50$。代理目标似乎鼓励无限放大好动作的概率。但这些 Advantage 是根据旧策略计算的；如果新策略改变很大，旧估计不一定再可信。

尤其在机器人操作中，动作分布的一次较大改变可能引发接触切换、掉落甚至完全不同的未来状态访问。故 L^{CPI} 增大**不保证**真实 J 增大。

11. KL Divergence 与 TRPO 约束

使用 KL Divergence 衡量同一状态下两策略的分布差异:

$$D_{\text{KL}}(\pi_{\text{old}}\|\pi_\theta)=\mathbb{E}_{a\sim\pi_{\text{old}}}\left[\log\frac{\pi_{\text{old}}(a\mid s)}{\pi_\theta(a\mid s)}\right].$$

TRPO 的实用优化形式是

$$\boxed{\begin{aligned}\max_\theta\quad & \mathbb{E}_t[r_t(\theta)\hat{A}_t]\\ \text{s.t.}\quad & \mathbb{E}_{s\sim d_{\pi_{\text{old}}}}[D_{\text{KL}}(\pi_{\text{old}}(\cdot\mid s)\|\pi_\theta(\cdot\mid s))]\le\delta.\end{aligned}}\tag{16}$$

δ 决定信赖域大小。其动机不是“策略改变越小越好”，而是**在一个代理目标足够可信的局部区域内寻找改进**。

TRPO 的理论分析给出含有最大状态策略差异惩罚项的性能下界，典型结构为

$$J(\pi_\theta)\ge J(\pi_{\text{old}})+\frac{L_{\text{sur}}(\theta)}{1-\gamma}-C\max_s D_{\text{KL}}(\pi_{\text{old}}\|\pi_\theta),\tag{17}$$

其中 C 依赖回报/Advantage 的界和 γ；具体常数以及 KL 方向取决于采用的界。**注意**：理论中的最坏状态差异控制，强于工程实现的平均 KL 约束；实际神经网络 TRPO 不能据此保证每次更新单调提升。

12. TRPO 局部二阶近似与 Natural Gradient

令 $\Delta\theta=\theta-\theta_{\text{old}}$。在旧参数附近进行泰勒展开:

$$L^{\text{CPI}}(\theta)\approx L^{\text{CPI}}(\theta_{\text{old}})+g^\top\Delta\theta,\quad g=\nabla_\theta L^{\text{CPI}}\big|_{\theta_{\text{old}}}.$$

KL 在相同策略处为零且一阶梯度为零，二阶近似为

$$\bar{D}_{\text{KL}}\approx\tfrac{1}{2}\Delta\theta^\top F\Delta\theta,$$

其中 F 是在旧参数处计算的 Fisher 信息矩阵。为避免歧义，先定义 $u_{\text{old}}(s,a)=\nabla_\theta\log\pi_\theta(a\mid s)|_{\theta=\theta_{\text{old}}}$，则

$$F=\mathbb{E}_{s\sim d_{\pi_{\text{old}}},\,a\sim\pi_{\text{old}}}[u_{\text{old}}(s,a)u_{\text{old}}(s,a)^\top].$$

局部问题变成

$$\boxed{\max_{\Delta\theta} g^\top \Delta\theta \quad \text{s.t.} \quad \tfrac{1}{2}\Delta\theta^\top F \Delta\theta \le \delta.} \tag{18}$$

写出 Lagrangian：

$$\mathcal{L}(\Delta\theta, \eta) = g^\top \Delta\theta - \eta\big(\tfrac{1}{2}\Delta\theta^\top F \Delta\theta - \delta\big).$$

一阶条件 $g - \eta F \Delta\theta = 0$ 导致 $\Delta\theta = \eta^{-1} F^{-1} g$。代入活动约束（假设 F 正定、$g \neq 0$）得

$$\boxed{\Delta\theta^* = \sqrt{\frac{2\delta}{g^\top F^{-1} g}}\, F^{-1} g.} \tag{19}$$

方向 $F^{-1}g$ 正是 Natural Policy Gradient 的核心形式。实际训练不能显式反演一个巨大矩阵，因此需要 Fisher-vector product、共轭梯度、线搜索等技术；非正定或病态情况下还需阻尼。**PPO 的设计动机**：保留保守策略更新的思想，但避免 TRPO 的复杂二阶/约束优化。

第五部分　PPO-Clip 的核心推导

13. 从 KL Penalty 到 Clipped Surrogate

最直接的一阶化方式是 PPO-Penalty：

$$L^{\text{KLPEN}}(\theta) = \mathbb{E}_t[r_t(\theta)\hat{A}_t] - \beta\, \mathbb{E}_t[D_{\text{KL}}(\pi_{\text{old}} \| \pi_\theta)]. \tag{20}$$

但 β 太小会限制不足，太大则更新过慢，还需要调整罚系数。PPO-Clip 改为限制“过大的概率比带来的额外目标收益”。设

$$\text{clip}(r, 1-\epsilon, 1+\epsilon) = \min\{1+\epsilon, \max\{r, 1-\epsilon\}\}.$$

最常用的 PPO 目标为

$$\boxed{L^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\Big(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\Big)\Big].} \tag{21}$$

常见起点为 $\epsilon = 0.1 \sim 0.2$，但应当根据环境、动作维度和监测的 KL 调整。**这里的 `min` 与 `clip` 必须一起理解**。

14. 正 Advantage：好动作不能无限被奖励

若 $\hat{A}_t > 0$，希望提高该动作概率，即使 $r_t > 1$。式 (21) 化简为

$$\boxed{L_t^{\text{CLIP}} = \min(r_t, 1+\epsilon)\hat{A}_t = \begin{cases} r_t \hat{A}_t, & r_t \le 1+\epsilon, \\ (1+\epsilon)\hat{A}_t, & r_t > 1+\epsilon. \end{cases}} \tag{22}$$

设 $\hat{A}_t = 2, \epsilon = 0.2$：

r_t	原始 $r_t\hat{A}_t$	PPO 目标
0.8	1.6	1.6
1.0	2.0	2.0
1.1	2.2	2.2

r_t	原始 $r_t\hat{A}_t$	PPO 目标
1.2	2.4	2.4
1.5	3.0	2.4
2.0	4.0	2.4

当 $r_t > 1.2$ 时，这个样本不再因为概率继续上升而增加代理目标，梯度在此方向上为零。**设计动机**：避免对看起来“好”的旧策略样本过度提高概率。

15. 负 Advantage：坏动作也不能无限被压低

若 $\hat{A}_t < 0$，希望降低动作概率，即降低 r_t。由于乘以负数会反转大小关系，

$$\boxed{L_t^{\text{CLIP}} = \max(r_t, 1-\epsilon)\hat{A}_t = \begin{cases}(1-\epsilon)\hat{A}_t, & r_t < 1-\epsilon, \\ r_t\hat{A}_t, & r_t \geq 1-\epsilon.\end{cases}} \tag{23}$$

设 $\hat{A}_t = -2, \epsilon = 0.2$：

r_t	原始 $r_t\hat{A}_t$	PPO 目标
0.5	-1.0	-1.6
0.7	-1.4	-1.6
0.8	-1.6	-1.6
1.0	-2.0	-2.0
1.2	-2.4	-2.4
1.5	-3.0	-3.0

PPO 最大化目标，因此负 Advantage 时降低 r 有利；但当 $r < 0.8$ 时不再额外奖励进一步压低该动作的概率。

16. 为什么一定要有 `min`，而不是只用 `clip`?

假设只有 $\text{clip}(r, 1-\epsilon, 1+\epsilon)\hat{A}$。若 $\hat{A} > 0$、$r = 0.5$、$\epsilon = 0.2$，说明好动作的概率已经被错误地大幅降低，但单独的 `clip` 会输出 $0.8\hat{A}$，在 $r < 0.8$ 区域梯度为零，无法纠正错误方向。

加入 `min` 后，由于 $r\hat{A} < 0.8\hat{A}$，实际使用原始项 $r\hat{A}$，因此仍有正梯度推动 r 回升。负 Advantage 下的错误方向同理。

更新方向正确但幅度过大：	停止提供额外优化收益；
更新方向不利于 Advantage：	保留纠偏梯度。

而且由 $\min(x, y) \leq x$，逐样本成立 $L^{\text{CLIP}} \leq L^{\text{CPI}}$：这是对**未裁剪代理目标**的保守下界，**不等于对真实回报 J 的严格下界**。

17. 从梯度看 PPO 在何处停止更新

因为旧策略分母在本次优化中不变，

$$\boxed{\nabla_\theta r_t(\theta) = r_t(\theta)\nabla_\theta \log \pi_\theta(a_t \mid s_t)}. \tag{24}$$

忽略折点处的不可微性（优化器采用次梯度），单样本梯度为

$$\boxed{\nabla_\theta L_t^{\mathrm{CLIP}} = \begin{cases} 0, & \hat{A}_t > 0,\ r_t > 1+\epsilon, \\ 0, & \hat{A}_t < 0,\ r_t < 1-\epsilon, \\ \hat{A}_t r_t \nabla_\theta \log \pi_\theta(a_t \mid s_t), & \text{其余区域}. \end{cases}} \tag{25}$$

当 $\theta = \theta_{\mathrm{old}}$，$r_t = 1$，因此初始局部梯度与 CPI/Advantage Policy Gradient 一致（差相应归一化常数）。**PPO 不改变初始策略改进方向，而是在重复更新、概率比逐渐偏离 1 时抑制过大更新收益**。

18. PPO 不能保证 Ratio 或 KL 被硬性约束

$\epsilon = 0.2$ **不意味着**更新后始终有 $0.8 \le r_t \le 1.2$。Clipping 修改的是损失，而不是把策略投影到可行集合。共享神经网络参数会同时影响其他状态和样本；多个 Mini-batch Epoch 后，平均 KL 依然可能过大。

因此常在训练中监测

$$\bar{D}_{\mathrm{KL}}(\pi_{\mathrm{old}} \| \pi_\theta)$$

并使用 Target-KL Early Stopping，必要时结合学习率调整、Gradient Clipping。需要分清三种裁剪：**Ratio Clipping** 控制代理收益；**Gradient Clipping** 控制梯度范数；**Action Clipping** 限制真实执行动作范围。三者不是一回事。

第六部分　GAE：Advantage 究竟如何估计?

19. Monte Carlo Advantage 的动机与代价

PPO 中使用的 $\hat{A}_t$ 不是凭空给定。按定义 $A^\pi = Q^\pi - V^\pi$，最直接的方法是把样本回报 G_t 作为 Q^π 的样本估计：

$$\boxed{\hat{A}_t^{\mathrm{MC}} = G_t - V_\phi(s_t)}, \qquad G_t = \sum_{l=0}^{T-t-1} \gamma^l r_{t+l}. \tag{26}$$

对完整终止轨迹，在正确策略分布下，G_t 的条件期望等于 $Q^\pi(s_t, a_t)$。它不需要用 Critic 对未来进行自举，因此目标相对直接。然而，如果抓取任务直到第 100 步成功才给出奖励，早期动作的 G_t 会受到大量后续随机因素影响，梯度方差可能很大。

问题：能否尽早利用较少步数的信息估计 Advantage，而不依赖整条未来轨迹?

20. 由 Bellman 方程得到 One-Step TD Advantage

Bellman 关系为

$$Q^\pi(s_t, a_t) = \mathbb{E}[r_t + \gamma V^\pi(s_{t+1}) \mid s_t, a_t].$$

因此，一步 TD Residual：

$$\boxed{\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)} \tag{27}$$

可用作 Advantage 估计：

$$\boxed{\hat{A}_t^{(1)} = \delta_t}. \tag{28}$$

它只需要一个转移样本 s_t, a_t, r_t, s_{t+1}。未来随机回报被 $V_\phi(s_{t+1})$ 代替，通常可以降低方差；代价是 Critic 不准确时会引入自举偏差。对于真正终止状态，未来价值为零；对于人为时间截断，则通常仍需 Bootstrap。

估计器	主要优点	主要缺点
Monte Carlo	减少未来价值自举偏差	长期回报导致高方差
One-Step TD	方差较低、反馈及时	更依赖 Critic 的预测准确性

21. 从 One-Step 推导 N-Step Advantage

一步估计：

$$\hat{A}_t^{(1)} = r_t + \gamma V(s_{t+1}) - V(s_t) = \delta_t.$$

两步估计：

$$\begin{aligned} \hat{A}_t^{(2)} &= r_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2}) - V(s_t) \\ &= [r_t + \gamma V(s_{t+1}) - V(s_t)] \\ &\quad + \gamma[r_{t+1} + \gamma V(s_{t+2}) - V(s_{t+1})] \\ &= \boxed{\delta_t + \gamma \delta_{t+1}}. \end{aligned} \tag{29}$$

三步估计同理：

$$\hat{A}_t^{(3)} = \delta_t + \gamma \delta_{t+1} + \gamma^2 \delta_{t+2}.$$

推广到 n 步：

$$\boxed{\hat{A}_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l \delta_{t+l}}. \tag{30}$$

较小 n 更多依赖 Value Bootstrap，较大 n 更多依赖真实未来奖励。**GAE 的出发点**：不要在单一的 n 上做硬选择，而是平滑组合多个 N-Step 估计器。

22. GAE 的指数权重与求和交换推导

令 $0 \le \lambda < 1$，定义

$$\boxed{\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \hat{A}_t^{(n)}}. \tag{31}$$

这里权重总和为 1：$(1-\lambda)\sum_{n\ge 1} \lambda^{n-1} = 1$。代入 N-Step 形式：

$$
\begin{aligned}
\hat{A}_t^{\mathrm{GAE}} &= (1-\lambda)\sum_{n=1}^{\infty}\lambda^{n-1}\sum_{l=0}^{n-1}\gamma^l\delta_{t+l} \\
&= \sum_{l=0}^{\infty}\gamma^l\delta_{t+l}(1-\lambda)\sum_{n=l+1}^{\infty}\lambda^{n-1} \\
&= \sum_{l=0}^{\infty}\gamma^l\delta_{t+l}\lambda^l.
\end{aligned}
$$

因此：

$$
\boxed{\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty}(\gamma\lambda)^l\delta_{t+l}}. \tag{32}
$$

也就是：

$$
\hat{A}_t = \delta_t + \gamma\lambda\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \cdots.
$$

两个极端：$\lambda = 0$ 时只剩 δ_t，即 One-Step TD；$\lambda \to 1$ 时趋于所有后续 TD Residual 之和。对于完整终止轨迹，$\lambda = 1$ 对应 $G_t - V(s_t)$。实用 PPO 常把 $\gamma = 0.99, \lambda = 0.95$ 作为起点。

GAE 并非数学上保证任何情况下方差都更低。它提供可调的偏差—方差折中：λ 较小通常更依赖 Critic，λ 较大通常更依赖长期采样回报。

(" & 的 $O(T)$ 反向递推

式 (32) 具有递归结构：

$$
\boxed{\hat{A}_t = \delta_t + \gamma\lambda\hat{A}_{t+1}}. \tag{33}
$$

若 $d_t = 1$ 表示 MDP 在该步真正终止：

$$
\boxed{\begin{aligned}
&\delta_t = r_t + \gamma(1-d_t)V_\phi(s_{t+1}) - V_\phi(s_t), \\
&\hat{A}_t = \delta_t + \gamma\lambda(1-d_t)\hat{A}_{t+1}.
\end{aligned}} \tag{34}
$$

这样从 $T-1$ 向前循环一次就能完成全部计算，复杂度为 $O(T)$。固定长度 Rollout 的最后一个状态若并未真正终止，应使用 $V(s_T)$ Bootstrap；而递归不能跨越一次 Episode Reset 错误地把下一条 Episode 的 Advantage 累加进来。实现时应区分 **terminated（真实终止）**、**truncated（外部时间限制）**和 **Rollout Boundary（采样段边界）**。

用 (" & 构造 $SLUDÀ 练目标

GAE 给出 Advantage 估计：

$$
\hat{A}_t \approx Q^{\pi_{\mathrm{old}}}(s_t, a_t) - V_{\mathrm{old}}(s_t).
$$

自然定义 TD(λ) Return：

$$
\boxed{\hat{R}_t^{\lambda} = \hat{A}_t^{\mathrm{GAE}} + V_{\phi_{\mathrm{old}}}(s_t)}. \tag{35}
$$

这里 $V_{\phi_{\mathrm{old}}}$ 是计算这一批 Rollout 目标时保存的旧 Value，整个 PPO 更新阶段必须视为常量。这一技巧让 Actor 的 Advantage 估计与 Critic 的回归目标保持一致。

第七部分　完整 Actor–Critic 损失

25. Actor Loss

本轮旧策略 Rollout 上，先计算并固定 $\hat{A}_t$（实践中经常标准化）。Actor 最大化

$$L^{\mathrm{CLIP}}(\theta)=\mathbb{E}_t\left[\min\left(r_t\hat{A}_t,\mathrm{clip}(r_t,1-\epsilon,1+\epsilon)\hat{A}_t\right)\right].$$

在 PyTorch 等以最小化为接口的框架中：

$$\boxed{\mathcal{L}_{\mathrm{actor}}=-L^{\mathrm{CLIP}}}. \tag{36}$$

如果 $\hat{A}_t$ 通过当前可训练 Critic 的计算图产生，应在 Actor Loss 中对其 `detach`，避免无意把 Actor 更新传播到 Advantage 目标。

26. Critic Loss

Critic 通过回归 $\hat{R}_t^{\lambda}$ 学习状态价值：

$$\boxed{\mathcal{L}_{\mathrm{critic}}(\phi)=\mathbb{E}_t[(V_\phi(s_t)-\hat{R}_t^{\lambda})^2]}. \tag{37}$$

也可以直接回归 Monte Carlo Return。某些实现另设 Value Clipping 或 $\frac{1}{2}$ 系数，但它们不是 PPO-Clip 目标的必要组成。Critic 过差会导致 Advantage 的符号或量级不准确，因此 PPO 的训练稳定性很大程度依赖 Value Function 的估计质量。

27. Entropy Bonus 的动机

策略可能过早收缩到狭窄动作分布，停止尝试其他抓取方式。使用策略熵

$$\boxed{H(\pi_\theta(\cdot\mid s))=-\mathbb{E}_{a\sim\pi_\theta}[\log\pi_\theta(a\mid s)]}. \tag{38}$$

鼓励合理探索。其平均值记为 $L^{\mathrm{entropy}}=\mathbb{E}_t[H(\pi_\theta(\cdot\mid s_t))]$。高熵并不总是更好，尤其是机器人部署阶段；Entropy Coefficient 可能取零，也可能随训练退火。

28. PPO 的最终训练目标

把 Actor、Critic、Entropy 组合，得到一个常见的联合最大化目标：

$$\boxed{L^{\mathrm{PPO}}(\theta,\phi)=L^{\mathrm{CLIP}}(\theta)-c_v\mathcal{L}_{\mathrm{critic}}(\phi)+c_eL^{\mathrm{entropy}}(\theta)}. \tag{39}$$

对应的**最小化**形式为

$$\boxed{\mathcal{L}_{\mathrm{total}}=-L^{\mathrm{CLIP}}+c_v\mathcal{L}_{\mathrm{critic}}-c_eL^{\mathrm{entropy}}}. \tag{40}$$

- c_v：控制 Value 回归损失权重；
- c_e：控制 Entropy Bonus 权重；
- 独立 Actor 与 Critic 参数时可分别更新；共享网络时需要考虑梯度耦合。

常见实现对每个训练 Mini-batch 做多轮更新，但旧 Log Probability、Advantage 和 Return Target 在该 Rollout 的 PPO Epoch 内保持固定。

第八部分　连续动作 PPO 与机器人控制

29. Gaussian Policy 的对数概率与 Ratio

机器人动作通常是连续向量，例如关节位置增量、目标速度或关节力矩。典型 Actor 使用对角 Gaussian：

$$\boxed{\pi_\theta(a \mid s) = \mathcal{N}(\mu_\theta(s), \operatorname{diag}(\sigma_1^2, \dots, \sigma_n^2))}. \tag{41}$$

则联合 Log Probability 为

$$\boxed{\log \pi_\theta(a \mid s) = -\frac{1}{2} \sum_{i=1}^{n} \left[\frac{(a_i - \mu_i)^2}{\sigma_i^2} + 2 \log \sigma_i + \log(2\pi) \right]}. \tag{42}$$

数值稳定的 Ratio 计算为

$$\boxed{r_t = \exp\left(\log \pi_\theta(a_t \mid s_t) - \log \pi_{\text{old}}(a_t \mid s_t)\right)}. \tag{43}$$

实现重点：先把各动作维度的 Log Probability 求和，得到整个动作向量的联合 Log Probability，再指数化算 Ratio；不能对各维 Ratio 分别 Clip 后当作标准 PPO 联合策略目标。若使用 Tanh-Squashed Gaussian，需按变量变换计算正确的 Log Probability；若环境直接将采样动作截断到硬边界，也要审慎区分“采样动作”和“实际执行动作”的密度。

30. Gaussian Policy 的梯度如何推动动作均值?

考虑一维固定方差模型：

$$a \sim \mathcal{N}(\mu_\theta(s), \sigma^2),$$

$$\boxed{\frac{\partial \log \pi_\theta(a \mid s)}{\partial \mu_\theta} = \frac{a - \mu_\theta(s)}{\sigma^2}}. \tag{44}$$

因此未裁剪区域的 Policy Gradient 对均值参数的主要贡献为

$$\boxed{\nabla_\theta J \propto \mathbb{E}\left[\hat{A} \frac{a - \mu_\theta(s)}{\sigma^2} \nabla_\theta \mu_\theta(s) \right]}. \tag{45}$$

当 $\hat{A} > 0$，更新推动分布均值朝采样动作移动；当 $\hat{A} < 0$，更新推动均值远离该动作。PPO 的 Clipping 再限制单个样本从此类概率变化中获得的额外收益。若 σ 可训练，还有关于方差的梯度项。

31. 机器人 Residual PPO 示例

假设有基础轨迹跟随控制器

$$a_t^{\text{base}} = \pi_{\text{base}}(s_t),$$

另外用 PPO 学习随机残差

$$\Delta a_t \sim \pi_\theta(\cdot \mid s_t),$$

最终执行

$$\boxed{a_t = a_t^{\text{base}} + \Delta a_t}. \tag{46}$$

残差可表示关节目标位置、速度、末端位姿或手指闭合动作的修正。在抓取并旋转螺丝刀任务中，基础轨迹负责大体动作，PPO Residual 学习补偿接触位置误差、初始姿态扰动、摩擦系数变化和抓取滑动。

此时要用**残差动作分布**计算

$$r_t = \frac{\pi_\theta(\Delta a_t \mid s_t)}{\pi_{\text{old}}(\Delta a_t \mid s_t)}.$$

如果残差经过缩放、非线性变换或安全过滤，必须明确哪些变量的密度用于策略优化、哪些是实际控制命令。PPO 不要求接触动力学可微，但需要可重复的环境 Rollout、合理奖励和足够准确的状态/价值估计。

第九部分　一次 PPO Iteration 的完整执行过程

32. 从采样到更新的十个步骤

1. **冻结旧策略**：令 $\pi_{\text{old}} = \pi_{\theta_k}$，用它采集若干 Episode 或定长 Rollout。
2. **保存采样信息**：记录 (s_t, a_t, r_t, s_{t+1})、终止标记、$\log \pi_{\text{old}}(a_t \mid s_t)$ 和 $V_{\text{old}}(s_t)$。
3. **计算 TD Error**：$\delta_t = r_t + \gamma V_{\text{old}}(s_{t+1}) - V_{\text{old}}(s_t)$，对真实终止进行正确 Mask。
4. **反向计算 GAE**：$\hat{A}_t = \delta_t + \gamma\lambda \hat{A}_{t+1}$，同时正确处理 Episode Boundary。
5. **计算 Critic Target**：$\hat{R}_t^{\lambda} = \hat{A}_t + V_{\text{old}}(s_t)$，之后固定该目标。
6. **可选标准化 Advantage**：$\tilde{A}_t = (\hat{A}_t - \mu_A)/(\sigma_A + \varepsilon_{\text{num}})$。
7. **在 Mini-batch 上计算新 Log Probability**：对于已采样的**旧动作** a_t，计算当前 $\log \pi_\theta(a_t \mid s_t)$。
8. **计算 Ratio 与 Clipped Actor Loss**：$r_t = \exp(\log \pi_\theta - \log \pi_{\text{old}})$，然后应用式 (21)。
9. **更新 Actor 与 Critic**：最小化联合损失或分别优化，在同一批数据上重复若干 Epoch，监测 KL、Clip Fraction、Value Loss。
10. **重新采样**：更新完毕后，用新策略生成下一批数据；不把任意久远的旧数据当作标准 On-Policy PPO 的普通 Rollout 重复训练。

33. 简洁且可直接改造的 PyTorch 实现

下面代码给出 PPO 核心 Actor Loss。输入 `new_log_prob`、`old_log_prob` 均为**动作向量的联合** Log Probability，且旧 Log Probability 和 Advantage 在本轮更新中固定。

```
import torch


def ppo_actor_loss(new_log_prob, old_log_prob,
                   advantages, clip_eps=0.2):
    # The old log probability and advantage are fixed rollout data.
    old_log_prob = old_log_prob.detach()
    advantages = advantages.detach()

    log_ratio = new_log_prob - old_log_prob
    ratio = torch.exp(log_ratio)

    objective_unclipped = ratio * advantages
    objective_clipped = torch.clamp(
        ratio, 1.0 - clip_eps, 1.0 + clip_eps
    ) * advantages
```

```
    return -torch.minimum(
        objective_unclipped, objective_clipped
    ).mean()
```

计算 GAE 的最小后向递推如下。此示例中的 `bootstrap_mask[t]` 控制 δ_t 是否使用下一状态价值；`continue_mask[t]` 控制 GAE 是否跨越此步递归，真实终止、超时与 Rollout 切分须由环境正确提供。

```
import torch


def compute_gae(rewards, values, next_values,
                bootstrap_mask, continue_mask,
                gamma=0.99, lam=0.95):
    """All tensors are indexed by rollout time."""
    T = rewards.shape[0]
    advantages = torch.zeros_like(rewards)
    running = torch.zeros_like(rewards[0])

    for t in reversed(range(T)):
        delta = (rewards[t]
                 + gamma * bootstrap_mask[t] * next_values[t]
                 - values[t])
        running = (delta
                   + gamma * lam * continue_mask[t] * running)
        advantages[t] = running

    returns = advantages + values
    return advantages, returns
```

如果使用并行环境，这些变量可以具有形状 `[T, N_env]`；`continue_mask` 不能跨不同 Episode 串接。用于生成 `returns` 的旧 `values` 和 `next_values` 应与计算 Rollout 目标时的冻结 Critic 一致。

对于对角 Gaussian：

```
# dist: torch.distributions.Normal, action shape [batch, action_dim]
new_log_prob = dist.log_prob(actions).sum(dim=-1)
# entropy shape [batch]
entropy = dist.entropy().sum(dim=-1)
```

这段代码只展示核心公式；完整训练还需要 Rollout Buffer、环境步进、网络结构、优化器、正确的 `terminated/truncated` 处理与指标记录。

第十部分　超参数与实际诊断

34. 常见起始超参数及其含义

参数	常见起点	控制什么
γ	0.99	未来奖励的时间折扣
GAE λ	0.95	Advantage 的偏差–方差折中
Clip ϵ	0.1–0.2	过大 Probability Ratio 的额外代理收益
Actor 学习率	3×10^{-4}	Actor 优化速度

参数	常见起点	控制什么
PPO Epoch 数	3–10	一批旧数据的重复更新次数
Target KL	0.01–0.03	是否提前停止 Actor 更新
Entropy 系数	0 或小正数	探索强度
Value 权重	约 0.5	Critic 回归损失相对权重

这些是便于启动实验的经验量级，不是普适最优参数。RSL-RL 等大规模并行运控实现可能使用不同学习率、Batch 尺寸、Clip 和 Epoch 设置。

γ 与 λ 的根本区别：γ 决定优化目标如何折扣远期奖励，而 λ 决定 Advantage 估计多大程度依赖远期采样 TD 信息。二者共同以 $(\gamma\lambda)^l$ 出现在 GAE 中，但理论角色不同。

重要诊断量还包括 Episode Return、Success Rate、Value Loss、Explained Variance、Entropy、Approximate KL、Clip Fraction、梯度范数。出现“代理 Loss 看似改善、真实成功率下降”时，应重点检查数据分布漂移、Critic 误差、KL 过大和奖励设计。

第十一部分　与 REINFORCE、Actor–Critic、TRPO 的关系

35. 算法演化：每一步解决一个问题

算法	关键公式 / 想法	主要动机
REINFORCE	$\nabla \log \pi_\theta \, G$	不对环境求导也能优化策略
Advantage Actor–Critic	$\nabla \log \pi_\theta \, \hat{A}$	用状态价值作为 Baseline 降低方差
TRPO	最大化 $\mathbb{E}[r\hat{A}]$，约束 KL	保持代理目标在可信的局部区域
PPO-Clip	$\min(r\hat{A}, \mathrm{clip}(r)\hat{A})$	用简单一阶更新抑制过大概率改变的收益
PPO + GAE	$\hat{A}_t = \sum_l (\gamma\lambda)^l \delta_{t+l}$	调节 Advantage 估计的偏差和方差

PPO 本质上依然是 **On-Policy Policy Gradient / Actor–Critic**。在每轮 Rollout 的初始点，它的 Clipped Objective 与普通 CPI 代理目标的一阶梯度相同；真正的区别发生在对同一批数据继续做多次更新、Probability Ratio 逐渐偏离 1 之后。

PPO 在仿真机器人控制中常见，是因为可以并行采样许多环境、训练结构简单、不要求动力学可微，且天然支持 Gaussian 连续动作。但在真实机器人上，样本采集昂贵，SAC 等 Off-Policy 算法可能更具交互样本效率。对于 Diffusion / Flow Matching Action Policy，标准 PPO 需要的显式且可计算 Log Probability 往往不易获得，通常需要专门改造。

第十二部分　最终公式总览与研究启示

核心公式 1：策略梯度

$$\nabla_\theta J \propto \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a \mid s) A^\pi(s,a)\right].$$

动机：不必对不可微动力学反向传播。

核心公式 2：重要性采样

$$r_t = \pi_\theta(a_t \mid s_t) / \pi_{\text{old}}(a_t \mid s_t).$$

动机：在旧策略动作样本上构造新策略的局部代理目标。

核心公式 3：TRPO 的信赖域思想

$$\bar{D}_{\text{KL}}(\pi_{\text{old}} \| \pi_\theta) \le \delta.$$

动机：控制代理目标因分布变化而失真的风险。

核心公式 4：PPO Clipped Objective

$$\boxed{L^{\text{CLIP}} = \mathbb{E}_t \left[\min \left(r_t \hat{A}_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) \hat{A}_t \right) \right]}.$$

动机：保守限制过大概率变化能获得的代理收益，保留修正不利更新方向的梯度。

核心公式 5：GAE

$$\boxed{\hat{A}_t = \sum_{l \ge 0} (\gamma\lambda)^l \delta_{t+l}}, \qquad \delta_t = r_t^{\text{reward}} + \gamma V(s_{t+1}) - V(s_t).$$

动机：在长期样本回报和短期 Bootstrap 间进行平滑折中。这里用 r_t^{reward} 区分即时奖励与 PPO 的概率比 $r_t(\theta)$。

最终最小化损失：

$$\boxed{\mathcal{L}_{\text{PPO}} = -L^{\text{CLIP}} + c_v \mathcal{L}_{\text{critic}} - c_e L^{\text{entropy}}}.$$

> **研究层面的核心结论**：PPO 并不是一个新的环境梯度计算方法。它使用 Policy Gradient 确定局部性能改进方向，用 Importance Sampling 在旧数据上构造代理目标，用 Clipping 抑制过度优化，并用 GAE 改善 Advantage 估计。GAE 主要解决**梯度估计质量**的问题；Clipping 主要解决**策略更新稳定性**的问题。二者功能不同、相互补充。

参考文献与延伸阅读

1. Schulman, J. et al. (2017). *Proximal Policy Optimization Algorithms*. arXiv:1707.06347. https://arxiv.org/abs/1707.06347
2. Schulman, J. et al. (2015). *Trust Region Policy Optimization*. ICML. https://arxiv.org/abs/1502.05477
3. Schulman, J. et al. (2015). *High-Dimensional Continuous Control Using Generalized Advantage Estimation*. ICLR 2016. https://arxiv.org/abs/1506.02438
4. OpenAI Spinning Up. *PPO Documentation*. https://spinningup.openai.com/en/latest/algorithms/ppo.html

---

编写说明：正文基于前述完整 PPO 推导整理，补充了重要性采样支持条件、折扣占用分布的严格记号、PPO-Clip 并非硬约束、以及 Rollout 截断与真实终止的区别。

SAC（Soft Actor-Critic）完整推导与设计动机

课程首页 · English · PDF

项目	说明
适用范围	现代连续动作 SAC；双 Critic 与自动温度调节；伪代码为主
文档核验日期	2026-10-18
许可	CC BY 4.0 · Xiong Zeng and contributors

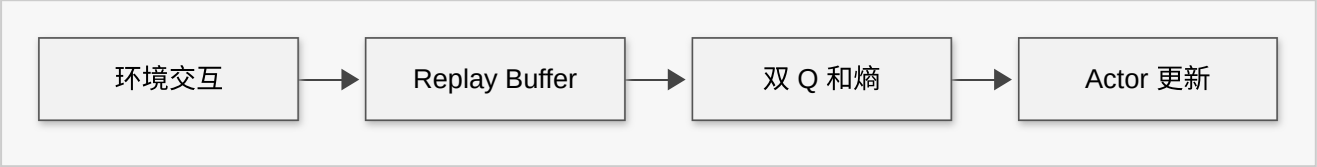
核验边界： 文档整理与上游引用快照核对；未进行教程环境、训练或硬件验证。 版本与固定源码引用见版本说明。原创教材采用 CC BY 4.0，第三方引用、图片及代码遵循其原有许可；注明来源不等于拥有再许可权。

目录

- 1. 从标准强化学习开始
- 2. 第一步：最大熵强化学习
  - 2.1 策略熵与温度参数
  - 2.2 把熵写成可采样的形式
- 3. 第二步：Soft Bellman 方程
  - 3.1 与普通 Bellman 方程对比
  - 3.2 定义 Soft V 和 Soft Q
  - 3.3 Soft Bellman Expectation Equation
- 4. 第三步：Soft Policy Improvement
  - 4.1 为什么普通贪心改进不够?
  - 4.2 变分法求 Boltzmann Policy
- 5. 第四步：把策略改进转为 KL 最小化
  - 5.1 计算 Boltzmann 分布为什么困难?
  - 5.2 为什么精确 Soft Policy Iteration 可以改进策略?
- 6. 第五步：从 Soft Policy Iteration 到 Actor-Critic
- 7. 第六步：为什么能够 Off-Policy 学习?
  - 7.1 PPO 的数据分布依赖
  - 7.2 Bellman 更新为何可以使用旧经验?
- 8. 第七步：完整推导 Critic Loss
  - 8.1 从 Soft Bellman Target 到 TD Regression
  - 8.2 Target Network：减缓目标漂移
  - 8.3 Clipped Double Q：缓解价值高估
  - 8.4 最终 Critic Target 和 Loss
- 9. 第八步：完整推导 Actor Loss
- 10. 第九步：Reparameterization Trick
  - 10.1 把依赖参数的随机采样变成可微函数

    - 10.2 Tanh-Squashed Gaussian Policy
    - 10.3 把 Actor Loss 改写为 Pathwise Objective
- 11. 第十步：Tanh Gaussian 的 Log Probability 修正
- 12. 第十一步：Automatic Entropy Tuning
    - 12.1 从固定温度到熵约束
    - 12.2 温度损失及其更新方向
    - 12.3 为什么学习 Log-Alpha?
    - 12.4 Target Entropy 的经验选择
- 13. 第十二步：完整的现代 SAC 算法
    - 13.1 模块与三个优化问题
    - 13.2 一次更新的完整步骤
    - 13.3 教学用伪代码
- 14. SAC v1 与现代 SAC 的区别
    - 14.1 原始 SAC v1：显式训练 V 网络
    - 14.2 现代 SAC：去掉显式 V 网络
- 15. SAC 与 PPO 的本质区别
    - 15.1 策略更新公式对比
    - 15.2 对比表
    - 15.3 为什么机器人仿真中 PPO 仍常见?
- 16. 最终总结：五组关键公式与理解框架
    - 最关键的 PPO / SAC 区别
- 参考资料

章节知识地图



1. 从标准强化学习开始

SAC（Soft Actor-Critic）是一种面向连续控制的 **Off-Policy、随机策略、Actor-Critic** 强化学习算法。其基本思路是：在最大化长期回报的同时鼓励策略维持适当的随机性，借助 Soft Bellman 递推学习动作价值，再用可微的 Q 函数优化 Actor。其主要组成包括最大熵目标、Soft Policy Iteration、Replay Buffer、双 Q 网络、目标网络、重参数化技巧和自动熵温度调节。

考虑折扣型 MDP:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma), \qquad 0 \le \gamma < 1.$$

其中 $s_t \in \mathcal{S}$ 是状态，$a_t \in \mathcal{A}$ 是动作，$P(s_{t+1} \mid s_t, a_t)$ 是环境转移核，$r(s_t, a_t)$ 是即时奖励，$\pi_\theta(a \mid s)$ 是参数为 θ 的策略。标准强化学习最大化:

$$\boxed{J_{\mathrm{RL}}(\pi)=\mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{\infty}\gamma^t r(s_t,a_t)\right].}$$

动机问题：只优化奖励为什么可能不够？ 以机器人抓取为例，假设当前 Critic 对三个动作给出如下估计：

候选动作	Critic 估计的 Q 值
从左侧抓取	10
从右侧抓取	9.8
从上方抓取	9.5

如果策略立即集中于左侧抓取，就可能放弃探索。更严重的是，这些估计可能不准确：例如当前 Critic 给出 $\hat{Q}(s,a_{\text{left}})=10$、$\hat{Q}(s,a_{\text{right}})=8$，但真实值却是 $Q(s,a_{\text{left}})=8$、$Q(s,a_{\text{right}})=10$。由于右侧抓取很少被尝试，估计错误可能长期无法纠正。

于是我们希望策略不只是选择当前估计的最佳动作，还可以在高价值动作附近保持适度的随机性。这就是最大熵强化学习（Maximum Entropy RL）的出发点。

2. 第一步：最大熵强化学习

2.1 策略熵与温度参数

离散动作的 Shannon 熵是 $\mathcal{H}(\pi(\cdot\mid s))=-\sum_a \pi(a\mid s)\log\pi(a\mid s)$；连续动作通常使用微分熵：

$$\boxed{\mathcal{H}(\pi(\cdot\mid s))=-\int \pi(a\mid s)\log\pi(a\mid s)\,da=\mathbb{E}_{a\sim\pi}[-\log\pi(a\mid s)].}$$

对一维正态策略 $\pi(a\mid s)=\mathcal{N}(\mu(s),\sigma^2(s))$：

$$\mathcal{H}(\pi)=\frac{1}{2}\log(2\pi e\sigma^2)=\log\sigma+\frac{1}{2}\log(2\pi e).$$

因此，方差越大，熵越大，动作采样也越分散。**注意**：连续分布的微分熵可以为负，而且依赖动作变量的尺度；这与离散熵不同。

引入温度 $\alpha\geq 0$，将目标改为：

$$\boxed{J_{\mathrm{ME}}(\pi)=\mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{\infty}\gamma^t\big(r(s_t,a_t)+\alpha\mathcal{H}(\pi(\cdot\mid s_t))\big)\right].}$$

α 控制高回报与高熵之间的权衡：$\alpha\to 0$ 时趋于普通 RL；α 较大时鼓励更分散的策略。需要单独的系数，是因为奖励与熵的数值尺度可能完全不同。例如 $r\approx 100$，而 $\mathcal{H}(\pi)\approx 5$，若直接相加，熵项可能影响很小；通过 α 才能控制相对权重。

2.2 把熵写成可采样的形式

由 $\mathcal{H}(\pi(\cdot\mid s))=\mathbb{E}_{a\sim\pi}[-\log\pi(a\mid s)]$，有：

$$\boxed{J_{\mathrm{ME}}(\pi) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{\infty}\gamma^t\big(r(s_t,a_t) - \alpha\log\pi(a_t\mid s_t)\big)\right].} \tag{1}$$

可以将 $\tilde{r}_t = r_t - \alpha\log\pi(a_t\mid s_t)$ 看成带熵正则的奖励表达。它依赖当前策略，**不是**与策略无关的固定环境奖励。改写的关键好处是：连续动作算法只需采样 $a\sim\pi$ 并计算 $\log\pi(a\mid s)$，而不必对整个动作空间显式积分求熵。

3. 第二步：Soft Bellman 方程

3.1 与普通 Bellman 方程对比

标准价值函数满足：

$$V^\pi(s) = \mathbb{E}_{a\sim\pi}[Q^\pi(s,a)], \quad Q^\pi(s,a) = \mathbb{E}_{s'\sim P}[r(s,a) + \gamma V^\pi(s')].$$

将两式合并：

$$Q^\pi(s,a) = \mathbb{E}_{s',a'}[r(s,a) + \gamma Q^\pi(s',a')].$$

最大熵 RL 需要把未来策略的熵也纳入价值计算。

3.2 定义 Soft V 和 Soft Q

采用 SAC 常用的一种一致约定：

$$\boxed{V^\pi(s) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty}\gamma^t\big(r_t - \alpha\log\pi(a_t\mid s_t)\big)\;\middle|\; s_0 = s\right],}$$

$$\boxed{Q^\pi(s,a) = \mathbb{E}_\pi\left[r_0 + \sum_{t=1}^{\infty}\gamma^t\big(r_t - \alpha\log\pi(a_t\mid s_t)\big)\;\middle|\; s_0 = s, a_0 = a\right].}$$

区别是：V^π 包含当前动作的熵项；Q^π 固定当前动作，因此把熵项从下一步开始计入。这种约定便于书写递推。由条件期望：

$$\boxed{V^\pi(s) = \mathbb{E}_{a\sim\pi}\big[Q^\pi(s,a) - \alpha\log\pi(a\mid s)\big].} \tag{2}$$

等价地：

$$\boxed{V^\pi(s) = \mathbb{E}_{a\sim\pi}[Q^\pi(s,a)] + \alpha\mathcal{H}(\pi(\cdot\mid s)).} \tag{3}$$

普通 V 衡量当前策略未来可获得的奖励；Soft V 同时衡量奖励与可保持的随机性。

3.3 Soft Bellman Expectation Equation

由定义和马尔可夫性质：

$$Q^\pi(s,a) = \mathbb{E}_{s'\sim P}[r(s,a) + \gamma V^\pi(s')].$$

代入式 (2)：

$$\boxed{Q^{\pi}(s,a)=\mathbb{E}_{\substack{s'\sim P(\cdot|s,a)\\ a'\sim\pi(\cdot|s')}}\left[r(s,a)+\gamma\left(Q^{\pi}(s',a')-\alpha\log\pi(a'\mid s')\right)\right].} \tag{4}$$

与普通 Bellman 方程相比，增加了 $-\alpha\log\pi(a'\mid s')$。这正是熵奖励通过下一状态的 Soft Value 进入 Critic Target 的方式。该公式给出了 SAC **策略评估** 的理论基础，但还需要推导策略改进。

4. 第三步：Soft Policy Improvement

4.1 为什么普通贪心改进不够?

普通策略迭代使用 $\pi_{\text{new}}(s)\in\arg\max_a Q^{\pi_{\text{old}}}(s,a)$。随机策略若只最大化 $\mathbb{E}_a Q$，最优解也可能退化为集中于最大 Q 动作的确定性分布。SAC 希望在改进动作价值的同时保留熵。

固定状态 s 和旧策略的 $Q_{\text{old}}(s,a)=Q^{\pi_{\text{old}}}(s,a)$，考虑:

$$\boxed{\pi_{\text{new}}=\arg\max_{\pi}\left\{\mathbb{E}_{a\sim\pi}[Q_{\text{old}}(s,a)]+\alpha\mathcal{H}(\pi(\cdot\mid s))\right\}.} \tag{5}$$

即最大化泛函:

$$F(\pi)=\int\pi(a\mid s)Q_{\text{old}}(s,a)\,da-\alpha\int\pi(a\mid s)\log\pi(a\mid s)\,da,$$

并约束 $\int\pi(a\mid s)\,da=1$。

4.2 变分法求 Boltzmann Policy

设 $\alpha>0$，并假定归一化常数有限。拉格朗日函数为:

$$\begin{aligned}\mathcal{L}(\pi,\lambda)=&\int\pi(a\mid s)Q_{\text{old}}(s,a)\,da\\&-\alpha\int\pi(a\mid s)\log\pi(a\mid s)\,da\\&+\lambda\left(\int\pi(a\mid s)\,da-1\right).\end{aligned}$$

对密度的变分导数:

$$\frac{\delta\mathcal{L}}{\delta\pi(a\mid s)}=Q_{\text{old}}(s,a)-\alpha(\log\pi(a\mid s)+1)+\lambda.$$

令其为零:

$$\log\pi(a\mid s)=\frac{Q_{\text{old}}(s,a)}{\alpha}+\frac{\lambda}{\alpha}-1.$$

取指数并用积分归一化，得到:

$$\boxed{\pi^{*}_{\text{new}}(a\mid s)=\frac{\exp(Q_{\text{old}}(s,a)/\alpha)}{Z_{\text{old}}(s)},\quad Z_{\text{old}}(s)=\int_{\mathcal{A}}\exp(Q_{\text{old}}(s,a')/\alpha)\,da'.} \tag{6}$$

它是 **Boltzmann Policy**：Q 越大概率越大，α 越大分布越平缓，$\alpha\to0^{+}$ 时概率集中于 Q 最大的动作。假如两个动作 $Q(a_1)=2,Q(a_2)=1$，则 $\alpha=0.5$ 时 $\pi(a_1)\approx0.881,\pi(a_2)\approx0.119$；$\alpha=2$ 时变为 $0.622,0.378$。

动机总结：最大熵策略改进不再是简单地选择 $\arg\max_a Q$，而是学习一个由价值函数决定的高价值动作分布。

5. 第四步：把策略改进转为 KL 最小化

5.1 计算 Boltzmann 分布为什么困难?

对高维连续动作，$Z(s) = \int e^{Q(s,a)/\alpha}\, da$ 通常无法高效计算，因而不能直接构造或采样归一化后的理想策略。SAC 通过参数化 Actor π_θ，让它 **投影到 Boltzmann 分布**，并消去 $Z(s)$。

令:

$$\pi_B(a \mid s) = \frac{\exp(Q_{\text{old}}(s,a)/\alpha)}{Z(s)}.$$

考虑前向方向为 $D_{\text{KL}}(\pi_\theta \| \pi_B)$ 的 KL 散度:

$$\begin{aligned} D_{\text{KL}}(\pi_\theta \| \pi_B) &= \mathbb{E}_{a\sim\pi_\theta}\left[\log \pi_\theta(a \mid s) - \log \pi_B(a \mid s)\right] \\ &= \mathbb{E}_{a\sim\pi_\theta}\left[\log \pi_\theta(a \mid s) - \frac{Q_{\text{old}}(s,a)}{\alpha}\right] + \log Z(s). \end{aligned}$$

Q_{old} 固定，$\log Z(s)$ 与 Actor 参数无关，于是:

$$\boxed{\arg\min_\theta D_{\text{KL}}(\pi_\theta \| \pi_B) = \arg\min_\theta \mathbb{E}_{a\sim\pi_\theta}\left[\alpha \log \pi_\theta(a \mid s) - Q_{\text{old}}(s,a)\right].} \tag{7}$$

这里便得到了 SAC 的 **Actor Loss 原型**：既无需计算 $Z(s)$，也不用对高维动作空间枚举积分。

更进一步:

$$\boxed{\mathbb{E}_{a\sim\pi_\theta}[Q(s,a)] + \alpha \mathcal{H}(\pi_\theta(\cdot \mid s)) = \alpha \log Z(s) - \alpha D_{\text{KL}}(\pi_\theta \| \pi_B).} \tag{8}$$

这表明：最大化 Soft Value 与逼近 Boltzmann Policy 是同一优化问题的两种表述。

5.2 为什么精确 Soft Policy Iteration 可以改进策略?

设 π_{old} 在允许的策略族中，且在每个状态上准确求解 KL 投影。根据最优性:

$$D_{\text{KL}}(\pi_{\text{new}} \| \pi_B) \le D_{\text{KL}}(\pi_{\text{old}} \| \pi_B).$$

使用式 (8) 得到:

$$\boxed{\mathbb{E}_{a\sim\pi_{\text{new}}}[Q^{\pi_{\text{old}}}(s,a) - \alpha \log \pi_{\text{new}}(a \mid s)] \ge V^{\pi_{\text{old}}}(s).} \tag{9}$$

定义固定策略 π 的 Soft Bellman 算子:

$$(\mathcal{T}^\pi Q)(s,a) = r(s,a) + \gamma \mathbb{E}_{s',a'\sim\pi}\left[Q(s',a') - \alpha \log \pi(a' \mid s')\right].$$

式 (9) 结合 Bellman 方程可得:

$$Q^{\pi_{\text{old}}} \le \mathcal{T}^{\pi_{\text{new}}} Q^{\pi_{\text{old}}}.$$

$\mathcal{T}^\pi$ 是单调算子，在适当的有界性假设下还满足压缩性质:

$$\|\mathcal{T}^\pi Q_1 - \mathcal{T}^\pi Q_2\|_\infty \le \gamma \|Q_1 - Q_2\|_\infty.$$

反复应用 $\mathcal{T}^{\pi_{\text{new}}}$:

$$Q^{\pi_{\text{old}}} \le \mathcal{T}^{\pi_{\text{new}}} Q^{\pi_{\text{old}}} \le \cdots \longrightarrow Q^{\pi_{\text{new}}}.$$

因此：

$$\boxed{Q^{\pi_{\text{new}}}(s,a) \ge Q^{\pi_{\text{old}}}(s,a).} \tag{10}$$

这说明精确的 **Soft Policy Evaluation + Soft Policy Improvement** 具有策略改进性质。**边界条件**：这里是精确评估与精确投影下的理论结论；神经网络、有限采样、随机梯度和函数逼近的实际 SAC 并不因此获得全局收敛保证。

6. 第五步：从 Soft Policy Iteration 到 Actor-Critic

至此得到了两步：

- **Policy Evaluation**：根据 $Q^{\pi}(s,a) = \mathbb{E}[r + \gamma(Q^{\pi}(s',a') - \alpha \log \pi(a' \mid s'))]$ 评估策略。
- **Policy Improvement**：根据 $\min_{\pi} \mathbb{E}_{a\sim\pi}[\alpha \log \pi(a \mid s) - Q(s,a)]$ 改进策略。

连续状态和动作无法用表格存储完整的 Q 和 π，因此用神经网络近似：

$$\boxed{\text{Actor: } \pi_\theta(a \mid s), \qquad \text{Critic: } Q_\phi(s,a).}$$

Actor 输入状态，输出动作分布；Critic 输入状态与动作，输出标量 Q 值。由 Soft Bellman 方程构造 TD 误差：

$$\delta = Q_\phi(s,a) - \left[r + \gamma \left(Q_\phi(s',a') - \alpha \log \pi_\theta(a' \mid s')\right)\right].$$

自然的 Critic 损失为 $\frac{1}{2}\mathbb{E}[\delta^2]$；Actor 则最小化 $\mathbb{E}[\alpha \log \pi_\theta(a \mid s) - Q_\phi(s,a)]$。这里的奖励信号通过 $r \to Q_\phi \to \pi_\theta$ 间接作用于 Actor，而不是直接让环境奖励反向传播。

注意：用一个不断变化的 Q_ϕ 同时预测与构造自身目标，可能造成训练不稳定；第 8 节会引入目标网络和双 Q。

7. 第六步：为什么能够 Off-Policy 学习?

7.1 PPO 的数据分布依赖

PPO 的 Clipped Objective 依赖旧策略产生的数据和概率比值：

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}.$$

因此标准 PPO 主要使用近期 On-Policy Rollouts；旧得太久的数据会造成严重分布偏移。

7.2 Bellman 更新为何可以使用旧经验?

在固定 MDP 中，环境的条件转移 $P(s' \mid s,a)$ 与**是哪一个行为策略采样到** (s,a) 无关。过去行为策略产生的样本 (s,a,r,s') 仍是有效的环境转移样本。只需在下一个状态 s'，使用**当前策略**重新采样 $a' \sim \pi_\theta(\cdot \mid s')$，就能构造当前策略的 Soft Bellman Target：

$$y = r + \gamma\big(Q(s',a') - \alpha \log \pi_\theta(a' \mid s')\big).$$

变量	数据来源
s, a, r, s'	Replay Buffer 中的历史转移
a'	当前 Actor $\pi_\theta(\cdot \mid s')$ 重新采样
$\log \pi_\theta(a' \mid s')$	当前 Actor 计算

维护经验池：

$$\boxed{\mathcal{D} = \{(s_t, a_t, r_t, s_{t+1}, d_t)\}.}$$

更新时从 $\mathcal{D}$ 随机抽取 Mini-Batch，而非必须收集一批新 Rollout。历史数据可反复复用，环境一步交互之后甚至可以做多次梯度更新。

重要限制：Off-Policy 不等于分布差异完全无害。若 Replay Buffer 对某些关键状态动作缺乏覆盖，神经网络 Q 在这些区域可能严重外推错误。有限数据、函数逼近与 Bootstrapping 的组合也可能引起不稳定，因此样本效率与可靠性之间仍需权衡。

8. 第七步：完整推导 Critic Loss

8.1 从 Soft Bellman Target 到 TD Regression

用当前策略采样 $a' \sim \pi_\theta(\cdot \mid s')$，理论单样本目标为：

$$y = r + \gamma \left[Q_\phi(s', a') - \alpha \log \pi_\theta(a' \mid s') \right].$$

对应回归目标是：

$$J_Q(\phi) = \frac{1}{2} \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(Q_\phi(s, a) - y \right)^2 \right].$$

但这还不是完整 SAC：此时目标也使用同一个正在更新的网络，且 Actor 会倾向利用 Critic 的高估误差。

8.2 Target Network：减缓目标漂移

为 Critic 维护慢更新参数 $\bar{\phi}$，构造 $Q_{\bar{\phi}}$，并使用 Polyak Averaging：

$$\boxed{\bar{\phi} \leftarrow (1 - \tau)\bar{\phi} + \tau \phi, \qquad 0 < \tau \ll 1.} \tag{11}$$

例如 $\tau = 0.005$。此举减慢 TD Target 的变化速度，而非解决估计偏差本身。

8.3 Clipped Double Q：缓解价值高估

如果 $\hat{Q}(s, a) = Q(s, a) + \epsilon(s, a)$，Actor 倾向选择 $\hat{Q}$ 较大的动作，即使该动作的高估主要来自误差 ϵ。例如两动作真实值都为 10，Critic 却给出 12 和 8，Actor 会错误地偏好前者，并可能进一步放大误差。

因此训练两个独立 Critic：

$$Q_{\phi_1}(s, a), \qquad Q_{\phi_2}(s, a),$$

及对应目标网络：

$$Q_{\bar{\phi}_1}(s,a), \qquad Q_{\bar{\phi}_2}(s,a).$$

用保守的最小值构造目标：

$$\boxed{Q_{\text{target}}(s',a') = \min_{j=1,2} Q_{\bar{\phi}_j}(s',a').}$$

取最小值有助于缓解过高估计，但可能引入低估，并**不保证** Q 估计无偏。

8.4 最终 Critic Target 和 Loss

令 $d=1$ 表示真实终止（而非单纯 Time-Limit Truncation），最终 Target 为：

$$\boxed{y = r + \gamma(1-d)\left[\min_{j=1,2} Q_{\bar{\phi}_j}(s',a') - \alpha \log \pi_\theta(a' \mid s')\right], \quad a' \sim \pi_\theta(\cdot \mid s').} \tag{12}$$

每个 Critic 分别拟合同一 Target：

$$\boxed{J_{Q_i}(\phi_i) = \frac{1}{2}\mathbb{E}_{\mathcal{D}}\left[\left(Q_{\phi_i}(s,a) - y\right)^2\right], \qquad i=1,2.} \tag{13}$$

$$\boxed{J_Q(\phi_1,\phi_2) = J_{Q_1}(\phi_1) + J_{Q_2}(\phi_2).} \tag{14}$$

执行 $\phi_i \leftarrow \phi_i - \eta_Q \nabla_{\phi_i} J_{Q_i}$。构造 Target 时必须停止目标分支梯度：$y \leftarrow \text{stopgrad}(y)$。对于**真实终止**，$d=1$ 从而 $y=r$；对于仅因最大步数被截断的非终止状态，通常仍应保留 Bootstrapping。

9. 第八步：完整推导 Actor Loss

第 5 节 KL 投影给出：

$$\pi_{\text{new}} = \arg\min_{\pi} \mathbb{E}_{a\sim\pi}\left[\alpha \log \pi(a \mid s) - Q(s,a)\right].$$

用 Actor π_θ 和 Critic Q_ϕ 近似，并从 Replay Buffer 提供状态：

$$\boxed{J_\pi(\theta) = \mathbb{E}_{\substack{s\sim\mathcal{D} \\ a\sim\pi_\theta(\cdot|s)}}\left[\alpha \log \pi_\theta(a \mid s) - Q_\phi(s,a)\right].} \tag{15}$$

使用双 Q 的常见版本：

$$\boxed{J_\pi(\theta) = \mathbb{E}_{\substack{s\sim\mathcal{D} \\ a\sim\pi_\theta(\cdot|s)}}\left[\alpha \log \pi_\theta(a \mid s) - \min_{i=1,2} Q_{\phi_i}(s,a)\right].} \tag{16}$$

也存在更新 Actor 时仅使用第一个 Critic 的实现。式 (16) 具有两个互补部分：最小化 $\alpha \log \pi_\theta$ 等效于鼓励高熵；最小化 $-Q_\phi$ 鼓励产生高价值动作。

关键区别：PPO 主要使用采样动作的 Advantage 和概率比更新策略；连续动作 SAC 则希望梯度穿过动作 a，利用 $\nabla_a Q_\phi(s,a)$ 对 Actor 提供局部方向信息。不过动作来自随机分布，普通“采样”不能直接写成可微的确定性计算，因此需要重参数化。

10. 第九步：Reparameterization Trick

10.1 把依赖参数的随机采样变成可微函数

令 Actor 输出均值和标准差：

$$\mu_\theta(s), \qquad \sigma_\theta(s).$$

令未压缩的动作：

$$u \sim \mathcal{N}\big(\mu_\theta(s), \mathrm{diag}(\sigma_\theta^2(s))\big).$$

直接描述为 $u \sim \pi_\theta$ 时，采样分布依赖于 θ。可等价改写为：

$$\boxed{u_\theta(s,\epsilon) = \mu_\theta(s) + \sigma_\theta(s) \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).} \tag{17}$$

此时噪声 ϵ 与参数无关，并且 u_θ 对 θ 可微。例如 $\partial u/\partial \mu = I$、$\partial u/\partial \sigma = \mathrm{diag}(\epsilon)$。

10.2 Tanh-Squashed Gaussian Policy

真实机器人动作一般有界（如 $[-1,1]^d$），而 Gaussian 支持整个实数域。SAC 使用逐维 $\tanh$：

$$\boxed{a_\theta(s,\epsilon) = \tanh\big(\mu_\theta(s) + \sigma_\theta(s) \odot \epsilon\big).} \tag{18}$$

对于实际动作区间 $[a_{\min,i}, a_{\max,i}]$，可以进一步做 $a_i^{\mathrm{env}} = c_i + b_i \tanh(u_i)$，其中 $c_i = (a_{\max,i} + a_{\min,i})/2$、$b_i = (a_{\max,i} - a_{\min,i})/2$。

10.3 把 Actor Loss 改写为 Pathwise Objective

记 $a = f_\theta(s,\epsilon)$，用固定噪声分布替换动作分布中的随机性：

$$\boxed{J_\pi(\theta) = \mathbb{E}_{\substack{s\sim\mathcal{D} \\ \epsilon\sim\mathcal{N}(0,I)}} \left[\alpha \log \pi_\theta(f_\theta(s,\epsilon) \mid s) - Q_\phi(s, f_\theta(s,\epsilon))\right].} \tag{19}$$

令 $a_\theta = f_\theta(s,\epsilon)$，对参数求全导数：

$$\boxed{\nabla_\theta J_\pi = \mathbb{E}\left[\alpha \frac{d}{d\theta} \log \pi_\theta(a_\theta \mid s) - \nabla_a Q_\phi(s,a_\theta) \frac{\partial a_\theta}{\partial \theta}\right].} \tag{20}$$

注意 $\log \pi_\theta(a_\theta \mid s)$ 既显式依赖 θ，又通过 a_θ 隐式依赖 θ：

$$\frac{d}{d\theta} \log \pi_\theta(a_\theta \mid s) = \left.\frac{\partial \log \pi_\theta(a \mid s)}{\partial \theta}\right|_{a=a_\theta} + \nabla_a \log \pi_\theta(a_\theta \mid s) \frac{\partial a_\theta}{\partial \theta}.$$

故：

$$\boxed{\begin{aligned}\nabla_\theta J_\pi = \mathbb{E}\Bigg[&\alpha \left.\frac{\partial \log \pi_\theta(a \mid s)}{\partial \theta}\right|_{a=a_\theta} \\ &+ \big(\alpha \nabla_a \log \pi_\theta(a_\theta \mid s) - \nabla_a Q_\phi(s,a_\theta)\big) \frac{\partial a_\theta}{\partial \theta}\Bigg].\end{aligned}} \tag{21}$$

其中最具有控制意义的部分是：

$$\underbrace{\nabla_a Q_\phi(s,a_\theta)}_{\text{动作朝哪个方向更好}} \qquad \underbrace{\frac{\partial a_\theta}{\partial \theta}}_{\text{如何改变 Actor 参数}}.$$

这使 SAC 与 DDPG 的确定性策略梯度有直接联系：SAC 在可重参数化的**随机策略**中利用 Critic 的动作梯度，并额外进行熵正则化。Pathwise Gradient 通常比直接使用 Score-Function Estimator 具有更低的方差，但效果仍依赖 Critic 的质量。

11. 第十步：Tanh Gaussian 的 Log Probability 修正

Gaussian 变量 u 与 $a=\tanh(u)$ 的分布不同，因此不能直接使用 $\log \mathcal{N}(a;\mu,\Sigma)$。对可逆变换 $a=g(u)$，由密度变换：

$$\pi(a \mid s)=p_U(u \mid s)\left|\det \frac{\partial a}{\partial u}\right|^{-1},$$

$$\log \pi(a \mid s)=\log p_U(u \mid s)-\log \left|\det \frac{\partial a}{\partial u}\right|.$$

对于逐维 $a_i=\tanh(u_i)$，有 $\partial a_i/\partial u_i=1-\tanh^2(u_i)$。Jacobian 是对角矩阵，其行列式为 $\prod_i(1-\tanh^2(u_i))$。所以：

$$\boxed{\log \pi_\theta(a \mid s)=\log \mathcal{N}(u;\mu_\theta(s),\Sigma_\theta(s))-\sum_{i=1}^{d}\log\left(1-\tanh^2(u_i)\right).} \tag{22}$$

如果将动作进一步缩放为 $a_i^{\mathrm{env}}=c_i+b_i\tanh(u_i)$，且 $b_i>0$：

$$\boxed{\log \pi_\theta(a^{\mathrm{env}} \mid s)=\log \mathcal{N}(u;\mu_\theta,\Sigma_\theta)-\sum_{i=1}^{d}\log\left(b_i(1-\tanh^2(u_i))\right).} \tag{23}$$

实现时为避免 $\tanh(u)$ 接近 ±1 引起的数值误差，可用稳定恒等式 $\log(1-\tanh^2 u)=2(\log 2-u-\mathrm{softplus}(-2u))$，或者加上数值稳定项。常见实现先在归一化动作空间计算 Log Probability，再将动作缩放给环境；重要的是整个熵与动作尺度约定保持一致。

动机总结：SAC 的 Actor 和温度调节都依赖 $\log\pi$；漏掉 Tanh 的 Jacobian Correction 会直接改变优化目标和梯度方向。

12. 第十一步：Automatic Entropy Tuning

12.1 从固定温度到熵约束

手工设定 α 对奖励缩放和动作维度敏感。与其直接选择熵正则化系数，不如要求平均策略熵达到目标水平。一个常用的概括形式为：

$$\boxed{\begin{aligned}\max_{\pi}\quad & J_R(\pi)\\ \text{s.t.}\quad & \bar{\mathcal{H}}(\pi)\geq \mathcal{H}_{\text{target}}.\end{aligned}} \tag{24}$$

$\bar{\mathcal{H}}$ 表示相关状态访问分布上的平均策略熵。原论文以逐时刻熵约束推导温度学习；这里采用等价设计思想的平均熵表述来简化解释。

引入非负乘子 α，拉格朗日函数为：

$$\boxed{\mathcal{L}(\pi,\alpha)=J_R(\pi)+\alpha\big(\bar{\mathcal{H}}(\pi)-\mathcal{H}_{\text{target}}\big).} \tag{25}$$

相应形式是：

$$\boxed{\max_{\pi} \min_{\alpha \geq 0} \mathcal{L}(\pi, \alpha).} \tag{26}$$

固定 α 优化 π，就回到最大熵策略优化；固定策略优化 α，则得到熵目标的对偶更新。

12.2 温度损失及其更新方向

用 $\mathcal{H}(\pi) = -\mathbb{E} \log \pi$，得：

$$\boxed{J_\alpha(\alpha) = \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi_\theta} \left[\alpha \left(-\log \pi_\theta(a \mid s) - \mathcal{H}_{\text{target}} \right) \right].} \tag{27}$$

等价地 $J_\alpha = -\mathbb{E}[\alpha(\log \pi_\theta(a \mid s) + \mathcal{H}_{\text{target}})]$。求导：

$$\boxed{\frac{\partial J_\alpha}{\partial \alpha} = \bar{\mathcal{H}}(\pi_\theta) - \mathcal{H}_{\text{target}}.} \tag{28}$$

所以：

$$\boxed{\begin{cases} \bar{\mathcal{H}}(\pi) < \mathcal{H}_{\text{target}} & \Rightarrow \alpha \uparrow, \\ \bar{\mathcal{H}}(\pi) > \mathcal{H}_{\text{target}} & \Rightarrow \alpha \downarrow. \end{cases}} \tag{29}$$

熵偏低时，提高 α 让 Actor 更重视探索；熵偏高时，降低 α 让 Actor 更集中于高价值动作。

12.3 为什么学习 Log-Alpha?

直接更新 α 可能导致负值。因此定义 $\eta = \log \alpha$、$\alpha = e^\eta > 0$。严格代入对偶损失为：

$$J_\eta(\eta) = -\mathbb{E} \left[e^\eta \left(\log \pi_\theta(a \mid s) + \mathcal{H}_{\text{target}} \right) \right].$$

它的梯度包含 $e^\eta = \alpha$。许多常见实现采用梯度尺度不同、但内部驻点相同的代理损失：

$$\boxed{J_\eta^{\text{surrogate}}(\eta) = -\mathbb{E} \left[\eta \left(\log \pi_\theta(a \mid s) + \mathcal{H}_{\text{target}} \right) \right].} \tag{30}$$

温度更新时必须将 Log Probability `detach`，避免温度损失反向更新 Actor。

12.4 Target Entropy 的经验选择

对于 d 维归一化连续动作，常见启发式为 $\mathcal{H}_{\text{target}} = -d$。例如 $d = 12$ 时可先试 -12。这不是理论上的最优公式；连续微分熵允许负值，且会受动作尺度与概率密度约定影响。机器人高维动作、手部关节或动作序列控制往往需要验证这一超参数，而不能机械套用。

13. 第十二步：完整的现代 SAC 算法

13.1 模块与三个优化问题

现代连续动作 SAC 一般包含以下模块：

模块	参数/对象	作用
Actor	θ	生成可重参数化的随机动作
Critic 1 / 2	ϕ_1, ϕ_2	预测 Soft Q，缓解过高估计
Target Critics	$\bar{\phi}_1, \bar{\phi}_2$	产生缓慢变化的 TD Target
Temperature	α 或 $\eta = \log \alpha$	调整熵权重
Replay Buffer	$\mathcal{D}$	保存历史 (s, a, r, s', d)

三个核心目标：

$$\boxed{\min_{\phi_1,\phi_2} J_Q, \qquad \min_{\theta} J_\pi, \qquad \min_{\alpha \geq 0} J_\alpha.}$$

13.2 一次更新的完整步骤

步骤 1：交互并写入 Buffer。 由当前状态 s_t 采样 $a_t \sim \pi_\theta(\cdot \mid s_t)$，执行动作，记录 $(s_t, a_t, r_t, s_{t+1}, d_t)$。初始训练阶段可以使用随机动作收集一定的覆盖数据。

步骤 2：历史采样。 从 $\mathcal{D}$ 抽取 Batch $B = \{(s_i, a_i, r_i, s'_i, d_i)\}_{i=1}^{N}$。

步骤 3：计算目标。 用当前 Actor 在下一状态采样 $a'_i \sim \pi_\theta(\cdot \mid s'_i)$，计算：

$$\boxed{y_i = r_i + \gamma(1 - d_i)\left[\min_{j=1,2} Q_{\bar{\phi}_j}(s'_i, a'_i) - \alpha \log \pi_\theta(a'_i \mid s'_i)\right].}$$

步骤 4：更新 Critic。 两个 Critic 使用相同目标，分别最小化：

$$J_{Q_j} = \frac{1}{2N} \sum_{i=1}^{N} (Q_{\phi_j}(s_i, a_i) - y_i)^2.$$

步骤 5：更新 Actor。 重新采样 $\epsilon_i \sim \mathcal{N}(0, I)$、$\tilde{a}_i = f_\theta(s_i, \epsilon_i)$，最小化：

$$\boxed{J_\pi = \frac{1}{N} \sum_{i=1}^{N} \left[\alpha \log \pi_\theta(\tilde{a}_i \mid s_i) - \min_j Q_{\phi_j}(s_i, \tilde{a}_i)\right].}$$

Actor 更新时冻结 Critic **参数**，但必须允许梯度通过 Critic 输入动作，即保留 $\nabla_a Q_\phi$。

步骤 6：更新温度。 根据 $-\log \pi_\theta$ 与目标熵之差调整 α，并在此步骤对策略概率进行 `detach`。

步骤 7：更新目标网络。 对每个 Critic 做 $\bar{\phi}_j \leftarrow (1 - \tau)\bar{\phi}_j + \tau \phi_j$。然后重复。

13.3 教学用伪代码

```
initialize actor pi_theta
initialize critics Q1, Q2
initialize target critics Q1_targ, Q2_targ from Q1, Q2
initialize replay buffer D and log_alpha

for each environment interaction:
    action = pi_theta.sample(obs)  # warm-up may use random actions
    next_obs, reward, terminated, truncated = env.step(action)
    D.add(obs, action, reward, next_obs, terminated)
```

```
    obs = env.reset() if (terminated or truncated) else next_obs

    for each gradient update:
        bs, ba, br, bs_next, bd = D.sample(batch_size)
        alpha = exp(log_alpha)

        # 1) Critic target: no gradient through target branch
        with no_grad():
            a_next, logp_next = pi_theta.rsample(bs_next)
            q_next = min(Q1_targ(bs_next, a_next),
                         Q2_targ(bs_next, a_next))
            y = br + gamma * (1 - bd) * (q_next - alpha * logp_next)

        loss_q1 = mean((Q1(bs, ba) - y) ** 2)
        loss_q2 = mean((Q2(bs, ba) - y) ** 2)
        update(Q1, loss_q1)
        update(Q2, loss_q2)

        # 2) Actor: freeze Q weights, keep dQ/da
        a_pi, logp_pi = pi_theta.rsample(bs)
        q_pi = min(Q1(bs, a_pi), Q2(bs, a_pi))
        loss_pi = mean(alpha.detach() * logp_pi - q_pi)
        update(pi_theta, loss_pi)

        # 3) Automatic entropy tuning (log-alpha surrogate)
        loss_alpha = -mean(log_alpha *
                           (logp_pi.detach() + target_entropy))
        update(log_alpha, loss_alpha)

        # 4) Polyak target update
        for Q, Q_targ in [(Q1, Q1_targ), (Q2, Q2_targ)]:
            Q_targ.params = (1 - tau) * Q_targ.params + tau * Q.params
```

> **实现警告**：上面是为了说明更新顺序而写的伪代码，不是可以直接运行的完整库。真实代码需正确执行优化器零梯度、参数冻结、动作缩放、概率变换、初始探索、环境 Reset 和 Batch Shape 处理；还需要区分 `terminated` 与 `truncated`。

14. SAC v1 与现代 SAC 的区别

14.1 原始 SAC v1：显式训练 V 网络

最初的 SAC 除双 Q 和 Actor 外，还维护状态价值网络 $V_\psi(s)$：

$$V^\pi(s) = \mathbb{E}_{a\sim\pi}\big[Q^\pi(s,a) - \alpha \log \pi(a \mid s)\big].$$

因此可以学习：

$$\boxed{J_V(\psi) = \frac{1}{2}\mathbb{E}_s\left[\left(V_\psi(s) - \left(\min_i Q_{\phi_i}(s,\tilde{a}) - \alpha \log \pi_\theta(\tilde{a} \mid s)\right)\right)^2\right].}$$

目标项在训练 V 时视为常数。其 Q Target 为：

$$\boxed{y_Q = r + \gamma(1-d)V_{\bar{\psi}}(s').}$$

14.2 现代 SAC：去掉显式 V 网络

由于：

$$V^\pi(s') = \mathbb{E}_{a'\sim\pi}\big[Q^\pi(s',a') - \alpha \log \pi(a' \mid s')\big],$$

可直接用下一动作采样近似：

$$V^\pi(s') \approx \min_j Q_{\bar{\phi}_j}(s',a') - \alpha \log \pi_\theta(a' \mid s').$$

这样 Q Target 直接写成式 (12)，不需要单独的 V 网络，减少额外拟合误差及网络复杂度。现在提及“标准 SAC”，通常是指该现代双 Q 版本，并经常搭配自动温度调节。

15. SAC 与 PPO 的本质区别

15.1 策略更新公式对比

PPO 使用采样动作的 Advantage 及策略概率比：

$$\boxed{J_{\text{PPO}}(\theta) = \mathbb{E}\left[\min\left(\rho_t(\theta)\hat{A}_t, \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right],}$$

其中 $\rho_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\text{old}}(a_t \mid s_t)$。优化器最大化该目标，并通过 Clipping 抑制剧烈的策略变化。典型策略梯度由 $\nabla_\theta \log \pi_\theta(a \mid s)\, A^\pi(s,a)$ 形成。

SAC 则最小化：

$$\boxed{J_{\text{SAC}}(\theta) = \mathbb{E}\left[\alpha \log \pi_\theta(a_\theta \mid s) - Q_\phi(s, a_\theta)\right], \quad a_\theta = f_\theta(s,\epsilon).}$$

其中包含 $\nabla_a Q_\phi(s,a_\theta)\, \partial a_\theta/\partial\theta$。PPO 通常无需 $\nabla_a Q$；SAC 标准连续 Actor 更新需要它。

15.2 对比表

对比维度	PPO	SAC
数据范式	On-Policy	Off-Policy
Actor	通常随机策略	可重参数化随机策略
Critic	常用 $V(s)$	常用 $Q(s,a)$
策略优化信号	Advantage、Probability Ratio	Q 值、动作梯度、熵
主要梯度估计	Score-Function	Pathwise / Reparameterization
历史数据复用	有限，主要使用新 Rollout	Replay Buffer 可重复采样
熵项	常用但非必需	核心目标
稳定机制	Clipped Policy Update	Double Q、Target Critics
样本效率	通常较低	通常较高
典型优势	大规模并行仿真吞吐量高	环境交互成本昂贵时可充分复用数据

对比维度	PPO	SAC
主要挑战	新样本需求大	Critic 外推、高估和 Bootstrap 稳定性

PPO Clipping 与 SAC 的 Double Q **不是同一类机制**：前者主要限制策略概率变化，后者主要缓解 Critic 高估；SAC 没有标准 PPO 式的概率比 Clipping。

15.3 为什么机器人仿真中 PPO 仍常见?

在 Isaac Lab 等平台上，大规模并行仿真可以以较低单位成本生成大量轨迹。PPO 通过高吞吐的并行 Rollout 获得出色的训练速度；SAC 虽复用样本，但 Critic Bootstrapping、频繁梯度更新和 Replay Buffer 访问也有成本。因此：

$$\boxed{\text{Sample Efficiency} \neq \text{Wall-Clock Efficiency.}}$$

高接触非线性、稀疏奖励、状态分布偏移也会增加 SAC 的价值学习难度。在真实机器人环境交互昂贵时，SAC 的 Off-Policy 数据复用通常更具吸引力，但仍需考虑安全探索和离线数据偏移。

16. 最终总结：五组关键公式与理解框架

（1）Maximum-Entropy Objective:

$$\boxed{J(\pi) = \mathbb{E}_{\tau}\left[\sum_{t=0}^{\infty} \gamma^t \big(r_t - \alpha \log \pi(a_t \mid s_t)\big)\right].}$$

动机：最大化回报，同时通过熵正则化维持探索。

（2）Soft Bellman Equation:

$$\boxed{Q^{\pi}(s,a) = \mathbb{E}\left[r + \gamma\big(Q^{\pi}(s',a') - \alpha \log \pi(a' \mid s')\big)\right].}$$

动机：将最大熵目标递归分解，使 Critic 能进行 TD 学习。

（3）KL-Based Soft Policy Improvement:

$$\boxed{\pi_{\text{new}} = \arg\min_{\pi} D_{\text{KL}}\left(\pi(\cdot \mid s) \,\middle\|\, \frac{\exp(Q^{\pi_{\text{old}}}(s,\cdot)/\alpha)}{Z(s)}\right),}$$

等价于最小化 $\mathbb{E}_{a\sim\pi}[\alpha \log \pi(a \mid s) - Q^{\pi_{\text{old}}}(s,a)]$。*动机*：逼近高价值 Boltzmann 分布而不计算归一化常数。

（4）现代 SAC 的 Critic 与 Actor:

$$\boxed{J_{Q_i} = \frac{1}{2}\mathbb{E}_{\mathcal{D}}\big[(Q_{\phi_i}(s,a) - y)^2\big], \quad y = r + \gamma(1-d)\big[\min_j Q_{\bar{\phi}_j}(s',a') - \alpha \log \pi_{\theta}(a' \mid s')\big],}$$

$$\boxed{J_{\pi} = \mathbb{E}_{s,\epsilon}\big[\alpha \log \pi_{\theta}(f_{\theta}(s,\epsilon) \mid s) - \min_j Q_{\phi_j}(s, f_{\theta}(s,\epsilon))\big].}$$

动机：用可训练网络、历史数据、双 Q 和 Pathwise Gradient 实现 Soft Policy Iteration。

（5）Automatic Entropy Tuning:

$$\boxed{J_\alpha = -\mathbb{E}\big[\alpha(\log \pi_\theta(a \mid s) + \mathcal{H}_{\text{target}})\big].}$$

动机：根据目标熵自动调节探索强度。

最关键的 PPO / SAC 区别

PPO 主要通过 $\nabla_\theta \log \pi_\theta(a \mid s) A^\pi(s,a)$ 调整**已采样动作的概率**；连续 SAC 使用 $\nabla_a Q_\phi(s,a)\, \partial a_\theta / \partial \theta$（同时有熵梯度），调整**可微动作生成函数**以提高动作价值。

完整逻辑链条为：

$$\boxed{\begin{aligned}&\text{Maximum-Entropy RL} \longrightarrow \text{Soft Bellman} \longrightarrow \text{KL Policy Improvement}\\ &\longrightarrow \text{Reparameterization} \longrightarrow \text{Off-Policy Actor-Critic}\end{aligned}}$$

Double Q、Target Critics 和 Automatic Temperature Tuning 则使上述理论更加适合复杂连续控制任务。

参考资料

1. Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). **Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor.** ICML. https://arxiv.org/abs/1801.01290 — Soft Policy Iteration、KL 策略改进和随机 Actor 的理论基础。
2. Haarnoja, T., Zhou, A., Hartikainen, K., et al. (2018). **Soft Actor-Critic Algorithms and Applications.** https://arxiv.org/abs/1812.05905 — 自动温度调节和现代 SAC 设计。
3. OpenAI Spinning Up. **Soft Actor-Critic.** https://spinningup.openai.com/en/latest/algorithms/sac.html — 现代 SAC 目标函数与工程实现。

文档说明：本 Markdown 与其同名 PDF 使用同一正文源文件生成；章节、公式、表格与参考资料保持一致。

数据采集与模仿学习

课程首页 · English · PDF

项目	说明
适用范围	示教数据、BC、DAgger、ACT、生成式动作块策略
文档核验日期	2026-10-18
许可	CC BY 4.0 · Xiong Zeng and contributors

核验边界： 方法与实验设计，未实际复现训练。 版本与固定源码引用见版本说明。原创教材采用 CC BY 4.0，第三方引用、图片及代码遵循其原有许可；注明来源不等于拥有再许可权。

目录

- 1. 从演示到可训练的数据
  - 1.1 一条轨迹应包含什么
  - 1.2 时间和坐标系先于网络结构
- 2. 数据采集与质量控制
  - 2.1 成功演示之外，还需要什么
  - 2.2 数据划分比指标数字更重要
- 3. 模仿学习：从 BC 到交互式纠错
  - 3.1 为什么训练误差低也会失败
  - 3.2 DAgger 是什么
- 4. 从单步动作到生成式动作块
  - 4.1 ACT 的核心思路
  - 4.2 Diffusion Policy 与 Flow Matching
- 5. 增强、再标定与仿真数据
  - 5.1 接触事件和恢复样本
  - 5.2 数据增强的安全边界
- 6. 统一 Panda 桌面抓取实验
  - 6.1 最小实验表
  - 6.2 常见失败排查
- 7. 课程衔接与参考资料

学习目标： 识别数据质量问题；比较 BC、ACT 与生成式动作策略；设计可复现的抓取实验。

1. 从演示到可训练的数据

模仿学习的目标不是保存一段看起来正确的视频，而是把观察、机器人状态、动作和任务条件组织成可对齐、可复现的监督信号。对于闭环机器人，最终学习的是条件策略 $\pi_\theta(a_t \mid o_{\le t}, g)$，其中 g 是目标。

1.1 一条轨迹应包含什么

建议每条 episode 至少保存：任务与场景 ID、相机时间戳、RGB 或 RGB-D、关节状态、执行动作、动作坐标系、控制模式、成功与否、异常事件。**动作标签必须表示实际发给机器人的命令**，不能把目标关节角、实际反馈位置和关节速度混为一谈。对视觉模型，还需注明相机外参是静态还是时变。

1.2 时间和坐标系先于网络结构

使用统一的时钟基准或经验证的时间对齐。标明 T_B^C、T_B^E、T_B^O 的定义，所有姿态矩阵使用相同的乘法约定。相机通常低频，机器人本体状态可能更高频；可采用与动作时刻对齐的最近帧或带明确延迟的插值，避免未来信息泄漏。

2. 数据采集与质量控制

典型数据源包括遥操作、拖动示教、脚本专家、仿真 RL 专家和人类视频重定向。它们在采样效率、动作精度、接触真实性和设备成本上各有差异。



实用原则： 一开始优先获得质量稳定且可复现的小数据集，而不是尽快积累大量未标注或不同步的视频。

2.1 成功演示之外，还需要什么

保留有效的失败轨迹、纠错动作与恢复片段，并标注事件边界，例如接触开始、滑落、重新抓取。不要把所有失败都作为“照做”的标签；区分应模仿的恢复策略与应避免的危险命令。明确数据使用权限、隐私、许可证及真实机器人安全审核。

2.2 数据划分比指标数字更重要

按对象实例、场景布局、操作者、初始姿态和采集日期划分训练与测试；**禁止同一连续轨迹切块后同时出现在训练集和测试集**。记录真实 episode 级成功率，而不仅是逐动作 MSE。多相机或多帧观测应整体划分，防止相邻帧泄漏。

3. 模仿学习：从 BC 到交互式纠错

Behavior Cloning（BC）直接优化数据上的动作预测。对于确定性回归可采用 $\mathcal{L}_{BC} = \mathbb{E}_{(o,a)\sim D}\|\pi_\theta(o) - a\|_2^2$；对于概率策略则最大化动作条件对数似然。这两种目标并不自动解决测试时的分布偏移。

3.1 为什么训练误差低也会失败

闭环执行中，小的动作误差会改变下一时刻的观测，机器人可能进入训练集从未覆盖的状态。应在测试时记录状态偏移、触碰失败、纠错次数，以及是否能从偏离轨迹的状态恢复。可加入安全受控的小扰动或恢复示教，扩大有效状态覆盖。

3.2 DAgger 是什么

DAgger 的思想是让当前策略收集会访问的状态，并由专家为这些状态提供适当动作，再聚合数据更新策略。对真实机器人要有明确的安全干预机制；如果无法在线查询专家，就不能简单地把普通离线训练称为 DAgger。

4. 从单步动作到生成式动作块

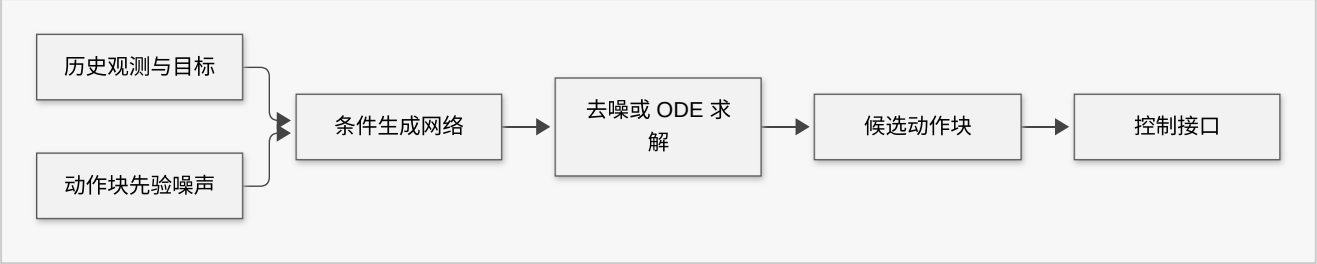
生成动作块可减少频繁重规划的开销，但会引入执行中观测过时的风险。区分 chunk horizon（预测长度）、execution horizon（执行长度）与 policy query rate（模型调用频率）。动作块的控制参考系必须和机器人的实际执行器接口一致。

4.1 ACT 的核心思路

ACT 使用 Transformer 预测一段连续的动作，并在论文实现中借助潜变量建模演示差异。与逐步 BC 相比，动作块适合示教数据中连续且结构化的操作，但需要正确处理 chunk 的对齐与组合。

4.2 Diffusion Policy 与 Flow Matching

Diffusion Policy 与 Flow Matching 都可用生成式机制表示多模态动作分布，但**扩散去噪过程、概率流或速度场求解与机器人关节速度是不同概念**。



在比较方法时，固定观察模态、示教数据量、推理预算、控制频率与评估环境；不能只比较离线训练损失。

5. 增强、再标定与仿真数据

在机器人操作中，几何变换必须一致地作用于图像、物体目标、轨迹参考和动作标签。仅把图像旋转而不调整动作的增强通常会产生错误监督。仿真专家轨迹可以扩充姿态覆盖，但接触参数、视觉渲染和执行器时延的 Sim-to-Real 差异需要单独验证。

5.1 接触事件和恢复样本

报告均匀采样基线，再评估接触切换、夹持失败和物体掉落附近的增采样。事件采样改变训练数据分布，需要明确其权重以及是否扭曲自然状态分布；也要评估无事件偏置的测试集。

5.2 数据增强的安全边界

不应把任意初始条件直接传播到不满足物理约束的接触轨迹。对旋转、平移、尺度、时序扰动都检查可达性、碰撞、接触一致性和控制限幅；可先做短时物理回放再收录。

6. 统一 Panda 桌面抓取实验

实验任务： 在 Isaac Lab 或 MuJoCo 中让 Panda 抓取随机放置的方块，先生成脚本专家或遥操作轨迹，再训练 BC / ACT / 生成式策略。

控制变量： 固定相机视角、训练任务数、机器人动作接口、控制频率、训练/测试对象划分、评估回合数。以简单 BC 作为所有生成式方法的必要基线。

指标： 成功率、接触失败率、轨迹偏差、恢复率、平均策略推理延迟、平均控制延迟与数据量。每种方法报告多个随机种子和失败模式，而非宣称某方法天然最优。

6.1 最小实验表

对照	目的
BC 单步	数据与控制接口是否正确
BC 动作块	排除 chunking 本身的贡献
ACT	检验 Transformer 行为生成
Flow/Diffusion	检验多模态动作建模
增加恢复示教	检验闭环扰动恢复

6.2 常见失败排查

优先检查时间对齐、动作/反馈混淆、相机外参、动作缩放、延迟、策略执行长度、数据划分与初始化分布，最后再考虑扩大模型。不能用只在训练集上计算的 action MSE 代替真机任务成功率。

7. 课程衔接与参考资料

本章连接相机标定、视觉状态估计、Flow Matching与GR00T / SONIC。

- ACT / ALOHA
- Diffusion Policy
- DAgger
- LeRobot

核验边界： 本章是方法与实验设计导论；没有声称这些算法已在本课程统一 Panda 环境复现。

' NPX . BUDI JOH：从数学原ã 到完整推导

课程首页 · English · PDF

项目	说明
适用范围	CFM 理论；OpenPI pi0.py 代码引用固定如下；示例不代表已完成微调实验
文档核验日期	2026-10-18
许可	CC BY 4.0 · Xiong Zeng and contributors

核验边界： 文档整理与上游引用快照核对；未进行教程环境、训练或硬件验证。 版本与固定源码引用见版本说明。原创教材采用 CC BY 4.0，第三方引用、图片及代码遵循其原有许可；注明来源不等于拥有再许可权。

目录

- 一、第一步：为什么需要 Flow Matching?
  - 1.1 生成模型究竟要解决什么问题?
  - 1.2 从容易采样的高斯分布开始
- 二、第二步：为什么引入 ODE?
  - 2.1 将复杂变换拆成连续的小变换
  - 2.2 从控制理论角度理解
- 三、第三步：为什么需要连续性方程?
  - 3.1 从粒子的运动出发
  - 3.2 连续性方程的推导
  - 3.3 连续性方程的物理含义
- 四、第四步：Flow Matching 的理想目标是什么?
  - 4.1 如果已知正确的速度场
  - 4.2 为什么理想目标不能直接训练?
- 五、第五步：Conditional Flow Matching 的构造
  - 5.1 构造两个端点
  - 5.2 直接得到可训练的损失函数
- 六、第六步：为什么条件速度回归能够得到全局速度场?
  - 6.1 区分单条路径速度与边缘速度
  - 6.2 严格推导边缘速度场
  - 6.3 用概率密度重新表达
- 七、第七步：为什么 MSE 能学到条件期望?
  - 7.1 最小二乘回归的正交分解
  - 7.2 为什么这是关键突破?
  - 7.3 为什么训练路径是直线，生成轨迹却不一定是直线?
- 八、第八步：为什么还要引入 Gaussian Conditional Path?
  - 8.1 一般高斯条件路径

    - 8.2 最简单的线性高斯路径
    - 8.3 推导速度场
- 九、第九步：Flow Matching 实际如何训练和生成?
    - 9.1 训练过程
    - 9.2 推理过程
- 十、第十步：为什么 Flow Matching 能生成多模态分布?
- 十一、第十一步：π0.5 如何使用 Flow Matching?
    - 11.1 生成对象是 Action Chunk
    - 11.2 条件 Flow Matching 如何生成动作块?
    - 11.3 重要实现细节：OpenPI 使用相反的时间方向
- 十二、第十二步：Flow Matching 如何用于强化学习微调?
    - 12.1 预训练 Flow Matching 学到了什么?
    - 12.2 为什么可以学习 Residual Velocity?
    - 12.3 为什么不能把 Reward 直接当作 Velocity Label?
    - 12.4 Advantage-Weighted Flow Matching 的一个自然设计
- 十三、总结：Flow Matching 的完整数学逻辑
    - 13.1 八个关键环节
    - 13.2 三个最重要的理解

本章知识地图



一、第一步：为什么需要 Flow Matching?

1.1 生成模型究竟要解决什么问题?

假设训练数据来自未知分布：

$$x \sim p_{\text{data}}(x).$$

例如：图像生成时，x 是一张图片；视频生成时，x 是一段视频；机器人控制时，x 可以是一个 Action Chunk；灵巧手控制时，x 可以是未来若干时刻的关节动作序列。

目标是学习生成模型，使得：

$$x_{\text{generated}} \sim p_{\text{data}}(x).$$

真实数据分布通常十分复杂。例如，机器人抓取杯子可能有左侧抓取、右侧抓取两种策略，对应多模态分布：

$$p_{\text{data}}(x) = \frac{1}{2}p_{\text{left}}(x) + \frac{1}{2}p_{\text{right}}(x).$$

如果直接训练确定性网络 $f_\theta(o) \to x$，使用平方损失：

$$\mathcal{L}(\theta) = \mathbb{E}\left[\|f_\theta(o) - x\|^2\right],$$

其理论最优解是条件均值：

$$f_\theta^*(o) = \mathbb{E}[x \mid o].$$

这个均值可能既不是左侧抓取，也不是右侧抓取。

动机： 我们希望学习整个数据分布，而不只是预测其均值。

1.2 从容易采样的高斯分布开始

从标准高斯分布采样非常简单：

$$\boxed{x_0 \sim p_0(x) = \mathcal{N}(0, I).}$$

因此可以换一种思路：不直接预测真实数据，而是学习一个变换

$$F_\theta : \mathbb{R}^d \to \mathbb{R}^d, \qquad x_1 = F_\theta(x_0),$$

使得

$$\boxed{(F_\theta)_{\#} p_0 = p_{\text{data}}.}$$

其中 $\#$ 表示 **Pushforward（推前分布）**。直观地讲，将许多高斯采样点通过 F_θ 后，输出点的总体分布应当与真实数据分布一致：

$$\underbrace{\mathcal{N}(0, I)}_{\text{简单高斯分布}} \xrightarrow{F_\theta} \underbrace{p_{\text{data}}}_{\text{复杂多模态分布}}.$$

关键问题变成：**怎样学习这个分布变换?**

二、第二步：为什么引入 ODE?

2.1 将复杂变换拆成连续的小变换

直接学习 $x_1 = F_\theta(x_0)$ 可能很困难。为此，引入连续的**虚拟生成时间**：

$$t \in [0, 1],$$

约定 $t = 0$ 时对应高斯分布，$t = 1$ 时对应真实数据分布：

$$t = 0: \ x_0 \sim p_0, \qquad t = 1: \ x_1 \sim p_{\text{data}}.$$

在两个端点之间，样本沿连续轨迹移动：

$$x_0 \longrightarrow x_t \longrightarrow x_1.$$

定义时间相关的速度场：

$$v_\theta(x, t) : \mathbb{R}^d \times [0, 1] \to \mathbb{R}^d,$$

并建立常微分方程（ODE）：

$$\boxed{\frac{dx_t}{dt} = v_\theta(x_t, t).} \tag{1}$$

初始条件为 $x_0 \sim \mathcal{N}(0, I)$。积分得到：

$$x_1 = x_0 + \int_0^1 v_\theta(x_t, t)\, dt.$$

动机： 神经网络不再一次性预测最终样本，而是回答“给定当前位置 x_t 和时间 t，样本下一步应该沿什么方向移动？”这将全局生成问题转化成局部速度预测问题。

这种模型属于 **Continuous Normalizing Flow（CNF，连续归一化流）**。

2.2 从控制理论角度理解

从动力系统角度，Flow Matching 学习非自治系统：

$$\dot{x} = v_\theta(x, t).$$

但它的目标不是单个状态收敛到平衡点，而是让**整体概率分布**按照设计的路径演化：

$$p_0 \longrightarrow p_t \longrightarrow p_1.$$

普通控制常关心 $x(0) \to x_{\text{target}}$；Flow Matching 更关心：

$$\boxed{p_0(x) \to p_{\text{target}}(x).}$$

无需预先规定每个噪声样本必须对应哪个真实数据点；最终生成分布正确才是核心要求。

三、第三步：为什么需要连续性方程？

3.1 从粒子的运动出发

设大量粒子 $x_t^{(1)}, \ldots, x_t^{(N)}$ 均满足：

$$\frac{dx_t^{(i)}}{dt} = v_t(x_t^{(i)}).$$

这些粒子共同构成概率密度 $p_t(x)$。粒子流入某一区域，该区域的密度增大；粒子流出该区域，密度减小。

因此，必须建立速度场与概率密度变化之间的关系。

3.2 连续性方程的推导

取任意光滑测试函数 $\phi : \mathbb{R}^d \to \mathbb{R}$，考虑：

$$\mathbb{E}[\phi(x_t)] = \int \phi(x) p_t(x)\, dx.$$

对时间求导（假设具备交换求导与积分的正则性）：

$$\frac{d}{dt}\mathbb{E}[\phi(x_t)] = \int \phi(x) \frac{\partial p_t(x)}{\partial t}\, dx.$$

另一方面，链式法则给出：

$$\frac{d}{dt}\phi(x_t) = \nabla\phi(x_t)^\top \frac{dx_t}{dt} = \nabla\phi(x_t)^\top v_t(x_t).$$

于是：

$$\frac{d}{dt}\mathbb{E}[\phi(x_t)] = \int \nabla\phi(x)^\top v_t(x) p_t(x)\, dx.$$

假设边界项为零，对右侧进行分部积分：

$$= -\int \phi(x) \nabla \cdot [p_t(x) v_t(x)]\, dx.$$

比较两种表达式：

$$\int \phi(x) \left[\frac{\partial p_t(x)}{\partial t} + \nabla \cdot (p_t(x) v_t(x)) \right] dx = 0.$$

由于对任意适当的 ϕ 都成立，得到：

$$\boxed{\frac{\partial p_t(x)}{\partial t} + \nabla \cdot [p_t(x) v_t(x)] = 0.} \tag{2}$$

这就是 **Continuity Equation（连续性方程）**。

3.3 连续性方程的物理含义

将散度项展开：

$$\frac{\partial p_t}{\partial t} + v_t^\top \nabla p_t + p_t \nabla \cdot v_t = 0.$$

其中 $v_t^\top \nabla p_t$ 表示密度的对流变化，$\nabla \cdot v_t$ 表示局部速度场的发散。

当 $\nabla \cdot v_t > 0$ 时，局部粒子趋向扩张，密度倾向降低；当 $\nabla \cdot v_t < 0$ 时，局部粒子趋向汇聚，密度倾向升高。

沿 ODE 轨迹还可得到：

$$\boxed{\frac{d}{dt} \log p_t(x_t) = -\nabla \cdot v_t(x_t).} \tag{3}$$

这是 CNF 能够计算样本对数密度的理论基础。

动机： ODE 描述单个粒子的运动，连续性方程描述由这些粒子组成的概率分布如何变化。两者建立了生成动力学与目标分布之间的联系。

四、第四步：Flow Matching 的理想目标是什么？

4.1 如果已知正确的速度场

假设我们事先选定了一条概率分布路径 $p_t(x)$，满足：

$$p_{t=0} = p_0, \qquad p_{t=1} = p_{\text{data}},$$

并且存在速度场 $u_t(x)$，使得：

$$\frac{\partial p_t}{\partial t} + \nabla \cdot (p_t u_t) = 0.$$

那么可以直接回归这个速度场：

$$\boxed{\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{\substack{t\sim\mathcal{U}[0,1] \\ x\sim p_t}} \left[\| v_\theta(x,t) - u_t(x) \|^2 \right].} \tag{4}$$

如果能够精确学到 $v_\theta = u_t$，由该速度场诱导的 ODE 就可实现选定的分布演化。

4.2 为什么理想目标不能直接训练？

真实情况下，我们通常只拥有：

$$x_1 \sim p_{\text{data}},$$

并没有观测到“从高斯逐渐演变为真实数据”的过程，因此既不知道中间的 $p_t(x)$，也不知道所需的 $u_t(x)$。

核心动机： 能否人为设计一条容易采样、速度又可直接计算的路径，并用这条路径给神经网络提供训练信号？

这就引出了 Conditional Flow Matching（CFM）。

五、第五步：Conditional Flow Matching 的构造

5.1 构造两个端点

从高斯分布与真实数据分布中分别采样：

$$x_0 \sim p_0 = \mathcal{N}(0, I), \qquad x_1 \sim p_{\text{data}}.$$

最简单的方式是令两个端点独立：

$$x_0 \perp x_1.$$

对于一对端点 (x_0, x_1)，人为指定匀速直线运动：

$$\boxed{x_t = (1-t)x_0 + t x_1.} \tag{5}$$

检查边界条件：$t=0$ 时 $x_t = x_0$；$t=1$ 时 $x_t = x_1$。

例如：

$$x_0 = \begin{bmatrix} -1 \\ 2 \end{bmatrix}, \quad x_1 = \begin{bmatrix} 3 \\ 4 \end{bmatrix}.$$

对 (5) 求导：

$$\boxed{\frac{dx_t}{dt} = x_1 - x_0.} \tag{6}$$

在上述例子中，目标速度是 $[4, 2]^\top$。它不依赖 t，因为我们选择了匀速直线插值。

为什么用直线？ 对固定端点而言它计算最简单，速度标签能够显式写出，且不需要训练额外模型或模拟中间轨迹。

5.2 直接得到可训练的损失函数

训练时随机采样：

$$x_0 \sim \mathcal{N}(0, I), \quad x_1 \sim p_{\text{data}}, \quad t \sim \mathcal{U}[0, 1].$$

再构造：

$$x_t = (1-t)x_0 + tx_1, \qquad u_t = x_1 - x_0.$$

用神经网络预测速度：

$$v_\theta(x_t, t) \approx x_1 - x_0.$$

得到：

$$\boxed{\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t, x_0, x_1}\left[\left\| v_\theta((1-t)x_0 + tx_1, t) - (x_1 - x_0) \right\|^2\right].} \tag{7}$$

这就是常用的**直线路径 Conditional Flow Matching**。

一轮训练只需采样一个数据点、一份噪声、一个随机时间，计算插值和目标速度，再做一次网络前向传播与均方误差（MSE）优化。

关键优点： 训练时不需要求解 ODE，因此这种训练方式通常称为 *simulation-free training*。

但是还有一个深层疑问：网络输入只有 (x_t, t)，并不知道真实端点 (x_0, x_1)。为什么回归 $x_1 - x_0$ 就能得到正确的全局速度场？

六、第六步：为什么条件速度回归能够得到全局速度场？

6.1 区分单条路径速度与边缘速度

给定端点 (x_0, x_1)，单条路径的速度是：

$$u_t(x \mid x_0, x_1) = x_1 - x_0.$$

它依赖于端点选择，可称为**条件速度**。

但推理时并不知道最终数据端点，因此网络必须预测只依赖当前位置和时间的速度场：

$$v_t(x).$$

同一个 (x, t) 可能被许多不同的直线路径经过；它们的速度可能并不相同。网络应该预测什么？答案是：**这些条件速度的条件期望**。

6.2 严格推导边缘速度场

用大写字母表示随机变量。定义：

$$Z = (X_0, X_1), \qquad X_t = (1-t)X_0 + tX_1,$$

以及：

$$U_t = X_1 - X_0.$$

由于 $dX_t/dt = U_t$，对于光滑测试函数 ϕ：

$$\frac{d}{dt}\mathbb{E}[\phi(X_t)] = \mathbb{E}[\nabla\phi(X_t)^\top U_t].$$

应用条件期望的塔式性质：

$$= \mathbb{E}\left[\nabla\phi(X_t)^\top \mathbb{E}[U_t \mid X_t]\right].$$

定义：

$$\boxed{v_t^*(x) = \mathbb{E}[U_t \mid X_t = x].} \tag{8}$$

于是：

$$\frac{d}{dt}\mathbb{E}[\phi(X_t)] = \int \nabla\phi(x)^\top v_t^*(x) p_t(x)\, dx.$$

通过分部积分：

$$= -\int \phi(x) \nabla \cdot [p_t(x) v_t^*(x)]\, dx.$$

另一方面：

$$\frac{d}{dt}\mathbb{E}[\phi(X_t)] = \int \phi(x)\partial_t p_t(x)\, dx.$$

因此：

$$\boxed{\partial_t p_t(x) + \nabla \cdot [p_t(x) v_t^*(x)] = 0.} \tag{9}$$

也就是说，条件期望构成了一个能够实现目标边缘概率路径的速度场：

$$\boxed{v_t^*(x) = \mathbb{E}[X_1 - X_0 \mid X_t = x].} \tag{10}$$

这是 CFM 的核心数学结论之一。

6.3 用概率密度重新表达

更一般地，令 z 为条件变量，$p_t(x \mid z)$ 为条件概率路径，$u_t(x \mid z)$ 为其条件速度。边缘分布为：

$$p_t(x) = \int p_t(x \mid z) q(z)\, dz.$$

可定义：

$$\boxed{v_t^*(x) = \frac{\int u_t(x \mid z) p_t(x \mid z) q(z)\, dz}{p_t(x)}.} \tag{11}$$

其本质仍是：

$$v_t^*(x) = \mathbb{E}[u_t(x \mid Z) \mid X_t = x].$$

对固定端点的确定性插值，条件路径退化为位于插值点的 Dirac 分布，使用上一节的随机变量推导更直接。

注意： 连续性方程可能允许不止一个满足要求的速度场。Flow Matching 并不是寻找一个与路径选择无关、唯一存在的“真实速度场”，而是在指定概率路径后，学习能够实现这条路径的速度场。

七、第七步：为什么 MSE 能学到条件期望？

7.1 最小二乘回归的正交分解

固定时间 t，定义：

$$m_t(X_t) = \mathbb{E}[U_t \mid X_t].$$

考虑训练损失：

$$\mathcal{L}(\theta) = \mathbb{E}[\|v_\theta(X_t, t) - U_t\|^2].$$

在括号内加上、减去 m_t：

$$v_\theta - U_t = (v_\theta - m_t) + (m_t - U_t).$$

展开平方：

$$\begin{aligned}\|v_\theta - U_t\|^2 = \|v_\theta - m_t\|^2 + \|m_t - U_t\|^2 \\ + 2(v_\theta - m_t)^\top (m_t - U_t).\end{aligned}$$

由于：

$$\mathbb{E}[m_t - U_t \mid X_t] = 0,$$

并且 $v_\theta(X_t, t) - m_t(X_t)$ 仅依赖于 X_t，交叉项取期望为零：

$$\mathbb{E}[(v_\theta - m_t)^\top (m_t - U_t)] = 0.$$

于是：

$$\boxed{\begin{aligned}\mathcal{L}_{\mathrm{CFM}}(\theta) = \mathbb{E}[\|v_\theta(X_t, t) - m_t(X_t)\|^2] \\ + \mathbb{E}[\|U_t - m_t(X_t)\|^2].\end{aligned}} \tag{12}$$

第二项与参数 θ 无关。因此，在相同采样分布和平方误差定义下：

$$\boxed{\arg\min_\theta \mathcal{L}_{\mathrm{CFM}}(\theta) = \arg\min_\theta \mathcal{L}_{\mathrm{FM}}(\theta).} \tag{13}$$

在可交换求导与积分的条件下，进一步有：

$$\boxed{\nabla_\theta \mathcal{L}_{\mathrm{CFM}} = \nabla_\theta \mathcal{L}_{\mathrm{FM}}.} \tag{14}$$

7.2 为什么这是关键突破？

我们想学的是：

$$v_t^*(x) = \mathbb{E}[U_t \mid X_t = x],$$

但这个条件期望难以显式计算。

CFM 的做法是：**不先计算条件期望，而是直接回归随机采样的 U_t**。 MSE 的理论性质会使最优预测自动逼近条件期望。

$$\underbrace{\text{难计算的边缘速度场}}_{v_t^*(x)} \quad \Longleftarrow \quad \underbrace{\text{可采样的条件速度}}_{X_1 - X_0}.$$

这是 Conditional Flow Matching 能被高效训练的根本原因。

7.3 为什么训练路径是直线，生成轨迹却不一定是直线?

训练时每对端点的路径确实是直线:

$$X_t = (1-t)X_0 + tX_1.$$

但最终学习的是条件平均速度:

$$v_t^*(x) = \mathbb{E}[X_1 - X_0 \mid X_t = x].$$

例如，两条路径在同一个位置相交，条件速度分别为:

$$u_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \qquad u_2 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}.$$

如果两条路径在该位置的条件权重相同，则:

$$v_t^*(x) = \frac{1}{2}u_1 + \frac{1}{2}u_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

这个速度不等于任意一条单独路径的速度。因此求解 $\dot{x}_t = v_t^*(x_t)$ 后得到的生成轨迹可以是弯曲的。

重要结论： 训练样本使用直线插值，不代表最终 ODE 生成轨迹必然是直线。

八、第八步：为什么还要引入 Gaussian Conditional Path?

前面的直线插值只是 Flow Matching 的一个特例。原始框架允许使用更一般的条件概率路径。

8.1 一般高斯条件路径

给定一个数据样本 x_1，定义:

$$\boxed{p_t(x \mid x_1) = \mathcal{N}\left(x; \mu_t(x_1), \sigma_t^2 I\right).} \tag{15}$$

其中 $\mu_t(x_1)$ 控制分布中心如何移动，σ_t 控制分布如何扩张或收缩。

该分布可以通过采样表示:

$$\boxed{X_t = \mu_t(x_1) + \sigma_t \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).} \tag{16}$$

对时间求导:

$$\frac{dX_t}{dt} = \dot{\mu}_t(x_1) + \dot{\sigma}_t \epsilon.$$

根据 (16)：

$$\epsilon = \frac{X_t - \mu_t(x_1)}{\sigma_t}.$$

代回可得条件速度场：

$$\boxed{u_t(x \mid x_1) = \dot{\mu}_t(x_1) + \frac{\dot{\sigma}_t}{\sigma_t}[x - \mu_t(x_1)].} \tag{17}$$

推导要求相应时刻 $\sigma_t > 0$；对于终点方差趋零的情形，应按极限理解端点。

动机： 只要人为指定 μ_t 和 σ_t，就能直接计算条件速度，使用同样的 MSE 框架训练模型。直线插值不是唯一可选路径。

8.2 最简单的线性高斯路径

选择：

$$\mu_t(x_1) = t x_1, \qquad \sigma_t = 1 - (1 - \sigma_{\min})t,$$

其中 $0 \le \sigma_{\min} \ll 1$。于是：

$$\boxed{X_t = t x_1 + [1 - (1 - \sigma_{\min})t]\epsilon.} \tag{18}$$

当 $t = 0$ 时，$X_0 = \epsilon \sim \mathcal{N}(0, I)$。

当 $t = 1$ 时，$X_1 = x_1 + \sigma_{\min}\epsilon$，即：

$$X_1 \mid x_1 \sim \mathcal{N}(x_1, \sigma_{\min}^2 I).$$

当 $\sigma_{\min} = 0$ 时，终点恰好为真实样本；若 $\sigma_{\min} > 0$，终点边缘分布是数据分布与一个小高斯的卷积，并非严格等于原始数据分布。

8.3 推导速度场

对 (18) 求导：

$$\frac{dX_t}{dt} = x_1 - (1 - \sigma_{\min})\epsilon.$$

因此目标速度为：

$$\boxed{u_t = x_1 - (1 - \sigma_{\min})\epsilon.} \tag{19}$$

令 $\sigma_{\min} = 0$，得到：

$$X_t = (1 - t)\epsilon + t x_1,$$

以及：

$$\boxed{u_t = x_1 - \epsilon.} \tag{20}$$

这正是之前的直线插值 Flow Matching。

总结： 常见的 $x_1 - \epsilon$ 速度回归目标可以看作一般 Gaussian Conditional Flow Matching 的一个特例。不同的概率路径会影响监督目标、学习难度与数值积分表现。

九、第九步：Flow Matching 实际如何训练和生成？

9.1 训练过程

给定真实数据样本：

$$x_1 \sim p_{\text{data}}.$$

依次执行以下操作：

1. 采样高斯噪声：$x_0 \sim \mathcal{N}(0, I)$。
2. 随机采样时间：$t \sim \mathcal{U}(0, 1)$。
3. 构造插值样本：$x_t = (1 - t)x_0 + tx_1$。
4. 构造目标速度：$u_t = x_1 - x_0$。
5. 网络预测速度：$\hat{u}_t = v_\theta(x_t, t)$。
6. 以 $\mathcal{L} = \|\hat{u}_t - u_t\|^2$ 优化参数。

整个训练过程不需要对生成 ODE 进行逐步积分。

下面是简化的 PyTorch 实现：

```
import torch


def flow_matching_loss(model, x1, cond=None):
    # x1 shape: [B, D] or [B, H, D]
    B = x1.shape[0]

    # 1. Gaussian noise
    x0 = torch.randn_like(x1)

    # 2. Random flow timestep
    t = torch.rand(B, device=x1.device)
    t_view = t.view(B, *([1] * (x1.ndim - 1)))

    # 3. Linear interpolation
    xt = (1 - t_view) * x0 + t_view * x1

    # 4. Target velocity
    target_v = x1 - x0

    # 5. Predict velocity
    pred_v = model(xt, t, cond)

    # 6. Mean squared error
    loss = ((pred_v - target_v) ** 2).mean()
    return loss
```

9.2 推理过程

生成时不再需要真实样本 x_1。首先采样：

$$x_0 \sim \mathcal{N}(0, I),$$

然后求解：

$$\frac{dx_t}{dt} = v_\theta(x_t, t).$$

最简单的积分方法是 Euler。将 $[0,1]$ 均分为 K 步，令 $\Delta t = 1/K$，则：

$$\boxed{x_{k+1} = x_k + \Delta t\, v_\theta(x_k, k\Delta t).} \tag{21}$$

最终 $x_K \approx x_1$。PyTorch 伪代码：

```
@torch.no_grad()
def sample_flow(model, shape, cond=None, K=10, device="cuda"):
    x = torch.randn(shape, device=device)
    dt = 1.0 / K

    for k in range(K):
        t = torch.full((shape[0],), k * dt, device=device)
        v = model(x, t, cond)
        x = x + dt * v

    return x
```

除了 Euler，还可以使用 Heun、Midpoint、Runge–Kutta 和自适应 ODE 求解器。高阶求解器可能降低数值误差，但单个步长往往需要更多次网络评估，因此需要在精度和 NFE（函数评估次数）之间权衡。

十、第十步：为什么 Flow Matching 能生成多模态分布？

假设相同观察 o 对应左右两种抓取方式：

$$p(A \mid o) = \frac{1}{2} p_{\text{left}}(A \mid o) + \frac{1}{2} p_{\text{right}}(A \mid o).$$

普通 MSE 策略 $f_\theta(o)$ 只看到观察 o，最优输出为条件均值 $\mathbb{E}[A \mid o]$，可能落在两种动作之间。

而 Flow Matching 的模型输入为：

$$v_\theta(x_t, t, o),$$

其中起始样本来自：

$$x_0 \sim \mathcal{N}(0, I).$$

不同初始噪声可以被传输到不同动作模态。

即使给定 x_0 和 o 后 ODE 的输出是确定的，X_0 自身仍是随机变量，因此最终：

$$X_1 = \Phi_1(X_0; o)$$

也仍是随机变量，其条件分布为：

$$\boxed{p_\theta(A \mid o) = (\Phi_1(\cdot; o))_{\#}\mathcal{N}(0, I).} \tag{22}$$

关键区别： Flow Matching 虽然也使用 MSE，但它回归的是传输速度场而非最终动作的单一均值，所以可以对多模态动作分布建模。

十一、第十一步：π0.5 如何使用 Flow Matching?

11.1 生成对象是 Action Chunk

机器人动作模型希望学习：

$$\boxed{p(A \mid o, \ell),}$$

其中：

$$A = [a_0, a_1, \ldots, a_{H-1}]$$

是未来一段动作序列；o 是视觉和机器人状态，ℓ 是语言或高层任务条件。

若单步动作维度为 d_a，则：

$$A \in \mathbb{R}^{H \times d_a}.$$

例如 $H = 50$、$d_a = 18$，则整个 Action Chunk 有 900 个连续变量。Flow Matching 把整个动作块作为高维变量进行生成，而不是将每一维动作逐个自回归采样。

11.2 条件 Flow Matching 如何生成动作块?

先沿用本讲义的时间约定：$t = 0$ 是噪声，$t = 1$ 是真实动作。令：

$$\epsilon \sim \mathcal{N}(0, I),$$

并构造：

$$\boxed{A_t = (1 - t)\epsilon + tA.} \tag{23}$$

目标速度：

$$\boxed{u_t = A - \epsilon.} \tag{24}$$

于是训练 Action Expert：

$$\boxed{\mathcal{L}_{\mathrm{FM}} = \mathbb{E}\left[\left\| v_\theta(A_t, t, o, \ell) - (A - \epsilon) \right\|^2\right].} \tag{25}$$

相比无条件 Flow Matching，只需将观察和语言条件输入速度场。理想条件速度场为：

$$\boxed{v_t(x \mid o, \ell) = \mathbb{E}[U_t \mid X_t = x, o, \ell].} \tag{26}$$

这表示网络学习：在给定观察和任务要求时，如何将高斯噪声变成合理的动作块。

11.3 重要实现细节：OpenPI 使用相反的时间方向

为了复现 π0.5，必须区分论文讲解中的时间约定和实际代码实现。

在 OpenPI 官方 `pi0.py` 的相关 Flow Matching 实现中，时间约定是：

$$t = 1：噪声， \qquad t = 0：真实动作.$$

具体插值为：

$$\boxed{A_t = t\epsilon + (1-t)A.} \tag{27}$$

因此：

$$\boxed{\frac{dA_t}{dt} = \epsilon - A.} \tag{28}$$

速度目标为：

$$\boxed{\mathcal{L} = \|v_\theta(A_t, t, o) - (\epsilon - A)\|^2.} \tag{29}$$

即预测的是 **noise - action**，而不是 **action - noise**。这两种定义都可以使用，但生成时间方向必须与速度目标保持一致。

生成从 $A_1 \sim \mathcal{N}(0, I)$ 开始，按 $t : 1 \to 0$ 积分。使用 K 步 Euler 时，$\Delta t = -1/K$：

$$A_{t+\Delta t} = A_t + \Delta t\, v_\theta(A_t, t, o),$$

即：

$$\boxed{A_{t-1/K} = A_t - \frac{1}{K} v_\theta(A_t, t, o).} \tag{30}$$

积分终点 A_0 就是生成的动作块。OpenPI 的相关默认配置常采用 10 个积分步，并可缓存视觉语言主干前缀的 KV，以降低每步重复计算的成本；具体参数应以使用的代码版本和配置为准。

Flow Matching Velocity 不等于机器人关节速度

必须区分两种时间：

- **生成时间 t：** ODE 从噪声生成 Action Chunk 的虚拟时间。
- **物理时间：** 机器人实际执行 $a_0, a_1, \ldots, a_{H-1}$ 的时间。

如果 $A_t \in \mathbb{R}^{50\times 18}$，则：

$$v_\theta(A_t, t, o) \in \mathbb{R}^{50\times 18}.$$

该速度描述的是整个 noisy action chunk 沿生成时间的变化，并非机械臂或灵巧手的物理关节速度。

十二、第十二步：Flow Matching 如何用于强化学习微调?

这一节把前面的推导联系到 π0.5、DSRL 和 **Residual Velocity Adaptation** 的研究问题。

12.1 预训练 Flow Matching 学到了什么?

模仿学习得到一个速度场:

$$v_{\text{pre}}(x, t, o),$$

它对应的生成分布大致与演示动作分布 $p_{\text{demo}}(A \mid o)$ 一致。

通过:

$$\frac{dA_t}{dt} = v_{\text{pre}}(A_t, t, o)$$

能够生成与训练演示数据相似的动作，但这些动作未必在当前任务中具有最高回报。

例如，相同物体有多种有效抓取方式，但成功率、稳定性和完成时间可能不同。强化学习后训练的目标是让生成分布更偏向高回报动作。

12.2 为什么可以学习 Residual Velocity?

一种自然的设计是冻结预训练速度场，只训练一个较小的残差:

$$\boxed{v_{\text{new}}(x, t, o) = v_{\text{pre}}(x, t, o) + \Delta v_\phi(x, t, o).} \tag{31}$$

生成 ODE 因此变成:

$$\boxed{\frac{dA_t}{dt} = v_{\text{pre}}(A_t, t, o) + \Delta v_\phi(A_t, t, o).} \tag{32}$$

动机类似控制理论中的 Residual Control：保留预训练策略的基础能力，同时通过增量修正适应新的奖励或任务目标。

但要注意，这是一种后训练算法设计，并不是原始 Flow Matching 理论必然要求的结构。

12.3 为什么不能把 Reward 直接当作 Velocity Label?

预训练中，条件速度目标是:

$$u_t = A - \epsilon.$$

这是由明确的路径方程求导得到的监督信号。

强化学习得到的通常是:

$$R(A, o),$$

它是对最终动作或执行轨迹的回报，不直接告诉我们每个中间生成状态上的残差速度:

$$\Delta v_\phi(A_t, t, o).$$

所以不能简单把 Advantage 解释成生成空间中的速度标签。需要建立 **Reward → 目标动作分布 → 速度场更新** 的合理联系。

12.4 Advantage-Weighted Flow Matching 的一个自然设计

假设在线探索得到数据：

$$\mathcal{D} = \{(o_i, A_i, R_i)\}_{i=1}^{N}.$$

为高回报动作赋予更大的正权重：

$$w_i = \exp\left(\frac{\widehat{\mathrm{Adv}}_i}{\beta}\right), \qquad \beta > 0.$$

定义加权残差速度回归目标：

$$\boxed{\begin{aligned}\mathcal{L}(\phi) = \mathbb{E}_{\mathcal{D},\epsilon,t}\Big[w(A,o)\big\| v_{\mathrm{pre}}(A_t,t,o) \\ + \Delta v_\phi(A_t,t,o) - (A-\epsilon)\big\|^2\Big],\end{aligned}} \tag{33}$$

其中：

$$A_t = (1-t)\epsilon + tA.$$

还可以加入残差正则化：

$$\lambda\,\mathbb{E}\left[\|\Delta v_\phi(A_t,t,o)\|^2\right]$$

来限制与预训练速度场的偏离。

为什么这个方案有数学动机?

回顾：MSE 回归的最优解是相应联合分布下条件速度的条件期望。

对固定观察 o，定义重新加权后的动作分布：

$$\boxed{p_w(A \mid o) = \frac{w(A,o)p_{\mathcal{D}}(A \mid o)}{Z(o)},} \tag{34}$$

其中：

$$Z(o) = \mathbb{E}_{A\sim p_{\mathcal{D}}(\cdot|o)}[w(A,o)].$$

在训练数据有足够覆盖、权重非负可积、模型容量和优化充分的前提下，加权 CFM 可以解释为对加权目标动作分布进行速度场学习。

因此研究逻辑可以写成：

$$\boxed{\text{Advantage} \longrightarrow \text{加权动作分布} \longrightarrow \text{Flow Matching} \longrightarrow \text{新速度场}.}$$

重要限制： 加权 Flow Matching 不自动等价于在线 Policy Gradient，也不保证策略性能单调改善。实际表现仍受到数据覆盖、Advantage 估计、分布偏移、数值积分以及约束设计的影响。

十三、总结：Flow Matching 的完整数学逻辑

13.1 八个关键环节

1. 确定生成目标。 从高斯分布生成真实数据：

$$p_0 = \mathcal{N}(0, I) \longrightarrow p_1 = p_{\text{data}}.$$

2. 用 ODE 定义连续变换。

$$\dot{x}_t = v_\theta(x_t, t).$$

其诱导的概率路径应满足：

$$\partial_t p_t + \nabla \cdot (p_t v_\theta) = 0.$$

3. 理想目标是回归全局速度场。

$$\mathcal{L}_{\text{FM}} = \mathbb{E}[\|v_\theta(x,t) - v_t^*(x)\|^2],$$

但 v_t^* 通常未知。

4. 构造可计算的条件路径。

$$x_t = (1-t)x_0 + t x_1, \qquad u_t = x_1 - x_0.$$

5. 证明边缘速度是条件期望。

$$\boxed{v_t^*(x) = \mathbb{E}[X_1 - X_0 \mid X_t = x].}$$

6. 利用 MSE 的 L^2 投影性质。

$$\boxed{\mathbb{E}\|v_\theta - U_t\|^2 = \mathbb{E}\|v_\theta - v_t^*\|^2 + C.}$$

7. 得到简单的训练损失。

$$\boxed{\min_\theta \mathbb{E}_{\substack{t\sim\mathcal{U}[0,1] \\ x_0\sim\mathcal{N}(0,I) \\ x_1\sim p_{\text{data}}}} \Big[\big\| v_\theta((1-t)x_0 + t x_1, t) - (x_1 - x_0) \big\|^2 \Big].}$$

8. 生成时通过 ODE 积分。

$$\boxed{\begin{cases} x_0 \sim \mathcal{N}(0, I), \\ \dot{x}_t = v_\theta(x_t, t), \\ x_1 \sim p_\theta \approx p_{\text{data}}. \end{cases}}$$

最后一行表示生成样本的总体分布近似真实数据分布，并不意味着每个生成样本必须等于训练集中某个已有样本。

13.2 三个最重要的理解

第一，Flow Matching 是学习概率分布的动力系统。

$$\boxed{\dot{x}_t = v_t(x_t) \quad \Longleftrightarrow \quad \partial_t p_t = -\nabla \cdot (p_t v_t).}$$

它将生成模型与连续时间动力系统联系起来。

第二，CFM 的核心是条件期望与 L^2 投影。

$$\boxed{v_t^*(x) = \mathbb{E}[U_t \mid X_t = x].}$$

这解释了为什么简单的随机路径监督就足以构造复杂分布的速度场学习问题。

第三，训练时的直线插值不等于推理时的直线轨迹。 神经网络学习的是全局边缘速度场，受条件路径与样本分布共同影响。

对于 Flow Matching Policy 的在线后训练，可以进一步研究：

$$\boxed{\text{改变动作分布} \quad \Longleftrightarrow \quad \text{改变概率传输的速度场.}}$$

如何利用 Reward 或 Advantage 稳定改变速度场、同时保留预训练生成能力，是一个独立于标准 Flow Matching、值得系统研究的优化问题。

文档说明： 本版仅保留 Flow Matching 基础推导、条件流匹配、实现流程、π0.5 和强化学习残差速度微调内容；已删除指定的两部分，并将后续章节与公式编号连续调整。

；形机器； System 1（VLA）与 System 0（½ 身运动控制）

课程首页 · English · PDF

项目	说明
适用范围	SONIC 论文集成：GR00T N1.5；文档 G1 工作流：N1.7、78 维动作、2.5 Hz 推理 / 50 Hz 动作流。Checkpoint 与观测配置须匹配
文档核验日期	2026-10-18
许可	CC BY 4.0 · Xiong Zeng and contributors

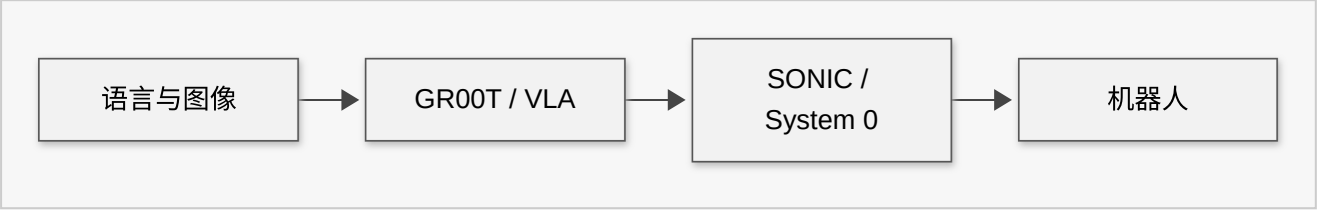
核验边界： 文档整理与上游引用快照核对；未进行教程环境、训练或硬件验证。 版本与固定源码引用见版本说明。原创教材采用 CC BY 4.0，第三方引用、图片及代码遵循其原有许可；注明来源不等于拥有再许可权。

目录

- 1. 整体架构：两个系统分别解决什么问题
  - 1.1 为什么不能只靠一个模型?
  - 1.2 核心区别
  - 1.3 两个闭环，而不是一个开环
- 2. System 1：GR00T 如何理解任务并生成动作
  - 2.1 VLA 是什么?
  - 2.2 GR00T 的两部分结构
  - 2.3 为什么一次预测多个动作?
  - 2.4 Flow Matching 的直觉解释
  - 2.5 VLA 输出可以如何表示?
  - 2.6 GR00T 的训练分为哪几步?
- 3. System 0：SONIC 如何学习全身运动控制
  - 3.1 什么是全身运动控制（WBC）?
  - 3.2 先认识一条常见替代路线：下半身 RL + 上半身 IK
  - 3.3 SONIC 的核心任务：Motion Tracking
  - 3.4 SONIC 的网络：编码器 + Motion Token + 控制解码器
  - 3.5 SONIC 如何从 token 得到关节控制?
  - 3.6 SONIC 的强化学习训练：PPO + 跟踪奖励
  - 3.7 SONIC 学到的是什么，又没有学到什么?
- 4. 两个系统如何通过 Motion Token 连接
  - 4.1 最关键的接口公式
  - 4.2 官方实例中的准确维度
  - 4.3 两种控制频率如何配合?

    - 4.4 为什么这种分层结构更容易泛化?
    - 4.5 分层接口必须处理的工程细节
- 5. 完整例子：走路、抓取、踩踏板、投放
    - 5.1 示例任务
    - 5.2 各模块分别做什么?
    - 5.3 一个容易混淆的边界
- 6. 从零训练、采集数据与真机部署
    - 6.1 Step 1：获得可用的 System 0
    - 6.2 Step 2：通过 SONIC 进行 VR 遥操作示教
    - 6.3 Step 3：微调 GR00T，让它输出 SONIC 动作
    - 6.4 Step 4：通过 PolicyServer + SONIC 部署
    - 6.5 Step 5：如何评价整体系统?
- 7. 研究方向、实验设计与常见误区
    - 7.1 应该分别改进什么?
    - 7.2 建议的最小消融实验
    - 7.3 五个常见误区
- 8. 总结与参考资料
    - 8.1 最值得记住的三条结论
    - 8.2 参考论文与官方资源

章节知识地图



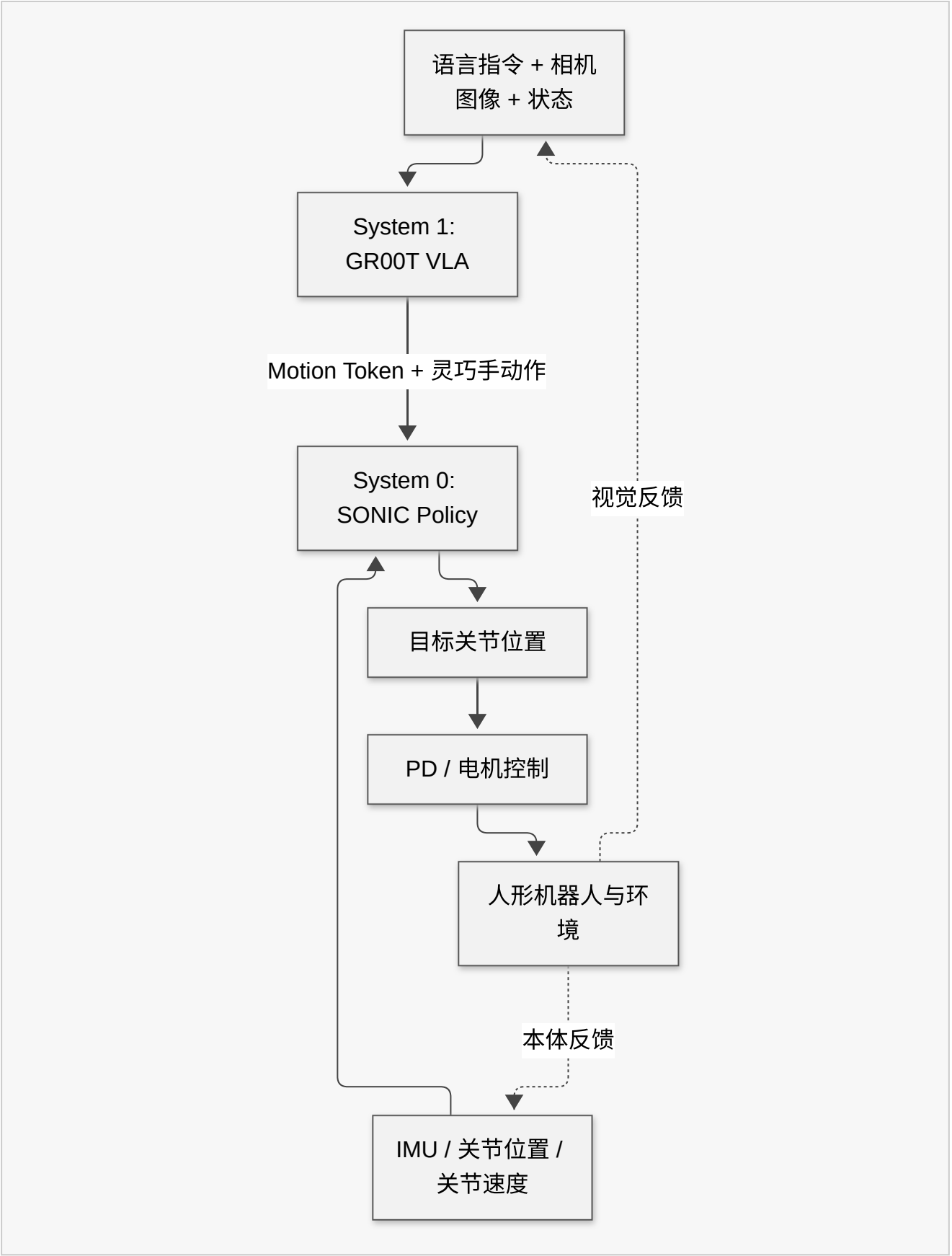
1. 整体架构：两个系统分别解决什么问题

1.1 为什么不能只靠一个模型?

人形机器人同时面临两类难题：一类是**语义与任务问题**，例如识别水杯、理解“把杯子放进箱子”，并选择合理的动作；另一类是**动力学与接触问题**，例如双腿站稳、重心移动、落脚和弯腰时不跌倒。

在实际系统中，一种有效的工程思路是将它们分成两个层次：

- **System 1 / VLA（高层）：** 处理相机图像、语言指令和机器人状态，预测未来动作或运动潜变量。
- **System 0 / WBC（低层）：** 处理运动参考、关节角、关节速度、IMU 等，实时生成全身关节控制目标。



重点： GR00T 不只是传统任务规划器，它可直接生成短时动作序列；SONIC 也不只是被动执行预设轨迹，它会结合实时机器人状态不断调整关节目标。

1.2 核心区别

对比维度	System 1：GR00T	System 0：SONIC
核心目标	理解任务，生成合适动作	跟踪动作，维持动态稳定
主要输入	图像、语言、机器人状态	运动目标、关节状态、IMU
主要输出	未来 Action Chunk	全身关节目标位置
典型模型	VLM + Flow Matching DiT	Motion Encoder + RL Controller
典型训练	大规模预训练、模仿微调	Motion Tracking + PPO
主要挑战	泛化、感知误差、长时任务	接触、平衡、运动耦合、抗扰
典型反馈	相机与任务状态，较低频	本体感觉，高频

1.3 两个闭环，而不是一个开环

VLA 可以根据新相机图像周期性地重规划；SONIC 则在每个控制周期利用新的关节和 IMU 状态调整动作。分层设计的目的，不是让高层忽略物理，而是使高层不必在每一次视觉推理中重新解决复杂的全身动态控制。

---

2. System 1：GR00T 如何理解任务并生成动作

2.1 VLA 是什么?

VLA = Vision-Language-Action Model（视觉-语言-动作模型）。 它比普通 VLM 多了一项能力：将感知和语言理解转换为机器人动作。

普通 VLM 往往输出文字:

$$(\text{Image}, \text{Language}) \longrightarrow \text{Text}.$$

VLA 则输出动作:

$$(\text{Image}, \text{Language}, \text{Robot State}) \longrightarrow \text{Action}.$$

例如指令“拿起桌子上的水杯”，VLA 需要判断杯子在哪里、机器人处于什么姿态，并预测伸手、接近、抓取等动作。

2.2 GR00T 的两部分结构

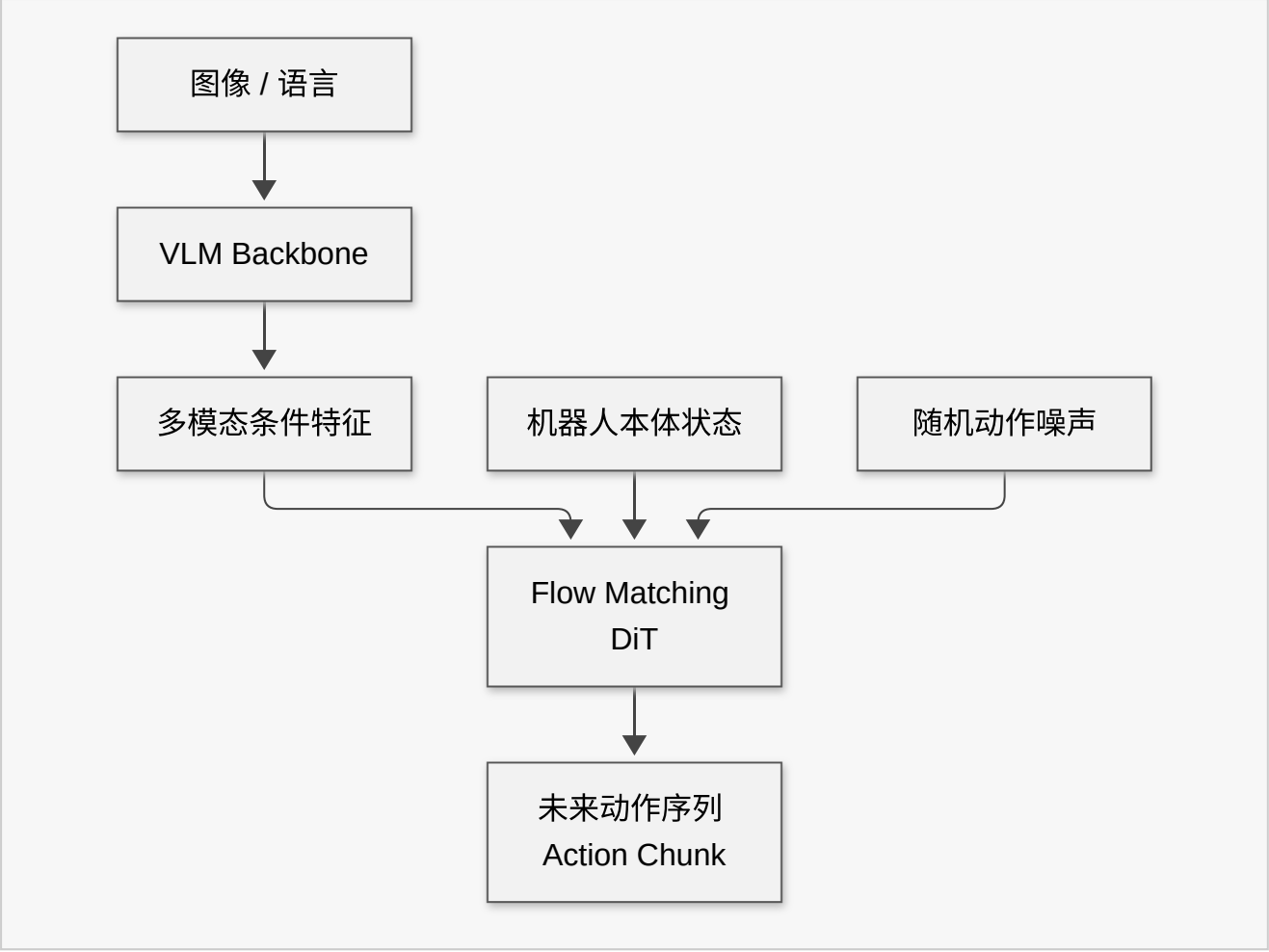
以 **GR00T N1.7** 为例，其主要由两个部分构成。[R3]

1. Vision-Language Backbone（VLM）

接收视觉和语言信息，提供多模态条件特征。N1.7 公开代码使用基于 Qwen3-VL 架构的 **Cosmos-Reason2-2B** 作为视觉语言骨干；它与早期 N1.5 / N1.6 的配置不同。

2. Flow Matching Action Head

以 VLM 特征和机器人本体状态为条件，用基于 **Diffusion Transformer（DiT）** 的动作头生成动作序列。这里的 Flow Matching 用来学习动作分布，而不是替代底层的物理控制器。



2.3 为什么一次预测多个动作?

GR00T 不是只能输出单个瞬时动作。动作头通常预测长度为 H 的短时序列:

$$A_t = [a_t, a_{t+1}, \ldots, a_{t+H-1}].$$

这里 a_t 表示一个动作时间步，H 表示 **Action Horizon**。N1.7 的默认长度为 **40 步**。[R3]

Action Chunk 的好处： 一次推理产生连续动作，减少每个电机控制周期都调用大型 VLA 的开销。**但不能理解为必须把 40 步全部执行完才再次观察环境**。 实际系统可以使用滚动时域控制（Receding Horizon）提前更新下一段动作。

2.4 Flow Matching 的直觉解释

Flow Matching 从噪声动作 x_0 出发，通过一个由网络预测的时间相关速度场，逐步得到符合当前图像和任务的动作 x_1:

$$\frac{\mathrm{d}x_\tau}{\mathrm{d}\tau} = v_\theta(x_\tau, \tau \mid I, l, s), \qquad \tau \in [0, 1].$$

可以把它想成：模型学习“怎样把随机动作噪声推向合理的动作轨迹”。这里的 τ 是**生成过程中的流时间**，不等于机器人执行动作的物理时间 t。

2.5 VLA 输出可以如何表示?

动作空间	含义	适用情况 / 限制
Joint Action	直接预测关节角或增量	接口直接；依赖具体机器人机构
End-Effector Pose	预测手部目标位姿	方便操作；需 IK / WBC 接口
Whole-Body Pose	预测全身姿态	表达力强；物理可执行性难保证
Latent Action	预测学得的运动潜变量	与预训练运控衔接；必须匹配解码器

GR00T + SONIC 的关键选择是 Latent Action。 VLA 不必直接回归每个全身关节，而是学习 SONIC 所定义的 **Motion Token 接口**，再由 SONIC 生成全身运动。

2.6 GR00T 的训练分为哪几步?

1. **预训练：** 用多机器人示教和人类视频学习视觉、语言及动作先验。N1.7 的公开介绍强调人类视频预训练和跨本体的相对末端动作表示；这些是通用 GR00T 能力，并不意味着所有本体都使用 SONIC latent action。[R3]
2. **任务微调：** 采集带有图像、语言、状态和动作标签的目标任务示教数据，使用监督式动作生成损失训练。
3. **部署评估：** 将新图像送入策略，不断生成与更新动作序列，记录任务成功率及失败原因。

训练数据可以抽象为:

$$\mathcal{D} = \{(I_t, l, s_t, a_t)\}_{t=1}^{N}.$$

在 SONIC 场景中，a_t 可以是 Motion Token 与双手动作的组合，而不是原始的全身关节角。

注意： 模仿学习是这一官方工作流的主要 VLA 微调方式；VLA 后训练也可以研究在线强化学习，但它不是部署 SONIC 的必要前提。

---

3. System 0：SONIC 如何学习全身运动控制

3.1 什么是全身运动控制（WBC）?

对固定机械臂而言，控制器主要保证末端执行器达到目标。对人形机器人而言，伸手时还要协调双腿、腰部、脚底接触以及全身重心；走路、爬行、弯腰、用脚踩踏板都要求身体各部分协同。

因此，**Whole-Body Control 不等于把所有关节 PID 放在一起**。它关心全身耦合动力学以及与环境的接触。

3.2 先认识一条常见替代路线：下半身 RL + 上半身 IK

NVIDIA 在 GR00T N1.5/N1.6 中提供过 **Decoupled WBC**：下半身强化学习用于行走和平衡，上半身逆运动学（IK）用于双臂跟踪。[R4]

其优点是模块清楚、容易调试；局限是上肢动作与步态可能需要额外协调，尤其在大幅弯腰、单腿支撑抓取和脚部操作时。

SONIC 则把多样全身运动映射到**统一的强化学习跟踪策略**。严格地说，SONIC 是学习式的通用全身运动跟踪控制器，并不是传统基于 QP 的 WBC 求解器。

3.3 SONIC 的核心任务：Motion Tracking

SONIC 的研究假设是：与其逐个设计“跑步策略”“蹲下策略”“爬行策略”，不如让同一个策略跟踪足够广泛的人体运动参考，逐渐学会通用的全身协调方式。[R1]

基本训练过程：

1. 收集人体动捕（Motion Capture）轨迹。
2. 对人体动作进行 **Retargeting**，转换为机器人的运动参考。
3. 在物理仿真中随机初始化机器人，要求策略跟踪参考运动。
4. 用 PPO 更新策略，使跟踪更准确、姿态更稳定、动作更平滑。
5. 加入动力学参数扰动和外力等 Domain Randomization，提高 Sim-to-Real 鲁棒性。

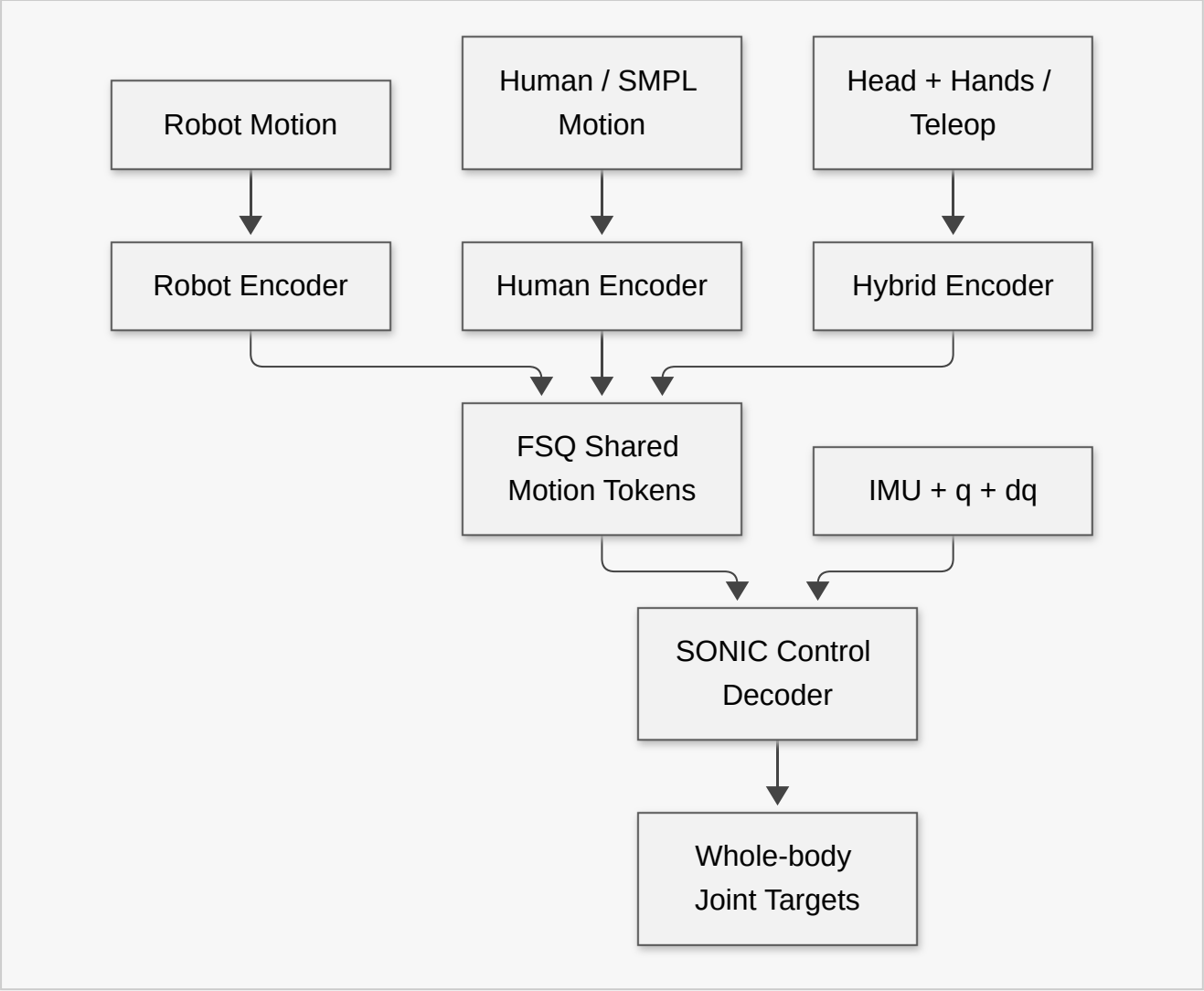
论文报告的规模约为 **700 小时动捕、超过一亿帧训练数据、最大约 4200 万参数、约 2.1 万 GPU 小时**。这些是论文的研究规模，不是复现一个可用控制器的最低门槛。[R1]

3.4 SONIC 的网络：编码器 + Motion Token + 控制解码器

SONIC 将不同来源的动作命令转换为共同的运动潜空间：

- **Robot Encoder：** 接收机器人关节运动参考。
- **Human Encoder：** 接收人体姿态 / SMPL 运动参考。
- **Hybrid / Teleop Encoder：** 接收头部、双手等关键点及其他运动约束。

这些编码器的输出由 **FSQ（Finite Scalar Quantization，有限标量量化）** 转换成共享的 **Universal Motion Token**。随后控制解码器根据 token 和机器人本体状态生成关节目标。[R1–R2]



最重要的直觉： 运动输入可以来自真人、VR、运动规划器或 VLA；只要它被编码为兼容的 Motion Token，就能调用同一个低层控制器。

3.5 SONIC 如何从 token 得到关节控制?

可以把 SONIC 写成:

$$\boxed{q_t^{\text{target}} = \pi_{\text{SONIC}}(s_t, z_t)}.$$

- s_t：当前本体感觉状态（关节位置、关节速度、IMU 等）。
- z_t：期望运动对应的 Motion Token。
- q_t^{target}：全身目标关节位置。

SONIC 在官方 Unitree G1 配置中控制 **29 个身体关节**。[R2]

随后底层伺服系统可以使用 PD 控制跟踪关节目标。若目标关节速度设为 0，一种简化表达为:

$$\tau_t = K_p(q_t^{\text{target}} - q_t) - K_d\dot{q}_t.$$

这只是**关节控制的示意公式**，不是 SONIC 网络本身的训练目标；实际电机控制还有安全限幅、硬件增益、通信延迟等因素。

3.6 SONIC 的强化学习训练：PPO + 跟踪奖励

SONIC 使用 PPO，通过密集的 Motion Tracking Reward 训练：

$$r_t \approx r_{\text{tracking}} - r_{\text{penalty}}.$$

跟踪奖励关注根节点、身体关键点、姿态和速度；惩罚项约束不平滑动作、关节越界和不合理接触。实际奖励由多个带权重的项组成，上式只表达总体思想。

训练中可采用 **Asymmetric Actor–Critic**：Actor 使用真机上能获取的观测；Critic 使用仿真中额外可见的精确状态，以提高训练效率。部署时只需 Actor / 控制解码器，不需要 Critic。

Domain Randomization 通常包含质量、摩擦、执行器响应、外部推力和初始姿态等变化。它有助于鲁棒性，但不能保证所有强扰动都不会跌倒。

3.7 SONIC 学到的是什么，又没有学到什么？

SONIC 学到的是：在特定机器人动力学约束下，对丰富运动参考进行全身协调跟踪。

SONIC **不会单凭语言自行完成目标识别、任务规划和物体检测**。这些任务仍需高层 VLA、运动生成器、规划器或遥操作人员提供目标。

---

4. 两个系统如何通过 Motion Token 连接

4.1 最关键的接口公式

GR00T 生成运动 token 与手部控制量；SONIC 接收 token 并输出身体关节目标：

$$\begin{aligned}(z_t, h_t) &= \pi_{\text{GR00T}}(I_t, l, s_t),\\ q_t^{\text{target}} &= \pi_{\text{SONIC}}(s_t, z_t).\end{aligned}$$

其中 h_t 是由动作接口单独提供给双手的指令，**不要把 14 维手部动作误当成 SONIC 的 29 维身体关节输出**。

4.2 官方实例中的准确维度

针对公开的 **Unitree G1 + SONIC + GR00T N1.7** 示例：[R4–R5]

$$\underbrace{a_t^{\text{VLA}}}_{78} = \big[\underbrace{z_t}_{64}, \underbrace{h_t^L}_{7}, \underbrace{h_t^R}_{7}\big].$$

项目	配置 / 含义
Motion Token	64 维全身运动潜表示
左手动作	7 维
右手动作	7 维
合计 VLA 输出	每步 78 维
Action Horizon	40 个动作步
SONIC 身体关节	29 个关节目标

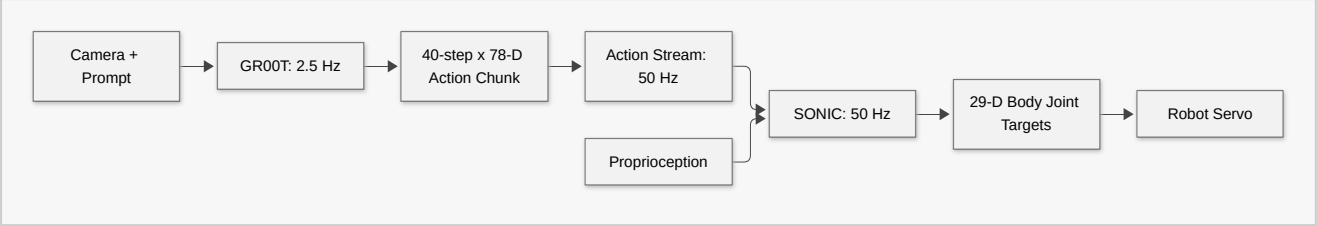
这里的 **64 维**是共享运动表示的向量维度，不是 64 个“走路、转身、蹲下”这样的固定技能标签；其潜变量由 SONIC 的量化编码器学习得到。

4.3 两种控制频率如何配合？

官方 N1.7 工作流提供以下示例配置：[R4–R5]

模块	示例频率	对应时间
GR00T VLA 推理	2.5 Hz	约每 400 ms 更新一次 Action Chunk
动作发布 / SONIC	50 Hz	约每 20 ms 更新一次运动目标与关节动作
Action Horizon	40 步	按 50 Hz 计，预测窗口 0.8 s
电机伺服	依硬件实现	可能比策略更高频，不应混同

注意 **0.8 s 是单次预测窗口，不是必须无反馈地执行 0.8 s**。例如每 0.4 s 更新一批新动作，上一批动作可被部分使用并由新动作接替。



4.4 为什么这种分层结构更容易泛化？

如果 VLA 直接预测全身关节角，那么它需要同时学会视觉条件动作、身体协调和动态平衡。Latent Action 接口将后两项中的许多能力交给已经预训练的 SONIC。

对于某些突然的外力或落脚误差，即使高层 token 没有变化，SONIC 仍可因 s_t 改变而生成不同的关节目标。**但如果 VLA 给出了不可执行的目标，SONIC 不能保证一定挽救任务**。

4.5 分层接口必须处理的工程细节

- **时间同步：** 图像、动作 chunk 和本体状态应有一致的时间戳。
- **关节顺序与单位：** 明确 rad / deg，左右手和 29 维关节映射。
- **模型版本匹配：** VLA 微调权重、SONIC checkpoint、编码器和观测配置必须相互兼容。
- **延迟与切换：** 新旧动作 chunk 切换时避免不连续；推理延迟过大时进入安全策略。
- **安全边界：** 实现急停、关节限位、失联保护、落地保护以及仿真先行测试。

---

5. 完整例子：走路、抓取、踩踏板、投放

5.1 示例任务

语言指令： “走到桌边拿起易拉罐，走向垃圾桶，用脚踩开踏板，把易拉罐投进去。”

这是一项典型的 **Loco-Manipulation（移动操作）**：机器人既需要双手操作，也要移动、保持平衡并进行脚部接触。

5.2 各模块分别做什么？

阶段	GR00T / System 1	SONIC / System 0
找到易拉罐	根据视觉确定目标与接近方向	不负责识别物体
走向桌子	生成短时全身动作序列	跟踪步态、协调支撑
伸手抓取	预测手臂动作与手指控制	协调躯干、双臂、双腿
走向垃圾桶	根据视觉持续更新动作	行走和平衡反馈
用脚踩踏板	给出相关脚部和身体动作意图	单腿支撑、姿态与接触协调
投入易拉罐	生成投放动作	稳定全身动作执行

SONIC 论文报告，该复杂的“易拉罐到垃圾桶”任务在 **10 次试验中成功 60%**；多类 VLA 移动操作任务的平均成功率约为 **75%**。这些结果展示了方案可行性，同时也表明其可靠性尚未达到不受约束的真实环境部署要求。[R1]

5.3 一个容易混淆的边界

SONIC 可以稳定执行“把脚移动到某个动作参考所对应的位置”，却不会仅凭一句“踩踏板”就自行完成视觉定位并推断落脚目标。高层提供什么动作意图，依然十分重要。

---

6. 从零训练、采集数据与真机部署

6.1 Step 1：获得可用的 System 0

推荐优先复用官方 SONIC checkpoint，在仿真与受控真机环境中验证，再决定是否从头训练。

若从头训练，核心路径是：

```
人体动捕 / 机器人运动
       ↓ Retargeting + 数据筛选
机器人参考运动库
       ↓ Isaac Lab 并行仿真
PPO Motion Tracking + Auxiliary Loss
       ↓ 验证 / Sim2Sim / Sim2Real
SONIC Actor + Motion Encoder
```

官方公开了 SONIC 的训练指南、配置以及 BONES-SEED 相关数据处理流程。[R2] **注意：** BONES-SEED 等公开资源和论文所述 700 小时完整动捕数据不应视为完全相同的数据集。

建议先检查：直立稳定、行走/转身、弯腰和单腿支撑、外力恢复、动作切换是否连续，再进入 VLA 数据采集。

6.2 Step 2：通过 SONIC 进行 VR 遥操作示教

人类操作员通过 VR / 动捕提供人体动作，SONIC Encoder 将其转换为 Motion Token；SONIC Controller 执行这些 token 驱动机器人。同步记录：

- 机器人相机图像（最好覆盖目标和手部接触区域）；
- 任务语言标签、时间戳、机器人状态；
- SONIC Motion Token（64 维）及双手动作（7+7 维）；
- 每一条示教的成败、复位与中断状态。

官方工作流支持导出 **LeRobot** 格式示教数据，并建议先采集足量成功轨迹，再进行清洗与任务微调。[R4]

6.3 Step 3：微调 GR00T，让它输出 SONIC 动作

目标从原始关节角监督变为：

$$(I_t, l, s_t) \longmapsto (z_t, h_t).$$

以官方 N1.7 的任务流程为例，需要使用匹配的 embodiment tag：

```
# 官方仓库中的示例参数，需先安装 Isaac-GR00T 并准备数据
uv run python gr00t/experiment/launch_finetune.py \
  --base-model-path nvidia/GR00T-N1.7-3B \
  --dataset-path /path/to/lerobot_dataset \
  --embodiment-tag UNITREE_G1_SONIC \
  --modality-config-path gr00t/configs/data/embodiment_configs.py \
  --num-gpus 4 \
  --output-dir /path/to/output
```

命令仅展示关键接口，具体其余训练参数、软件依赖和脚本路径，以当前官方文档和代码版本为准。[R4]

6.4 Step 4：通过 PolicyServer + SONIC 部署

可以使用 GPU 计算机运行 GR00T PolicyServer；机器人侧读取相机、本体反馈，接收动作块并通过 SONIC 控制器执行。[R5]

```
GPU 计算机：GR00T PolicyServer
     ↕ 视觉 / 状态 / Action Chunk
机器人计算单元：VLA Client + SONIC C++ 推理
     ↕ 高频本体感知、关节目标
电机伺服 + 人形机器人
```

真实部署时先做 **Sim2Sim**，再做低速、受限空间的真机验证。官方提供了 MuJoCo 仿真流程和 C++ 推理栈，不建议将首次运行直接放到无保护的真实环境。[R2, R5]

6.5 Step 5：如何评价整体系统？

层次	建议评价指标	主要帮助定位的问题
VLA	Task Success Rate、泛化、动作生成延迟	是否理解并选择正确动作
Motion Token	重建误差、接口适配、动作平滑性	潜表示是否表达目标运动

层次	建议评价指标	主要帮助定位的问题
SONIC	Motion Tracking Success、关键点误差、跌倒率	动作是否被稳定执行
系统	端到端成功率、延迟、恢复能力、安全事件	模块交互与真实部署可靠性

排查失败要分层： 先看 VLA 是否生成正确运动，再看 token 是否能表达该运动，最后看 SONIC 是否稳健地执行。不能把所有任务失败都归咎于 VLA。

---

7. 研究方向、实验设计与常见误区

7.1 应该分别改进什么？

研究目标	优先改进模块	常用思路
少示教、跨物体泛化	System 1	数据增强、预训练、模仿微调
长时任务与视觉纠错	System 1	反馈重规划、任务级后训练
更自然的全身动作	System 0	大规模 Motion Tracking
更好的抗扰 / Sim-to-Real	System 0	PPO、Domain Randomization、系统辨识
脚手协同的复杂接触	两层与接口	接触丰富的数据、联合评价
更稳定的高低层衔接	接口与部署	Token 设计、时序对齐、安全过滤

7.2 建议的最小消融实验

消融 A：动作表示

保持数据量、VLA 骨干和底层控制预算尽量一致，比较：1. 直接关节动作；2. 明确全身姿态；3. SONIC Motion Token。关注成功率、动作平滑、对不可执行指令的敏感性。注意不同表示可能改变可学习性和控制空间，结论不能简单归因于“压缩维度更小”。

消融 B：闭环频率

保持底层 SONIC 不变，比较不同 VLA 更新频率、动作 chunk 执行长度以及推理延迟。关注动态目标变化、接触切换和失败恢复。

消融 C：低层鲁棒性

固定高层 token，改变仿真质量/摩擦、IMU 噪声和外部扰动，比较成功率、姿态误差及跌倒率。

7.3 五个常见误区

1. **“System 1 是语言规划器，System 0 才生成动作。”** 不完全正确。VLA 本身可以直接生成短时动作序列。
2. **“SONIC 能理解任务和识别物体。”** 不正确。SONIC 是运动控制模型，通常不负责高层视觉语义。
3. **“Motion Token 就是预定义的技能编号。”** 不正确。这里是学习得到的量化运动潜空间。
4. **“一个 40 步 Action Chunk 就是 0.8 秒开环控制。”** 不正确。窗口长度与实际执行长度可不同。

5. **“换一个 SONIC checkpoint，VLA 输出可以直接通用。”** 不能保证。编码器、潜空间和观测配置改变会造成接口不匹配。

---

8. 总结与参考资料

8.1 最值得记住的三条结论

第一，分工： GR00T 将视觉和语言条件转换为机器人未来动作，SONIC 根据实时状态稳定地执行全身运动。

第二，接口： Motion Token 是连接高层智能和低层运动技能的桥梁。在已公开的 G1 + SONIC 配置中，它是 64 维；加上双手动作后，VLA 每步输出 78 维。

第三，训练： SONIC 主要依靠大规模 Motion Tracking + PPO 学习全身运动能力；GR00T 主要通过多模态预训练和任务示教微调学习高层动作生成。

可将整个系统概括为：

$$\boxed{(I_t, l, s_t) \xrightarrow{\text{GR00T}} (z_t, h_t) \xrightarrow[s_t]{\text{SONIC}} q_t^{\text{target}} \xrightarrow{\text{Servo}} \text{Robot}}.$$

8.2 参考论文与官方资源

- **[R1] SONIC 论文与项目主页：** Luo et al., *SONIC: Supersizing Motion Tracking for Natural Humanoid Whole-Body Control*, *Science Robotics* 11(117), 2026. 项目主页 · arXiv · DOI
- **[R2] SONIC 官方文档与训练指南：** GR00T-WholeBodyControl 文档 · Training Guide
- **[R3] GR00T N1.7 官方仓库：** NVIDIA/Isaac-GR00T
- **[R4] GR00T + SONIC 官方 VLA 工作流：** VLA Workflow: Collect, Fine-tune, Deploy
- **[R5] SONIC VLA 推理与部署：** VLA Inference Guide
- **[R6] WBC 代码库：** NVlabs/GR00T-WholeBodyControl

整理日期：2026 年 10 月 16 日。本文的尺寸、频率与模型版本针对上述公开实现，不代表所有人形机器人系统的统一标准。